book 7

javascript es2026 and typescript 7

a complete manual for two layers

Raeliosol

version 6.1, compiled 10 2026

Contents

1. javascript and its toolchain ... 8
   1.1. editions, approval against shipping ... 8
   1.2. the test framework ... 10
   1.3. the no build step thesis ... 11
2. values and lexical structure ... 13
   2.1. declarations and the dead zone ... 13
   2.2. the words ... 14
   2.3. seven primitives, one lying operator ... 15
   2.4. literals ... 16
   2.5. two equalities and the same-value operator ... 17
   2.6. truthiness ... 17
3. functions, scope, and closures ... 19
   3.1. three forms, one value ... 19
   3.2. this binds by call site ... 19
   3.3. parameters ... 21
   3.4. closures and the module pattern ... 21
   3.5. the iife, retired ... 22
   3.6. higher order functions ... 23
   3.7. recursion and the tail call promise ... 24
4. objects, prototypes, and classes ... 26
   4.1. literals and key order ... 26
   4.2. descriptors ... 26
   4.3. delegation ... 27
   4.4. classes ... 28
   4.5. private fields ... 30
   4.6. spread and rest ... 32
5. iteration and collections ... 33
   5.1. the iterator protocol ... 33
   5.2. generators ... 34
   5.3. lazy pipelines ... 35
   5.4. keyed collections ... 37
   5.5. set algebra ... 39
   5.6. weak references ... 40
   5.7. binary data ... 41
6. async javascript ... 44
   6.1. the loop, one run through ... 44
   6.2. promises from scratch ... 44
   6.3. async and await, and Promise.try ... 45
   6.4. the four combinators ... 47
   6.5. async iteration ... 48
   6.6. cancellation ... 49
7. modules and resources ... 52
   7.1. the export forms, and live bindings ... 52
   7.2. dynamic import ... 53
   7.3. json modules and import attributes ... 54
   7.4. import.meta and module relative paths ... 54
   7.5. explicit resource management ... 55
8. the standard library by edition ... 58

8.1. change by copy, and grouping ... 58
8.2. escaping patterns ... 59
8.3. errors that identify themselves ... 60
8.4. json that keeps its digits ... 61
8.5. temporal, measured live ... 62
9. the heap and the collector ... 66
9.1. the heap as v8 reports it ... 66
9.2. the scavenger: a semispace copy ... 66
9.3. marking, sweeping, and orinoco ... 67
9.4. external memory and the large object space ... 68
9.5. weak references under a real collector ... 68
9.6. the particle kernel: numbers against boxes ... 69
9.7. the flags, measured on a child run ... 71
10. profiling and the event-loop tail ... 73
10.1. the profile file, parsed ... 73
10.2. the heap profile, sampled ... 74
10.3. the inspector, from inside the process ... 75
10.4. the loop, and where delay is born ... 76
10.5. percentiles for a handler loop ... 77
10.6. the row kernel at ten million rows ... 79
10.7. which tool when ... 82
10.8. across the six books ... 82
11. typescript 7: the type layer ... 84
11.1. a checker, not a runtime ... 84
11.2. the native 7 compiler ... 85
11.3. the honest es2026 story ... 85
11.4. tsconfig anatomy ... 86
11.5. the two stage verify chain ... 87
12. type syntax ... 89
12.1. annotations and inference ... 89
12.2. primitives against their runtime values ... 90
12.3. literal and union literal types ... 91
12.4. arrays and tuples ... 91
12.5. aliases and typeof queries ... 92
13. the type landscape ... 94
13.1. the two tops and two bottoms ... 94
13.2. structural typing ... 95
13.3. assignability ... 96
13.4. narrowing, previewed ... 97
14. typing functions ... 99
14.1. overloads ... 99
14.2. parameter forms ... 99
14.3. generics with constraints ... 100
14.4. function type compatibility ... 101
14.5. the checked this ... 102
15. typing objects ... 103
15.1. interfaces against aliases ... 103
15.2. optional and readonly ... 104
15.3. excess properties ... 104
15.4. index signatures ... 105
15.5. extension and merging ... 106

16. unions and narrowing 107
16.1. built in guards 108
16.2. user defined guards and assertions 108
16.3. intersections 110
17. type-level programming 111
17.1. querying types 111
17.2. mapped types 111
17.3. conditional types, infer, and recursion 112
17.4. template literal types 113
17.5. variadic tuples and the library operators 113
18. classes and decorators 115
18.1. abstract classes 115
18.2. this types 116
18.3. generic classes and structure 117
18.4. decorators 118
18.5. implements and protected 120
19. declarations, namespaces, and emit 122
19.1. resolution for the type layer 122
19.2. namespaces against modules 123
19.3. hand written declarations 124
19.4. what emit leaves behind 125
19.5. node's type stripping 127
19.6. the emitted companions 128
20. typescript idioms 130
20.1. branding 130
20.2. the exhaustiveness alarm 130
20.3. errors as values 131
20.4. freezing deep 132
20.5. state machines 132
20.6. typed events 133
20.7. defaults 134
20.8. satisfies and const parameters 135
20.9. overloads against unions, and generics economy 135
21. react 19: components as functions 137
21.1. a function that returns elements 137
21.2. hooks and their rules 139
21.3. derive first, memo on evidence 140
21.4. what 19 changed 142
21.5. forms as actions 143
21.6. boundaries catch, but not here 144
21.7. the island seam 145
21.8. the server as oracle 147
21.9. the honest trade 148
22. svelte 5: runes and compiled reactivity 149
22.1. a compiler, not a runtime 149
22.2. runes, outside any component 150
22.3. stores against runes 151
22.4. props, callbacks, and bindable 153
22.5. snippets 154
22.6. the island seam 155
22.7. transitions 155

22.8. the server as oracle ... 156
22.9. when svelte beats react ... 157
23. api kernel ... 159
23.1. the handler contract ... 159
23.2. the route table ... 160
23.3. the error envelope ... 160
23.4. the wire codec ... 161
23.5. request ids ... 162
23.6. the app skeleton and the composition file ... 163
23.7. testing the kernel ... 164
24. middleware ... 166
24.1. the chain type ... 166
24.2. the access log ... 166
24.3. the handler deadline ... 167
24.4. the exception-to-envelope adapter ... 168
24.5. wiring order and why ... 169
24.6. testing middleware ... 170
25. the user resource ... 172
25.1. the representation ... 172
25.2. decoding with strictness ... 173
25.3. validation from first principles ... 174
25.4. patch merge semantics ... 174
25.5. keyset pagination ... 175
25.6. the status contract ... 176
25.7. golden handler tests ... 177
26. authn ... 179
26.1. password storage with scrypt ... 179
26.2. login and timing flattening ... 180
26.3. server-side sessions ... 181
26.4. cookie mechanics ... 181
26.5. jwt from scratch with hmac ... 182
26.6. jwt with eddsa ... 183
26.7. refresh rotation and reuse detection ... 184
26.8. testing auth without walls ... 185
27. authz and rbac ... 186
27.1. authentication versus authorization ... 186
27.2. roles and the grant table ... 186
27.3. policies as functions ... 187
27.4. 401 versus 403 ... 188
27.5. object-level authorization ... 188
27.6. deny by default with a route audit ... 189
27.7. the rbac matrix test ... 191
28. store ... 193
28.1. node:sqlite from first principles ... 193
28.2. migrations test driven ... 193
28.3. one statement, prepared once ... 194
28.4. the transaction idiom ... 195
28.5. the snapshot sqlite actually gives you ... 195
28.6. the lost update and the compare-and-swap answer ... 196
28.7. analytics in one query ... 197
28.8. store tests without docker ... 198

29. concurrency control 199
29.1. one thread, many interleavings 199
29.2. strong etags over the exact bytes 199
29.3. the duel: two patchers, one version 200
29.4. idempotency keys and stored snapshots 201
29.5. the in-flight window 202
29.6. the reports route over the wire 202
30. caching 204
30.1. what caching promises 204
30.2. LRU over one Map 204
30.3. TTL beside LRU 205
30.4. the stampede guard: singleflight from a promise map 206
30.5. invalidation on write 206
30.6. what never gets cached 207
31. rate limiting 209
31.1. why limits exist 209
31.2. the token bucket from first principles 210
31.3. one bucket per key 210
31.4. the 429 contract 212
31.5. what to key on 212
31.6. limits under test 213
32. observability 215
32.1. what a service owes its operators 215
32.2. the metrics registry 215
32.3. the prometheus text format 216
32.4. recording every request 217
32.5. the internal listener and the inspector 218
32.6. one request fully observed 219
33. load and p99 221
33.1. the event loop is the worker 221
33.2. graceful shutdown with drain 222
33.3. admission control 223
33.4. the load harness 224
33.5. p99 from nearest rank 225
33.6. backpressure and writable sockets 226
33.7. no orphans left behind 226
34. ship it 228
34.1. the image 228
34.2. compose one command 229
34.3. the docker lane as a gate 231
34.4. the shipped checklist 232
35. the test suite 234
35.1. what the suite holds 234
35.2. the surface 234
35.3. doubles without a framework 235
35.4. the fake clock owns every deadline 236
35.5. properties as seeded loops 237
35.6. dated and declined 237
35.7. coverage, opt-in 239
35.8. what the suite proves 239
36. the typed layer 241

36.1. one vehicle, two languages 241
36.2. the strict tsconfig 242
36.3. types at the kernel seams 242
36.4. the store port as interfaces 243
36.5. what the types prove 244
37. the typed suite 246
37.1. the same suite over the ts sources 246
37.2. what strictness catches 247
37.3. what the vectors still own 248
38. capstone: the query engine, microfrontend 250
38.1. layer one: conditions without a compiler 250
38.2. the lazy chain 252
38.3. snapshots that outlive the process 255
38.4. layer two: the typed facade 258
38.5. the third face: one engine, two islands 262
38.6. the domain and the two islands 262
38.7. the server, plain node:http 264
38.8. profiled, and why 266
38.9. the suite and the gate 268
38.10. the service part 270
39. appendices: coverage and sources 271
39.1. keyword coverage 271
39.2. stdlib coverage 273
39.3. pinned sources 275

1. javascript and its toolchain

This book is two books in one language, with a third layer on top. Part one, chapters 1 through 10, is javascript itself: values, functions, objects, iteration, async, modules, the standard library, the heap, and the profiler, written as plain `.mjs` files that node 26 executes directly with no compiler, no bundler, and no configuration file. Part two, chapters 11 through 20, is the typescript layer on top of the same runtime, and it adds exactly one tool, `tsc`. Part three, chapters 21 and 22, is the framework layer, react and svelte, and it adds the second tool, `vite`. The splits are the language's own shape: typescript is a checker whose annotations erase, and every rule in part two is a promise about code that part one already explains, while the browser target of part three is the first place in this book where two engines with two module graphs have to be reconciled at all.

The runtime under both parts is measured, not assumed. This book runs node v26.3.0 with V8 14.6.202.34-node.20 and npm 12.0.2 on windows, probed 2026-09-13, and the first sample reads its facts from the process itself:

```
// the honest version report: the process and its engine, read from the
// runtime rather than transcribed. npm ships its own version through its
// binary, so it stays a prose fact of this book, not a probe
export function runtimeInfo() {
  return {
    node: process.version,
    v8: process.versions.v8,
  };
}
```

listing 1: the runtime report, read from the process rather than transcribed (source: javascript/samples/src/ch01-toolchain.mjs, lines 4-12)

Everything a chapter claims about engine behavior is either executed by a test in the samples project or marked as a fragment and never executed. Where the specification and this node disagree, the chapter says so out loud, and the disagreements start early.

1.1. editions, approval against shipping

The language ships as annual editions. Ecma's tc39 committee approves one each june, and the edition then fans out: spec text first, engine implementations on their own schedules, node releases picking up V8 versions on a third schedule. The seven editions this book's runtime spans:

```
export const EDITIONS = [
  ["es2020", "optional chaining, nullish coalescing, bigint, dynamic import"],
  ["es2021", "logical assignment, string replaceAll, promise any"],
  ["es2022", "class fields, top level await, object hasOwn"],
  ["es2023", "array findLast, toSorted, toReversed, hashbang"],
  ["es2024", "regexp v flag, resizable array buffers, promise withResolvers"],
  ["es2025", "iterator helpers, set methods, import attributes, regexp escape"],
  ["es2026", "seven features, the matrix above, approval 2026-06-30"],
];
```

listing 2: the edition timeline as data, es2020 through es2026 (source: javascript/samples/src/ch01-toolchain.mjs, lines 80-88)

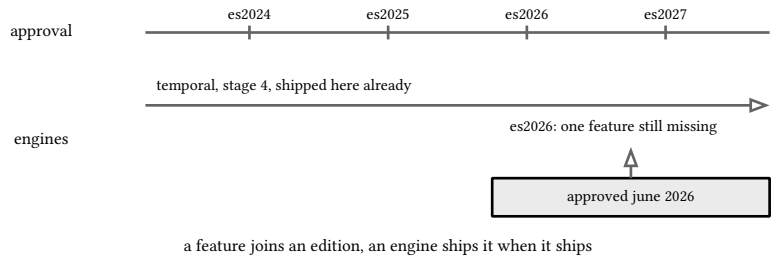


figure 1: two clocks, approval in june, engines trailing on their own schedules

ES2026 was approved 2026-06-30 with seven features: `Array.fromAsync`, `Error.isError`, `Math.sumPrecise`, base64 and hex codecs on `Uint8Array`, `Iterator.concat`, JSON source text access with `JSON.rawJSON`, and `Map` and `WeakMap` `getOrInsert`. On this node, five of the seven are present, one is half shipped, and one is absent, and the matrix says which is which:

```
export function featureMatrix() {
  const probes = [
    ["Array.fromAsync", typeof Array.fromAsync],
    ["Error.isError", typeof Error.isError],
    ["Math.sumPrecise", typeof Math.sumPrecise],
    [
      "Uint8Array base64 and hex",
      typeof Uint8Array.fromHex === "function" &&
        typeof Uint8Array.fromBase64 === "function"
        ? "function"
        : "undefined",
    ],
    ["Iterator.concat", typeof Iterator?.concat],
    ["JSON.rawJSON", typeof JSON.rawJSON],
    [
      "Map and WeakMap getOrInsert",
      typeof Map.prototype.getOrInsert,
    ],
  ];
  return probes.map(([feature, kind]) => ({
    feature,
    status: kind === "function" ? "present" : "absent",
  }));
}
```

listing 3: the es2026 matrix, each feature probed by typeof against this runtime (source: javascript/samples/src/ch01-toolchain.mjs, lines 27-50)

`Math.sumPrecise` is the absent one, `typeof Math.sumPrecise` is `"undefined"` here. The json feature is the half: `JSON.rawJSON` and `JSON.isRawJSON` exist, but `JSON.parse` still takes two parameters, text and reviver, so the spec's source text access through a third argument does not run on this node. The arity is the whole evidence:

```
// json.parse source access is the one es2026 feature this node half
// shipped: rawJSON is here, the reviver's third argument is not, and
// the arity is the evidence
export function jsonParseArity() {
  return JSON.parse.length;
}

// temporal reached stage 4 and is slated for the es2027 edition; this
// node already ships it unflagged, an engine ahead of its edition
export function temporalStatus() {
  return {
    shipped:
      typeof Temporal === "object" &&
      typeof Temporal?.Instant === "function",
    edition: "es2027",
  };
}
```

listing 4: the arity probe, and temporal shipped ahead of its edition (source: javascript/samples/src/ch01-toolchain.mjs, lines 52-68)

Temporal runs the other direction: stage 4 and slated for the ES2027 edition, yet fully present and unflagged on this runtime, which the part one chapters on dates use without ceremony. A chapter states what this node does, names the edition a feature belongs to, and marks spec text that cannot execute here as a fragment.

1.2. the test framework

Part one tests with `node:test`, the runner built into the runtime: no dependency, no configuration, discovery by glob. The samples project opened with one smoke suite that pinned two edition facts:

```
import assert from "node:assert/strict";
import { test } from "node:test";

// measured on node 26.3.0: temporal ships without a flag and
// Temporal.Now.instant() hands back a Temporal.Instant
test("Temporal.Now.instant() returns a Temporal.Instant", () => {
  const instant = Temporal.Now.instant();
  assert.ok(instant instanceof Temporal.Instant);
  assert.match(instant.toString(), /^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}/);
});

// Map.prototype.getOrInsert inserts on a miss and returns the
// existing value on a hit without touching the entry
test("Map.prototype.getOrInsert inserts once and hits thereafter", () => {
  const counts = new Map();
  assert.equal(counts.getOrInsert("a", 0), 0);
  assert.equal(counts.getOrInsert("a", 99), 0);
  assert.equal(counts.size, 1);
});
```

listing 5: the smoke suite, temporal and getOrInsert measured on day one (source: javascript/samples/test/smoke.test.mjs, lines 1-20)

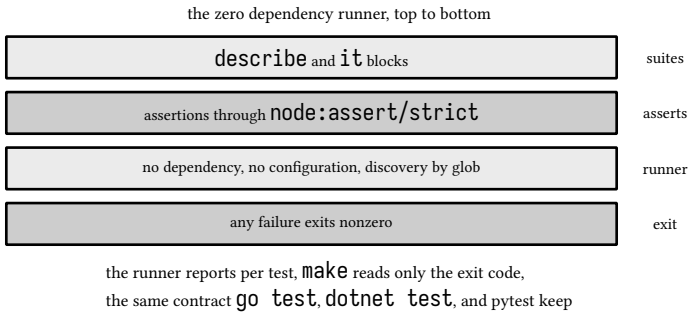


figure 2: the test framework stacked, blocks to assertions to the exit code

Suites grow into `describe` and `it` blocks asserting through `node:assert/strict`, the runner reports per test, and any failure exits nonzero. That exit code is the only interface `make` needs, the same contract every other book in this corpus keeps with its own runner, `go test`, `dotnet test`, or pytest.

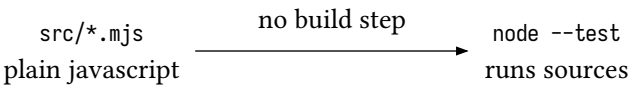
1.3. the no build step thesis

Part one's project is `javascript/samples/`, and its whole manifest is fifteen lines:

```
{
  "name": "jsbook-samples",
  "private": true,
  "type": "module",
  "engines": {
    "node": "≥26"
  },
  "scripts": {
    "test": "node --test \"test/**/*.test.mjs\"",
    "verify": "node --test \"test/**/*.test.mjs\""
  }
}
```

listing 6: the part i manifest, no dependencies, no build (source: javascript/samples/package.json, lines 1-13)

`type: module` makes every file an es module, `engines` records the node floor, and `verify` is one stage: node's own test runner over the sources. Nothing compiles, nothing is emitted, and the file a listing shows is byte for byte the file the test executed. Part two's `tssamples` project keeps the two stage chain instead, `tsc` typechecks and emits, then node tests the emitted `dist`, and that contrast is deliberate: the typescript chapters are about what a checker adds, so they test what the checker produced.



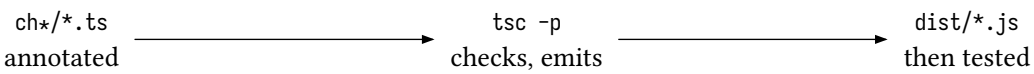


figure 3: the two verify chains, one stage for part i, two for part ii

Two more projects arrive with their own chapters, part three's fwsamples project for react and svelte and the capstone project of chapter 38, each reusing the chain its layer built. Every chapter of part one adds one module

under `samples/src/` and one suite under `samples/test/`, and the sample reads its own manifest to keep the contract honest:

```
// the toolchain contract of part i: sources are .mjs, node runs them
// directly, the test command needs no build stage. this file reads
// its own project manifest to show what that contract looks like
export function projectScripts() {
  const pkg = JSON.parse(
    readFileSync(new URL("../package.json", import.meta.url), "utf8"),
  );
  return { name: pkg.name, type: pkg.type, verify: pkg.scripts.verify };
}
```

listing 7: the project manifest read from the sample that runs under it (source: javascript/samples/src/ch01-toolchain.mjs, lines 70-78)

NOTE: the two tools, and why a bundler waits for part three

Tooling enters this book twice, once per layer that needs it. Part two adds `tsc`, a checker whose annotations erase, and part three adds `vite`, a bundler. The honest argument for the wait: a book pinned to one runtime has nothing to reconcile, node 26 runs the editions of the last seven years natively, and the one feature it lacks cannot be polyfilled anyway, since `Math.sumPrecise` is a built in whose absence is the fact being reported. A browser island target is the first place a bundler reconciles anything real: the island ships jsx the browser cannot parse and a component graph the server never runs, and one build step turns both into a single esm file a script tag can load. Vite stays a build step, never a runtime, and `node --test` remains the only runner through the last page.

sources: nodejs.org api and release documentation, tc39.es ecma 262 edition list and finished proposals, developer.mozilla.org edition guides, accessed 2026-09-13. Runtime behavior verified live with node v26.3.0, V8 14.6.202.34-node.20, npm 12.0.2 on windows, 10 tests green through `npm run verify` in `javascript/samples`.

2. values and lexical structure

Values come before types. This chapter is the runtime half of the language's foundation, how bindings come into existence, what the primitive types are and how `typeof` reports them, how literals are spelled, and what the two equality operators actually compare. The typescript spellings for these same values, annotations and generics and overloads, live in part two, chapter 12 onward; nothing in this chapter requires a compiler, and every fact below is executed by a test.

2.1. declarations and the dead zone

Three declaration forms survived into modern javascript, and each makes a different promise. `var` is function scoped and redeclares silently. `let` is block scoped, one binding per name per block. The difference is observable before either line even runs, because `let` and `const` bindings exist from the top of their scope but stay uninitialized until their declaration line, and reading them inside that window throws:

```
// the temporal dead zone as an observable fact: the let binding exists
// from the top of its scope, reads before the declaration line throw,
// and the probe hands the error class back for the test to pin
export function tdzProbe() {
  try {
    return `read ${sneaky}`;
  } catch (e) {
    return e.constructor.name;
  }
  let sneaky = 1;
}
```

listing 8: the temporal dead zone, caught rather than described (source: javascript/samples/src/ch02-values.mjs, lines 3-13)

The `let sneaky` line below the `catch` is unreachable, but it is still part of the function's scope, which is exactly why the read above it throws instead of resolving to an outer name. `var` has no such window, it hoists initialized to `undefined`, and `const` behaves like `let` with one addition:

```
// var against let against const, one function of facts: var redeclares
// silently and scopes to the function, let scopes to the block, const
// freezes the binding and not the value
export function declarationKinds() {
  const facts = [];
  var note = "first declaration";
  var note = "second declaration wins";
  {
    let shadow = "inner block";
    facts.push(shadow);
  }
  facts.push(note);
  const counter = { hits: 0 };
  counter.hits = 1;
  facts.push(`const object mutated to ${counter.hits}`);
  return facts;
}

export function constReassignProbe() {
  const fixed = 1;
  try {
    fixed = 2;
    return "reassigned";
  } catch (e) {
    return e.constructor.name;
  }
}
```

listing 9: var redeclares, let shadows per block, const freezes the binding not the value (source: javascript/samples/src/ch02-values.mjs, lines 15-41)

The third fact is the one that surprises: `const` makes the binding unreassignable, not the value immutable. A `const` object still mutates, and the mutation is ordinary code, while rebinding the name is a `TypeError`. This book uses `const` for almost every binding, `let` only inside loops that rebind, and `var` only when quoting it.

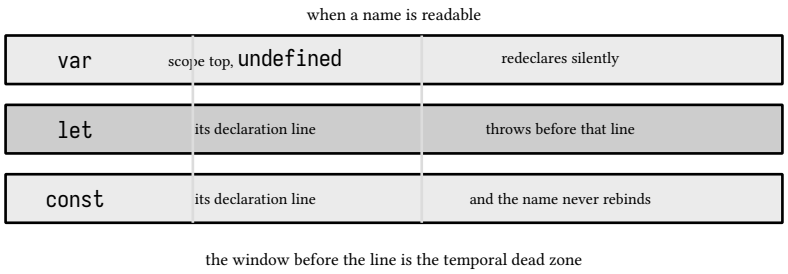


figure 4: the binding lifecycle of each declaration form

2.2. the words

The grammar freezes its vocabulary in strata. 35 words are reserved outright, `if` and `typeof` and `with`, and none of them can ever name a binding. 4 more, `let`, `static`, `yield`, and `await`, are reserved only inside strict mode and module code, which is every file this book runs. 7 are held for the future, `enum`, `implements`, `interface`, `package`, `private`, `protected`, and `public`, kept out of circulation so the language can grow into them. Chapter 24's appendix counts the full census word by word. Automatic semicolon insertion, the parser's power to end a statement at a line break, is a non-issue in this book's module code: every statement carries its own semicolon.

2.3. seven primitives, one lying operator

Every value in the language is either a primitive or an object, and there are exactly seven primitives: `undefined`, `null`, boolean, number, bigint, string, and symbol. `typeof` names six of them honestly:

```
// the seven primitive types, each carried as a value and reported
// through typeof, whose table has exactly one famous lie in it
export function typeofTable() {
  const entries = [
    ["undefined", undefined],
    ["null", null],
    ["boolean", true],
    ["number", 1],
    ["bigint", 1n],
    ["string", "s"],
    ["symbol", Symbol("s")],
  ];
  return entries.map(([name, value]) => ({
    primitive: name,
    typeofName: typeof value,
  }));
}
```

listing 10: the seven primitives carried as values through typeof (source: javascript/samples/src/ch02-values.mjs, lines 43-59)

The seventh row is the famous lie, `typeof null` is `"object"`, a compatibility decision from the language's first implementation that the spec has carried ever since. Two more readings complete the operator's behavior, functions report `"function"` even though function is not a primitive type but an object kind, and `typeof` on a name that was never declared anywhere returns `"undefined"` without throwing, the one undeclared read the language allows:

```
export function typeofQuirks() {
  return {
    nullReports: typeof null,
    functionReports: typeof (() => 1),
    // typeof on an undeclared name is the one safe read in the language
    undeclaredReports: typeof notDeclaredAnywhereEver,
  };
}
```

listing 11: the null lie, the function kindness, the undeclared safe read (source: javascript/samples/src/ch02-values.mjs, lines 61-68)

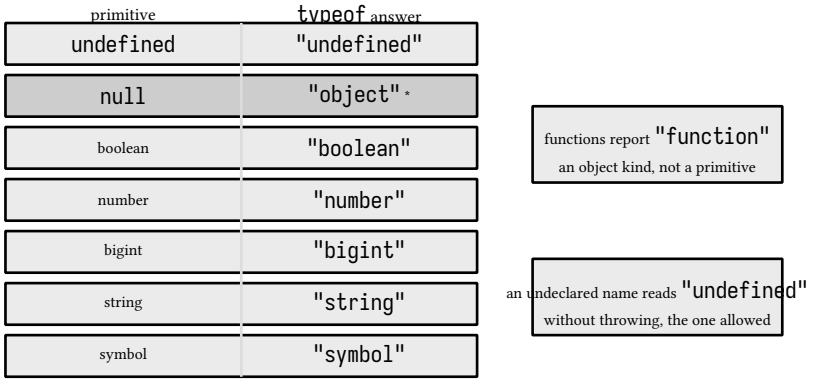


figure 5: typeof against the seven primitives, six honest answers, the 1995 lie starred

The seventh primitive is the one with no literal spelling, a symbol is a name minted at run time: `Symbol("s")` returns a fresh unique value on every call, two calls never compare equal, and a symbol used as a property key never

shows up in `Object.keys`. The well-known symbols, `Symbol.iterator`, `Symbol.hasInstance`, and `Symbol.dispose`, are the protocol hooks chapters 4 through 7 build on.

Primitives are immutable and compared by value, objects are compared by identity, and that split drives the equality section below. `Object.freeze` and the object half of the language belong to chapter 4.

2.4. literals

Numbers read in four radices, decimal, hexadecimal with the `0x` prefix, octal with `0o`, binary with `0b`, and digit separators group long decimals. Past 2^53 the `number` type, an ieee 754 double, has run out of exact integers, rounds to even, and `Number.isSafeInteger` draws the line, while a `bigint` literal carries its trailing `n` into arbitrary precision:

```
// numbers in four radices with separators, and the ieee 754 boundary
// where number rounds to even and bigint holds the exact value
export function numberLiterals() {
  return {
    hex: 0xff,
    octal: 0o10,
    binary: 0b101,
    separated: 1_000_000,
    roundsPastSafe: 2 ** 53 + 1 === 2 ** 53,
    safeEdge: [Number.isSafeInteger(2 ** 53 - 1), Number.isSafeInteger(2 ** 53)],
    bigintExact: (2n ** 53n + 1n).toString(),
  };
}
```

listing 12: four radices, separators, and the 2^53 boundary against bigint (source: javascript/samples/src/ch02-values.mjs, lines 70-82)

Strings are single or double quoted or template literals, and template literals interpolate arbitrary expressions, not just variables. String contents are utf-16 code units, so one emoji spans two units and `length` reports 2 while `codePointAt` still reaches the full code point. The escape grammar spells both, a four digit code unit escape for the é and a braced code point escape for the emoji, and `String.raw` keeps a template's backslashes for paths and patterns:

```
// template literals interpolate expressions, strings are utf-16 code
// units so one emoji spans two, and raw templates keep their backslashes
export function stringFacts(name) {
  const emoji = "😀";
  return {
    interpolated: `hi ${name.toUpperCase()}, ${1 + 2} ways`,
    emojiUnits: emoji.length,
    emojiCodePoint: emoji.codePointAt(0).toString(16),
    escapes: "caf\u00e9 \u{1F600}",
    raw: String.raw`c:\new\table`,
  };
}
```

listing 13: interpolation, utf-16 units against code points, escapes, raw (source: javascript/samples/src/ch02-values.mjs, lines 84-95)

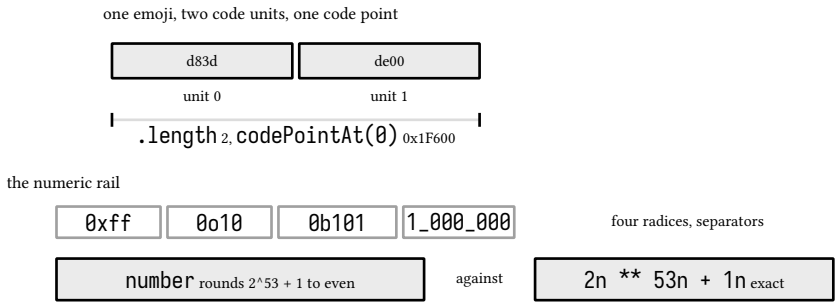


figure 6: one emoji in utf-16 memory, and the numeric rail beside it

2.5. two equalities and the same-value operator

== runs a coercion algorithm before comparing, === does not, and the honest summary is a table of pairs rather than a rule of thumb:

```
// loose equality runs a coercion algorithm, strict equality does not,
// and nan is unequal to everything including itself under both
export function equalityPairs() {
  const box = {};
  return {
    coercive: [null == undefined, 0 == "", 0 == false, 1 == "1"],
    strict: [null === undefined, 0 === "", 0 === false, 1 === "1"],
    nan: [NaN == NaN, NaN === NaN, Number.isNaN(NaN)],
    identity: [box == box, box === box, {} == {}, [] == []],
    signedZero: [0 === -0, Object.is(0, -0), Object.is(NaN, NaN)],
  };
}
```

listing 14: coercive pairs against strict pairs, nan, identity, signed zero (source: javascript/samples/src/ch02-values.mjs, lines 97-108)

The algorithm has one step the table cannot show, because no pinned pair mixes an object with a primitive: when only one operand is an object, it is asked for a primitive first, `valueOf` before `toString`, and only then do the pair rules apply. Three readings fall out. Loose equality's one cross type pair is `null == undefined`, both meaning absence, equal to each other and to nothing else. `NaN` violates reflexivity under both operators, which is why `Number.isNaN` exists and why `indexOf` hunts of nan fail. Objects compare by identity, two fresh {} are two different objects, and the one operator with different opinions is `Object.is`, the same-value comparison, which distinguishes the two zeros and identifies the two nans:

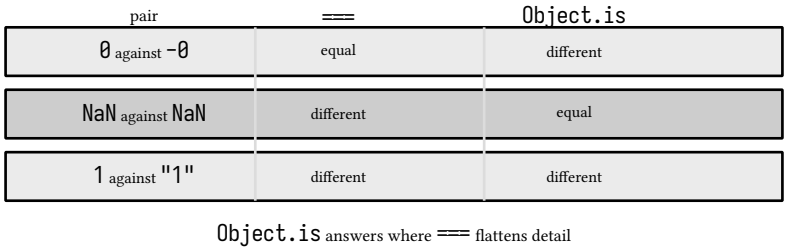


figure 7: three verdicts for the hard pairs

This book writes === everywhere and == only in the table above, where the coercion itself is the subject.

2.6. truthiness

Exactly eight values are falsy. Every other value is truthy, and the surprises are all strings and objects: `"0"` and `"false"` are non-empty strings, `[]` and {} are objects, and objects are always truthy no matter how empty:

```
// exactly eight values are falsy, everything else is truthy,
// including "0", "false", [] and {}
export const FALSY = [false, 0, -0, 0n, "", null, undefined, NaN];

export function truthiness(samples) {
  return samples.map((v) => !!v);
}
```

listing 15: the eight falsy values, and truthiness as one operator (source: javascript/samples/src/ch02-values.mjs, lines 110-116)

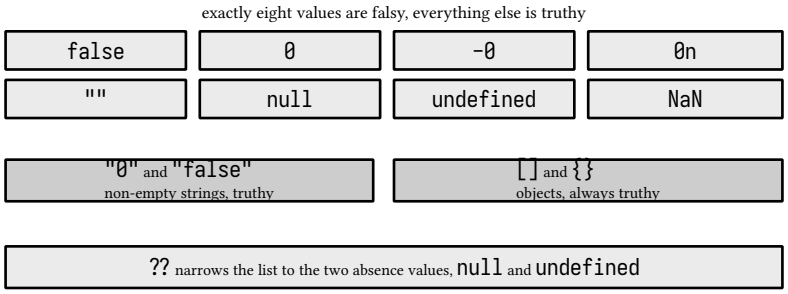


figure 8: the eight falsy values as a closed list, the surprises that stay truthy

Default parameters and the ?? operator in chapter 3 build directly on this list, ?? defaults exactly the two absence values rather than all eight.

sources: developer.mozilla.org grammar and types pages, tc39.es ecma 262 sections on declarations and equality, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows, 23 tests green through `npm run verify` in `javascript/samples`, 13 of them this chapter's.

3. functions, scope, and closures

A function in javascript is a value first: it can be stored, passed, returned, compared by identity, and it carries two pieces of metadata the engine maintains for free, its name and its arity. Around that value the language builds scope, the rules for which bindings a function can see, and closures, the mechanism by which those bindings outlive the call that created them. Everything in this chapter runs without a compiler on node 26.

3.1. three forms, one value

Functions come written three ways. A declaration is a statement with a name, hoisted and initialized before the module body runs. An expression is a value assigned to a binding. An arrow is an expression with a shorter syntax, a lexical `this`, and no `arguments` object of its own:

```
// declaration, expression, arrow: one callable value each, with a name
// and an arity the engine tracks. default parameters fill in but do not
// count toward length, which counts parameters before the first default
export function identityFacts() {
  function declared(a, b) {
    return a + b;
  }
  const expressed = function named(a, b = 1) {
    return a + b;
  };
  const arrow = (a) => a * 2;
  return [
    [declared.name, declared.length],
    [expressed.name, expressed.length],
    [arrow.name, arrow.length],
  ];
}
```

listing 16: the three forms, with the name and arity the engine tracks (source: javascript/samples/src/ch03-functions.mjs, lines 4-20)

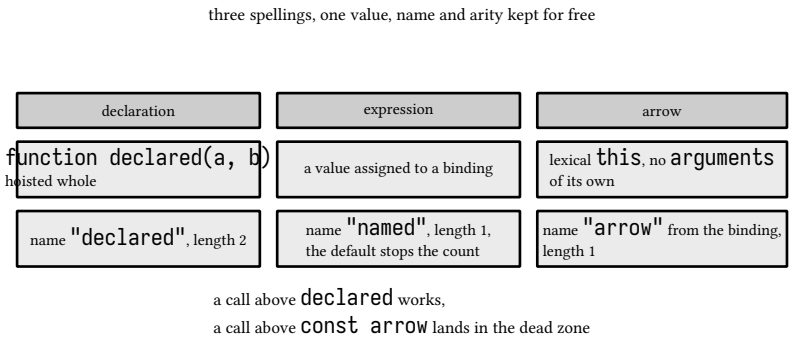


figure 9: the three forms as columns, one callable value with free metadata

`length` counts parameters up to the first default, so `expressed.length` is 1, and inferred names like `arrow` come from the binding when the expression has no name of its own. Declarations hoist whole, which chapter 2's timeline already drew: a call above the declaration line works, while a call above a `const` arrow lands in the dead zone.

3.2. this binds by call site

The `this` of an ordinary function is decided by how it is called, not where it is written. Called through its receiver, a method sees the receiver. Detached and called bare in module code, it sees `undefined`, and the property read throws. `call`, `apply`, and `bind` reattach a receiver explicitly, `bind` permanently, and the arrow sidesteps the whole mechanism by not having a `this` at all:

```
// this binds by call site: a method called through its receiver gets the
// receiver, the same function detached gets undefined in module code and
// the property read throws, and call, apply, and bind reattach it while
// the arrow never had a this of its own to reattach
export function receiverFacts() {
  const logger = {
    prefix: "log:",
    write(message) {
      return `${this.prefix} ${message}`;
    },
  };
  const facts = [logger.write("attached")];
  const detached = logger.write;
  let detachedError = "none";
  try {
    detached("detached");
  } catch (e) {
    detachedError = e.constructor.name;
  }
  const bound = logger.write.bind({ prefix: "bound:" });
  return facts.concat([
    detachedError,
    bound("reattached"),
    logger.write.call({ prefix: "called:" }, "explicit"),
    logger.write.apply({ prefix: "applied:" }, ["spread"]),
  ]);
}
```

listing 17: five calls, five receivers, one detached call that throws (source: javascript/samples/src/ch03-functions.mjs, lines 22-48)

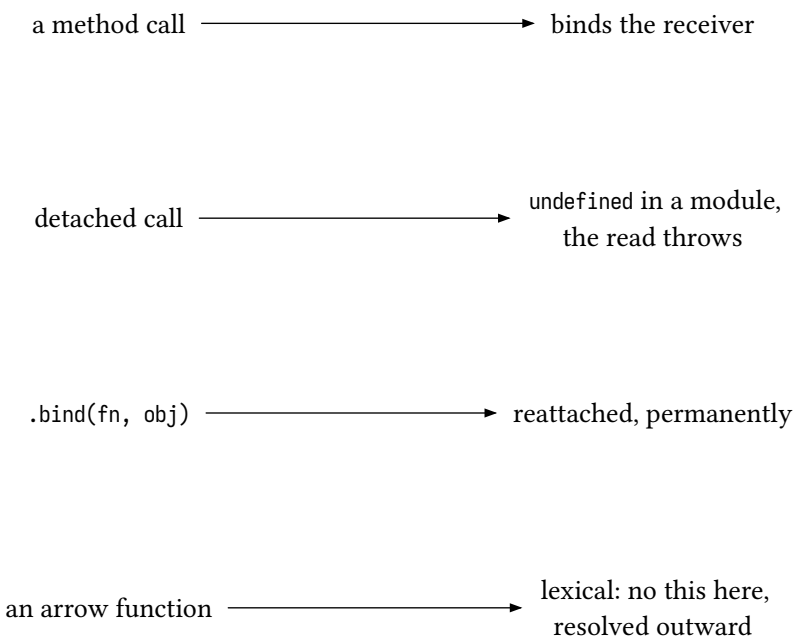


figure 10: receiver resolution, four ways a call finds its this

Arrows exist largely for this rule. A callback that must keep seeing the enclosing `this` is an arrow, a method that needs its receiver is an ordinary function, and the two forms split work along exactly that line.

3.3. parameters

Default parameters may read the parameters before them, one rest parameter collects everything after its position into a true array, and the `arguments` object belongs to ordinary functions, arrows resolve the name outward instead:

```
// default parameters may read earlier parameters, one rest parameter
// collects the remainder into a true array, and the arguments object
// belongs to ordinary functions only
export function paramsFacts() {
  const pair = (a, b = a * 2) => [a, b];
  const join = (head, ...rest) => [head, rest.length, Array.isArray(rest)];
  function legacyArguments() {
    return arguments.length;
  }
  return { pair: pair(3), join: join("h", "a", "b"), legacy: legacyArguments(1, 2, 3) };
}
```

listing 18: defaults reading earlier parameters, rest as a true array, legacy arguments (source: javascript/samples/src/ch03-functions.mjs, lines 50-60)

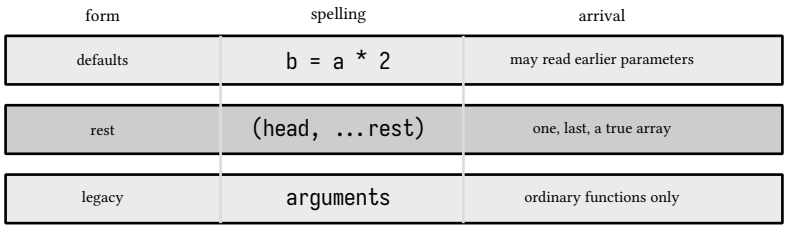


figure 11: the three parameter forms, what each spells and what arrives

3.4. closures and the module pattern

A closure is a function plus the bindings it kept. The counter below closes over `count`, and after `createCounter` returns, `count` is unreachable except through the two functions that captured it. That is encapsulation without classes, and at file scale it is the module pattern: the module's top level is the function, the exports are the returned object, and everything else is private:

```
// a closure is a function plus the bindings it kept alive: the counter's
// count is unreachable except through the two returned functions, which
// is the module pattern in miniature
export function createCounter(start = 0) {
  let count = start;
  return {
    increment: () => ++count,
    value: () => count,
  };
}

export function createWallet() {
  let balance = 0; // hidden state, no property reads it
  return {
    deposit: (amount) => {
      balance += amount;
      return balance;
    },
    balance: () => balance,
  };
}
```

listing 19: a counter and a wallet, state visible only through the returned functions (source: javascript/samples/src/ch03-functions.mjs, lines 62-82)

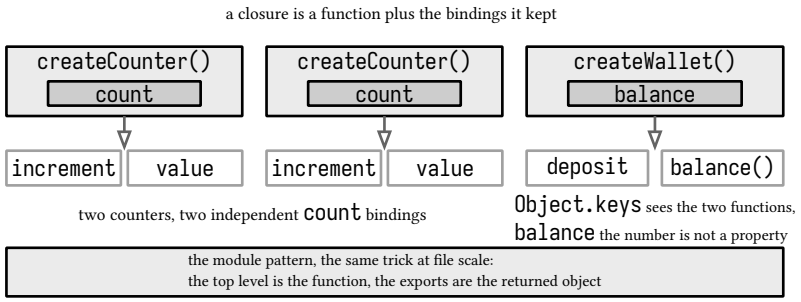


figure 12: the closure kept per call, and the module as the same trick at file scale

Two counters close over two independent bindings, which the tests pin, and `Object.keys` on the wallet sees the two functions and nothing else, `balance` the number does not exist as a property at all. Chapter 4 shows the same privacy built from `#private` fields, and part two's chapter on modules shows the file scale version.

3.5. the iife, retired

The immediately invoked function expression was the pre-2015 way to buy a scope, wrap the whole file in a function, call it at once, keep the private names inside. Block scoped `let` and `const` bought the same scope with two characters, and modules bought the file scale version natively:

```
// the iife: a function that exists only to be called immediately, the
// pre-2015 way to buy a scope. blocks and modules bought that scope
// cheaper, so the form survives mostly in older code and interview
// questions
export function iifeFacts() {
  const legacy = (function () {
    var privateToTheCall = 41;
    return privateToTheCall + 1;
  })();
  let fresh = 0;
  {
    let scoped = 41;
    fresh = scoped + 1;
  }
  return { legacy, block: fresh };
}
```

listing 20: the iife and the block that replaced it, buying the same scope (source: javascript/samples/src/ch03-functions.mjs, lines 84-99)

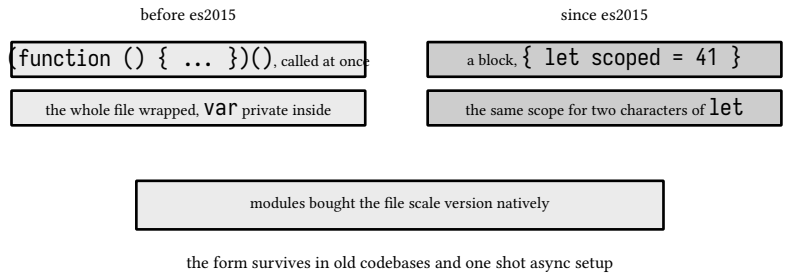


figure 13: the iife against the block that replaced it, the same scope re-priced

The form survives in older codebases and as an idiom for one-shot async setup, but this book writes blocks and modules instead.

3.6. higher order functions

A higher order function takes functions as arguments or returns them. Composition threads one function's output into the next function's input, and memoization wraps a function with a cache keyed by argument, one line on top of ES2026's `getOrInsertComputed`, which calls its callback only on a cache miss:

```
// higher order functions take or return functions: compose threads one
// output into the next input, memoize wraps a function with a Map cache
// keyed by argument. getOrInsertComputed calls its callback only on a
// miss, which is memoization's one line
export function compose(f, g) {
  return (x) => f(g(x));
}

export function memoize(fn) {
  const cache = new Map();
  return (arg) => cache.getOrInsertComputed(arg, () => fn(arg));
}

export function twice(fn) {
  return (x) => fn(fn(x));
}
```

listing 21: compose, memoize over getOrInsertComputed, twice (source: javascript/samples/src/ch03-functions.mjs, lines 101-116)

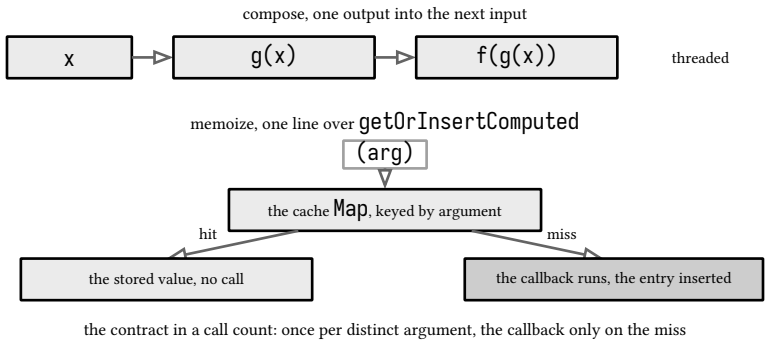


figure 14: compose threading one lane, memoize branching on the cache

The test proves the memoized function calls its wrapped function once per distinct argument, twice for two arguments, which is the whole contract of a cache. Higher order functions are how the standard library's `map`, `filter`, and `reduce` are built, and chapter 5 leans on them heavily.

3.7. recursion and the tail call promise

Functions may recurse directly or mutually with no forward declaration. The es2015 specification also requires proper tail calls, a tail position call reusing the caller's frame, and the engines split on it: javascriptcore implemented the requirement, v8 and spidermonkey declined it, citing debugging and stack trace costs. On this runtime the spec's promise does not hold, and the probe measures the fact rather than trusting either side:

```
// recursion works in every form. the spec's proper tail calls, required
// since es2015, are implemented by javascriptcore and declined by the
// other engines, so on v8 a tail recursive countdown of a million frames
// is still a million frames deep and overflows, measured here rather
// than asserted from the spec
export function tailCallOverflow(depth = 1_000_000) {
  try {
    const descend = (n) => (n === 0 ? "bottom" : descend(n - 1));
    return descend(depth);
  } catch (e) {
    return e.constructor.name;
  }
}
```

listing 22: a tail recursive countdown of a million frames, overflowing on v8 (source: javascript/samples/src/ch03-functions.mjs, lines 118-130)

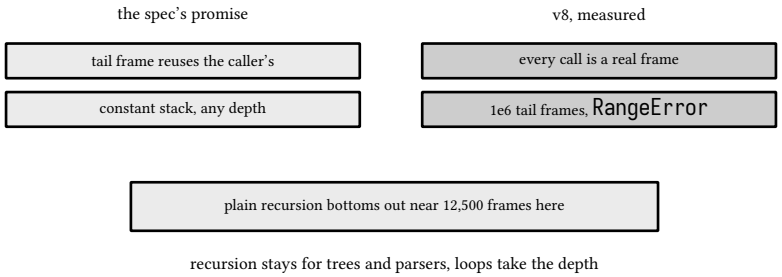


figure 15: the tail call promise against this engine's stack

The practical rule follows from the measurement: recursion is for structures naturally shaped like trees, where depth is logarithmic, and a loop is for anything that might go deep. javascriptcore is the one engine where the spec's promise holds, so portable code assumes it holds nowhere.

sources: developer.mozilla.org function and arrow function documentation, tc39.es es2015 tail calls clause, v8 dev blog on tail call elimination, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows, 36 tests green through `npm run verify` in `javascript/samples`, 13 of them this chapter's.

4. objects, prototypes, and classes

Everything in the language that is not a primitive is an object, and objects carry three systems at once: literal syntax that builds them in place, a prototype chain that lets one object delegate to another, and a class syntax that packages constructors, prototypes, and encapsulation into one declaration. The class keyword adds no new object model, it is the same delegation underneath, plus one mechanism literals never had, `#private` fields. The typescript typing of all of this, interfaces and structural shapes, belongs to chapter 15.

4.1. literals and key order

Object literals accept shorthand names, computed keys in brackets, and method shorthand, and they lay out their keys under one rule: integer like keys first in ascending order, then string keys in insertion order, then symbol keys:

```
// one literal exercising shorthand, a computed key, and a method, plus
// the key order rule: integer like keys ascending first, then strings
// and symbols in insertion order
export function literalFacts() {
  const kind = "k" + 1;
  const value = 5;
  const mixed = {
    title: "order",
    2: "two",
    1: "one",
    tag: "t",
    [kind]: value,
    shout() {
      return `${this.title}!`;
    },
  };
  return { keys: Object.keys(mixed), computed: mixed.k1, shout: mixed.shout() };
}
```

listing 23: shorthand, a computed key, method shorthand, and the key order they produce (source: javascript/samples/src/ch04-objects.mjs, lines 4-21)

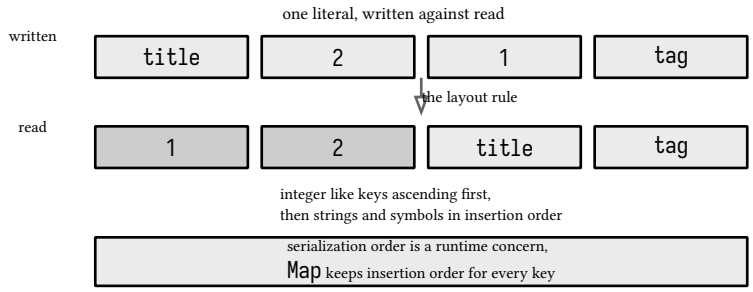


figure 16: the written order against the read order, one rule between them

The written order `title, 2, 1, tag` comes back as `1, 2, title, tag`, which is why serialization order is a runtime concern and why `Map`, which keeps insertion order for every key, exists for the cases where that order must be exact.

4.2. descriptors

Every property sits on a property descriptor with three flags, `writable`, `enumerable`, and `configurable`, and the two ways to create a property disagree about their defaults. Literal properties are open on all three. `Object.defineProperty` defaults every flag to false, which makes it the tool for locking a property down:

```
// every property sits on a descriptor. literal properties are writable,
// enumerable, and configurable; defineProperty defaults are the opposite,
// and a non-writable data property throws on assignment in module code
export function descriptorFacts() {
  const literal = { a: 1 };
  const defined = {};
  Object.defineProperty(defined, "a", { value: 1 });
  const pick = (d) => [d.writable, d.enumerable, d.configurable];
  let writeError = "none";
  try {
    defined.a = 2;
  } catch (e) {
    writeError = e.constructor.name;
  }
  return {
    literal: pick(Object.getOwnPropertyDescriptor(literal, "a")),
    defined: pick(Object.getOwnPropertyDescriptor(defined, "a")),
    writeError,
  };
}
```

listing 24: the two default worlds, and the assignment a closed descriptor rejects (source: javascript/samples/src/ch04-objects.mjs, lines 23-42)

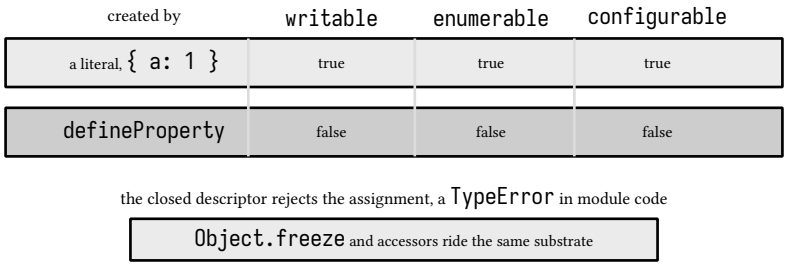


figure 17: the two default worlds, three flags each way

The rest of the object machinery, `Object.freeze` sealing everything shut, getters and setters as accessor descriptors, lives on this one substrate. This book touches descriptors where the standard library requires them and uses classes for structure.

4.3. delegation

An object holds a link to another object, its prototype, and every property read walks that link on a miss. `Object.create` builds the link directly and is the whole mechanism in one call:

```
// objects delegate along a prototype chain. Object.create builds the
// link directly, lookups walk outward, own against inherited is the
// hasOwnProperty question, and a null prototype object delegates nowhere
export function prototypeFacts() {
  const base = {
    who: "base",
    greet() {
      return `hi ${this.who}`;
    },
  };
  const child = Object.create(base);
  child.who = "child";
  const dictionary = Object.create(null);
  dictionary.key = "value";
  return {
    link: Object.getPrototypeOf(child) === base,
    shadowed: child.greet(),
    ownVsInherited: [Object.hasOwn(child, "who"), Object.hasOwn(child, "greet")],
    inWalks: "greet" in child,
    nullProtoDelegates: typeof dictionary.toString,
    nullProtoOwn: Object.hasOwn(dictionary, "key"),
  };
}
```

listing 25: a chain built by hand, shadowing, own against inherited, and the null prototype (source: javascript/samples/src/ch04-objects.mjs, lines 44-66)

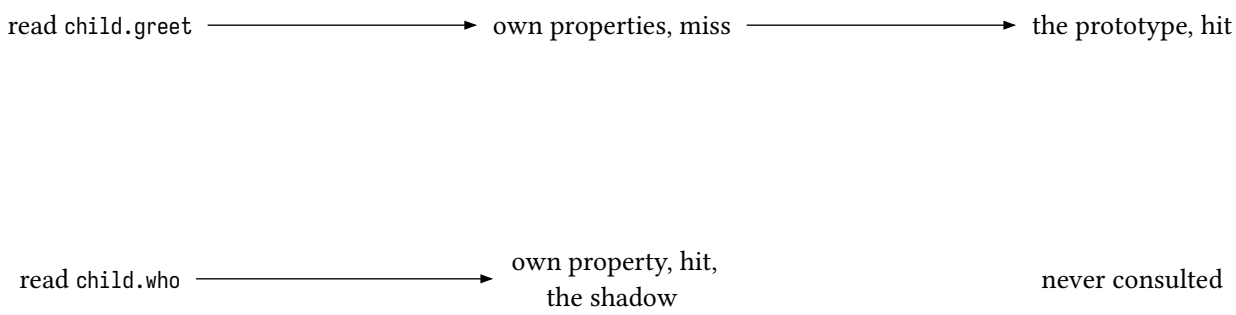


figure 18: one property read walking a two link chain

`this` inside the inherited method is the object the read started from, which is why `child.greet()` says `hi child` even though `greet` lives on `base`. `Object.hasOwn` is the modern own against inherited question, and `Object.create(null)` builds an object that delegates nowhere, no `toString`, no `hasOwnProperty`, a clean slate dictionary that foreign property reads cannot pollute.

4.4. classes

A class packages the same delegation with dedicated syntax: a constructor, methods on the prototype, accessors as getter setter pairs, statics that belong to the constructor, and private fields that are a runtime brand rather than a property:

```
// static evaluation order, visible once: fields and blocks run in
// source order at class definition time, before any instance exists
export const STATIC_EVAL_ORDER = [];
STATIC_EVAL_ORDER.push("module body");

export class Meter {
  static instances = 0;
  static {
    STATIC_EVAL_ORDER.push("static block");
  }
  static origin = STATIC_EVAL_ORDER.concat("static field");

  #reading = 0; // a runtime brand, not a property

  constructor(start = 0) {
    this.#reading = start;
    Meter.instances += 1;
  }
  get reading() {
    return this.#reading;
  }
  set reading(value) {
    if (value < 0) throw new RangeError("negative reading");
    this.#reading = value;
  }
  bump(by = 1) {
    this.#reading += by;
    return this.#reading;
  }
}
```

listing 26: statics with their once only block, an accessor pair, and one hash private field (source: javascript/samples/src/ch04-objects.mjs, lines 68-97)

Static fields and static blocks run once, at class definition time and in source order, before any instance exists, which the order array records and the test pins. `extends` links two classes, both their prototypes and their constructors, `super` reaches the base implementation, and statics inherit along the constructor chain:

```
// inheritance links prototypes and constructors both: super reaches the
// base implementation, statics inherit, and instanceof walks the chain
class Calibrated extends Meter {
  constructor(start, factor) {
    super(start);
    this.factor = factor;
  }
  bump(by = 1) {
    return super.bump(by * this.factor);
  }
}

export function staticEvalOrder() {
  return [...STATIC_EVAL_ORDER];
}

export function classFacts() {
  const before = Meter.instances;
  const m = new Meter(10);
  m.reading = 20;
  const c = new Calibrated(1, 10);
  c.bump(2); // 1 + 2 * 10
  return {
    staticsInherit: Calibrated.origin,
    instances: Meter.instances - before,
    accessor: m.reading,
    superCall: c.reading,
    chain: [c instanceof Calibrated, c instanceof Meter, m instanceof Calibrated],
  };
}
```

listing 27: extends and super, statics inherited, the instanceof chain (source: javascript/samples/src/ch04-objects.mjs, lines 99-128)

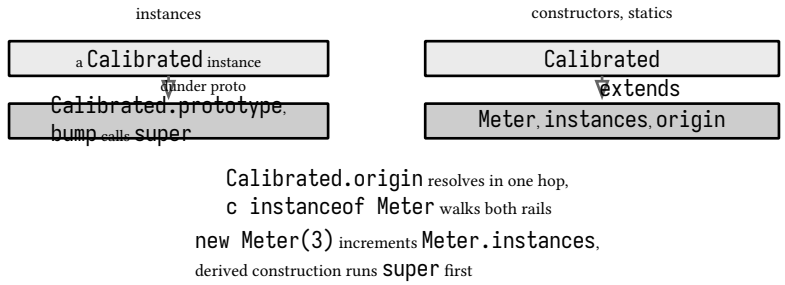


figure 19: two links extends builds, and what each carries

4.5. private fields

A # name is not a property. It lives in a per class brand slot, Object.keys cannot see it, no cast can reach it, and a method invoked with a foreign receiver throws rather than reads garbage:

```
// #reading is invisible to Object.keys and unreachable from outside,
// and a method invoked with a foreign receiver cannot open it either
export function privateFacts() {
  const m = new Meter(1);
  const imposter = Object.create(Meter.prototype);
  let brandError = "none";
  try {
    Meter.prototype.bump.call(imposter, 1);
  } catch (e) {
    brandError = e.constructor.name;
  }
  return {
    keys: Object.keys(m),
    brandError,
  };
}
```

listing 28: the brand against keys and against an imposter receiver (source: javascript/samples/src/ch04-objects.mjs, lines 130-145)

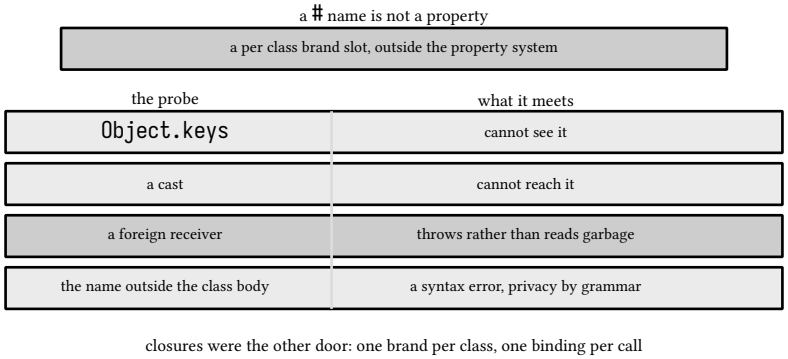


figure 20: the hash name as a brand, what each probe meets

Outside the class body the name `#reading` is a syntax error, so privacy is enforced by the grammar, not by convention. The getter is the door, and chapter 3's closures were the other door to the same privacy, one brand per class against one binding per call.

`instanceof` closes the chapter because it answers the question all this machinery raises, what is this object. The operator consults `Symbol.hasInstance`, and any object can implement that method, so the check is a protocol rather than a hardcoded operation:

```
// instanceof asks the constructor's Symbol.hasInstance, and any object
// can answer it, which is how the operator stays honest about duck types
const evens = {
  [Symbol.hasInstance](v) {
    return typeof v === "number" && v % 2 === 0;
  },
};

export function instanceofFacts() {
  return [4 instanceof evens, 3 instanceof evens, new Meter() instanceof Meter];
}
```

listing 29: a custom hasInstance answering for plain values (source: javascript/samples/src/ch04-objects.mjs, lines 147-157)

4.6. spread and rest

Object spread copies own enumerable properties from each source in order, later sources win, and getters are read once and copied as plain data properties, so a live getter flattens into a snapshot. Rest destructuring collects the keys not named:

```
// spread copies own enumerable properties, later sources win, getters
// are read once and copied as plain data, and rest destructuring
// collects the keys not named
export function spreadFacts() {
  const source = {
    get live() {
      return "computed";
    },
    keep: 1,
  };
  const merged = { a: 1, ...{ a: 2, b: 3 }, ...source };
  const { live, ...rest } = merged;
  const descriptor = Object.getOwnPropertyDescriptor(merged, "live");
  return {
    laterWins: merged.a,
    getterBecameData: [live, descriptor.get === undefined, descriptor.value],
    restCollects: Object.keys(rest).sort(),
  };
}
```

listing 30: later sources winning, a getter becoming data, rest collecting the rest (source: javascript/samples/src/ch04-objects.mjs, lines 159-177)

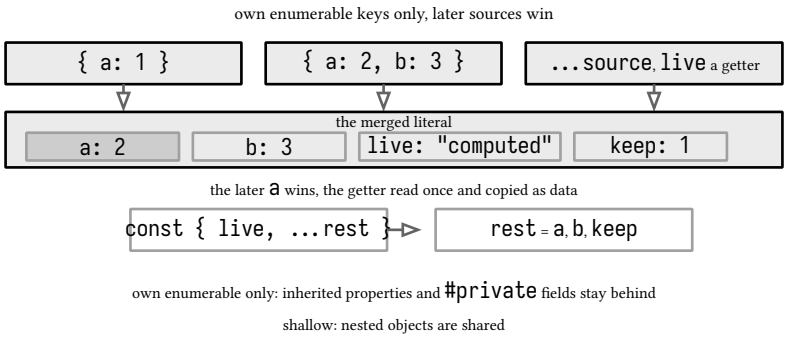


figure 21: spread merging three sources into one snapshot, rest collecting the unnamed

Spread is the literal's merge operator and the standard way to copy with changes, `{ ...state, field: next }`. It copies own enumerable properties only, so inherited properties and `#private` fields stay behind, and it is shallow, nested objects are shared, which is the fact to remember before building anything immutable on it.

sources: developer.mozilla.org working with objects, classes, and private properties pages, tc39.es class fields proposal history, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows, 47 tests green through `npm run verify` in `javascript/samples`, 11 of them this chapter's.

5. iteration and collections

Iteration in javascript is a protocol, not an array feature. Anything that exposes `Symbol.iterator` can flow through `for..of`, spread, and destructuring, and since ES2025 the protocol carries its own lazy pipeline methods, `map`, `filter`, `take`, and friends, on the `Iterator` prototype. This chapter builds the protocol by hand, then spends it on the collections: Map and Set with their ES2025 algebra and ES2026 `getOrInsert`, the weak references, and the binary end of the standard library, `Float16Array` and the `Uint8Array` codecs.

5.1. the iterator protocol

`for..of` does not consume arrays, it consumes a shape: an iterable is an object whose `Symbol.iterator` method returns an iterator, and an iterator is an object with a `next` method that produces `{ value, done }` rows. `Range` below satisfies the shape directly, with no array anywhere in it:

```
// a hand built iterable: Symbol.iterator returns an iterator, an
// object whose next() produces { value, done } rows until done
export class Range {
  #from;
  #to;
  constructor(from, to) {
    this.#from = from;
    this.#to = to;
  }
  [Symbol.iterator]() {
    let current = this.#from;
    const to = this.#to;
    return {
      next() {
        return current < to
          ? { value: current++, done: false }
          : { value: undefined, done: true };
      },
    };
  }
}

// everything that iterates consumes the protocol, not arrays: spread,
// for..of, destructuring, and Iterator.from, which wraps any iterable
// in the Iterator prototype that carries the helpers
export function protocolFacts() {
  const range = new Range(1, 4);
  const spread = [...range];
  const looped = [];
  for (const n of range) looped.push(n);
  const [first] = range;
  const doubled = [...Iterator.from(range).map((n) => n * 2)];
  return { spread, looped, first, doubled, helperOnArray: typeof [].drop };
}
```

listing 31: a hand built iterable feeding spread, for..of, destructuring, and Iterator.from (source: javascript/samples/src/ch05-iteration.mjs, lines 4-37)

Three facts the tests pin. A fresh call to `Symbol.iterator` starts over, which is why the same `Range` can be spread twice. `Iterator.from` wraps any iterable in the prototype that carries the helper methods, and plain arrays do not carry them, `typeof [].drop` is `undefined`: the helpers belong to iterators, not to arrays.

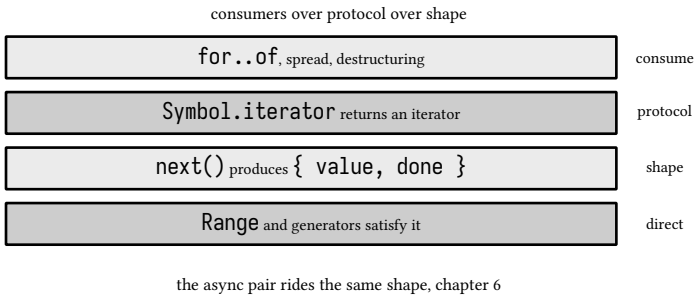


figure 22: the protocol, stacked

5.2. generators

A generator function is syntax for writing an iterator: the body pauses at each `yield` and resumes on the next pull. `yield*` delegates to another iterable, and the delegated generator's return value is the value of the whole `yield*` expression, which is how a delegation chain reports its own summary. `next(value)` is two way: after the first call, each call's argument becomes the value of the suspended `yield` expression:

```
// a generator function pauses at each yield and resumes on demand.
// yield* delegates to another iterable, and the delegated generator's
// return value is the value of the yield* expression itself
export function* Countdown(from, sends = "go") {
  for (let n = from; n > 0; n--) yield n;
  return sends;
}

export function* Chained() {
  const tail = yield* Countdown(3, "done");
  yield tail;
}

// next(value) is two way: the first call starts the body, and each
// later call's argument becomes the value of the suspended yield
export function* Echo() {
  const a = yield "ready";
  const b = yield a * 2;
  return b * 3;
}

export function twoWayFacts() {
  const gen = Echo();
  const started = gen.next();
  const second = gen.next(10);
  const third = gen.next(100);
  return [started.value, second.value, third.value, third.done];
}
```

listing 32: delegation with a return value, and two way next(value) (source: javascript/samples/src/ch05-iteration.mjs, lines 39-66)

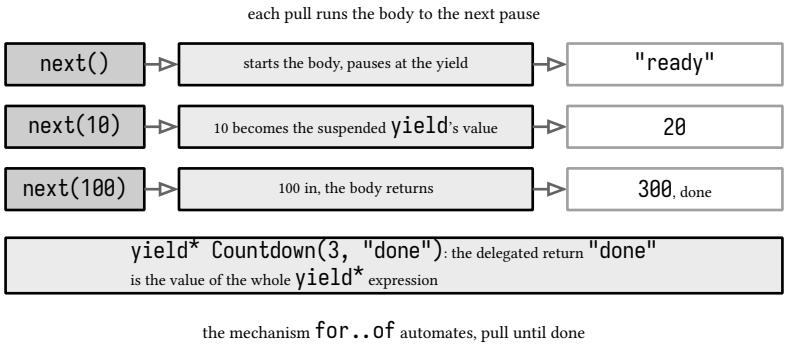


figure 23: the body paused and resumed, three pulls of one generator

Driven by hand, `Echo` answers `"ready"`, then `20` for the resent 10, then returns `300` and is done. The test pins all four observations. Most code never drives a generator by hand, but the mechanism is what makes generators usable as coroutines in older codebases, and it is the same mechanism `for..of` automates.

5.3. lazy pipelines

An infinite generator is safe because nothing iterates it whole. The helper methods, part of ES2025, pull exactly as many values as the consumer keeps. The pipeline below squares an endless stream, filters to odd squares, and keeps three. `map` runs five times, for the squares 1, 4, 9, 16, and 25, and never again: `take(3)` stopped the pull, and the generator is still willing to continue:

```
// an infinite generator is safe because nothing iterates it whole:
// consumers pull exactly as many values as they ask for
export function* Naturals(from = 0) {
  for (let n = from; ; n++) yield n;
}

// the helper chain is lazy: map runs once per value the consumer
// keeps, never for the values filter discards or take never asks for
export function lazyPipelineFacts() {
  let mapCalls = 0;
  const pipeline = Naturals(1)
    .map((n) => {
      mapCalls += 1;
      return n * n;
    })
    .filter((square) => square % 2 === 1)
    .take(3);
  const squares = [...pipeline];
  return { squares, mapCalls, isIterator: pipeline instanceof Iterator };
}
```

listing 33: an infinite source through map, filter, take: five calls, three kept (source: javascript/samples/src/ch05-iteration.mjs, lines 68-87)

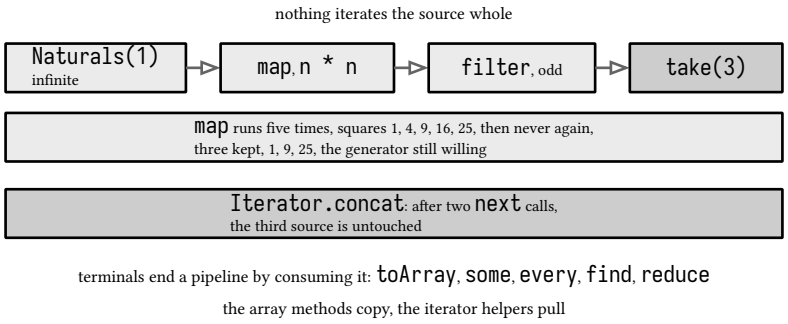


figure 24: the pull discipline, each helper an iterator wrapping the last

The same discipline on an array would build every intermediate array. Here nothing materializes: each helper is an iterator wrapping the one before it, and the final spread is the first full traversal. The rest of the family works the same way:

```
// the rest of the family: drop skips values, flatMap expands each
// value into many, reduce folds the stream without materializing it
export function helperFacts() {
  const dropped = [...Iterator.from([1, 2, 3, 4, 5]).drop(2)];
  const expanded = [...Iterator.from([1, 2]).flatMap((n) => [n, n * 10])];
  const summed = Naturals(1)
    .map((n) => n * 2)
    .take(4)
    .reduce((a, b) => a + b, 0);
  return { dropped, expanded, summed };
}
```

listing 34: drop skips, flatMap expands, reduce folds without materializing (source: javascript/samples/src/ch05-iteration.mjs, lines 88-99)

`reduce` and the other terminals, `toArray`, `some`, `every`, `find`, end a pipeline because they consume it. Between terminals the chain stays lazy, which is the whole point: the array methods copy, the iterator helpers pull.

`Iterator.concat`, one of the ES2026 seven, chains iterables of any kind into one iterator, left to right, and stays lazy: tapping every source shows that after two `next` calls only the first two sources have been touched at all:

```
// Iterator.concat (ES2026) chains iterables of any kind into one
// iterator, left to right, still lazy: each source is pulled only
// when the chain reaches it
export function concatFacts() {
  const pulled = [];
  const tapped = (iterable) => ({
    [Symbol.iterator]() {
      const inner = iterable[Symbol.iterator]();
      return {
        next() {
          const step = inner.next();
          if (!step.done) pulled.push(step.value);
          return step;
        },
      };
    },
  });
  const chained = Iterator.concat(tapped([1]), tapped([9, 9]), tapped("ab"));
  const firstTwo = [chained.next().value, chained.next().value];
  const pulledEarly = pulled.length; // the third source is untouched
  const rest = [...chained];
  return { seen: [...firstTwo, ...rest], pulledEarly, pullOrder: pulled };
}
```

listing 35: three tapped sources concatenated: two pulls in, the third source untouched (source: javascript/samples/src/ch05-iteration.mjs, lines 101-123)

5.4. keyed collections

A Map tracks insertion order, admits any value as a key, and finds keys by SameValueZero, the equality under which `NaN` equals itself and the two zeros stay equal, the one difference from `Object.is`. `set` on an existing key updates in place and keeps the slot, while delete followed by set re-enters the key at the end, which the key order in the test makes visible:

```
// ch05 part two: Map and Set, the getOrInsert family, set algebra,
// weak collections, Float16Array, and the Uint8Array text codecs

// Map keeps insertion order and admits any key by SameValueZero, the
// equality under which NaN is findable even though NaN === NaN is
// false. setting an existing key updates in place, delete then set
// moves the key to the end
export function mapFacts() {
  const map = new Map([
    ["a", 1],
    ["b", 2],
    ["c", 3],
  ]);
  map.set("a", 9); // a keeps its slot
  map.delete("b");
  map.set("b", 9); // b re-enters at the end
  map.set(NaN, "not a number");
  const anyKey = {};
  map.set(anyKey, "object key");
  return {
    order: [...map.keys()].filter((k) => typeof k === "string"),
    nanFindable: [map.has(NaN), NaN === NaN],
    objectKey: map.get(anyKey),
  };
}
```

listing 36: order kept, set in place, delete and set at the end, NaN findable (source: javascript/samples/src/ch05-collections.mjs, lines 1-25)

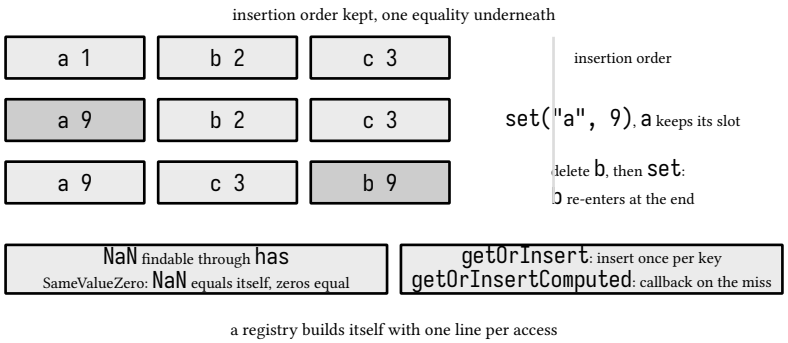


figure 25: the map slots through three operations, order as the visible fact

`getOrInsert`, from ES2026, returns the existing entry on a hit and inserts on a miss, so a registry builds itself with one line per access and the insert happens exactly once. `getOrInsertComputed` takes a callback and runs it only on the miss, which is the memoizer from chapter 3 in one method. `WeakMap` carries the same pair for object keys:

```
// getOrInsert (ES2026) returns the existing entry on a hit and
// inserts the given value on a miss, so the insert happens exactly
// once; getOrInsertComputed runs its callback only on the miss
export function getOrInsertFacts() {
  const registry = new Map();
  const first = registry.getOrInsert("db", { url: "postgres://a" });
  const second = registry.getOrInsert("db", { url: "postgres://b" });
  let computedCalls = 0;
  const memo = new Map();
  const square = (n) =>
    memo.getOrInsertComputed(n, () => {
      computedCalls += 1;
      return n * n;
    });
  square(3);
  square(3);
  square(4);
  return {
    insertOnce: [first === second, registry.size],
    computedCalls,
    weakVariant: typeof new WeakMap().getOrInsert,
  };
}
```

listing 37: insert once, compute once per key, the weak variant present (source: javascript/samples/src/ch05-collections.mjs, lines 26-49)

5.5. set algebra

Sets gained seven methods in ES2025, all over SameValueZero. Four produce new sets, three answer predicates, and every one accepts not just a Set but any set like object with `size`, `has`, and `keys`:

```
// the ES2025 set algebra family, seven methods over SameValueZero.
// each one takes a Set or any set like object with size, has, keys
export function setAlgebraFacts() {
  const left = new Set([1, 2, 3]);
  const right = new Set([2, 3, 4]);
  const setLike = {
    size: 2,
    has: (v) => v === 2,
    *keys() {
      yield 2;
      yield 99;
    },
  };
  return {
    union: [...left.union(right)],
    intersection: [...left.intersection(right)],
    difference: [...left.difference(right)],
    symmetricDifference: [...left.symmetricDifference(right)],
    predicates: [
      new Set([2]).isSubsetOf(left),
      left.isSupersetOf(new Set([2])),
      new Set([7]).isDisjointFrom(left),
    ],
    acceptsSetLike: [...new Set([1, 2]).union(setLike)],
    sourcesUntouched: [left.size, right.size],
  };
}
```

listing 38: the seven methods, plus a set like object riding union (source: javascript/samples/src/ch05-collections.mjs, lines 73-100)

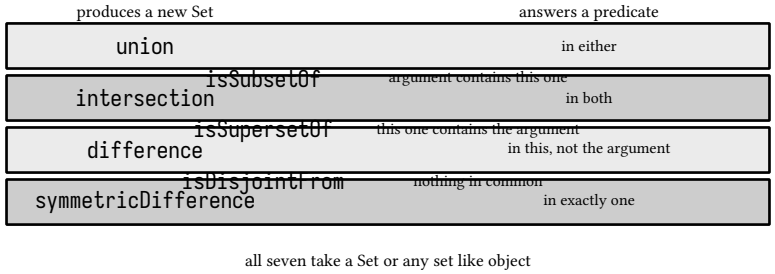


figure 26: the seven set methods, four producers and three predicates

5.6. weak references

A `WeakMap` holds entries whose keys it does not keep alive: when nothing else references a key, the entry can go. The observable, deterministic facts are narrower than the garbage collector promise: keys must be objects, primitives throw, entries are not enumerable and the collection has no `size`, and a `WeakRef` dereferences its target for as long as a strong reference exists somewhere:

```
// WeakMap keys must be objects, entries are not enumerable and do not
// keep their keys alive; WeakRef holds a target without owning it.
// the deterministic facts: primitive keys throw, and deref sees the
// target while a strong reference still exists elsewhere
export function weakFacts() {
  const meta = new WeakMap();
  const target = { name: "session" };
  meta.set(target, { opened: true });
  const ref = new WeakRef(target);
  let primitiveKey = "accepted";
  try {
    meta.set("scalar", 1);
  } catch (e) {
    primitiveKey = e.constructor.name;
  }
  return {
    get: meta.get(target).opened,
    notEnumerable: [typeof meta.size, typeof meta[Symbol.iterator]],
    derefSees: ref.deref().name,
    primitiveKey,
  };
}
```

listing 39: primitive keys throw, no size, no iteration, deref sees the target (source: javascript/samples/src/ch05-collections.mjs, lines 51-72)

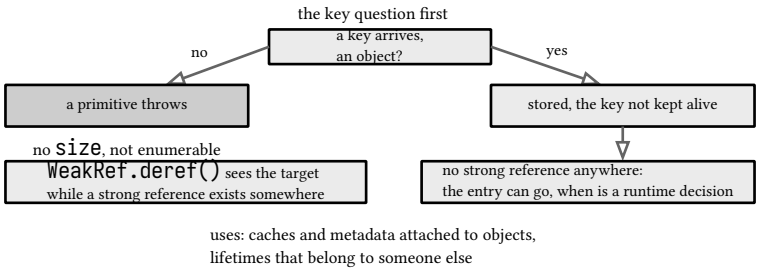


figure 27: the weak side, what is deterministic and what the runtime owns

When collection actually happens is a runtime decision, so the tests claim only the deterministic parts. The uses are caches and metadata attached to objects whose lifetime belongs to someone else, the pattern chapter 3's memoizer would want if its keys were DOM nodes.

5.7. binary data

`Float16Array`, ES2025, is the half precision end of the typed array family, two bytes per element. It keeps exactly what fits in eleven bits of significand and rounds the rest, `Math.f16round` performs the rounding without the array, and `DataView` reads and writes the format at a byte offset:

```
// Float16Array stores half precision: 1 + 1/1024 survives exactly,
// pi rounds to the nearest f16, Math.f16round does the rounding
// without the array, and DataView reads and writes the format
export function float16Facts() {
  const exact = new Float16Array(1);
  exact[0] = 1.0009765625; // 1 + 1/1024, representable in f16
  const pi = new Float16Array(1);
  pi[0] = 3.14159;
  const view = new DataView(new ArrayBuffer(2));
  view.setFloat16(0, 1.5);
  return {
    exactKept: exact[0],
    piRounds: pi[0],
    f16round: [Math.f16round(3.14159), Math.f16round(0.5)],
    dataViewRoundTrip: view.getFloat16(0),
    bytesPerElement: Float16Array.BYTES_PER_ELEMENT,
  };
}
```

listing 40: an exact f16, a rounded pi, f16round, and a DataView round trip (source: javascript/samples/src/ch05-collections.mjs, lines 101-119)

The `Uint8Array` codecs, part of the ES2026 seven, end the era of hand rolled base64: `toBase64` and `setFromBase64`, `toHex` and `setFromHex`, one method per direction, each pair a byte exact round trip:

```
// Uint8Array gained text codecs in ES2026: toBase64 and setFromBase64,
// toHex and setFromHex, one method per direction of each conversion
export function codecFacts() {
  const bytes = new Uint8Array([0xde, 0xad, 0xbe, 0xef]);
  const hex = bytes.toHex();
  const base64 = bytes.toBase64();
  const fromHex = new Uint8Array(4);
  fromHex.setFromHex(hex);
  const fromBase64 = new Uint8Array(4);
  fromBase64.setFromBase64(base64);
  const hi = new TextEncoder().encode("hi");
  return {
    hex,
    base64,
    hexRoundTrip: [...fromHex],
    base64RoundTrip: [...fromBase64],
    textRoundTrip: hi.toBase64() === "aGk=" && new TextDecoder().decode(hi) === "hi",
  };
}
```

listing 41: hex and base64 round trips, and text through the same codecs (source: javascript/samples/src/ch05-collections.mjs, lines 120-139)

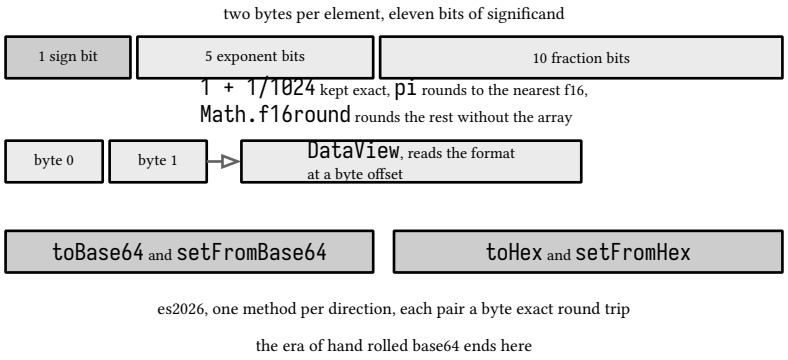


figure 28: f16 memory, the view over it, and the codec pairs

The pattern to hold onto: iteration is pull based and the helpers keep it pull based, the keyed collections ride one equality, SameValueZero, and the binary additions round out what used to need userland libraries. Chapter 6 puts the same protocol to work async, and chapter 8 places each addition on its edition shelf.

NOTE: what this chapter measured

Iterator helpers, Set methods, and `Float16Array` are ES2025; `Iterator.concat`, `getOrInsert`, and the `Uint8Array` base64 and hex codecs are three of the ES2026 seven. All of it measured live on node v26.3.0, the seven set methods one by one, the laziness claim by call counting, and the f16 values by exact comparison.

sources: developer.mozilla.org iteration protocols, iterator helpers, set methods, typed array, and base64 pages, tc39.es es2025 iterator helpers and set methods clauses, tc39 proposals for concat, getOrInsert, and uint8array base64, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows, 67 tests green through `npm run verify` in `javascript/samples`, 20 of them this chapter's.

6. async javascript

Async code here is one thread and two queues. Synchronous code runs to completion, the microtask queue drains after it, and timers and other macrotasks fire one turn later. Promises are the currency of that loop: a value that settles once, consumed by `then` chains or by `await`, with errors propagating through the chain like values. Everything in this chapter runs on node 26 with no build step.

6.1. the loop, one run through

The ordering is deterministic and the sample pins it. Synchronous code first, then every microtask in front of the timers, and a zero millisecond timer fires in a later macrotask than all of them. `await` suspends to the microtask queue even when the awaited value is not a promise:

```
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// one run through the loop: sync code first, then the microtask queue
// drains, and a zero millisecond timer fires in a later macrotask,
// after every microtask in front of it. awaiting a non promise still
// yields to the microtask queue
export async function loopOrder() {
  const order = [];
  order.push("sync");
  queueMicrotask(() => order.push("microtask"));
  await null; // suspends even though null is not a promise
  order.push("after-await");
  await sleep(2);
  order.push("after-timer");
  setTimeout(() => order.push("timer"), 0);
  await sleep(2);
  return order;
}
```

listing 42: sync, microtask, await, and timer order, recorded in sequence (source: javascript/samples/src/ch06-async.mjs, lines 3-21)

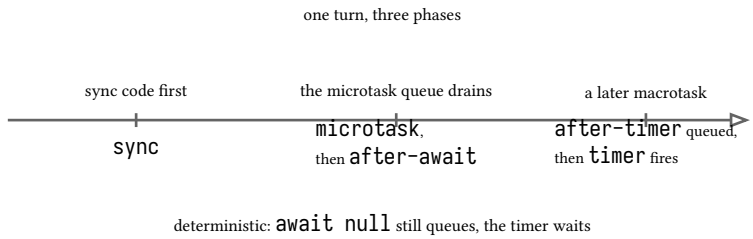


figure 29: one run through the queues, in order

The recorded order is `sync`, `microtask`, `after-await`, `after-timer`, `timer`, every run, because microtasks scheduled during the turn run before the next macrotask begins. Node also drains a `process.nextTick` queue ahead of promise microtasks, a third lane this book stays off by scheduling with `queueMicrotask`.

6.2. promises from scratch

A promise takes an executor that runs immediately with two settlers, and the contract is settle once: the first `resolve` or `reject` wins and every later call is ignored. That single fact is what makes a promise safe to hand to any number of consumers:

```
// a promise settles exactly once: the second resolve and a later
// reject are both ignored, the first settlement wins
export function settleOnceFacts() {
  const log = [];
  const promise = new Promise((resolve, reject) => {
    resolve("first");
    resolve("second");
    reject(new Error("late"));
  });
  return promise.then((v) => {
    log.push(v);
    return log;
  });
}
```

listing 43: a second resolve and a late reject, both ignored (source: javascript/samples/src/ch06-async.mjs, lines 23-36)

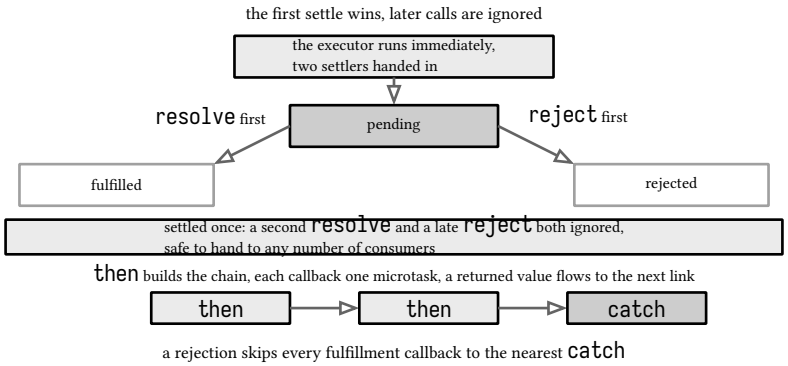


figure 30: the settle once machine, the chain riding beneath

then builds the chain. Each callback runs as a microtask, a returned value flows to the next link, and a rejection skips every fulfillment callback until the nearest catch, which is why the test's chain sees only the caught message. A callback or an async function that returns a promise makes the next link wait for that promise's settlement, then-adoption in the spec, and await unwraps thenables the same way.

6.3. async and await, and Promise.try

An async function is syntax over promise construction: every return resolves the result, every throw rejects it. await is syntax over then, with the same microtask suspension. The gap the syntax left was the sync throw: wrapping callback style code meant a try block around the call. Promise.try, from ES2025, lifts any function into the chain, a returned value resolves and a synchronous throw rejects:

```
// Promise.try (ES2025) lifts any function into a promise: a returned
// value resolves, a synchronous throw becomes a rejection, so
// callback style code enters a chain without a try block
export async function promiseTryFacts() {
  const value = await Promise.try(() => 41 + 1);
  let thrown = "resolved";
  try {
    await Promise.try(() => {
      throw new Error("sync");
    });
  } catch (e) {
    thrown = "rejected:" + e.message;
  }
  return { value, thrown };
}
```

listing 44: Promise.try on a value and on a synchronous throw (source: javascript/samples/src/ch06-async.mjs, lines 38-52)

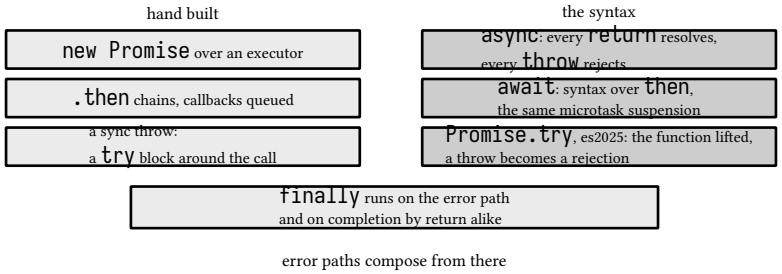


figure 31: the syntax against the hand built forms it replaces

Error paths compose from there. A throw inside an async function is a rejection awaiting its catch, an awaited rejection propagates to the nearest handler, and `finally` runs on the error path and on completion by return alike:

```
// error paths: a throw inside an async function is a rejection, an
// awaited rejection propagates to the nearest catch, and finally runs
// on both paths
export async function errorPathFacts() {
  const log = [];
  const boom = async () => {
    throw new Error("inner");
  };
  let caught = "none";
  try {
    await boom();
  } catch (e) {
    caught = e.message;
  } finally {
    log.push("finally");
  }
  async function completes() {
    try {
      return "returned";
    } finally {
      log.push("finally-on-return");
    }
  }
  return { caught, finallyOnReturn: await completes(), log };
}
```

listing 45: rejections propagating, finally on both paths (source: javascript/samples/src/ch06-async.mjs, lines 145-169)

6.4. the four combinators

The static combinators differ only in failure policy: `all` rejects on first rejection, `allSettled` never rejects and reports every outcome, `race` takes the first settlement of any kind, and `any` takes the first fulfillment, ignoring rejections unless all fail, which rejects with an `AggregateError` carrying every reason:

```
// the four combinators differ only in failure policy. all resolves in
// argument order whatever the completion order, allSettled reports
// every outcome, race takes the first settlement of any kind, and any
// takes the first fulfillment, ignoring rejections unless all fail
export async function combinatorFacts() {
  const argumentOrder = await Promise.all([
    sleep(9).then(() => "a"),
    sleep(1).then(() => "b"),
    sleep(5).then(() => "c"),
  ]);
  const outcomes = await Promise.allSettled([
    Promise.resolve(1),
    Promise.reject(new Error("no")),
    sleep(1).then(() => 3),
  ]);
  const settled = outcomes.reduce(
    (acc, o) => ((acc[o.status] += 1), acc),
    { fulfilled: 0, rejected: 0 },
  );
  const raced = await Promise.race([
    Promise.resolve("fast"),
    sleep(40).then(() => {
      throw new Error("slow rejected");
    }),
  ]);
  const anyUsable = await Promise.any([
    Promise.reject(new Error("broken")),
    sleep(2).then(() => "usable"),
  ]);
  let allFail = "no";
  try {
    await Promise.any([Promise.reject(new Error("a")), Promise.reject(new Error("b"))]);
  } catch (e) {
    allFail = e.constructor.name; // AggregateError carries every reason
  }
  return { argumentOrder, settled, raced, anyUsable, allFail };
}
```

listing 46: the four combinators, one policy each, all five outcomes pinned (source: javascript/samples/src/ch06-async.mjs, lines 54-90)

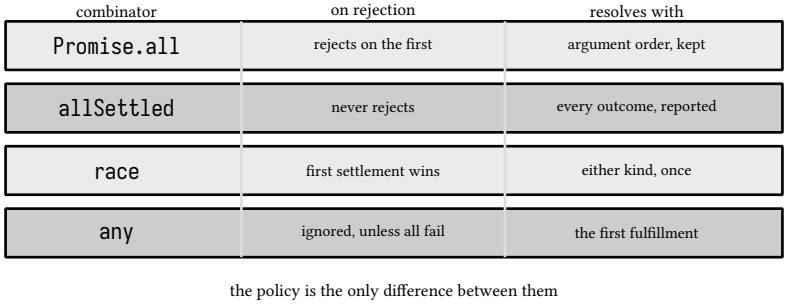


figure 32: the four combinators, one failure policy each

`Promise.all` resolves in argument order whatever the completion order, which the reversed delays in the sample prove, and the racing test shows an already resolved promise beating a rejection that never gets a say.

6.5. async iteration

The iterator protocol of chapter 5 has an async pair: `Symbol.asyncIterator` returns an iterator whose `next` yields promises of `{ value, done }`, and `for await..of` consumes it, suspending between steps. Hand built first, then as syntax:

```
// the async iterator protocol: Symbol.asyncIterator returns an
// iterator whose next() yields promises of { value, done }, and
// for await..of consumes it, awaiting each step
export async function handBuiltAsyncIterable() {
  const countdown = {
    n: 3,
    [Symbol.asyncIterator]() {
      return {
        next: async () =>
          this.n > 0
            ? { value: this.n--, done: false }
            : { value: undefined, done: true },
      };
    },
  };
  const seen = [];
  for await (const tick of countdown) seen.push(tick);
  return seen;
}
```

listing 47: a hand built async iterable (source: javascript/samples/src/ch06-async.mjs, lines 106-124)

```
// an async generator awaits between yields, which is the protocol
// above written as syntax
export async function asyncGeneratorFacts() {
  const log = [];
  async function* stream(items, ms) {
    for (const item of items) {
      await sleep(ms);
      log.push(`yield:${item}`);
      yield item;
    }
  }
  const collected = [];
  for await (const item of stream(["a", "b"], 1)) {
    log.push(`body:${item}`);
    collected.push(item);
  }
  return { collected, log };
}
```

listing 48: an async generator awaiting between yields, the same protocol as syntax (source: javascript/samples/src/ch06-async.mjs, lines 126-143)

The generator's log interleaves `yield` and `body` lines, which is the interleaving the test asserts: each `yield` suspends, the loop body runs, then the generator resumes. `Array.fromAsync`, one of the ES2026 seven, collects the whole stream into one array, and accepts an iterable of plain promises too:

```
// Array.fromAsync (ES2026) collects anything awaitable into one
// array: an async iterable, or an iterable of promises
export async function fromAsyncFacts() {
  async function* pages() {
    for (const page of [1, 2, 3]) {
      await sleep(1);
      yield page;
    }
  }
  const fromGenerator = await Array.fromAsync(pages());
  const fromPromises = await Array.fromAsync([Promise.resolve(1), 2]);
  return { fromGenerator, fromPromises };
}
```

listing 49: fromAsync over an async generator and over promises (source: javascript/samples/src/ch06-async.mjs, lines 92-104)

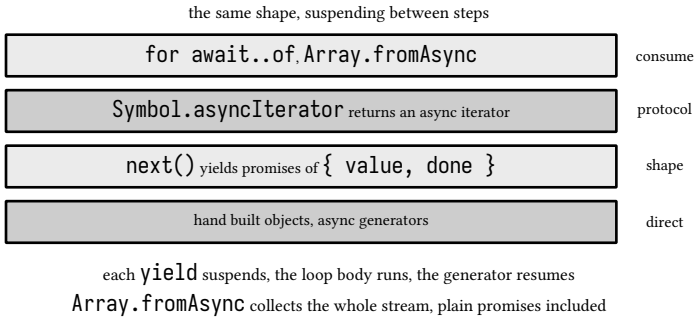


figure 33: the async pair of the protocol, stacked

6.6. cancellation

A promise cannot be cancelled from outside, and that gap is what `AbortSignal` fills: the work listens, the caller signals. The signal carries a reason, an explicit abort produces a `DOMException` named `AbortError`, and `AbortSignal.timeout` builds a self aborting signal whose reason is named `TimeoutError`. `AbortSignal.any` merges sources into one:

```
// a promise cannot be cancelled from outside: AbortSignal is the
// convention the work listens to. the signal carries a reason, a
// timeout shortcut builds one, and AbortSignal.any merges sources
export function abortableWork(signal) {
  return new Promise((resolve, reject) => {
    signal.addEventListener("abort", () => reject(signal.reason));
    setTimeout(() => resolve("done"), 500);
  });
}

export async function abortFacts() {
  const controller = new AbortController();
  const work = abortableWork(controller.signal);
  controller.abort("stop");
  let rejected = "resolved";
  try {
    await work;
  } catch (e) {
    rejected = e;
  }
  const timed = new Promise((resolve, reject) => {
    const signal = AbortSignal.any([AbortSignal.timeout(2), new AbortController().signal]);
    signal.addEventListener("abort", () => reject(signal.reason.name));
  });
  let timedOut = "no";
  try {
    await timed;
  } catch (e) {
    timedOut = e;
  }
  return { rejected, timedOut };
}
```

listing 50: abortable work, an explicit abort, a timeout, and a merged signal (source: javascript/samples/src/ch06-async.mjs, lines 171-202)

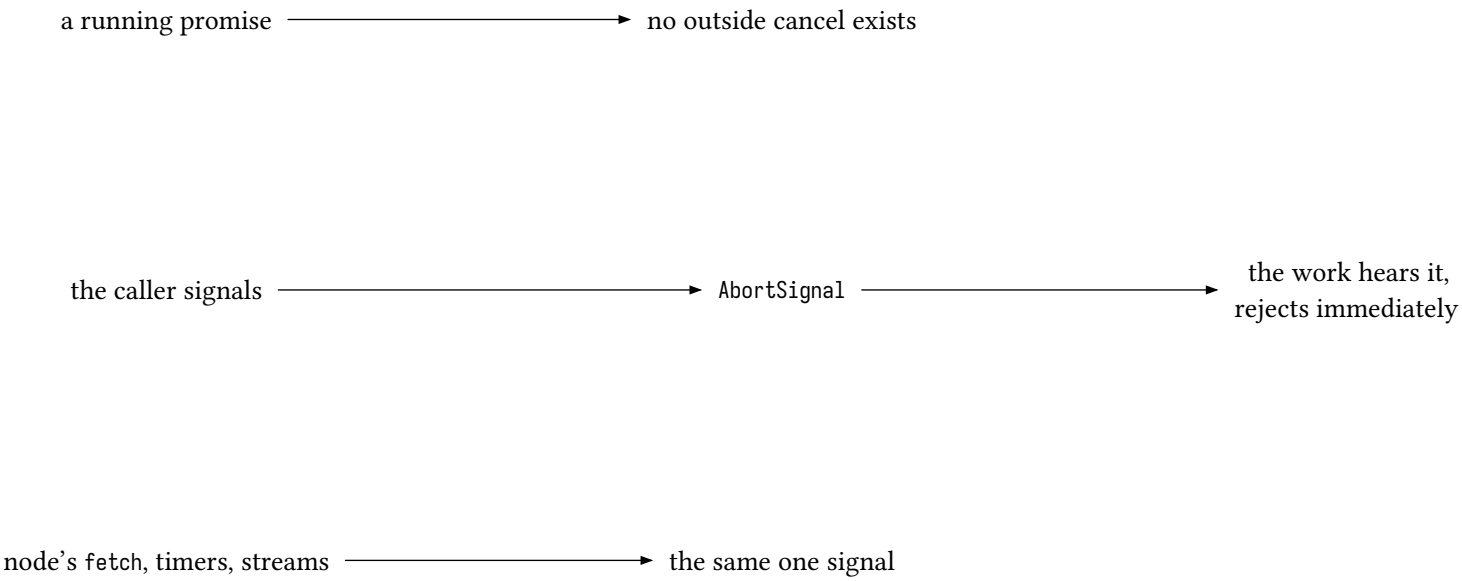


figure 34: cancellation, the signal the work listens to

The test aborts immediately and the half second of work rejects at once with the given reason. Node threads signals through its fetch, timers, and streams, so this one object is the cancellation currency across the runtime.

sources: developer.mozilla.org on promises, microtasks, the event loop, for await..of, and abort controller, nodejs.org event loop guide, tc39.es promise.try clause, tc39 array.fromAsync proposal, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows, 86 tests green through `npm run verify` in `javascript/samples`, 19 of them this chapter's.

7. modules and resources

A module is a file with its own scope, evaluated once, exporting names it chooses. Since 2015 that system is native, and since node 22 it is the runtime's default answer: `.mjs` is always a module, `.cjs` is always a script, and `.js` follows the package manifest's `type` field, which the samples project sets to `"module"`. The second half of the chapter spends the same discipline on resources: `using` declarations tie a value's release to the end of its block, the ES2025 answer to `finally` cleanup.

One sentence of scope on the script side: importing a `.cjs` file yields its `module.exports` as the default export, named imports from it work only where static analysis finds them on that object, and `require` of a module is not this book's path.

7.1. the export forms, and live bindings

One module, four ways to publish: a named `const`, a mutable named `let` with its mutator, and a default, which is just another export with the special name `default`:

```
// ch07 second module: the export forms a consumer can reach, plus a
// live binding whose value changes without any reimport

export const PI2 = Math.PI * 2;

export let tally = 0;

export function bump() {
  tally += 1;
  return tally;
}

export default function area(r) {
  return Math.PI * r ** 2;
}
```

listing 51: the exporter: a constant, a live let, a function, a default (source: javascript/samples/src/ch07-shapes.mjs, lines 1-16)

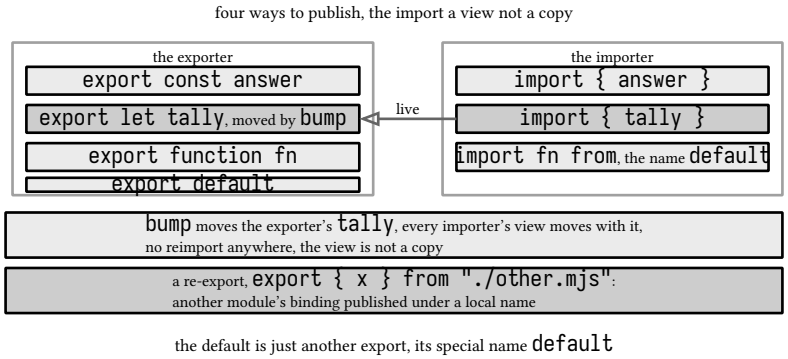


figure 35: the exporter's bindings, and the importer's live views

The consumer picks names with braces and the default without. Named imports are live views of the exporter's bindings, not copies, which is the part that surprises programmers from other languages: `bump` moves the exporter's `tally`, and every importer's view of `tally` moves with it, no reimport anywhere. A re-export publishes another module's binding under a local name:

```
import defaultExport, { PI2, bump, tally as tallyView } from "./ch07-shapes.mjs";
import config from "./ch07-data.json" with { type: "json" };

// a re-export publishes another module's binding under a local name
export { PI2 as TAU } from "./ch07-shapes.mjs";

// named imports are live views of the exporter's bindings, not
// copies: bump moves the exporter's tally and this module's view of
// it moves with it, with no reimport anywhere
export function importFacts() {
  bump();
  return {
    defaultIsFn: typeof defaultExport,
    area: defaultExport(2),
    tau: PI2,
    liveView: tallyView,
  };
}
```

listing 52: named and default imports, a live view, a re-export (source: javascript/samples/src/ch07-modules.mjs, lines 7-24)

7.2. dynamic import

`import()` is an expression that returns a promise of the module namespace, evaluates the module on first call, and returns the same namespace object on every later call, one instance per specifier per process. It is how code loads on demand, and how a module that might fail can be wrapped in a normal `try`:

```
// dynamic import returns the module namespace as a promise. the
// first call evaluates the module, every later call returns the same
// namespace object, one instance per specifier per process
export async function dynamicFacts() {
  const first = await import("./ch07-shapes.mjs");
  const second = await import("./ch07-shapes.mjs");
  return {
    sameNamespace: first === second,
    defaultPresent: typeof first.default,
    named: typeof first.bump,
  };
}
```

listing 53: two dynamic imports of one specifier, one namespace object (source: javascript/samples/src/ch07-modules.mjs, lines 32-43)

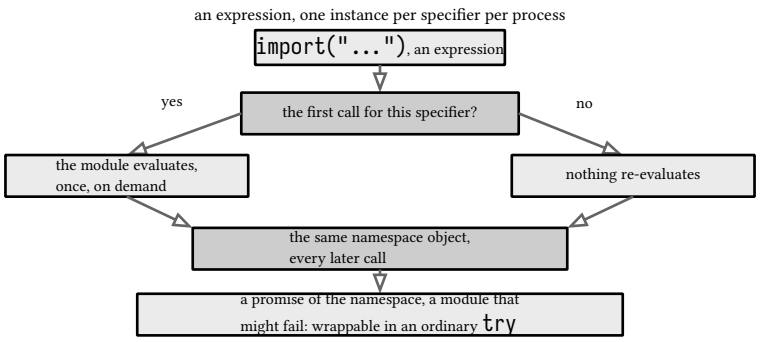


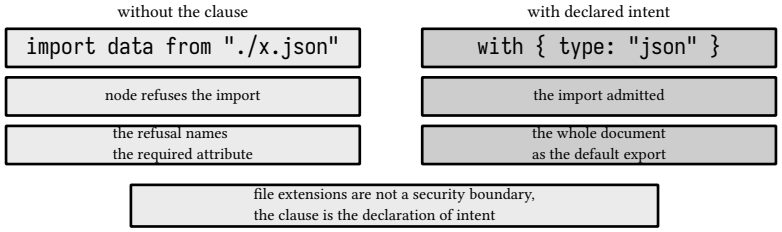
figure 36: the dynamic import, one namespace per specifier

7.3. json modules and import attributes

A JSON module's whole document arrives as its default export, and the `with` clause is what admits it. The attribute exists because file extensions are not a security boundary: without the declaration of intent, node refuses the import, and the refusal message says exactly what is missing:

```
// a JSON module's whole document arrives as the default export, and
// the attribute in the import statement is what admits it
export function jsonFacts() {
  return { title: config.title, edition: config.edition, tags: config.tags };
}
```

listing 54: a json module, imported with its type attribute (source: javascript/samples/src/ch07-modules.mjs, lines 26-30)



the legacy `assert` keyword: a `TypeError` on this runtime, stage 4 with es2025

figure 37: the with clause, declared intent against the refusal

Two adjacent facts the tests pin. Dropping the `with` clause rejects the import with a message naming the required attribute, and the legacy `assert` keyword that carried this job before ES2025 is gone on this runtime, it throws a `TypeError`. Import attributes reached stage 4 with the ES2025 edition.

7.4. import.meta and module relative paths

Inside a module, `import.meta` carries the module's own url, and node adds `dirname` and `filename`. `fileURLToPath` converts to a native path, and the `node:path` functions build on it, which is the portable recipe for "a file next to this module", posix and windows alike:

```
// import.meta carries the module's own url, node adds dirname and
// filename, and fileURLToPath with path build module relative paths
// that work on posix and windows alike
export function metaFacts() {
  const file = fileURLToPath(import.meta.url);
  const dir = path.dirname(file);
  return {
    urlEndsMjs: import.meta.url.endsWith(".mjs"),
    dirnameMatches: import.meta.dirname === dir,
    siblingResolve: path.join(dir, "ch07-data.json") === fileURLToPath(new URL("./ch07-data.json",
import.meta.url)),
  };
}
```

listing 55: url, dirname, and a sibling resolved two equivalent ways (source: javascript/samples/src/ch07-modules.mjs, lines 45-56)

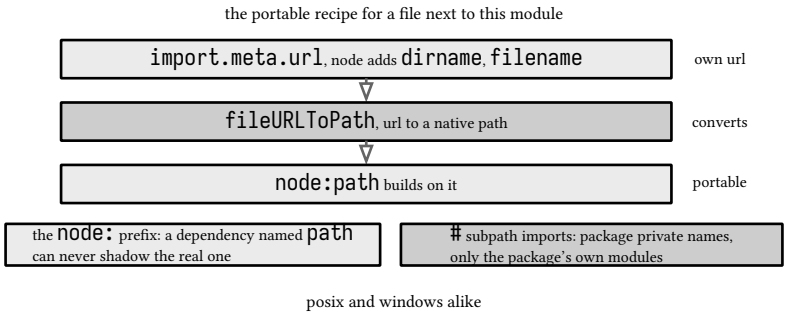


figure 38: the module's own address, made native

The same `node:` prefix that marks those two builtins marks all of them, `node:url`, `node:path`, `node:util`, and it exists so that a project dependency named `path` can never shadow the real one. The package manifest can also define private subpath imports, names beginning with `#` that only the package's own modules can use:

```
// package.json
{
  "imports": {
    "#cfg": "./src/config.mjs",
    "#internal/*": "./src/internal/*.mjs"
  }
}

// any module in the package
import { answer } from "#cfg";
import { hidden } from "#internal/lib";
```

fragment, illustrative, not executed

Subpath imports were measured on this node, in a scratch package with exactly that manifest: both specifiers resolve, and the samples project keeps a minimal manifest instead, so chapter 1's manifest listing stays as it was.

7.5. explicit resource management

`Symbol.dispose` marks a value as releasable. A `using` declaration binds one and runs its disposer at the closing brace, in reverse declaration order, on the normal path and the throw path alike, and a `null` or `undefined` binding disposes nothing. With `await using` and `Symbol.asyncDispose`, the block waits for the release:

```
// explicit resource management (ES2025): Symbol.dispose marks a
// releasable, using declares one, disposal runs at the closing brace
// in reverse declaration order, a throw inside still disposes before
// the error escapes, and null or undefined dispose nothing
export function usingFacts() {
  const log = [];
  const resource = (name) => ({
    [Symbol.dispose]() {
      log.push(name);
    },
  });
  {
    using a = resource("a");
    using b = resource("b");
    log.push("body");
  } // b disposed, then a
  let caught = "none";
  try {
    using c = resource("c");
    throw new Error("boom");
  } catch (e) {
    caught = e.message;
  }
  {
    using skipped = null;
    log.push("null ok");
  }
  return { log, caught };
}
```

listing 56: using: reverse order, the throw path still disposes, null allowed (source: javascript/samples/src/ch07-modules.mjs, lines 58-86)

```
// the async pair: Symbol.asyncDispose marks a resource whose release
// awaits, await using declares one, and the block waits for the
// disposal before continuing
export async function asyncUsingFacts() {
  const log = [];
  async function open(name) {
    log.push(`open:${name}`);
    return {
      [Symbol.asyncDispose]() {
        log.push(`close:${name}`);
        return Promise.resolve();
      },
    };
  }
  {
    await using conn = await open("db");
    log.push("query");
  }
  return log;
}
```

listing 57: await using: an async connection opened, used, awaited closed (source: javascript/samples/src/ch07-modules.mjs, lines 88-107)

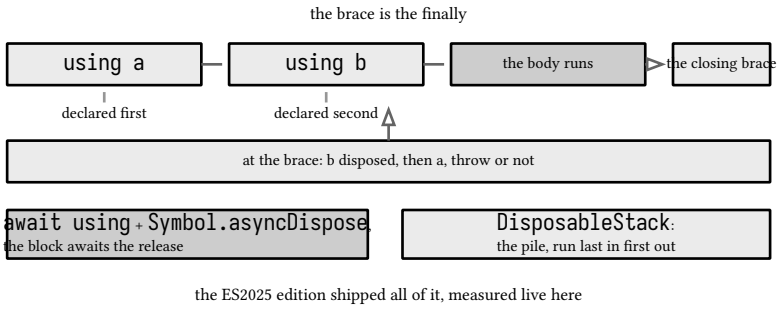


figure 39: the block scope as a resource scope

`DisposableStack` is the imperative form, a pile of disposers run last in first out, `defer` pushes a function, `use` pushes a disposable, `adopt` pairs an arbitrary value with a disposer, and `move` hands the whole pile to a fresh stack:

```
// DisposableStack collects disposers imperatively, last in first
// out, use wraps a resource, adopt pairs a value with a disposer,
// and disposed reports whether the pile has already run
export function stackFacts() {
  const log = [];
  const stack = new DisposableStack();
  stack.defer(() => log.push("defer"));
  stack.use({
    [Symbol.dispose]() {
      log.push("use");
    },
  });
  stack.adopt("config", (v) => log.push(`adopt:${v}`));
  const before = stack.disposed;
  stack.dispose();
  return { log, before, after: stack.disposed };
}
```

listing 58: defer, use, adopt, dispose, exactly once (source: javascript/samples/src/ch07-modules.mjs, lines 109-125)

The using declaration form belongs to the common case, one function, one scope, one resource. The stack belongs to code that assembles resources dynamically, the way a server builds a request's cleanup list as it goes.

sources: developer.mozilla.org on modules, import attributes, and symbol.dispose, nodejs.org esm documentation and api for url, path, and util, tc39.es explicit resource management and import attributes clauses, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows, 97 tests green through `npm run verify` in `javascript/samples`, 11 of them this chapter's.

8. the standard library by edition

The standard library grows by edition, one approved snapshot each June, and a runtime implements what it implements. This chapter reads the library the way a working programmer meets it: by edition shelf, then by what node 26.3.0 actually runs. Everything below was measured on this runtime, the ES2026 edition approved on 2026-06-30, the seventeenth, and the two ES2026 items this runtime does not carry are named as absent, with the probe that says so.

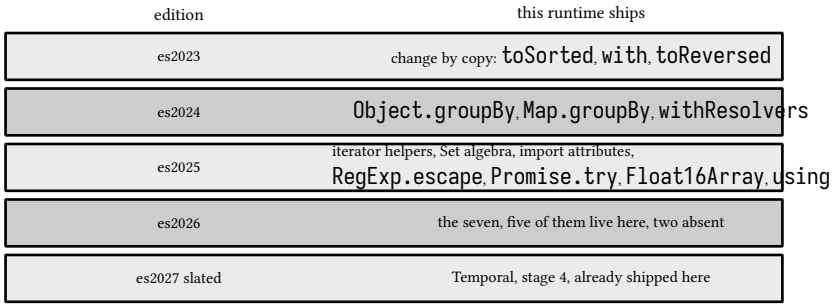


figure 40: the shelves, what node 26.3.0 runs of each edition

8.1. change by copy, and grouping

The ES2023 copy family mutates nothing: `toSorted`, `with`, and `toReversed` return new arrays and leave the source alone, which removed the spread-then-mutate dance from ordinary code. ES2024's `groupBy` partitions by a key callback, and its result is a null prototype object, no `toString`, no inherited anything, which a spread repairs when a plain object is wanted:

```
// the ES2023 copy family mutates nothing: toSorted, with, and
// toReversed each return a new array and leave the source alone
export function changeByCopyFacts() {
  const original = [3, 1, 2];
  return {
    original,
    sorted: original.toSorted(),
    replaced: original.with(1, 9),
    reversed: original.toReversed(),
  };
}

// Object.groupBy (ES2024) partitions by a key callback into a null
// prototype object, no toString, no inherited anything, which a
// spread repairs when a normal object is wanted; Map.groupBy keeps
// map semantics
export function groupByFacts() {
  const grouped = Object.groupBy([1, 2, 3, 4], (n) => (n % 2 ? "odd" : "even"));
  const byMap = Map.groupBy([1, 2, 3, 4], (n) => (n % 2 ? "odd" : "even"));
  return {
    odd: grouped.odd,
    protoIsNull: Object.getPrototypeOf(grouped) === null,
    repaired: { ...grouped }.odd,
    mapOddSize: byMap.get("odd").length,
  };
}
```

listing 59: the copy family against one original, then groupBy and its null prototype (source: javascript/samples/src/ch08-edition.mjs, lines 5-30)

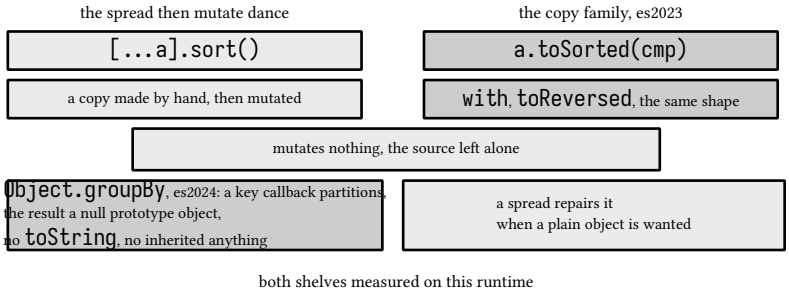


figure 41: the copy family, and the grouping it left to repair

8.2. escaping patterns

`RegExp.escape`, from ES2025, makes any string safe as a literal in a pattern. The escape rules were measured on this runtime, character by character: regexp syntax characters take a backslash, characters that could bind to surrounding context, the hyphen in a class range for instance, take a two digit hex escape, and letters, digits, and the underscore pass bare, except in first position. The first character is always escaped, a leading letter becomes `\x7a`, so an escaped fragment can follow a backslash in a host pattern without completing somebody else's escape sequence:

```
// RegExp.escape (ES2025) makes any string a literal: regexp syntax
// characters take a backslash, characters that could bind to context
// like the hyphen take a hex escape, letters and digits pass bare
// except in first position, where the first character is always
// escaped so the fragment can follow a backslash safely
export function escapeFacts() {
  const literal = (text) => new RegExp(RegExp.escape(text));
  return {
    dotStarEscaped: RegExp.escape("a.b*c"),
    firstAlwaysEscaped: [RegExp.escape("z"), RegExp.escape("5")],
    bareMidString: RegExp.escape("az9"),
    hyphenHex: RegExp.escape("a-b"),
    matchesItself: ["x.a*b".includes("a*b"), literal("a*b").test("xxa*bxx")],
    anchorsDoNotFire: [literal("^start$").test("the ^start$ line")],
  };
}
```

listing 60: the measured escape classes, and the output matching itself literally (source: javascript/samples/src/ch08-edition.mjs, lines 32-47)

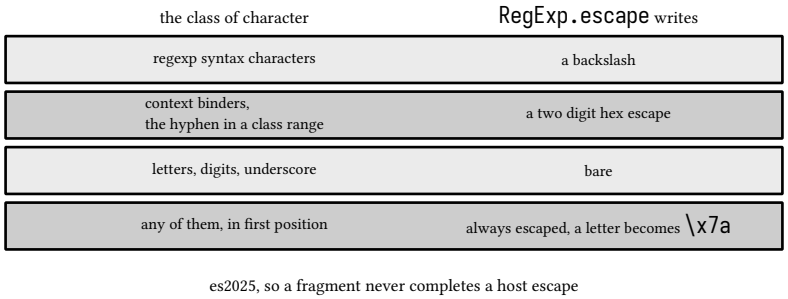


figure 42: the measured rule classes, character by character

8.3. errors that identify themselves

`Error.isError`, one of the ES2026 seven, answers the question `instanceof Error` gets wrong the moment an error crosses a realm or a lookalike object carries the right names. The `cause` option, the ES2022 neighbor, chains context through every level and the test walks it to the root:

```
// Error.isError (ES2026) answers the type question a lookalike
// object fools instanceof into getting wrong, and the cause option
// chains context through every level
export function errorFacts() {
  const lookalike = { name: "Error", message: "fake", stack: "nope" };
  const root = new Error("root");
  const middle = new Error("middle", { cause: root });
  const top = new Error("top", { cause: middle });
  return {
    isError: [Error.isError(top), Error.isError(lookalike), Error.isError("nope")],
    causeChain: top.cause.cause.message,
  };
}
```

listing 61: isError against a real error, a lookalike, and a cause chain (source: javascript/samples/src/ch08-edition.mjs, lines 49-61)

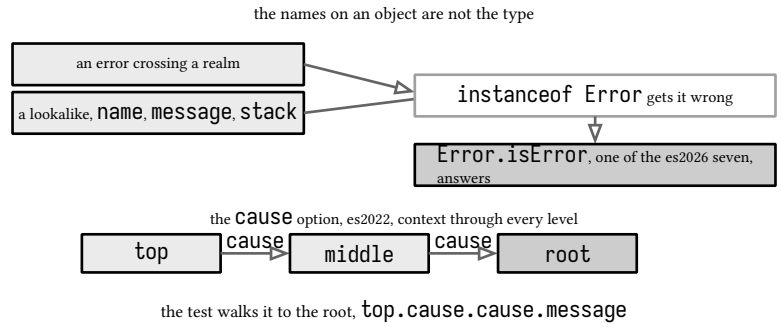


figure 43: the type question, and the chain that carries context

8.4. json that keeps its digits

`JSON.stringify` writes doubles, and doubles past 2^53 round. The second of the ES2026 seven to live here, `JSON.rawJSON`, pins source text: an integer embedded as raw JSON serializes to exactly the digits written, where the same literal as a number loses its trailing digit on the way out. `JSON.isRawJSON` reports which values carry the pin. The boundary is honest and the test states it: parsing the pinned text still produces the double, `9007199254740993` written and `9007199254740992` parsed, because the parse side of the source-text proposal is not on this runtime:

```
// JSON.rawJSON (ES2026) pins source text: a big integer embedded as
// raw JSON serializes to exactly the digits written, where the same
// number as a double loses the trailing digits on the way out.
// isRawJSON reports which values carry the pin. parsing the text
// still yields a double, that boundary is the parse side's to cross
export function rawJsonFacts() {
  const pinned = JSON.rawJSON("9007199254740993");
  const text = JSON.stringify({ id: pinned });
  const asDouble = JSON.stringify({ id: 9007199254740993 });
  return {
    text,
    asDouble,
    isRawJson: [JSON.isRawJSON(pinned), JSON.isRawJSON(9007199254740993)],
    parsesTo: JSON.parse(text).id,
  };
}
```

listing 62: raw digits surviving stringify, and the double that comes back from parse (source: javascript/samples/src/ch08-edition.mjs, lines 63-78)

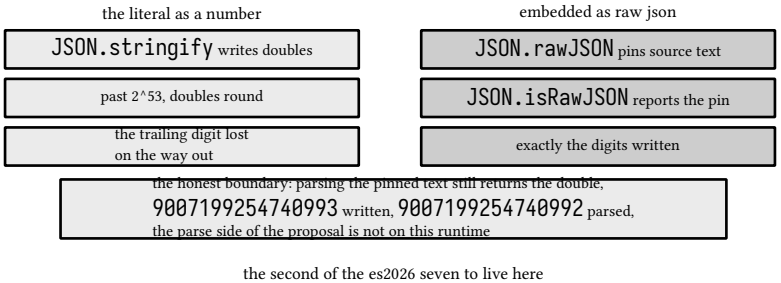


figure 44: the pin against the double, the same digits two ways

NOTE: in the ES2026 spec, absent on node 26.3.0

Two ES2026 items do not run here, both probed on 2026-09-13: `Math.sumPrecise` is `undefined` on this runtime, and `JSON.parse` takes two arguments here, its arity is 2, so the spec's third `{ source: true }` argument is silently ignored and the reviver never sees source text. Both absences are pinned by test, the day this runtime ships them the suite says so.

```
// in the ES2026 spec, not on node 26.3.0 (measured)
Math.sumPrecise([1, 1e100, 1, -1e100]);
// 2, exactly, any order
// [0.1, 0.2] still yields 0.30000000000000004, midpoint tie to even

// spec: a third argument opts into source text
JSON.parse("9007199254740993", null, { source: true });
// node 26: the third argument is ignored, a plain number back
```

fragment, illustrative, not executed

The remaining shelves live in their chapters: `Array.fromAsync` in chapter 6, the `Uint8Array` codecs and `Iterator.concat` and `getOrInsert` in chapter 5, all three part of the seven. Around the edges, two older additions keep earning their keep, `structuredClone` deep copies containers and refuses functions, and numeric collation sorts `a2` ahead of `a10`, which every lexicographic sort gets wrong:

```
// adjacent load bearing additions: structuredClone deep copies plain
// values and containers and refuses functions, and numeric collation
// sorts a2 ahead of a10, which every lexicographic sort gets wrong
export function adjacentFacts() {
  const cloned = structuredClone(new Map([["a", 1]]));
  let cloneError = "none";
  try {
    structuredClone(() => 1);
  } catch (e) {
    cloneError = e.name;
  }
  const numeric = new Intl.Collator("en", { numeric: true });
  return {
    mapCloned: cloned.get("a"),
    cloneError,
    numericSort: ["a100", "a10", "a2"].sort(numeric.compare),
  };
}
```

listing 63: structuredClone's boundary, and collation that reads numbers as numbers (source: javascript/samples/src/ch08-edition.mjs, lines 92-109)

8.5. temporal, measured live

Temporal is the largest addition to the library in a decade, and its status is worth stating plainly: Stage 4 in the tc39 process, slated for the ES2027 edition, and already shipped unflagged in this runtime, node 26, measured, and in Firefox 139 and Chrome 144. It replaces the `Date` object's mutable, ambiguous, local-time API with types that name what they hold. `Instant` is a point on the utc line. `PlainDate` is a calendar date with no time and no zone. `ZonedDateTime` is a wall clock reading pinned to a zone, and `Temporal.Now` is the one clock reading entry point:

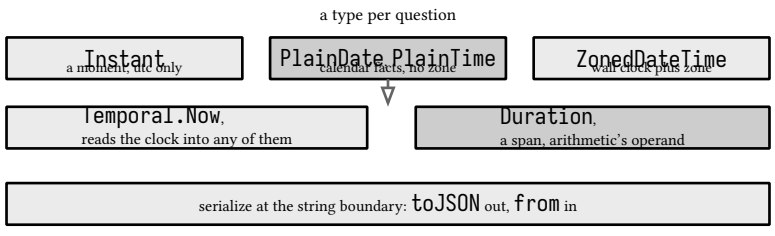


figure 45: the temporal types, what each one holds

`Instant` parses from an iso string, round trips through its string form at nanosecond precision, reads the epoch count, shifts by a duration, and compares with a total order:

```
// the string form, read the epoch count, shift by a duration, and
// compare with a total order instead of relational operators
export function instantFacts() {
  const launch = Temporal.Instant.from("2026-09-13T12:00:00Z");
  const roundTrip = Temporal.Instant.from(launch.toString());
  const earlier = Temporal.Instant.from("2026-01-01T00:00:00Z");
  return {
    iso: launch.toJSON(),
    epochMilliseconds: launch.epochMilliseconds,
    roundTripEquals: roundTrip.equals(launch),
    shifted: launch.add({ hours: 1 }).toString(),
    compare: [
      Temporal.Instant.compare(earlier, launch),
      Temporal.Instant.compare(launch, earlier),
      Temporal.Instant.compare(launch, launch),
    ],
  };
}
```

listing 64: an instant: parse, round trip, epoch count, shift, total order (source: javascript/samples/src/ch08-temporal.mjs, lines 5-22)

`PlainDate` arithmetic respects the calendar: a day over a month end lands correctly, differences come in the largest unit asked for, and `compare` orders dates that relational operators would corrupt:

```
// a PlainDate is a calendar date with no time and no zone: field
// access, arithmetic that respects the calendar, differences in the
// units asked for, and compare as the ordering
export function plainDateFacts() {
  const day = Temporal.PlainDate.from("2026-09-13");
  const later = Temporal.PlainDate.from("2027-09-13");
  return {
    fields: [day.year, day.month, day.day, day.dayOfWeek, day.daysInMonth],
    plusDays: day.add({ days: 1 }).toString(),
    plusMonths: day.add({ months: 1 }).toString(),
    differenceDefault: day.until(later).toString(),
    differenceInMonths: day.until(Temporal.PlainDate.from("2026-12-13"), { largestUnit:
"months" }).toString(),
    compare: Temporal.PlainDate.compare("2026-01-01", "2026-03-01"),
  };
}
```

listing 65: a calendar date: fields, arithmetic, differences, order (source: javascript/samples/src/ch08-temporal.mjs, lines 24-38)

`ZonedDateTime` carries its offset as part of the value. Converting the zone keeps the instant, the epoch nanoseconds are equal, and moves the wall clock, nine in New York is ten in the evening in Tokyo, and the plain views split the value apart for free:

```
// a ZonedDateTime is a wall clock reading pinned to a zone: the
// offset is part of the value, converting the zone keeps the instant
// and moves the wall clock, and the plain views split it apart
export function zonedFacts() {
  const opening = Temporal.ZonedDateTime.from("2026-09-13T09:00:00[America/New_York]");
  const tokyo = opening.withTimeZone("Asia/Tokyo");
  return {
    full: opening.toString(),
    offset: opening.offset,
    tokyoWallClock: tokyo.toString(),
    sameInstant: tokyo.epochNanoseconds === opening.epochNanoseconds,
    toInstant: opening.toInstant().toString(),
    toPlainDate: opening.toPlainDate().toString(),
  };
}
```

listing 66: a zoned wall clock: offset, zone conversion, plain views (source: javascript/samples/src/ch08-temporal.mjs, lines 40-54)

Durations are their own type, constructed from fields, totalled into a unit as a number, and rounded into larger units. `Temporal.Now` hands back the full precision types, never a mutable `Date`:

```
// Temporal.Now is the only clock reading entry point, and it hands
// back the full precision types, never a mutable Date
export function nowFacts() {
  const instant = Temporal.Now.instant();
  return {
    type: instant instanceof Temporal.Instant,
    zonedHere: Temporal.Now.zonedDateTimeISO().timeZoneId,
  };
}

// Durations are their own type: construct from fields, total into a
// unit as a number, round into larger units
export function durationFacts() {
  const shift = Temporal.Duration.from({ hours: 90, minutes: 30 });
  return {
    iso: shift.toString(),
    totalHours: shift.total({ unit: "hours" }),
    roundedToDays: shift.round({ largestUnit: "days" }).toString(),
  };
}
```

listing 67: the clock, and a duration across total and round (source: javascript/samples/src/ch08-temporal.mjs, lines 56-75)

Serialization is where teams lose dates, so the advice is one sentence: keep values in Temporal types in memory, cross boundaries as strings, `toJSON` writes rfc 9557 text through `JSON.stringify` with no replacer, `from` revives, and the relational operators are gone, `<` on two plain dates throws a `TypeError`, `compare` methods are the ordering:

```
// serialization: every Temporal type stringifies to rfc 9557 text
// through toJSON, so JSON.stringify carries it without a replacer,
// and from() revives. relational operators are gone, < and > throw,
// compare methods are the ordering
export function serializeFacts() {
  const day = Temporal.PlainDate.from("2026-09-13");
  let relational = "allowed";
  try {
    // eslint-disable-next-line no-unused-expressions
    day < Temporal.PlainDate.from("2027-01-01");
  } catch (e) {
    relational = e.constructor.name;
  }
  const envelope = JSON.stringify({ ships: day });
  const revived = Temporal.PlainDate.from(JSON.parse(envelope).ships);
  return {
    envelope,
    revivedEquals: revived.equals(day),
    relational,
    plainTime: Temporal.PlainTime.from("09:30:15").toString(),
    plainDateTime: Temporal.PlainDateTime.from("2026-09-13T09:30:15").toString(),
  };
}
```

listing 68: the string boundary, and the operator that no longer exists (source: javascript/samples/src/ch08-temporal.mjs, lines 77-99)

sources: tc39.es es2025 and es2026 draft clauses for iterator helpers, set methods, regexp escape, promise.try, error.isError, json raw json, and the temporal proposal at stage 4, developer.mozilla.org temporal and standard library pages, v8 and node release notes for temporal in chrome 144 and node 26, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows, 117 tests green through `npm run verify` in `javascript/samples`, 20 of them this chapter's.

9. the heap and the collector

Part one so far treated memory as if it were free. Values came from literals, arrays grew on demand, and no chapter asked where the bytes lived or who took them back. This chapter opens that box. The heap under node 26 is v8's, it is split into spaces with different jobs, and two collectors move objects between them while the program runs. Every number below was measured on this machine, 2026-09-13, node v26.3.0 with V8 14.6.202.34-node.20 on windows, and the workload is the particle kernel this book shares with the other language books: eight float64 fields per particle, a pure arithmetic step, sixteen steps per measurement, one stated prng.

9.1. the heap as v8 reports it

`node:v8` exposes the spaces directly. `getHeapSpaceStatistics` returns one record per segment of the heap, and the api promises neither an ordering nor that any named space exists, so the sample matches by name and the tests only floor the census. On this build the call reports 15 spaces, among them `read_only_space`, `new_space`, `old_space`, `code_space`, `trusted_space`, and `large_object_space` with its per-kind variants:

```
// the heap as v8 reports it: names, sizes, used. neither ordering
// nor availability is guaranteed by the api, so callers match by name
export function heapSpaces() {
  return v8.getHeapSpaceStatistics().map((s) => ({
    name: s.space_name,
    size: s.space_size,
    used: s.space_used_size,
  }));
}

// named lookups over the space list, the shape the tests assert on
export function spaceNamed(name) {
  return heapSpaces().find((s) => s.name === name);
}
```

listing 69: the spaces read from the live process, matched by name (source: javascript/samples/src/ch09-heap.mjs, lines 164-177)

An idle test process measured `new_space` at 0.5 MB and `old_space` at 3 MB used. The names carry the collector's division of labor. `new_space` is the nursery, two semispaces where every object is born and most die. `old_space` holds survivors and pre-tenured allocations and is the major collector's territory. `code_space` keeps jitted machine code, `read_only_space` holds objects frozen at snapshot time, and anything larger than v8's regular object limit goes to `large_object_space` instead, never moving.

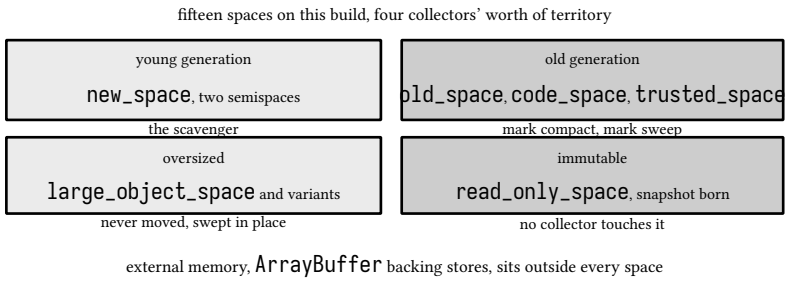


figure 46: the spaces and the collector that owns each

9.2. the scavenger: a semispace copy

Objects start in one half of the nursery, `from-space`. When it fills, the minor collector, the scavenger, copies every reachable young object into the other half, `to-space`, and the two swap names. Copying is the whole algorithm: nothing is marked, nothing is compacted, the dead simply never get copied. The walk is the classic interleaved

scan, evacuate a survivor, bepointer it, follow the next pointer, with a forwarding address left in the old copy so every other reference can be updated to the new location. Roots are the stack and globals plus the old-to-new references the write barrier recorded, so a scavenge never traces the old generation, and survivors of a second scavenge are promoted to old space instead of copied again. V8's write-up of the parallel scavenger, which gives helper threads and the main thread disjoint pointer sets with atomic forwarding pointers, measures 20 to 50 percent less total main thread young generation time than the sequential one.

A child run on this machine shows the shape in the trace. Each line is one collection, heap before and after, the pause, and the trigger:

```
24 ms: Scavenge 6.3 (7.4) → 6.0 (8.2) MB, pooled: 0.0 MB,
  10.62 / 0.00 ms (average mu = 1.000, current mu = 1.000)
  allocation failure;
25 ms: Scavenge 7.3 (9.4) → 7.3 (10.7) MB, ... 1.04 ms ...
```

fragment, illustrative, not executed

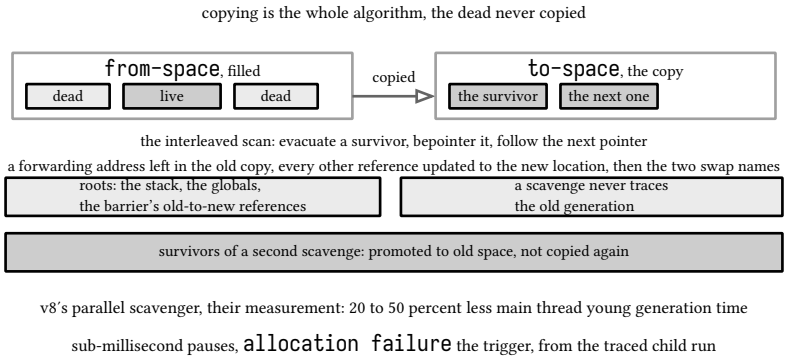


figure 47: the nursery as one copy, from space to to space

Sub-millisecond pauses, heap barely moving, `allocation failure` as the trigger: that is a nursery doing its job on short-lived objects. The flags section at the end of this chapter runs the same churn at three semi space sizes and counts the lines.

9.3. marking, sweeping, and orinoco

Old space needs a collector that can claim garbage without moving everything: mark and sweep. Marking is a graph walk from the roots in which every object is white until discovered, grey once queued, and black once its outgoing edges are all visited; when no grey objects remain, the whites are garbage. Since 2011 v8 has done that walk incrementally, interleaved with the program in paced chunks, which trades a write barrier on every field store for pauses of milliseconds instead of the several hundred a full stop could cost. The barrier is Dijkstra style, it greys a white value stored into a black object, which upholds the strong tri-color invariant: no black object ever points at a white one, so nothing live can be hidden from the walk. Since Chrome 64 and node 10 the marking itself is mostly concurrent, worker threads drain the worklist while javascript runs, and v8 measured 60 to 70 percent less main thread marking time from it.

Sweeping returns the dead pages to the allocator, and compaction evacuates fragmented pages. Orinoco is v8's project to make all of the old generation work parallel and concurrent: compaction parallelized at page level, remembered sets processed from per-page bitmaps instead of store buffers, and black allocation giving promoted objects dedicated pages that skip sweeping entirely. V8's own measurements there: compaction 75 percent faster, from about 7 ms to under 2 ms, and the gmail benchmark's worst compacting pause cut from 42 ms to 23 ms. The explicit `gc()` child run shows the major collector in the same trace format, with the pause and the mutation budget's `mu` gauge:

```
37 ms: Mark-Compact 16.5 (21.9) -> 16.0 (21.9) MB, pooled: 0.0 MB,
  4.86 / 0.00 ms (average mu = 0.855, current mu = 0.855)
  testing; GC in old space requested
```

fragment, illustrative, not executed

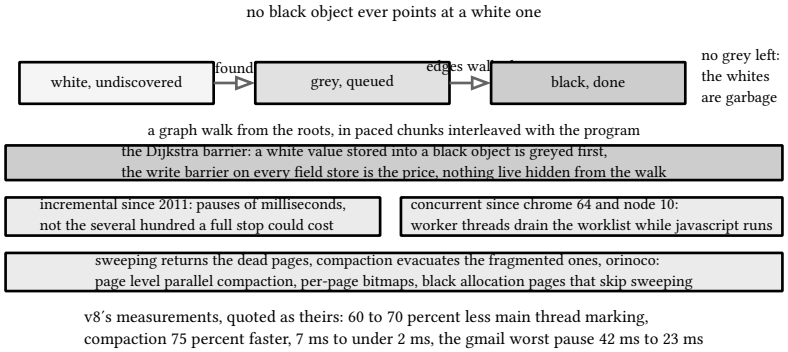


figure 48: the tri-color walk, and the barrier holding it honest

The pause is still single-digit milliseconds on a 16 MB heap, and the interleaving is visible in real runs as mark steps of well under a millisecond between program slices.

9.4. external memory and the large object space

`process.memoryUsage()` separates the javascript heap from the memory the process owns outside it: `heapUsed` counts the spaces above, `external` counts c++ allocations, and `arrayBuffers` counts the subset that is typed array backing stores. Measured on this machine, allocating a 256 MiB `ArrayBuffer` moved `arrayBuffers` by exactly 256 MB and `heapUsed` by 0 MB: the bytes never enter any v8 space, which is why the particle kernel below holds a million particles in 61 MB of `arrayBuffers` while `heapUsed` stays flat. v8 counts that memory toward the heap limit through `getHeapStatistics().external_memory` anyway, so an out-of-memory in a typed array program can arrive from the outside accounting.

Oversized objects get their own lane inside the heap. A plain array of ten million numbers measured `large_object_space` growing from 7.76 MB to 160.36 MB, the elements of one array sitting together on pages nothing else shares, and large object pages are never compacted because nothing else can fragment them.

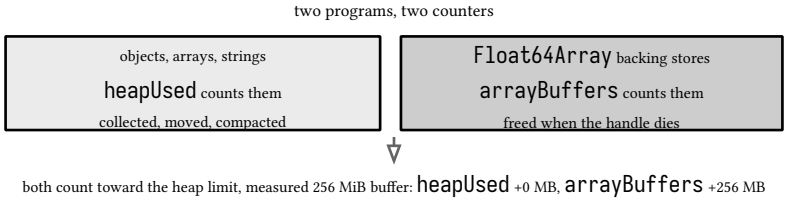


figure 49: where a byte of state lands

9.5. weak references under a real collector

Chapter 5 pinned the deterministic side of `WeakRef` and `FinalizationRegistry`, object keys only, no enumeration, collection itself a runtime decision. With a real collector in hand the interesting part is the job boundary: `deref` keeps the target alive for the rest of the job it runs in, so a program that dereferences while collecting can keep its own target alive. The first draft of this chapter's child did exactly that and measured `[object Object]` after two `gc()` calls, the target survived because the same job had touched it. The corrected child creates the target inside a function so its frame is gone, and observes from a later macrotask:

```
// ch09 child target: run under `node --expose-gc` so a real collector
// clears a WeakRef and fires a FinalizationRegistry callback. two
// details make it deterministic: the target is created inside a
// function so its frame is gone when the block ends, and deref, which
// keeps the target alive for the rest of its job, only runs in a
// later macrotask, after the collection
const registry = new FinalizationRegistry((held) => {
  console.log(`finalizer ran: ${held}`);
});
let ref = null;
function makeTarget() {
  const target = { payload: "ch09 target" };
  ref = new WeakRef(target);
  registry.register(target, "ch09 target");
}
makeTarget();
await new Promise((r) => setTimeout(r, 10));
globalThis.gc();
globalThis.gc();
await new Promise((r) => setTimeout(r, 10));
console.log(`after two gc calls and a macrotask: deref is ${ref.deref()}`);
await new Promise((r) => setTimeout(r, 50));
process.exit(0);
```

listing 70: a WeakRef cleared and a finalizer delivered, run under `--expose-gc` (source: javascript/samples/src/ch09-gc-child.mjs, lines 1-24)

Run under `node --expose-gc` on this machine the child prints `finalizer ran: ch09 target` and then `after two gc calls and a macrotask: deref is undefined`, deterministically across repeated runs, and the suite spawns that exact child and asserts both lines. The finalizer callback runs on the collector's schedule, never inline with the collection, which is why the registry is a cleanup hook and not a destructor: by the time it fires the object is already gone and only the held value remains.

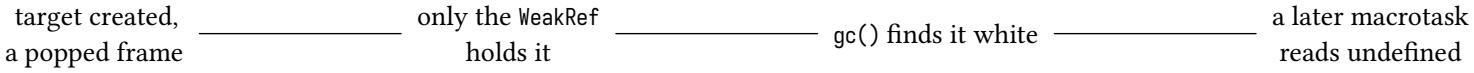


figure 50: one target, one job boundary

9.6. the particle kernel: numbers against boxes

The corpus-wide kernel gives every language book the same workload: N particles times eight float64 fields, `x`, `y`, `z`, `vx`, `vy`, `vz`, `m`, `q`, one pure arithmetic step per field with an analytic mass-weighted force, K of 16, and a stated seeded prng so runs are reproducible. The javascript question is only the representation: eight `Float64Array` columns, or one object literal per particle.

```
// the corpus-wide particle contract: 8 float64 fields per particle,
// positions, velocities, mass, charge, a pure arithmetic step with an
// analytic force, K steps per measurement, one seeded prng stated in
// the file: splitmix32, seed 0x12345678
export function splitmix32(seed) {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x9e3779b9) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 16), 0x21f0aaad);
    t = Math.imul(t ^ (t >>> 15), 0x735a2d97);
    return ((t ^ (t >>> 15)) >>> 0) / 4294967296;
  };
}
```

listing 71: splitmix32, the stated prng, seed 0x12345678 (source: javascript/samples/src/ch09-heap.mjs, lines 7-20)

```
// struct of arrays: one Float64Array per field, the whole state in
// eight contiguous backing stores
export function makeSoA(n, seed = 0x12345678) {
  const rnd = splitmix32(seed);
  const f = {};
  for (const name of ["x", "y", "z", "vx", "vy", "vz", "m", "q"]) {
    f[name] = new Float64Array(n);
  }
  for (let i = 0; i < n; i++) {
    f.x[i] = rnd() * 2 - 1;
    f.y[i] = rnd() * 2 - 1;
    f.z[i] = rnd() * 2 - 1;
    f.vx[i] = rnd() * 0.2 - 0.1;
    f.vy[i] = rnd() * 0.2 - 0.1;
    f.vz[i] = rnd() * 0.2 - 0.1;
    f.m[i] = 0.5 + rnd();
    f.q[i] = rnd() * 2 - 1;
  }
  return f;
}
```

listing 72: the SoA state, one contiguous column per field (source: javascript/samples/src/ch09-heap.mjs, lines 25-44)

```
// the step is pure arithmetic, mass weighted harmonic force, no
// branches, so both representations execute the same operations in
// the same order and land on identical checksums
export function stepSoA(f, n, k = K, dt = DT) {
  const half = 0.5 * dt * dt;
  for (let s = 0; s < k; s++) {
    for (let i = 0; i < n; i++) {
      const im = 1 / f.m[i];
      const ax = -f.x[i] * im;
      const ay = -f.y[i] * im;
      const az = -f.z[i] * im;
      f.x[i] += f.vx[i] * dt + ax * half;
      f.y[i] += f.vy[i] * dt + ay * half;
      f.z[i] += f.vz[i] * dt + az * half;
      f.vx[i] += ax * dt;
      f.vy[i] += ay * dt;
      f.vz[i] += az * dt;
    }
  }
}
```

listing 73: the step, pure arithmetic, no branches, identical operations in both representations (source: javascript/samples/src/ch09-heap.mjs, lines 46-65)

At one million particles, measured on this machine through the opt-in `samples/bench/particle.mjs`: the array-of-structs form initializes in 178.41 ms and costs 216.75 MB of `heapUsed`, one box per particle on the collector's heap. The struct-of-arrays form initializes in 21.1 ms and costs 61.04 MB of `arrayBuffers`, eight contiguous backing stores, with `heapUsed` moving 0.28 MB. Stepping 16 steps: 198.46 ms boxed, 12.4 ns per particle per step, against 75.48 ms flat, 4.72 ns, a 2.63x difference, and 2.63x to 2.82x across three runs of the same binary. The checksum pass proves the two representations compute the same eight sums, field by field in particle order, because the step executes the same operations in the same order either way.

The scale ladder is honest about its ceiling. One billion particles is 64 GB of columns, arithmetic only, not measured on this machine. One hundred million was measured once, alone, with nothing else running: 6103.65 MB of `arrayBuffers` and an rss of 6159.15 MB, init 2.31 s, 16 steps in 7.42 s, 4.63 ns per particle per step, checksum pass 379.14 ms, and the memory returned when the process exited.

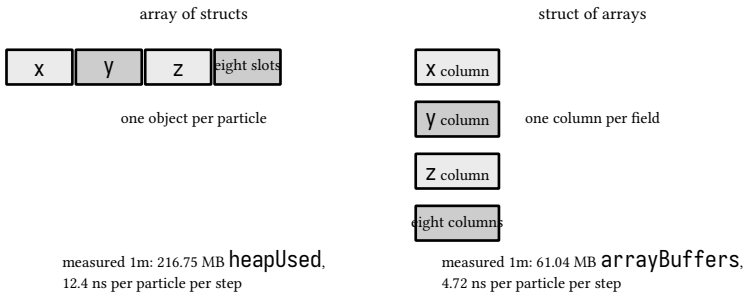


figure 51: the same state, two shapes of memory

9.7. the flags, measured on a child run

Three v8 flags matter for reading this chapter's collectors, and node passes them through. `--trace-gc` prints one line per collection. `--expose-gc` installs a `globalThis.gc()` that runs the major collector on demand, the child above depends on it. And `--max-semi-space-size` caps the young generation semispace in MiB. The child run is a churn workload, 400 rounds of 20,000 short-lived objects, and the count is the scavenge lines in the trace:

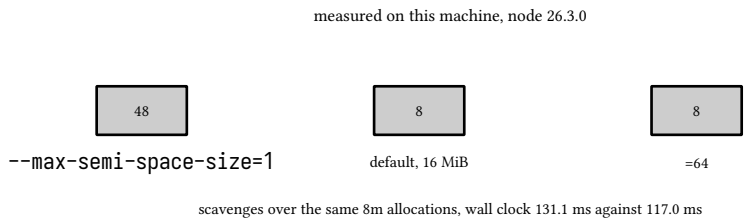


figure 52: scavenge count against semi space size, one churn workload

A six-times smaller nursery scavenged six times more often, 48 collections against 8, for the same work. The wall clock difference on this workload is noise, 131.1 ms against 117.0 ms, because each scavenge costs a fraction of a millisecond, but the shape is the lever: raise the semispace and short-lived garbage dies in larger, rarer batches, lower it and the pauses shrink while their count climbs. The opt-in benches behind this chapter are `samples/bench/particle.mjs` for the kernel at any N and `samples/bench/rows.mjs` for the sqlite row kernel of chapter 10, and neither is ever run by `make verify`.

NOTE: what this chapter measured

Every number above came off this machine on 2026-09-13: the space census through `node:v8`, the trace lines from real child runs spawned by the chapter's own scripts, the footprint deltas from `process.memoryUsage`, and the kernel timings from the committed bench scripts. The scavenger, marking, and Orinoco internals are v8's published account, cited below, with v8's own measurements quoted as theirs: 20 to 50 percent less young generation main thread time from the parallel scavenger, 60 to 70 percent less main thread marking time from concurrent marking, compaction 75 percent faster under Orinoco. Nothing in between was inferred.

sources: nodejs.org v8 module and cli options pages, nodejs.org process memoryUsage, v8.dev trash talk on the parallel scavenger, v8.dev orinoco on parallel compaction and black allocation, v8.dev concurrent marking on incremental marking and the tricolor walk, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows: 125 tests green through `npm run verify` in `javascript/samples`, 8 of them this chapter's, three consecutive clean runs, plus the measured child and bench runs quoted above.

10. profiling and the event-loop tail

Chapter 9 asked where the bytes live. This chapter asks where the time goes. Node ships two sampling profilers, a cpu profiler and an allocation profiler, both writable from a flag and both readable as plain json, and a histogram that measures the event loop's own lateness. The chapter runs all of them on this machine, 2026-09-13, node v26.3.0, parses what they wrote, and closes on the corpus row kernel, ten million sqlite rows inserted, aggregated in SQL, and walked by hand, with the commit tail read as percentiles. The distinction that organizes everything: a profiler explains where cpu or allocation went, a latency histogram explains when the loop was late, and they answer different questions.

10.1. the profile file, parsed

`node --cpu-prof` starts V8's sampling profiler at boot and writes a `.cpuprofile` before exit. The file is json: a tree of `nodes`, each carrying a `callFrame` with `functionName`, `url`, and line, plus a `hitCount`, and two parallel arrays, `samples`, the node id sampled at each tick, and `timeDeltas`, the microseconds between ticks. The default sampling interval is 1000 us. Parsing it is ordinary file work, so the suite does exactly that, against a child whose workload lives in a named function, because an anonymous top-level loop carries no frame name worth asserting on:

```
// ch10 child target: a named busy function the samplers can find.
// anonymous top level loops carry no findable frame name, so the
// workload lives in an exported, named function
export function grind(n) {
  let acc = 0;
  for (let i = 1; i ≤ n; i++) acc += Math.sqrt(i) % 1;
  return acc;
}

const n = Number(process.argv[2] ?? 5_000_000);
const scratch = [];
for (let r = 0; r < 30; r++) {
  const row = new Array(20_000);
  for (let i = 0; i < row.length; i++) row[i] = { i, v: grind(40) + i * 1.5 };
  scratch.push(row[0]);
}
console.log(`grind(${n}) = ${grind(n).toFixed(6)}, kept ${scratch.length}`);
```

listing 74: the child target, a named function the samplers can find (source: javascript/samples/src/ch10-busy.mjs, lines 1-18)

```
// spawn a child under --cpu-prof, parse the .cpuprofile it wrote, and
// return the facts the tests and the chapter quote: node count, hit
// counts by frame name, and the summed time deltas
export function captureCpuProfile(entry, args = []) {
  const dir = mkdtempSync(join(tmpdir(), "cpuprof-"));
  const run = spawnSync(process.execPath, [
    "--cpu-prof", `--cpu-prof-dir=${dir}`, "--cpu-prof-name=p.cpuprofile",
    entry, ...args.map(String),
  ], { encoding: "utf8" });
  if (run.status !== 0) {
    rmSync(dir, { recursive: true, force: true });
    throw new Error(run.stderr || `child exited ${run.status}`);
  }
  const profile = JSON.parse(readFileSync(join(dir, "p.cpuprofile"), "utf8"));
  const hitsByName = new Map();
  let totalHits = 0;
  for (const node of profile.nodes) {
    const name = node.callFrame.functionName;
    const hits = node.hitCount ?? 0;
    totalHits += hits;
    hitsByName.set(name, (hitsByName.get(name) ?? 0) + hits);
  }
  const micros = profile.timeDeltas.reduce((a, d) => a + d, 0);
  rmSync(dir, { recursive: true, force: true });
  return { nodes: profile.nodes.length, hitsByName, totalHits, micros, samples: profile.samples.length };
}
```

listing 75: spawn the child, read the file, fold hits by frame name (source: javascript/samples/src/ch10-profiling.mjs, lines 27-53)

One measured run of that capture, on this machine: 41 nodes, one `grind` frame, 22 of the run's 27 total hits on that frame, the summed `timeDeltas` 52.27 ms. That is the whole attribution story in one number, the profiler did not measure the program, it sampled it, and 22 samples out of 27 landed inside one function because that function held the thread.

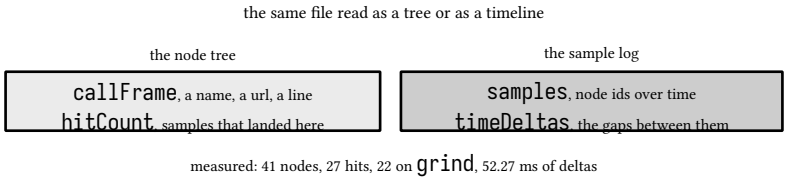


figure 53: one profile, two views

10.2. the heap profile, sampled

`--heap-prof` writes a `.heapprof` file, and it is a different instrument: a sampling allocation profiler. It records one tree of call frames where every node carries a `selfSize`, the bytes allocated by that frame that were still live when the profile was taken, and a `samples` array of `{size, nodeId, ordinal}` records. The default sampling interval is 512 KiB, one recorded allocation per half megabyte allocated, and it can be lowered for finer attribution. Measured on this machine at a 16 KiB interval against the same busy child: 42 samples, 514224 total sampled bytes, and the top frames are the runtime's own bootstrap, `createContext` at 230400 bytes and `compileForInternalLoader` at 115512, with `(IDLE)` at 69104. The honest reading: a sampling heap profile attributes by allocation site, and on a short-lived child the startup dominates the retained set, so the workload's own allocations have to outlive or outweigh bootstrap to rise to the top. For a leak hunt the comparison of two captures, same code path run twice, is the signal, not one capture's absolute numbers.

```
// spawn a child under --heap-prof, parse the .heapprofile: a sampling
// allocation profile, one tree of call frames with selfSize per node
export function captureHeapProfile(entry, args = [], interval = 16384) {
  const dir = mkdtempSync(join(tmpdir(), "heapprof-"));
  const run = spawnSync(process.execPath, [
    "--heap-prof", `--heap-prof-dir=${dir}`, "--heap-prof-name=p.heapprof",
    `--heap-prof-interval=${interval}`,
    entry, ...args.map(String),
  ], { encoding: "utf8" });
  if (run.status !== 0) {
    rmSync(dir, { recursive: true, force: true });
    throw new Error(run.stderr || `child exited ${run.status}`);
  }
  const profile = JSON.parse(readFileSync(join(dir, "p.heapprof"), "utf8"));
  let totalSelf = 0;
  const walk = (node) => {
    totalSelf += node.selfSize ?? 0;
    (node.children ?? []).forEach(walk);
  };
  walk(profile.head);
  rmSync(dir, { recursive: true, force: true });
  return { samples: profile.samples.length, totalSelf, head: profile.head };
}
```

listing 76: the heap capture, parsed, self sizes summed over the frame tree (source: javascript/samples/src/ch10-profiling.mjs, lines 54-76)

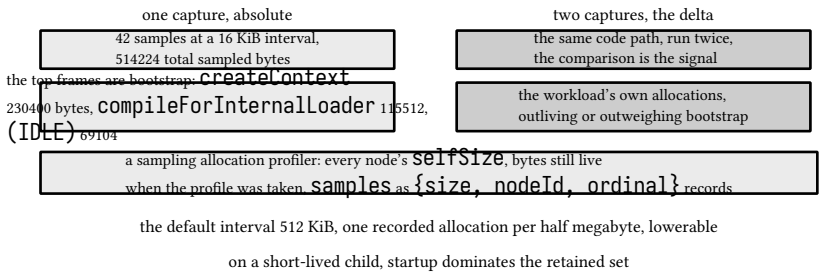


figure 54: one capture against two, the delta as the signal

10.3. the inspector, from inside the process

The flags write files at exit. The inspector writes the same profiles from inside a running process: `node:inspector` exposes a `Session` connected to the V8 back-end on the same thread, `post` sends a Chrome DevTools Protocol method, and the `Profiler` domain does enable, set sampling interval, start, stop. The session is callback shaped, so the sample wraps `post` in a promise and awaits the stops:

```
// the inspector session: the same profiler the devtools protocol
// drives, from inside the process, sampling at a stated interval.
// the frame name to count is stated by the caller because a wrapped
// closure samples under its own anonymous name
export async function profileWithInspector(work, frameName = work.name, samplingIntervalUs = 100) {
  const session = new inspector.Session();
  session.connect();
  const post = (method, params) =>
    new Promise((resolve, reject) =>
      session.post(method, params, (err, res) => (err ? reject(err) : resolve(res))));
  await post("Profiler.enable");
  await post("Profiler.setSamplingInterval", { interval: samplingIntervalUs });
  await post("Profiler.start");
  const value = work();
  const { profile } = await post("Profiler.stop");
  session.disconnect();
  let namedHits = 0;
  for (const node of profile.nodes) {
    if (node.callFrame.functionName === frameName) namedHits += node.hitCount ?? 0;
  }
  return {
    value,
    nodes: profile.nodes.length,
    samples: profile.samples.length,
    timeDeltas: profile.timeDeltas.length,
    namedHits,
  };
}
```

listing 77: the in-process session, the frame name stated because a closure samples anonymously (source: javascript/samples/src/ch10-profiling.mjs, lines 78-105)

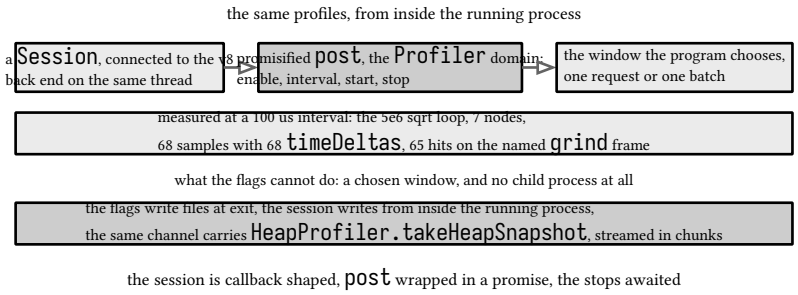


figure 55: the session, a pipeline from connect to the awaited stop

Measured on this machine at a 100 us interval: the 5e6 sqrt loop produced 7 nodes and 68 samples with 68 `timeDeltas`, 65 of the hits on the named `grind` frame. The in-process session costs a little setup and buys two things the flags cannot do: a profile of a window the program chooses, started and stopped around one request or one batch, and no child process at all. The same channel carries `HeapProfiler.takeHeapSnapshot`, streamed out in chunks, for the full snapshot graphs the devtools draw.

10.4. the loop, and where delay is born

Between timers and the next turn sits the loop itself, and node runs it through libuv in phases: timers, pending callbacks, idle and prepare, poll, check, and close callbacks, one turn through all of them per iteration. Since libuv 1.45, the version node 20 raised the floor to, timers run after the poll phase of the same iteration. The poll phase is where the loop waits for io, and it is also where delay is born: a timer is a threshold, not an appointment, the guide's own example is a 100 ms timer made to fire at 105 ms because a file callback held the thread for 10 ms

past the poll wait. When the poll queue is empty and immediates are scheduled the loop leaves poll for the check phase, otherwise it blocks for more io.

`monitorEventLoopDelay` from `node:perf_hooks` measures exactly that lateness. A timer of the stated resolution fires every tick, and each sample is how late it ran, in nanoseconds, so the histogram is the loop's own tail. The sample alternates roughly one millisecond of busy work with a `setImmediate` yield:

```
// monitorEventLoopDelay: a histogram of how late the loop's own timer
// fired, one sample per resolution tick, nanoseconds throughout
export async function measureEventLoopDelay(windowMs = 400, burst = 80_000) {
  const histogram = monitorEventLoopDelay({ resolution: 1 });
  histogram.enable();
  const t0 = performance.now();
  for (let round = 0; performance.now() - t0 < windowMs; round++) {
    grind(burst);
    await new Promise((r) => setImmediate(r));
  }
  const elapsed = performance.now() - t0;
  histogram.disable();
  return {
    elapsedMs: elapsed,
    count: histogram.count,
    p50ns: histogram.percentile(50),
    p99ns: histogram.percentile(99),
    meanNs: histogram.mean,
    maxNs: histogram.max,
  };
}
```

listing 78: the histogram run over a busy and yield pattern (source: javascript/samples/src/ch10-profiling.mjs, lines 107-127)

Measured over a 0.4 s window with 1 ms resolution on this machine: 400 samples, p50 1.07 ms, p99 1.26 ms, mean 1.00 ms, max 1.46 ms. The mean matches the busy budget, the p50 sits just above it, and the p99 is where a gc pause or an os scheduling hiccup landed. On an idle loop the same histogram reads near zero, the loop is never late when nothing holds it.

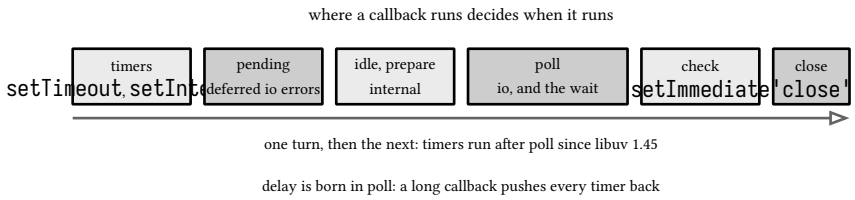


figure 56: the loop's phases, one turn

10.5. percentiles for a handler loop

Averages hide the tail, so this book reads latency as percentiles, nearest rank: sort the samples, take the value at the ceiling of the quantile times the count. The sample runs an async handler loop, one thousand chunks, each chunk an awaited `setImmediate` turn plus a bounded piece of work, and records the per-chunk latency:

```
// nearest-rank percentile over an already sorted array, the reading
// this book quotes for every latency distribution
export function percentile(sorted, q) {
  const rank = Math.ceil((q / 100) * sorted.length);
  return sorted[Math.min(sorted.length - 1, Math.max(1, rank) - 1)];
}
```

listing 79: nearest rank, the one percentile rule this book quotes (source: javascript/samples/src/ch10-profiling.mjs, lines 13-18)

```
// the async handler loop: per-chunk latency over a stated chunk count,
// the distribution the row kernel's commit tail is read through
export async function chunkLatencies(count = 1000, workUnits = 500) {
  const latencies = [];
  for (let c = 0; c < count; c++) {
    const start = performance.now();
    await new Promise((r) => setImmediate(r));
    const records = [];
    for (let i = 0; i < workUnits; i++) records.push({ id: i, v: Math.sqrt(i) % 1 });
    latencies.push(performance.now() - start);
  }
  latencies.sort((a, b) => a - b);
  return {
    count: latencies.length,
    p50: percentile(latencies, 50),
    p95: percentile(latencies, 95),
    p99: percentile(latencies, 99),
    p999: percentile(latencies, 99.9),
    max: latencies[latencies.length - 1],
  };
}
```

listing 80: the handler loop, one latency per chunk over a thousand chunks (source: javascript/samples/src/ch10-profiling.mjs, lines 129-149)

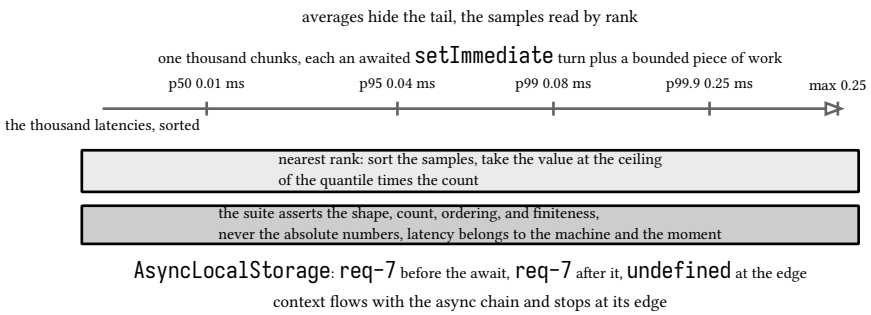


figure 57: the sorted thousand, read by rank

Measured on this machine: p50 0.01 ms, p95 0.04 ms, p99 0.08 ms, p99.9 0.25 ms, max 0.25 ms. The suite asserts the shape, count, ordering, and finiteness, never the absolute numbers, because latency is a property of the machine and the moment. The same turn structure carries request attribution: `AsyncLocalStorage` from `node:async_hooks` gives every callback a store without threading a parameter, and the store survives `await`:

```
// request tracing: one store per request, readable across awaits and
// inside every callback the request spawned, absent outside
export async function requestTrace() {
  const als = new AsyncLocalStorage();
  const trace = [];
  await als.run({ requestId: "req-7" }, async () => {
    trace.push(als.getStore().requestId);
    await new Promise((r) => setTimeout(r, 1));
    trace.push(als.getStore().requestId);
  });
  trace.push(String(als.getStore()));
  return trace;
}
```

listing 81: one request id, readable across awaits, absent at the boundary (source: javascript/samples/src/ch10-profiling.mjs, lines 151-164)

The measured trace is `req-7` before the await, `req-7` after it, and `undefined` once the run's scope ends, which is the whole contract: context flows with the async chain and stops at its edge. The lower-level `async_hooks` module underneath it, with its init, before, after, and destroy callbacks, can build the same thing by hand, at the cost of a callback per promise job, and the chapter uses the storage abstraction because the callback form is the instrument you reach for only when the storage is not enough.

10.6. the row kernel at ten million rows

The corpus-wide row kernel gives the database side of the measurement: one table, `station TEXT NOT NULL` plus eight `REAL` columns, 412 stations `st000` through `st411`, seeded values, 10k rows per transaction through one prepared statement, then the same aggregation twice, SQL `GROUP BY` against a manual cursor walk. The engine is node's own `node:sqlite`, measured at exactly 3.53.4 on node 26.10.0, asserted as a string by the suite the way the capstone pins it. The 3.53.1 reading the book first recorded on node 26.3.0, the one delta against the corpus's 3.53.4 pins, closed when the runtime's bundled engine caught the pin.

```
export function insertRows(db, n, seed = 0x5eed0000) {
  const names = Array.from({ length: STATIONS }, (_, i) => `st${String(i).padStart(3, "0")}`);
  const rnd = splitmix32(seed);
  const vals = new Float64Array(8);
  const ins = db.prepare(
    "INSERT INTO readings (station, v0, v1, v2, v3, v4, v5, v6, v7) VALUES (?,?,?,?,?,?,?,?,?)");
  const chunkMs = [];
  const start = performance.now();
  db.exec("BEGIN");
  for (let i = 0; i < n; i++) {
    const s = i % STATIONS;
    for (let v = 0; v < 8; v++) vals[v] = rnd() * 2 - 1;
    ins.run(names[s], vals[0], vals[1], vals[2], vals[3], vals[4], vals[5], vals[6], vals[7]);
    if ((i + 1) % CHUNK === 0) {
      const commitStart = performance.now();
      db.exec("COMMIT");
      chunkMs.push(performance.now() - commitStart);
      db.exec("BEGIN");
    }
  }
  db.exec("COMMIT");
  chunkMs.sort((a, b) => a - b);
  return {
    elapsedMs: performance.now() - start,
    rowsPerSecond: n / ((performance.now() - start) / 1000),
    chunks: chunkMs.length,
    p50: percentile(chunkMs, 50),
    p95: percentile(chunkMs, 95),
    p99: percentile(chunkMs, 99),
    p999: percentile(chunkMs, 99.9),
    max: chunkMs[chunkMs.length - 1] ?? 0,
  };
}
```

listing 82: the seeded insert, one prepared statement, the commit timed per 10k chunk (source: javascript/samples/src/ch10-rows.mjs, lines 38-70)

```
export function aggregateWalk(db) {
  const start = performance.now();
  const acc = new Map();
  for (const row of db.prepare(
    "SELECT station, v0, v1, v2, v3, v4, v5, v6, v7 FROM readings").iterate()) {
    let a = acc.get(row.station);
    if (a === undefined) {
      a = {
        n: 0,
        lo: new Float64Array(8).fill(Infinity),
        hi: new Float64Array(8).fill(-Infinity),
        sum: new Float64Array(8),
      };
      acc.set(row.station, a);
    }
    a.n++;
    for (let v = 0; v < 8; v++) {
      const x = row[`v${v}`];
      if (x < a.lo[v]) a.lo[v] = x;
      if (x > a.hi[v]) a.hi[v] = x;
      a.sum[v] += x;
    }
  }
  return { acc, elapsedMs: performance.now() - start };
}
```

listing 83: the manual walk, per station accumulators behind one cursor (source: javascript/samples/src/ch10-rows.mjs, lines 85-109)

Measured on this machine, 2026-09-13, through the opt-in `samples/bench/rows.mjs`: the insert of ten million rows took 18.85 s, 530.42k rows per second, in one thousand transactions, and the db file weighed 829.77 MB. The commit tail over those thousand chunks: p50 11.595 ms, p95 13.628 ms, p99 17.914 ms, p99.9 111.236 ms, max 111.236 ms, the tail being where sqlite flushed a full transaction's pages at once. The aggregation passes: SQL `GROUP BY` in 11.75 s, 851k rows per second, the manual cursor walk in 17.78 s, 562.3k rows per second, SQL ahead by a third on this workload because the engine aggregates inside its own scan while the walk pays for a javascript object per row. Both passes agree exactly on min and max and to a float epsilon on the average, and the suite checks that agreement at 1k rows in memory, under the one second kit budget.

The one-off above this scale was run once, alone, and deleted after: one hundred million rows, a 8330.77 MB db file, insert 197.05 s at 507.5k rows per second, commit tail p50 11.831 ms, p99 21.665 ms, p99.9 107.442 ms over ten thousand chunks, the manual walk 197.83 s at 505.47k rows per second, and the SQL pass 334.56 s at 298.9k rows per second, slower than at ten million by more than the row count explains: an 8.3 GB table does not fit this machine's free memory, so the scan rate is a property of what the os cached, and an earlier run of the same kernel with a warmer cache measured the same pass in 134.82 s. One billion rows is arithmetic only, on the order of 85 GB at this row shape, not measured on this machine.

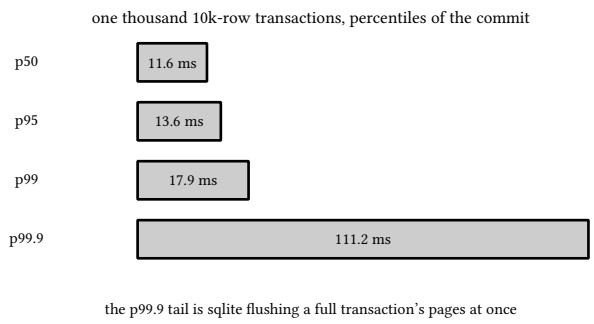


figure 58: the commit tail of the ten million row insert

10.6.1. the same kernel on duckdb

The corpus also ran this kernel on duckdb, from c and from go, and those two chapters hold the engine-side numbers. The c twin, same 412 stations and rows bit-identical to its sqlite lane, measured the appender at 1.05 to 1.12 million rows a second against sqlite's batched 464,047, the group by at 79.6 to 83.3 million against 555,402, 143x to 150x, and the same rows stored in 449.8 to 450.8 MB against this chapter's 829.77, while the go lane appended 774,437 rows a second with its group by at 124,918,491 over the engine's own thread pool, all 2026-09-20 on duckdb 1.5.5. `node:sqlite` stays right for this book's kernel anyway: the tail read here is a commit, the p99 of one 10k-row transaction, and the capstone's snapshots are one transaction each, lanes the appender does not offer because it opens no transaction and hands the batch boundary to the engine. Which engine a workload wants is measured in book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine) and chosen in book 13, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine).

10.7. which tool when

Each instrument answers one question, and the cheapest one that answers it wins. For "which function is hot", `--cpu-prof` on a child, no code changes, sampled attribution. For "which call path allocates", `--heap-prof`, read as a delta between two captures. For "this one batch, right now", the inspector session, started and stopped around the window. For "is the loop late", the histogram, and for "which request did this", the storage. For the store itself, timings and the aggregation comparison above. The row kernel's own numbers land in each tool's terms: the insert rate is throughput, the commit tail is latency, and the two aggregation passes are an algorithm choice measured, not argued.

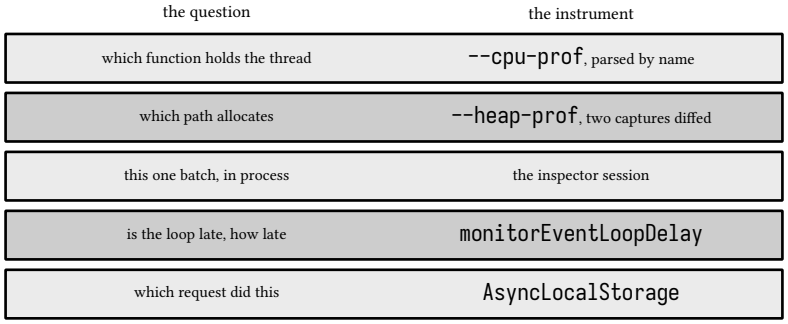


figure 59: one symptom, one instrument

sources: nodejs.org inspector, perf_hooks, async_hooks, and cli profiling options pages, the nodejs.org event loop timers and nexttick guide, and the nodejs.org sqlite page, accessed 2026-09-13. Behavior verified live with node v26.3.0 on windows: 133 tests green through `npm run verify` in `javascript/samples`, 8 of them this chapter's, three consecutive clean runs, and every quoted number above is from a child, session, histogram, or bench run executed on this machine, the opt-in benches under `samples/bench` never run by `make verify`.

10.8. across the six books

The same two kernels ran in all six language books under one contract, N particles of 8 float64 fields stepped 16 times and 10 million readings rows of one station key plus 8 real columns, every number below quoted from the chapter that measured it on this machine, 2026-09-13. The first two columns are the particle step at 1m in each language's row-of-objects repr and its flat column repr, and they disagree on purpose: the flat repr wins 1.4x in c, 2.6x in javascript, and 21x in python, the object repr wins 2.3x in c# and 3.8x in go because the step touches all eight fields of every particle and one cache line per particle beats eight streams in flight, and lua's ffi cdata wins 11.5x over boxed tables. The last four columns are the shared sqlite contract at 10m rows, insert through 10k-row batched transactions, the sql group by, the manual cursor walk, and the p99 of one 10k-row commit. Lua's p99 cell sits empty on purpose: its os.clock ticks at 1 ms and the chapter refuses to quote a percentile it cannot read, the boundary every book states in its own clock.

book	objects ns	flat ns	insert/s	group/s	walk/s	p99 ms
c, clang -O2	6.15 to 6.75	4.53 to 4.91	468,803	584,696	2,242,885	28.3
c#, .net 11	5.7	13.1	339,091	723,248	501,820	48.5
go 1.27	8.2	30.9	267,942	324,614	610,720	67.957
javascript, node 26	12.4	4.72	530.42k	851k	562.3k	17.914
python 3.14	345.7	16.18	327,703	536,378	307,199	39.60
lua 5.5 + luajit	114.1	9.9	504,765	573,915	580,353	

Table 1: the two kernels across the six language books: particle step ns at 1m in each book's object and flat repr, then the 10m row kernel with insert, group by, and walk in rows per second and p99 in milliseconds per 10k-row commit, every cell quoted from the chapter that measured it

The ladder above the table stays in its own book: the 100m particle one-off measured 5.768 ns flat and 7.229 ns object in c, 21.815 ns flat in c# (memory bound, soa only), 8.898 ns flat in go, 4.63 ns flat in javascript, 16.88 ns flat in python, and 8.2 ns ffi in lua, with the boxed 100m priced at 13.9 GB and refused. The 100m row one-off held its 10m insert rate within 5 percent in c, javascript, and lua, gained 4 percent in python, and paid 8 percent in go and 21 percent in c#. One billion is arithmetic in every book, 64 GB of particle state and roughly 81 to 87 GB of rows, not measured on this machine.

11. typescript 7: the type layer

Part one ran javascript with no build step: `.mjs` files, `node --test`, and nothing between the source and the runtime. Part two adds exactly one tool, `tsc`, and the tool is compile time only. It reads annotated javascript, checks it against promises the annotations make, then erases every one of those annotations and emits plain javascript that node 26 runs exactly as part one taught. The split is the language's own shape: typescript is a checker whose types vanish, so every rule in these chapters is a claim about code whose runtime behavior chapters 1 through 10 already own, and the emit side of that contract, declarations and namespaces and what survives compilation, is chapter 19′s subject. This chapter installs the compiler, configures it, and proves the erasure on the record.

11.1. a checker, not a runtime

The founding asymmetry: annotations are checked, then deleted. A parameter typed `string` compiles, and a caller who smuggles `null` past the checker with a cast still meets the plain runtime the annotation promised to describe. No check is injected, because the type never survived compilation:

```
// types vanish at emit. the test smuggles null past the compiler with
// a cast and the runtime TypeError proves no check was injected.
export function Shout(s: string): string {
  return s.toUpperCase() + "!";
}
```

listing 84: greet is ordinary typed code, shout is the erasure probe (source: javascript/tssamples/ch11/tstoolchain.ts, lines 44-48)

The test passes `null as unknown as string`, the emitted javascript throws `Cannot read properties of null` at the `toUpperCase` call, and the second observation is made against the emitted file itself: the module reads its own compiled output from `dist` and reports whether the annotation text is still present. It is not. The needles are assembled from halves so the probe cannot vouch for its own source:

```
// the erasure observation, checked rather than asserted in prose:
// the emitted javascript beside this file carries none of the
// annotations above, because none of them survived compilation. the
// needles are assembled from halves so this probe's own emitted
// source cannot contain them and vouch for itself.
export function EmittedErasure(): { emitted: boolean; annotated: boolean } {
  const emitted = readFileSync(join(import.meta.dirname, "tstoolchain.js"), "utf8");
  return {
    emitted: emitted.includes("function Shout" + "(s)"),
    annotated: emitted.includes("function Shout" + "(s: string)"),
  };
}
```

listing 85: the erasure observed in the emitted file rather than asserted in prose (source: javascript/tssamples/ch11/tstoolchain.ts, lines 50-61)

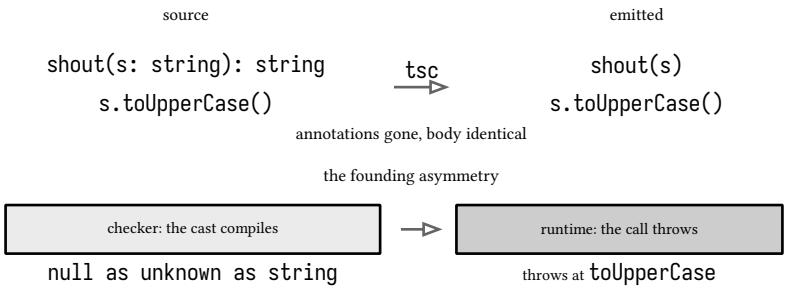


figure 60: the founding asymmetry, annotations erased at emit while the null cast reaches the runtime

That asymmetry, checked by the machine rather than argued in prose, is the mental model for everything in part two. When a type must be enforced in running code, the tool is a runtime check plus the `unknown` type of chapter 13, never a fancier annotation.

11.2. the native 7 compiler

TypeScript 7, released july 2026, is the native port: the compiler rewritten in Go, compiled ahead of time, reporting 8 to 12 times faster full builds and a fraction of the memory. For a command line user almost nothing moved. The compiler still installs as the `typescript` package and still answers to `tsc`, nightlies still live on the `next` tag, and `tsconfig.json` is still the configuration surface. What moved is everything underneath: the old `@typescript/native-preview` package is retired, and the js api of 5.x and 6.x is gone from the stable entry point, whose root export carries nothing beyond the version string. A compiler api does ship in 7.0.2, but as `unstable` subpath exports, sync, async, fs, proto, and ast entry points under `typescript/unstable`. This book builds on none of that unstable surface and waits for the stable api promised for 7.1. Projects that need the old api run 6.0 side by side through the `@typescript/typescript6` package and its `tsc6` binary.

The project pins `typescript ^7.0.2` and the measurement is part of the contract: `npx tsc --version` in `tssamples` prints `Version 7.0.2`, probed 2026-09-13, and the sample reads the same string from the installed package manifest so the suite fails if the compiler under it changes.

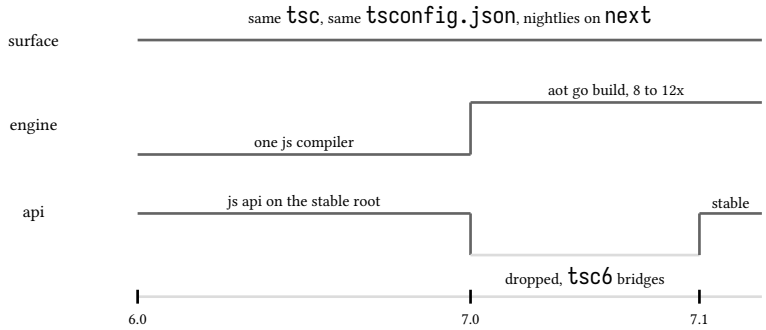


figure 61: the 7.0 port as a release timeline, three rails against the version axis

11.3. the honest es2026 story

Part one closed on an edition gap: es2026 was approved june 2026, node 26 ships most of it, and `Math.sumPrecise` is still absent. The type layer has its own gap, measured the same way, and it runs the other direction. `tsc` 7.0.2 knows no es2026 at all: `--target es2026` answers `TS6046` and lists every target it does accept, ending at `es2025` then `esnext`, and `--lib es2026` answers the same error over the library list. The default target is `es2025`, the newest edition the compiler knows, so this book's node runtime is ahead of its checker's edition vocabulary, not behind it.

The bridge is the lib fragment. A `lib` entry names a library description file, and the `esnext.*` fragments carry es2026 era api declarations ahead of an edition lib: `esnext.temporal` for Temporal, `esnext.disposable` for `using` and `Disposable`, `esnext.typedarrays` for the `Uint8Array` base64 and hex codecs, and `esnext.collection` for `Map.getOrInsert`, each fragment listed by the same `TS6046` message. The worked example is Temporal, and it is the honest case because the runtime ships it while the node type package does not: `@types/node` 26 declares no Temporal globals, so without the fragment the file below fails with `TS2503`, cannot find namespace `Temporal`, measured live, and with the fragment it typechecks and runs:

```ts
// temporal is fully shipped on this runtime, chapter 8 of part one
// uses it unflagged, but @types/node 26 declares no temporal globals,
// so the esnext.temporal lib fragment is what lets this file
// typecheck at all. dropping the fragment is measured TS2503, cannot
// find namespace 'Temporal', on the very line below.
export function WeekdayOf(iso: string): string {
  const days = ["mon", "tue", "wed", "thu", "fri", "sat", "sun"];
  const date = Temporal.PlainDate.from(iso);
  return days[date.dayOfWeek - 1];
}

// until is signed: forward dates count up, backward dates down.
export function DaysBetween(from: string, to: string): number {
  return Temporal.PlainDate.from(from).until(Temporal.PlainDate.from(to)).days;
}
```

listing 86: temporal typed through the esnext.temporal fragment, the api part one's chapter 8 already runs (source: javascript/tssamples/ch11/tstoolchain.ts, lines 24-38)

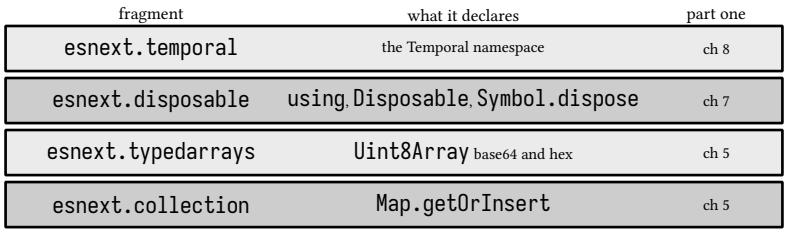


figure 62: es2026 era apis reaching the checker one fragment at a time

11.4. tsconfig anatomy

Every chapter of part two keeps its samples in one npm project under `javascript/tssamples/`, and the whole configuration is one file. `tsc --showConfig` against it prints the resolved configuration and exposes two settings 7.0 forces on with no opt out: `isolatedModules`, every file must be transpilable alone, which is what the native compiler's parallel checking assumes, and `preserveConstEnums`, const enums emit like plain enums:

```json
{
  "compilerOptions": {
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "target": "es2025",
    "lib": ["es2025", "esnext.array", "esnext.temporal"],
    "types": ["node"],
    "resolveJsonModule": true,
    "outDir": "dist",
    "rootDir": ".",
    "sourceMap": true,
    "verbatimModuleSyntax": true
  },
  "include": ["ch*/**/*.ts"]
}
```

listing 87: the part ii configuration, one file, node's resolution, es2025 with two esnext fragments (source: javascript/tssamples/tsconfig.json, lines 1-16)

The `nodenext` pair opts into node's own module resolution over the `esnext` default, which is what makes `import x from "./x.json" with`

`{ type: "json" }` check without ceremony. The `types` line is the migration trap: its default changed to an empty list, so installing `@types/node` no longer auto includes node types, and imports from `node:test` fail to resolve until `"types": ["node"]` says so out loud. `strict` is on without being asked, inherited from the 6.0 default move, and does not appear in the resolved output because nothing here overrides it.

`verbatimModuleSyntax` is the load bearing discipline for a book about erasure: whatever the import spelling, that exact spelling is emitted. A type imported plainly would emit an import of a name that no longer exists, so type-only imports must say `import type`, and the flag turns every silent violation into a build error. The chapters ahead use it everywhere, which is why their test files import interfaces with `import type` and everything else plainly.

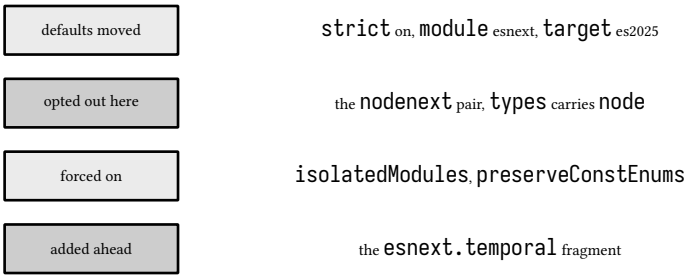


figure 63: tsconfig options by what 7.0 lets a project still turn

WARNING: removed in 7.0

The native compiler dropped a pile of legacy surface: `target:` `es5` and `downlevelIteration`, `moduleResolution: node10` and `classic`, the `amd`, `umd`, `systemjs`, and `none` module formats, and `baseUrl`, whose `paths` are now project root relative. Import `asserts` became `with`, the `module` keyword inside a namespace declaration is an error, and the `esnext` target list itself stops at `es2025`, verified live against 7.0.2.

11.5. the two stage verify chain

The manifest declares the module system and one script, and the script is two stages: `tsc` typechecks and emits, then node's own runner tests the emitted `dist`. The script opens by deleting `dist`, because `tsc` never removes old outputs itself: after this book's renumber moved `erasure.ts` out of ch17, a stale emitted `dist/ch17/erasure.test.js` still matched the test glob and failed the gate on files that no longer existed. Testing the emitted output rather than the sources is deliberate. Part one's file is byte for byte the file that ran, and part two's honest equivalent is the file `tsc` produced, annotations gone, which is also what lets enum and namespace samples, which node's own type stripping cannot handle, still run:

```
{
  "name": "jsbook-tssamples",
  "private": true,
  "type": "module",
  "scripts": {
    "verify": "node -e \"require('fs').rmSync('dist',{recursive:true,force:true})\" && tsc -p tsconfig.json
&& node --enable-source-maps --test \"dist/**/*.test.js\""
  },
  "devDependencies": {
    "@types/node": "^26.0.0",
    "typescript": "^7.0.2"
  }
}
```

listing 88: the part ii manifest, one script, two stages (source: javascript/tssamples/package.json, lines 1-13)



figure 64: the two stage verify chain, sources tested only as emitted output

Around `tsc -p` sit the inspection commands this chapter used while writing: `tsc --version` to pin the build, `tsc --showConfig` for the resolved configuration and the forced options, and the new parallelism knobs, `--checkers` for the count of parallel type checking threads, `--builders` for parallel project references under `--build`, and `--singleThreaded` to disable both for profiling.

The harness itself is `node:test` with `describe` and `it`, asserting through `node:assert/strict`, the same runner part one uses on sources, here aimed at `dist`:

```
import { describe, it } from "node:test";
import assert from "node:assert/strict";
import {
  DaysBetween,
  EmittedErasure,
  Greet,
  NodeMajor,
  Shout,
  TypeScriptVersion,
  WeekdayOf,
} from "./tstoolchain.js";

describe("typescript toolchain", () => {
  it("runs on node 26 or newer", () => {
    assert.ok(NodeMajor() ≥ 26, `node major was ${NodeMajor()}`);
  });

  it("typechecks with typescript 7.0.2", () => {
    assert.equal(TypeScriptVersion(), "7.0.2");
  });

  it("greets by name", () => {
    assert.equal(Greet("ada"), "hello ada");
  });
```

listing 89: the node:test harness every part two chapter reuses (source: javascript/tssamples/ch11/tstoolchain.test.ts, lines 1-24)

NOTE: no stable api means no introspection

The go book found its compiler version through `runtime.Version` and the scanner through `go/token`. Typescript 7 offers no stable equivalent: the package root exports nothing beyond its version, and the compiler api that does ship lives under `unstable` exports this book does not build on. The sample reads the installed package manifest from disk, which is also the honest answer to which compiler checked this file.

sources: devblogs.microsoft.com/typescript announcing typescript 7.0, typescriptlang.org tsconfig reference and compiler options in js, nodejs.org api test runner docs, accessed 2026-09-13. Toolchain behavior verified live with tsc 7.0.2 (`npx tsc --version` prints `Version 7.0.2`) and node 26.3.0 on windows, the es2026 rejections and the `TS2503` temporal failure reproduced against the cli, 7 tests green in ch11 through `npm run verify` in `javascript/tssamples`.

12. type syntax

Typescript inherits the ecmascript grammar whole and adds type syntax in positions javascript leaves empty: after a colon, inside angle brackets, between `type` and `=`. The runtime side of every value here is already chapter 2's, the seven primitives, the lying `typeof`, the two equalities, so this chapter is deliberately slim: what an annotation is, what inference does when you omit one, how literal and union literal types refine the primitives, how arrays and tuples are spelled, and how aliases and `typeof` queries give names to shapes. None of it survives compilation, chapter 11 proved that against the emitted file, so every test below exercises the value the annotation promised to describe.

The added layer is contextual, never reserved. Words like `keyof`, `satisfies`, `readonly`, and the type words `string`, `number`, `never` parse as keywords only where type syntax is expected, and as ordinary identifiers everywhere else:

```
// type layer words are contextual, never reserved: outside type
// position they bind as ordinary identifiers.
export function ContextualWords(): string {
  const keyof = "contextual";
  const satisfies = "ordinary";
  return `${keyof} ${satisfies}`;
}
```

listing 90: type words bind as identifiers outside type position (source: javascript/tssamples/ch12/typesyntax.ts, lines 81-87)

12.1. annotations and inference

An annotation is a promise attached to a binding, checked at compile time and erased at emit. Inference is the checker deriving the same promise from the initializing expression, and the interesting case is the literal. A `let` binding widens: `let mode = "read"` infers `string`, because the binding may be reassigned, so writing `"write"` back into it compiles. A `const` binding keeps the literal type, because nothing can change it:

```
// a let binding widens: the literal "read" is inferred as string, so
// reassigning "write" compiles. the counterfactual, passing this
// widened variable to a literal union parameter, is a compile error
// the chapter shows as a fragment.
export function WidenedLet(): string {
  let mode = "read";
  mode = mode === "read" ? "write" : "read";
  return mode;
}

// a const binding keeps its literal type, which is why the call below
// typechecks against "read" | "write" without a cast.
export type Mode = "read" | "write";

export function ModeLabel(mode: Mode): string {
  return mode === "read" ? "reading" : "writing";
}

export function NarrowConst(): string {
  const mode = "read";
  return ModeLabel(mode);
}
```

listing 91: let widens to the primitive, const keeps the literal, and only the narrow one calls the union (source: javascript/tssamples/ch12/typesyntax.ts, lines 15-36)

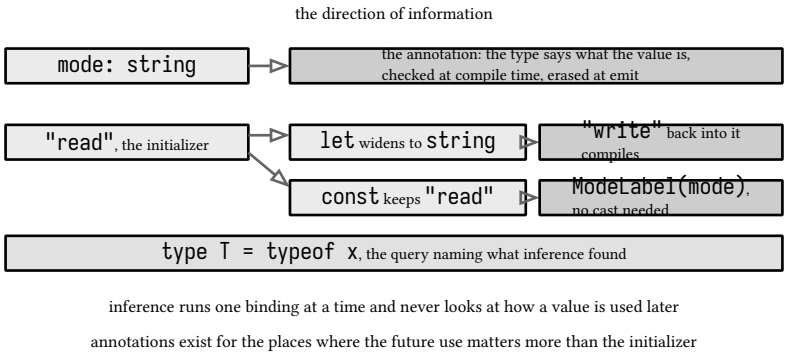


figure 65: annotation and inference, let widening against const keeping

The call in `NarrowConst` typechecks without a cast because the checker still knows `mode` is exactly `"read"`. The counterfactual does not compile, and that is the point:

```
export function Broken(): string {
  let mode = "read";      // inferred string, widened
  return ModeLabel(mode);  // error: string is not "read" | "write"
}
```

fragment, illustrative, not executed

Inference runs one binding at a time and never looks at how a value is used later; annotations exist for the places where the future use matters more than the initializing expression.

12.2. primitives against their runtime values

The primitive annotations correspond to runtime values chapter 2 already measured, and the correspondence has one asymmetry worth a table: `typeof null` is `"object"`, the 1995 lie, while the annotation for the value is `null`:

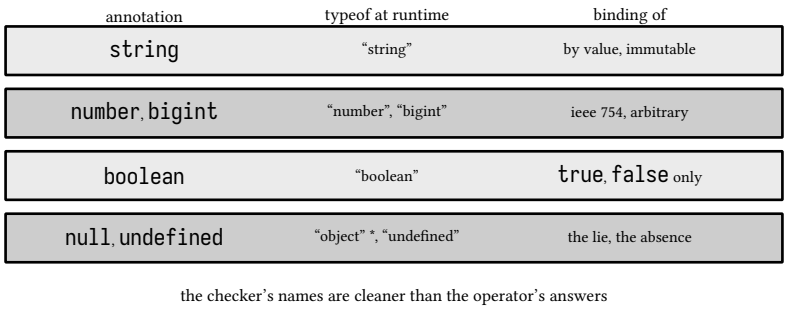


figure 66: annotation against typeof, the one starred row is chapter 2’s lie

The boxed forms, `String` and `Number` and friends, are object types, not primitive ones: two boxes over the same primitive are two identities, never equal, and the test pins it:

```
// the boxed forms are objects: two boxes over the same primitive are
// two identities, never equal, and the checker keeps `string` and
// `String` distinct so the confusion has to be typed out on purpose.
export function BoxedIdentity(): boolean {
  return new String("a") === new String("a");
}
```

listing 92: the box is an object, identity comparison fails, and the checker keeps string and String apart (source: javascript/tssamples/ch12/typesyntax.ts, lines 38-43)

12.3. literal and union literal types

A literal type admits exactly one value, and a union of literals is the cheapest enum-like checking there is: no runtime footprint, the members are the values themselves. `Mode` below is two string literals, and inside `ModeLabel` the `===` comparison narrows which one the parameter is, a preview of chapter 16's full treatment:

```
// a const binding keeps its literal type, which is why the call below
// typechecks against "read" | "write" without a cast.
export type Mode = "read" | "write";

export function ModeLabel(mode: Mode): string {
  return mode === "read" ? "reading" : "writing";
}
```

listing 93: the union literal type and its consumer (source: javascript/tssamples/ch12/typesyntax.ts, lines 25-31)

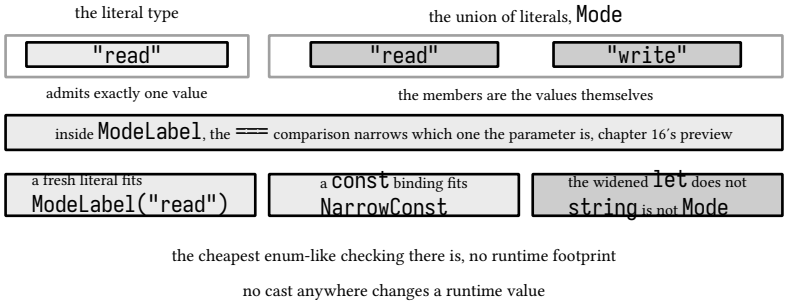


figure 67: literal and union literal types, the members are the values themselves

Literal unions compose with the widening rule above: a fresh literal argument fits, a `const` binding fits, a widened `let` does not, and no cast anywhere changes a runtime value.

12.4. arrays and tuples

Arrays carry one element type in two spellings, `number[]` and `Array<number>`, and the spellings are the same type: the second function below calls the first with its argument, unchecked and uncast, because they agree:

```
export function Total(ns: number[]): number {
  return ns.reduce((a, b) => a + b, 0);
}

// the two array spellings are one type: Array<number> is number[].
export function TotalAliased(ns: Array<number>): number {
  return Total(ns);
}
```

listing 94: one array type, two spellings, one calling the other (source: javascript/tssamples/ch12/typesyntax.ts, lines 6-13)

A tuple is an array with a fixed shape: the arity is part of the type, each position has its own element type, and the labels are names for humans and error messages. The length claim is enforced by the checker alone, part one's arrays know nothing about it, which is why `OptionalTail` can report a length the tuple's full form would forbid:

```ts
// a tuple fixes arity and per-position types; the labels are for
// humans and error messages, not runtime properties.
export function PairSummary(pair: [label: string, value: number]): string {
  const [label, value] = pair;
  return `${label}=${value}`;
}

export function OptionalTail(): number {
  const pair: [name: string, age?: number] = ["ada"];
  return pair.length;
}
```

listing 95: labeled tuple positions, and an optional tail one short (source: javascript/tssamples/ch12/typesyntax.ts, lines 45-55)

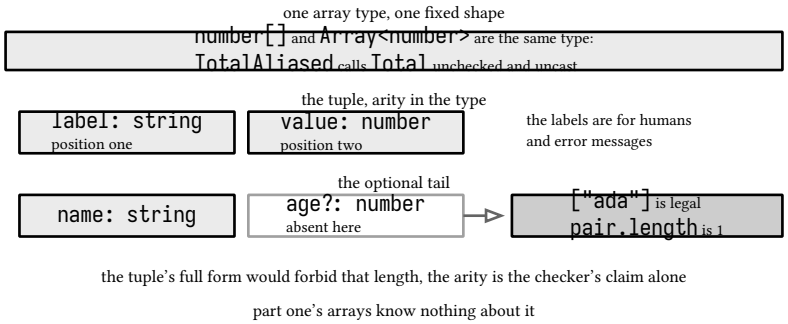


figure 68: arrays in two spellings, the tuple's fixed shape

12.5. aliases and typeof queries

A type alias is a name, not a new type. `type ID = string | number` adds a word to the checker's vocabulary and nothing to the runtime, and narrowing an `ID` is narrowing the union it names:

```ts
// an alias is a name, not a new type: narrowing an ID narrows the
// union it names.
export type ID = string | number;

export function IDKind(id: ID): "string" | "number" {
  return typeof id === "string" ? "string" : "number";
}
```

listing 96: an alias to a union, narrowed by the runtime operator (source: javascript/tssamples/ch12/typesyntax.ts, lines 57-63)

The `typeof` query is the bridge from values to types: in type position, `typeof x` is the type the checker currently has for `x`. Deriving a type from one object literal and consuming it elsewhere keeps the shape and the value from drifting apart, and querying a function yields its full signature, which another binding must then match exactly:

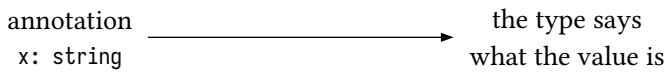
```ts
// a typeof query in type position reads a value's inferred type: the
// checker derives Defaults from the object literal, and DescribeDefaults
// consumes the derived type.
const defaults = { host: "localhost", port: 8080 };
export type Defaults = typeof defaults;

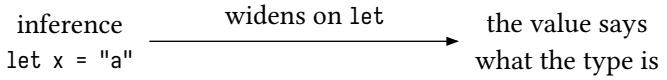
export function DescribeDefaults(d: Defaults): string {
  return `${d.host}:${d.port}`;
}

// the same query works on functions: TotalFn is the type of Total,
// and another function annotated with it must match exactly.
export type TotalFn = typeof Total;

export const TotalAgain: TotalFn = (ns) => ns.reduce((a, b) => a + b, 0);
```

listing 97: typeof deriving a shape from a value, and a signature from a function (source: javascript/tssamples/ch12/typesyntax.ts, lines 65-79)





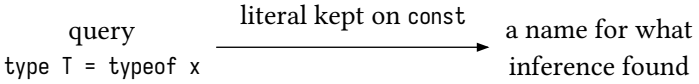


figure 69: the direction of information, annotation, inference, and query

NOTE: what this chapter is not

This is not the lexical chapter it replaced. The reserved word strata and the semicolon insertion note live in part one's chapter 2, the full census of all 75 words in chapter 39's appendix, and the private name lexeme in chapter 4's classes. The typescript specific residue, that the type layer's words are contextual rather than reserved, is the one grammar fact kept here, pinned by the `ContextualWords` test.

sources: typescriptlang.org handbook everyday types and more on functions, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 10 tests in ch12 green through `npm run verify`, the widening counterfactual confirmed by `tsc --noEmit` before being shown as a fragment.

13. the type landscape

Chapter 12 built the syntax of one type at a time. This chapter is about the relations between types: which values a type admits, which types substitute for which, and what the checker compares when it decides. Four special types anchor the map, `any`, `unknown`, `never`, and `void`, and one rule organizes everything else: typescript compares shape, not name.

13.1. the two tops and two bottoms

`any` is the escape hatch. Assignments in, member access out, all of it unchecked, and the failure moves from compile time to runtime. The call below compiles, the member does not exist, and the program dies at the call site. Real codebases lint `any` toward zero:

```
export function AnyHole(): void {
  const loosely: any = { n: 1 };
  loosely.boom();
}
```

listing 98: any: the hole in the checking, demonstrated by falling into it (source: javascript/tssamples/ch13/types.ts, lines 5-8)

`unknown` is the safe top type. Anything assigns into it, which makes it the honest type for data from outside the program, and nothing reads out of it without a runtime guard. The guards are not new machinery, they are part one's own operators, and each guard narrowing the value inside its branch is the preview of chapter 16:

```
// unknown is the safe top: anything assigns into it, nothing reads
// out of it without a runtime guard, and the guards are part one's
// own operators, typeof and Array.isArray.
export function LengthOf(value: unknown): number {
  if (typeof value === "string") return value.length;
  if (Array.isArray(value)) return value.length;
  return 0;
}
```

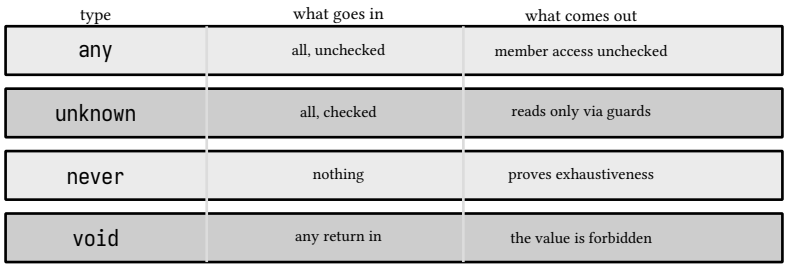
listing 99: unknown consumed only through the runtime guards of part one (source: javascript/tssamples/ch13/types.ts, lines 10-17)

`never` is the bottom type, inhabited by nothing. A function whose body always throws returns it, the return type is not a formality, and chapter 16 uses it to prove switches exhaustive. `void` is the return contract, and it is weaker than it looks: a function typed `() => void` may be assigned one that returns 42, the call still evaluates to 42, and only the type of the call forbids using the value, which the sample has to smuggle out through a double cast to observe:

```
export function Boom(): never {
  throw new Error("boom");
}

// the void contract is checker only: assigning a function that
// returns 42 is legal, the call still evaluates to 42, and only the
// type of the call forbids using the value. the cast smuggles the
// value out to observe what the runtime already knows.
export function VoidDiscards(): number {
  const f: () => void = () => 42;
  return f() as unknown as number;
}
```

listing 100: never throws, void only forbids (source: javascript/tssamples/ch13/types.ts, lines 19-30)



two tops, two bottoms: only the checker sees the difference

figure 70: the two tops and two bottoms, by what each one admits

The rule of thumb: `unknown` at boundaries, ordinary types inside, `never` for the impossible, `any` for the legacy file you have not migrated yet.

13.2. structural typing

A value is a `Printable` because it has the members a `Printable` needs. No name, no declaration lineage, no nominal identity. The interface and the alias below are interchangeable in every position, the class implements neither, and its instances satisfy both anyway, because structure is the only thing the checker compares:

```ts
// structural typing: no name, no declaration lineage, no nominal
// identity. the interface and the alias below are interchangeable,
// and the class declares neither, yet its instances satisfy both
// because they carry the member.
export interface Printable {
  label: string;
}

type Titled = {
  label: string;
};

export class Page {
  constructor(public label: string) {}
}

export function Print(p: Printable): string {
  return `[${p.label}]`;
}

export function TitleOf(t: Titled): string {
  return t.label.toUpperCase();
}

export function StructuralRoundtrip(label: string): string {
  const page = new Page(label);
  return `${Print(page)} ${TitleOf(page)}`;
}
```

listing 101: an interface, its alias twin, a class that declares neither, and one value through both consumers (source: javascript/tssamples/ch13/types.ts, lines 32-59)

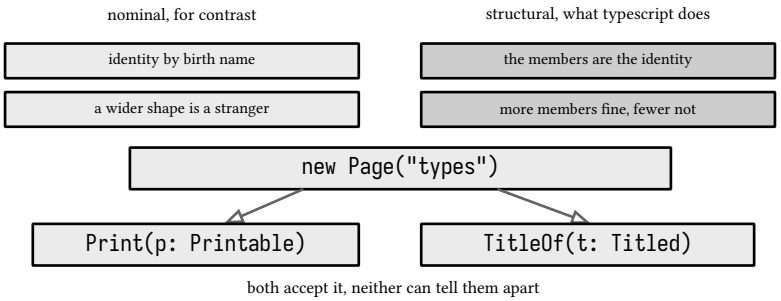


figure 71: nominal against structural, one value through both worlds

The contrast is with the nominal systems of the other books in this corpus, where a `UserId` and an `OrderId` over the same primitive are different by declaration. Structural typing trades that safety for openness, and when identity matters the fix is branding, chapter 20's idiom.

13.3. assignability

Substitution is the checker's daily work, and width subtyping is the rule: more members than required is assignable, fewer is not. The extra member is not stripped, because nothing at runtime ever strips properties, part one's chapter 4 established that, so the wide value keeps its `speed` after passing through the narrow parameter:

```ts
// assignability is width subtyping: more members than required is
// assignable, fewer is not, and nothing at runtime strips the extra.
export interface Reader {
  name: string;
}

type FastReader = Reader & { speed: number };

export function GreetReader(r: Reader): string {
  return `hi ${r.name}`;
}

export function WideAssigns(): string {
  const fast: FastReader = { name: "ada", speed: 2 };
  return `${GreetReader(fast)} at ${fast.speed}`;
}
```

listing 102: width subtyping, the wider value assigning down and keeping its extra member (source: javascript/tssamples/ch13/types.ts, lines 61-76)

Direction matters as much as width. A member of a union assigns to the union, a literal assigns to its primitive, `never` assigns to everything, and none of the reverse moves compile without a narrowing step in between:

```ts
// the direction table in one function: a member of a union assigns
// to the union, a fresh literal assigns to a literal type, and never
// assigns to everything, which is why it can end an exhaustive
// switch in chapter 16.
export type Mode = "read" | "write";

export function AcceptsMode(mode: Mode): string {
  return mode === "read" ? "reading" : "writing";
}

export function DirectionDemo(): string {
  const narrow: "read" = "read";
  const asUnion: Mode = narrow;
  return AcceptsMode(asUnion);
}
```

listing 103: the direction table as code: literal into union, never into all (source: javascript/tssamples/ch13/types.ts, lines 78-92)

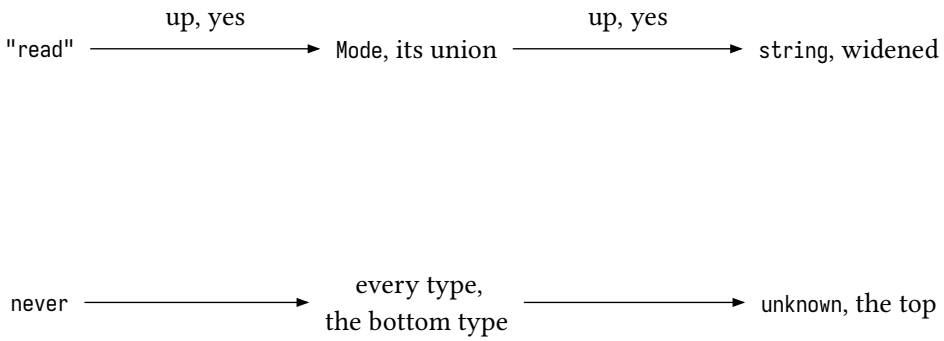


figure 72: assignability's one way streets

The reverse of every arrow in that diagram needs either a guard that narrows or a cast that lies, and the cast is exactly chapter 11's `null as unknown as string` in miniature: it silences the checker and adds no check.

13.4. narrowing, previewed

The `unknown` section already did it: a runtime guard, `typeof` or `Array.isArray`, and inside the branch the value's type shrinks to what the guard proved. Chapter 16 is the full treatment, union types and the discriminator pattern and exhaustiveness, and its runtime half is entirely part one's: the guards are operators the language always had, the checker merely tracks what each branch proved and forgets it when control flow can no longer guarantee it.

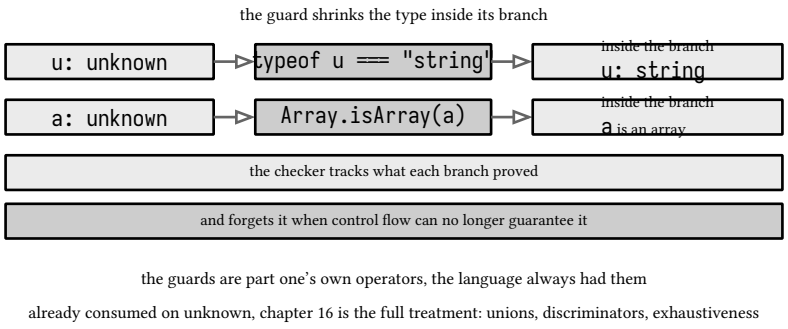


figure 73: narrowing previewed, the guard shrinking the type inside its branch

NOTE: what moved out of this chapter

> The typeof table now lives in part one's chapter 2, where the operator is measured on runtime values. Arrays and tuples, literal unions, and aliases are chapter 12's syntax. Enums, the one construct that emits an object, wait for the emit chapters, and their suite cases went with them.

sources: typescriptlang.org handbook more on functions and narrowing, typescriptlang.org any vs unknown documentation, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 10 tests in ch13 green through `npm run verify`.

14. typing functions

A typescript function is a javascript function: one object, one `typeof` category, callable, holdable, comparable only by identity, and every rule about receivers, closures, and recursion is part one's chapter 3. What the type layer adds is the signature surface: parameter and return forms, overloads, generics, and the compatibility rule that decides when one function substitutes for another. All of it erases, chapter 11 proved that against the emitted file, so a typescript call site compiles to the same call part one already runs.

14.1. overloads

Overloads are separate signatures stacked above one implementation. The implementation signature is invisible to callers, and resolution takes the first matching overload:

```
// overloads: separate signatures stacked above one implementation.
// callers see the signatures, never the implementation's, and
// resolution takes the first match.
export function CreatePoint(x: number, y: number): Point;
export function CreatePoint(label: string): Point;
export function CreatePoint(a: number | string, b?: number): Point {
  if (typeof a === "string") return { x: 0, y: 0, label: a };
  return { x: a, y: b ?? 0 };
}
```

listing 104: two overloads, one implementation (source: javascript/tssamples/ch14/typefunctions.ts, lines 7-15)

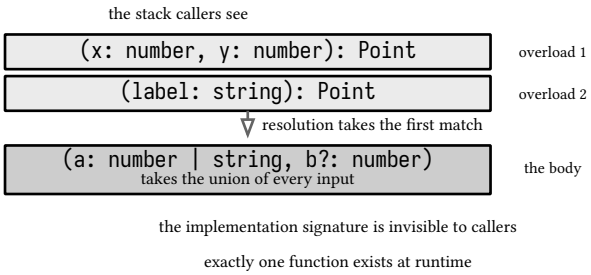


figure 74: the overload stack, two signatures callers see over one body

The body has to accept the union of all overload inputs, which is why the implementation takes `number | string`. Overloads are a documentation and precision tool, not polymorphism: there is exactly one function at runtime, and the emitted javascript keeps only the implementation.

14.2. parameter forms

Optional parameters are `?` suffixed or defaulted, and only one rest parameter may appear, last. Defaults make their parameter optional for free, and the rest parameter is a true array:

```
export function Greet(name: string, greeting = "hello", ...punct: string[]): string {
  return `${greeting} ${name}${punct.join("")}`;
}
```

listing 105: default and rest parameters (source: javascript/tssamples/ch14/typefunctions.ts, lines 17-19)

form	optionality	what arrives
`name: string`	required	the argument itself
`greeting = "hello"`	optional for free	the default, filled in
`...punct: string[]`	one, and last	a true array

the `?` marks a parameter optional, a default marks and fills it

figure 75: the arity table, what each parameter form asks of the caller

14.3. generics with constraints

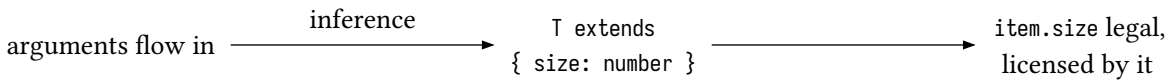
Type parameters may be constrained to a structural shape, and the constraint is what lets the body read `item.size` at all. Inference from the argument list means callers never spell the parameter out:

```
// a constrained generic: the extends clause is what licenses
// item.size in the body, and inference from the argument list means
// callers never spell the parameter out.
export function Largest<T extends { size: number }>(items: T[]): T | undefined {
  let best: T | undefined;
  for (const item of items) {
    if (best === undefined || item.size > best.size) best = item;
  }
  return best;
}

export function Duplicate<T>(value: T): [T, T] {
  return [value, value];
}

export function Compose<A, B, C>(f: (b: B) => C, g: (a: A) => B): (a: A) => C {
  return (a) => f(g(a));
}
```

listing 106: a constrained generic, a plain one returning a tuple, and composition in three parameters (source: javascript/tssamples/ch14/typefunctions.ts, lines 21-38)



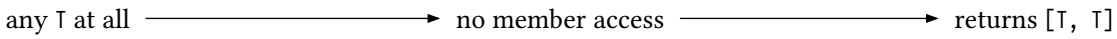


figure 76: the inference path, constraint to licensed member access

Contextual typing runs the other direction: annotate the variable with the function type and the lambdas inside infer their parameters, which is how a table of operations stays typed without a single parameter annotation in sight:

```ts
// contextual typing runs the other direction: the record's value
// type supplies the parameter types, so the lambdas name their
// parameters without annotating them.
export const Ops: Record<string, (a: number, b: number) => number> = {
  add: (a, b) => a + b,
  sub: (a, b) => a - b,
};
```

listing 107: parameters inferred from the contextual type (source: javascript/tssamples/ch14/typefunctions.ts, lines 40-46)

14.4. function type compatibility

The checker compares function types by three rules, and the three demos below each compile one legal direction and leave the illegal reverse to a fragment. Arity: fewer parameters than expected is fine, the function ignores the rest, while more parameters than expected does not compile. Returns: a narrower return type substitutes where a wider one is expected. Parameters: under the strict function type checking 7.0 defaults to, a function accepting a wider parameter substitutes where a narrower one is expected, which is contravariance, the same direction as chapter 13's assignability arrows read backwards:

```ts
// compatibility, arity: a function with fewer parameters assigns
// where more are expected, because it simply ignores the rest. the
// reverse, more parameters than expected, does not compile.
export function FewerWhereMoreExpected(): number {
  const takes: (a: number, b: number) => number = (a) => a * 2;
  return takes(21, 100);
}

// compatibility, returns: a function returning a narrower type
// assigns where a wider return is expected.
export function NarrowReturnWhereWideExpected(): string {
  const gives: () => string | number = () => "literal";
  return String(gives());
}

// compatibility, parameters: a function accepting a wider parameter
// assigns where a narrower one is expected, contravariance, under
// the strict function type checks 7.0 defaults to.
export function WideParameterWhereNarrowExpected(): number {
  const wants: (s: string) => number = (v: string | number) => Number(v);
  return wants("42");
}
```

listing 108: arity, return covariance, and parameter contravariance, all three compiling (source: javascript/tssamples/ch14/typefunctions.ts, lines 48-69)

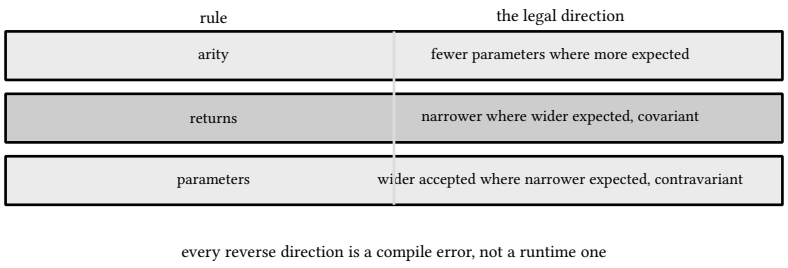


figure 77: the three compatibility rules, each one way

```
const more: (a: number) => number = (a: number, b: number) => a + b;
// error TS2322: type '(a: number, b: number) => number' is not
// assignable to type '(a: number) => number'
```

fragment, illustrative, not executed

14.5. the checked this

One piece of the receiver story belongs to the signature: a `this` parameter, checked like any other. `this: void` documents that a function reads no receiver at all, and the checker then rejects any `this` access inside its body, so a free function cannot silently grow dependent on a call site. The runtime rules of `this`, method calls, detached calls throwing in a module, `bind`, lexical arrows, are part one's chapter 3:

```
// a this parameter of void documents that a function reads no
// receiver at all, and the checker rejects this access in its body.
export function ThisFree(this: void, x: number): number {
  return x * 2;
}
```

listing 109: a this void signature, callable as a free function (source: javascript/tssamples/ch14/typefunctions.ts, lines 71-75)

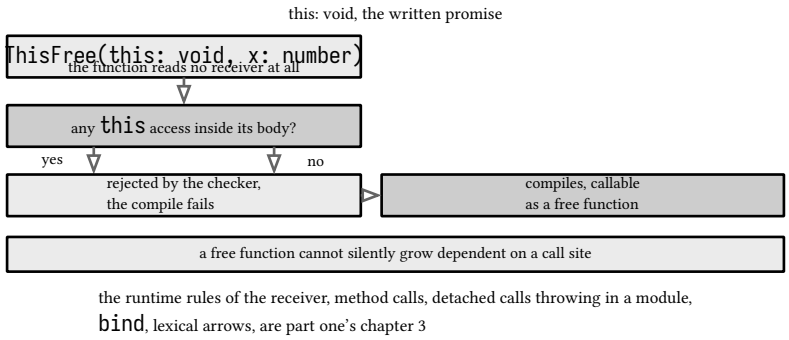


figure 78: the checked this, a written promise the checker enforces

NOTE: what moved out of this chapter

The old functions chapter demonstrated receivers, closures, and hoisting at runtime. All of that is part one's chapter 3 now, typed samples included, and the suite cases that pinned it went with them. What stays here is the signature layer: overloads, parameter forms, generics, compatibility, and the checked `this`.

sources: typescriptlang.org handbook more on functions and typescript handbook function compatibility, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 10 tests in ch14 green through `npm run verify`, the arity counterfactual confirmed by `tsc --noEmit` before being shown as a fragment.

15. typing objects

Chapter 13 established the rule, the checker compares shape, not name. This chapter applies it to the object: how a member is declared optional or readonly, what an index signature trades away, where the interface and the type alias part ways, and the one place shape checking tightens, the fresh object literal. The runtime side of objects, literal forms, key order, spread and rest, is part one's chapter 4, and nothing here changes it: every modifier and every declaration below erases at emit.

15.1. interfaces against aliases

An interface and a type alias with the same members are the same type to the checker. The test proves it by passing one value to consumers typed with each, and neither consumer can tell which declaration its parameter came from:

```
export interface Book {
  readonly isbn: string;
  title: string;
  pages?: number;
}

// a type alias with the same members: assignable to Book and from
// Book in both directions, because the checker compares shape, not
// declaration kind.
type Article = {
  readonly isbn: string;
  title: string;
  pages?: number;
};

export function Describe(b: Book): string {
  return `${b.title} ${b.pages ?? 0}p`;
}

export function DescribeArticle(a: Article): string {
  return `${a.title} ${a.pages ?? 0}p`;
}
```

listing 110: an interface, its alias twin, and two consumers that cannot tell them apart (source: javascript/tssamples/ch15/typeobjects.ts, lines 6-27)

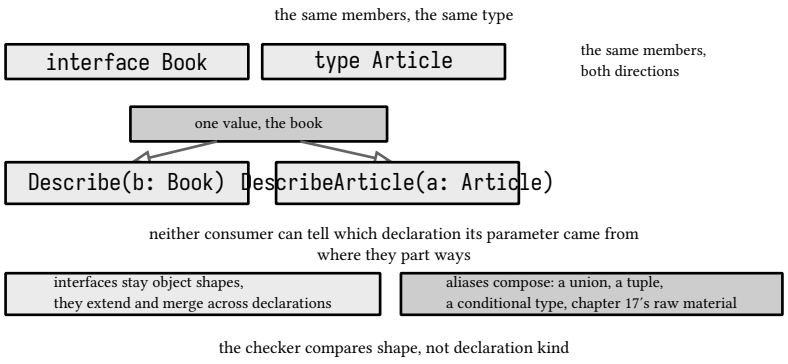


figure 79: interfaces against aliases, the same members as the same type

The declarations part ways in three places, and only the third matters for object shapes. Interfaces extend and merge, aliases compose: an alias can be a union, a tuple, a conditional type, the raw material of chapter 17, while an

interface is always an object shape. And interfaces can be augmented across declarations, which is the capability below.

15.2. optional and readonly

An optional member, `pages?`, may be absent, and the consumer defaults it with `??`, because reading it yields `undefined` at runtime exactly as part one's property access rules say. A readonly member is a checker promise with no runtime side: the cast below compiles because it routes through a type without the modifier, and the mutation runs:

```
// readonly is a checker promise: the cast compiles because it goes
// through a type without the modifier, and the mutation runs.
export function MutateIsbn(b: Book): string {
  (b as { isbn: string }).isbn = "mutated";
  return b.isbn;
}
```

listing 111: readonly members mutate through casts, exactly like the readonly members above (source: javascript/tssamples/ch15/typeobjects.ts, lines 37-42)

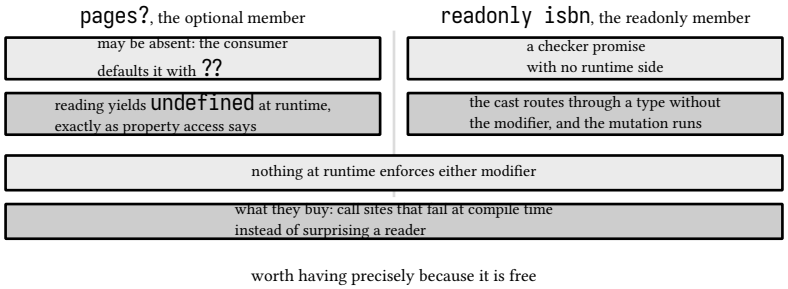


figure 80: the two modifiers, no runtime side on either

Nothing at runtime enforces either modifier. What they buy is call sites that fail at compile time instead of surprising a reader, and the discipline is worth having precisely because it is free.

15.3. excess properties

The one place shape checking tightens is the fresh object literal, which gets excess property checks so a typo in a property name fails at the assignment instead of riding along forever. The check applies to literals only. A variable whose type is `Book & { extra }` is a perfectly valid `Book`, and the extra member survives every assignment, because nothing at runtime ever strips properties:

```
// excess property checks apply to fresh literals only. an object
// typed Book & { extra } is a perfectly valid Book, and the extra
// member survives every assignment, because nothing at runtime ever
// strips properties.
export function Sneak(b: Book): unknown {
  return (b as { extra?: unknown }).extra;
}
```

listing 112: extra members ride along through variables, the literal check never sees them (source: javascript/tssamples/ch15/typeobjects.ts, lines 29-35)

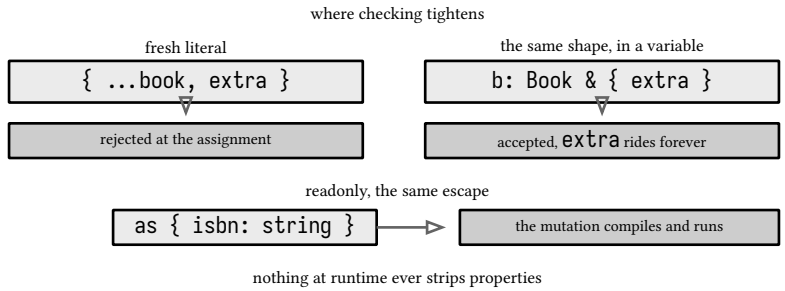


figure 81: excess property checks and readonly, where the tightening stops

```
const typo: Book = { isbn: "i1", title: "go", pagse: 321 };
// error TS2561: object literal may only specify known properties,
// but 'pagse' does not exist in type 'Book'. Did you mean 'pages'?
```

fragment, illustrative, not executed

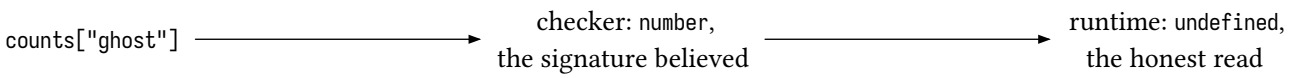
15.4. index signatures

An index signature types every key at once, which trades precision for openness. The cost is the missing key: the signature says `number`, the runtime says `undefined`, and the checker believes the signature:

```
export function CountsSum(counts: { [key: string]: number }): number {
  return Object.values(counts).reduce((a, b) => a + b, 0);
}

// the index signature promises number for every key, and the checker
// believes it: reading a missing key types as number and evaluates
// to undefined. noUncheckedIndexedAccess would type it number |
// undefined instead, at ergonomic cost.
export function MissingKey(): string {
  const counts: { [key: string]: number } = {};
  return String(counts["ghost"]);
}
```

listing 113: index signatures, and the missing key that types as number (source: javascript/tssamples/ch15/typeobjects.ts, lines 44-55)



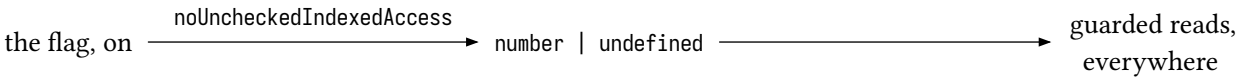


figure 82: the index signature's promise against the missing key

`noUncheckedIndexedAccess`, off by default in this book, would type the read `number | undefined` instead and force every indexed access to be guarded. The honest middleware is a `Map` when the key set is open and unknown, part one's chapter 5, or a guard at the boundary when it is not.

15.5. extension and merging

Interfaces extend one another and also merge: two declarations of the same name union their members. Merging is how `Array` grows extension methods in library code, and it is the one capability a type alias cannot copy, aliases are single declarations and redeclaring them is an error:

```
export interface PrintBook extends Book {
  press: string;
}

// duplicate interface declarations merge their members. this is the
// one capability a type alias cannot copy, aliases are single
// declarations and redeclaring them is an error.
export interface Box {
  load(items: string[]): void;
}

export interface Box {
  unload(): string[];
}

export function NewBox(): Box {
  const items: string[] = [];
  return {
    load(add: string[]) {
      items.push(...add);
    },
    unload() {
      return items.splice(0);
    },
  };
}
```

listing 114: extension, declaration merging, and one implementation (source: javascript/tssamples/ch15/typeobjects.ts, lines 57-82)

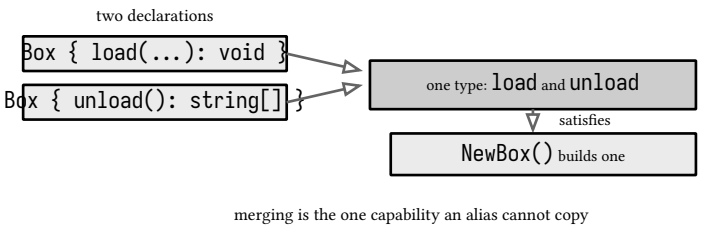


figure 83: declaration merging, two boxes into one type

NOTE: when shape is not enough

Structural typing admits any object with the right members, so a `UserId` and a `OrderId` both typed `string` substitute for each other freely. When identity matters, the idiom is branding, a nominal marker carried in the type. Chapter 20 builds it from the tools of this part.

sources: typescriptlang.org handbook object types and interfaces, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 8 tests in ch15 green through `npm run verify`, the excess property counterfactual confirmed by `tsc --noEmit` before being shown as a fragment.

16. unions and narrowing

A union type lists the shapes a value may take, and narrowing is how the language lets you use one. Nothing here is new machinery at runtime: the guards are part one's own operators, `typeof` from chapter 2, `instanceof` from chapter 4, and a switch is a switch. The checker's whole contribution is bookkeeping, remembering what each branch proved and applying it to the types inside that branch. The discriminator pattern, one literal member shared by every arm of the union, is the central data modeling tool of typescript, and the closest cousin elsewhere in this corpus is book 6, ch 6 (c# 15, f# 11, and .net 11, pattern matching and control flow), where a closed set of shapes is matched over a class hierarchy instead:

```
export type Circle = { kind: "circle"; r: number };
export type Rect = { kind: "rect"; w: number; h: number };
export type Triangle = { kind: "triangle"; base: number; height: number };
export type Shape = Circle | Rect | Triangle;
```

listing 115: a discriminated union of three shapes (source: javascript/tssamples/ch16/unions.ts, lines 7-10)

Inside a `switch` over the discriminator, each case body sees the exact member, `s.r` in the circle arm, `s.w` and `s.h` in the rect arm, with no casts anywhere. The default arm is where the language pays out its strongest guarantee: after every case, `s` has type `never`, and assigning it to a `never` variable compiles. Add a fourth shape to the union and that one line stops compiling, which converts every forgotten switch in the codebase into a build error:

```
export function Area(s: Shape): number {
  switch (s.kind) {
    case "circle":
      return Math.PI * s.r ** 2;
    case "rect":
      return s.w * s.h;
    case "triangle":
      return (s.base * s.height) / 2;
    default: {
      // unreachable by the types: after the three cases, s is never.
      // adding a Shape member turns this assignment into a compile
      // error in this exact spot.
      const unreachable: never = s;
      throw new Error(`exhaustive: ${String(unreachable)}`);
    }
  }
}
```

listing 116: the exhaustive switch, with its never default (source: javascript/tssamples/ch16/unions.ts, lines 12-28)

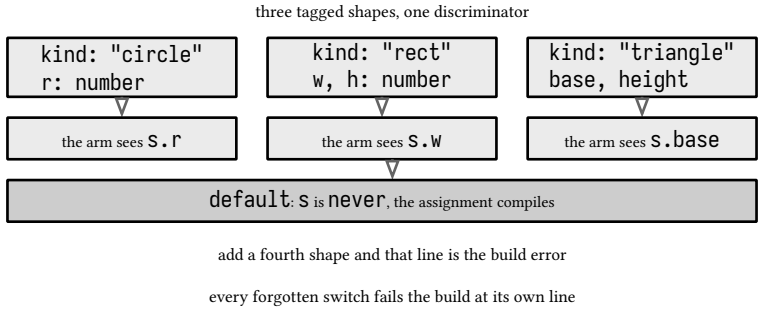


figure 84: the exhaustive switch over three tagged shapes

The runtime half is honest too: the test smuggles a fake `hexagon` member in through a double cast, and the default throws, so even a lie stays inside the guarded arm. The emitted switch is the same switch part one runs, with no injected checks.

16.1. built in guards

`typeof` narrows to the primitive categories, `Array.isArray` to arrays, `in` to members, and `instanceof` down class lines. `in` asks own or inherited anywhere on the chain, and throws on a primitive right side. The `instanceof` chain must run most specific first, since `SyntaxError` and `TypeError` are both `Error` subclasses and the supertype test would swallow them, a runtime fact part one's chapter 4 established when it walked the prototype chain:

```
export function Describe3(value: string | number | string[]): string {
  if (typeof value === "string") return `${value.length} chars`;
  if (typeof value === "number") return "a number";
  return `${value.length} items`;
}

type Bird = { name: string; fly: string };
type Fish = { name: string; swim: string };

export function Move(p: Bird | Fish): string {
  if ("fly" in p) return `${p.name} flies ${p.fly}`;
  return `${p.name} swims ${p.swim}`;
}

export function Explain(e: Error | TypeError | SyntaxError): string {
  if (e instanceof SyntaxError) return "syntax";
  if (e instanceof TypeError) return "type";
  return "generic";
}
```

listing 117: typeof, in, and instanceof narrowing (source: javascript/tssamples/ch16/unions.ts, lines 30-48)

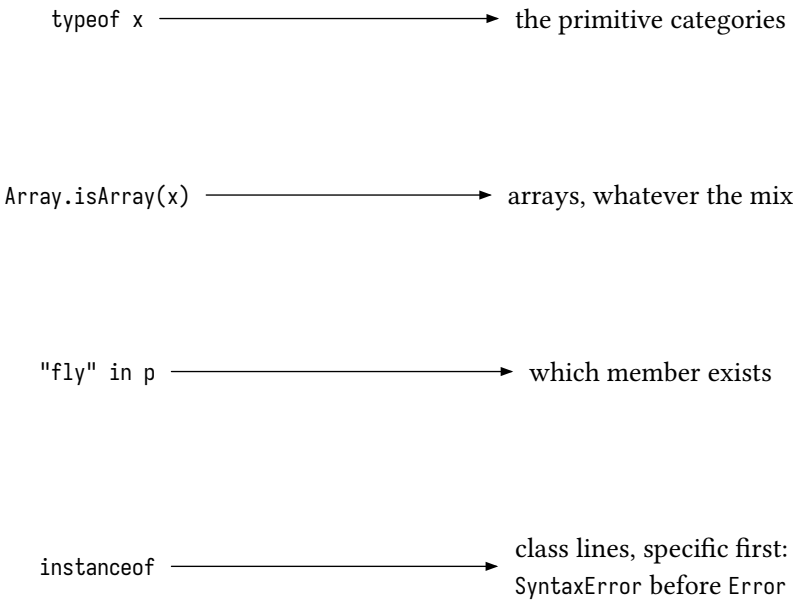


figure 85: the guard ladder, four ways to narrow one value

16.2. user defined guards and assertions

When the check is a shape test, the guard moves into a function with a predicate return type, `value is BookShape`, which teaches the checker what the runtime proved. This is the standard boundary function: `unknown` in, chapter 13's safe top, validated type out, `undefined` for the rest:

```ts
interface BookShape {
  isbn: string;
  title: string;
}

// a user defined type guard: the return type predicate teaches the
// checker what the runtime check proved.
function IsBook(value: unknown): value is BookShape {
  if (typeof value !== "object" || value === null) return false;
  const book = value as Record<string, unknown>;
  return typeof book.isbn === "string" && typeof book.title === "string";
}

export function ParseBook(text: string): BookShape | undefined {
  let parsed: unknown;
  try {
    parsed = JSON.parse(text);
  } catch {
    return undefined;
  }
  return IsBook(parsed) ? parsed : undefined;
}
```

listing 118: a type predicate parsing untrusted json (source: javascript/tssamples/ch16/unions.ts, lines 50-71)

The assertion function is the throwing sibling: `asserts value is T` means "if this returns, the narrowing holds". The assertion return type must be written out, which an inferred arrow binding cannot carry, and after the call the `undefined` arm is gone without a cast:

```ts
// an assertion function narrows by throwing: after the call, value is
// T, not T | undefined.
export function AssertDefined<T>(
  value: T | undefined,
  name: string,
): asserts value is T {
  if (value === undefined) throw new Error(`${name} was undefined`);
}
```

listing 119: an assertion function that narrows by throwing (source: javascript/tssamples/ch16/unions.ts, lines 73-80)

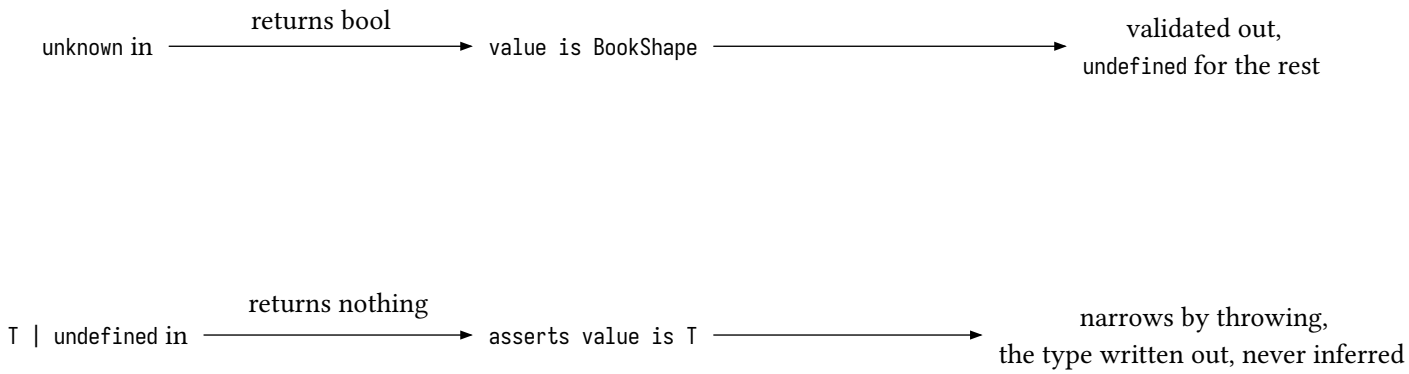


figure 86: two boundary functions, predicate against assertion

16.3. intersections

The dual of the union is the intersection, every member of both: `Named & Aged` demands `name` and `age` together. Intersecting disjoint primitives yields `never`, the type with no values, which is usually a modeling bug announcing itself:

```
type Named = { name: string };
type Aged = { age: number };

export function GreetPerson(p: Named & Aged): string {
  return `${p.name} is ${p.age}`;
}

export function ModeLabel(mode: "read" | "write"): string {
  return mode === "read" ? "reading" : "writing";
}
```

listing 120: an intersection and a literal equality narrowing (source: javascript/tssamples/ch16/unions.ts, lines 82-91)

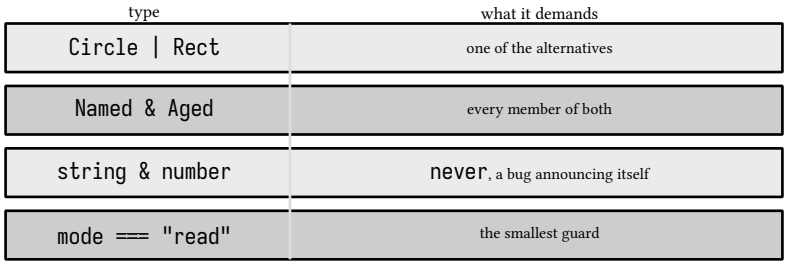


figure 87: the duality table, union against intersection

Narrowing is control flow analysis: assignments, returns, throws, truthiness, and literal equality all refine, and the refinements reset after closures capture the variable or `await` intervenes. The compiler forgets on purpose rather than guessing, and the escape hatches are the casts of chapter 11, which silence the checker but add no check.

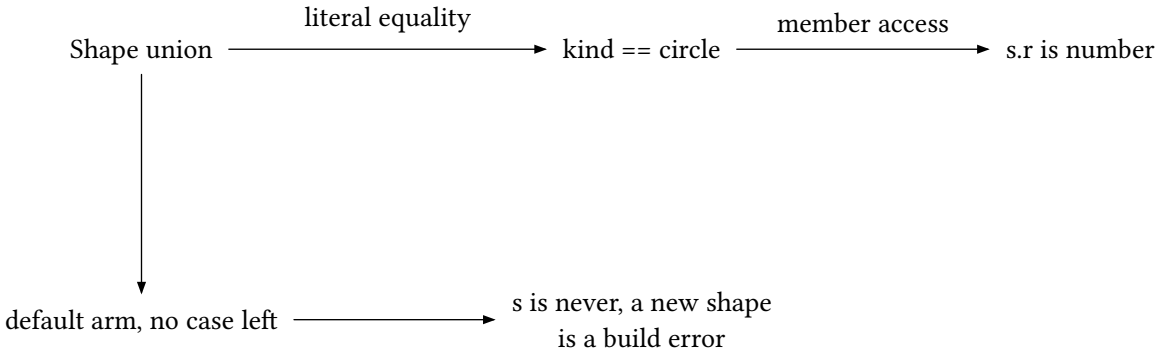


figure 88: a value narrowing as it flows through guards, the default arm to never

sources: typescriptlang.org handbook narrowing and instanceof guards, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 9 tests in ch16 green through `npm run verify`.

17. type-level programming

Everything before this chapter ran types alongside values. This chapter runs types alone: querying them, transforming them, and computing new ones from old, all erased to nothing at emit. The split is the discipline of the whole subject. A type-level construct is a function the checker evaluates between parsing and emit, its inputs and outputs are types, and it cannot read a value, call a function, or branch on anything the program did. So every construct below keeps a runtime companion beside it, `keyof Book` beside `Object.keys`, a recursive conditional beside a loop, because a type utility nobody can call is a page of prose, not a tool.

17.1. querying types

`keyof T` is the union of member names, `T[K]` is the member type at `K`, and `typeof value` lifts a value into the type world. The generic `GetField` uses indexed access with a constrained key, so reading `title` types as `string` and reading `pages` types as `number | undefined`, one function, per member precision:

```
export interface Book {
  readonly isbn: string;
  title: string;
  pages?: number;
}

export function KeysOf(book: Book): (keyof Book)[] {
  return Object.keys(book) as (keyof Book)[];
}

export function GetField<K extends keyof Book>(book: Book, key: K): Book[K] {
  return book[key];
}
```

listing 121: keyof and indexed access against one interface (source: javascript/tssamples/ch17/typelevel.ts, lines 4-16)

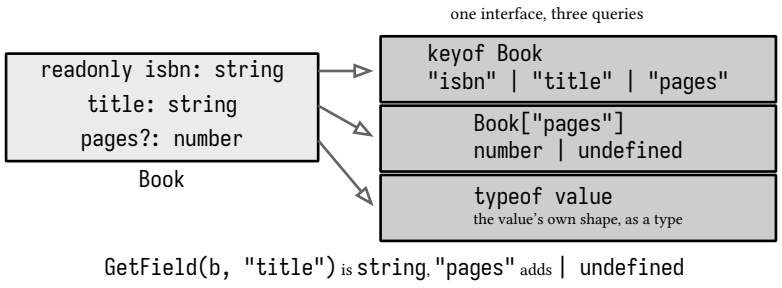


figure 89: keyof, indexed access, and typeof against one interface

The pairing in `KeysOf` is the honest one: `Object.keys` returns `string[]` at runtime, because javascript has no type to consult, and the cast back to `(keyof Book)[]` marks exactly where the checker's knowledge ends. That cast is the seam of the whole chapter, and it appears in every companion below.

17.2. mapped types

A mapped type iterates the members of `T` and rebuilds them, and the `as` clause remaps names while iterating. `Getters` renames every member `m` to `getM` through the `Capitalize` intrinsic, and the runtime companion builds the matching object, key by key:

```ts
type Getters<T> = {
  [K in keyof T as `get${Capitalize<K & string>}`]: () => T[K];
};

export function BuildGetters<T extends object>(obj: T): Getters<T> {
  const out: Record<string, () => unknown> = {};
  for (const [key, value] of Object.entries(obj)) {
    out[`get${key.charAt(0).toUpperCase()}${key.slice(1)}`] = () => value;
  }
  return out as Getters<T>;
}

export function MakePartial<T>(obj: T): Partial<T> {
  return { ...obj };
}
```

listing 122: a homomorphic mapped type with key remapping, and its builder (source: javascript/tssamples/ch17/typelevel.ts, lines 18-32)

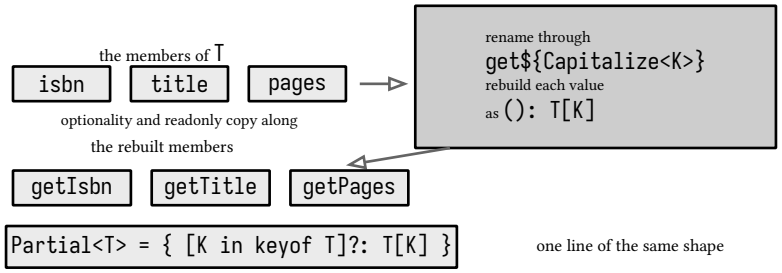


figure 90: the mapped type pipeline, rebuild and rename every member

Homomorphic mapped types copy modifiers, optionality and readonly, from the source, which is why `Partial<T>` in the library is one line of this shape, `{ [K in keyof T]?: T[K] }`, and the sample's `MakePartial` is its runtime shadow. Chapter 20 deepens the same move into a recursive `DeepReadonly` walking a frozen value.

17.3. conditional types, infer, and recursion

The conditional type is a type-level ternary, and `infer` names a position inside the matched branch. The recursive `Unwrap` peels arrays until a non array remains, `[[5]]` unwrapping to `5`, and the runtime `Flat` loop does the same walk over values:

```ts
// recursive conditional: unwrap arrays until a non array appears.
type Unwrap<T> = T extends readonly unknown[] ? Unwrap<T[number]> : T;

export function Flat<T>(value: T): Unwrap<T> {
  let current = value as unknown;
  while (Array.isArray(current)) current = current[0];
  return current as Unwrap<T>;
}
```

listing 123: a recursive conditional with infer (source: javascript/tssamples/ch17/typelevel.ts, lines 34-41)

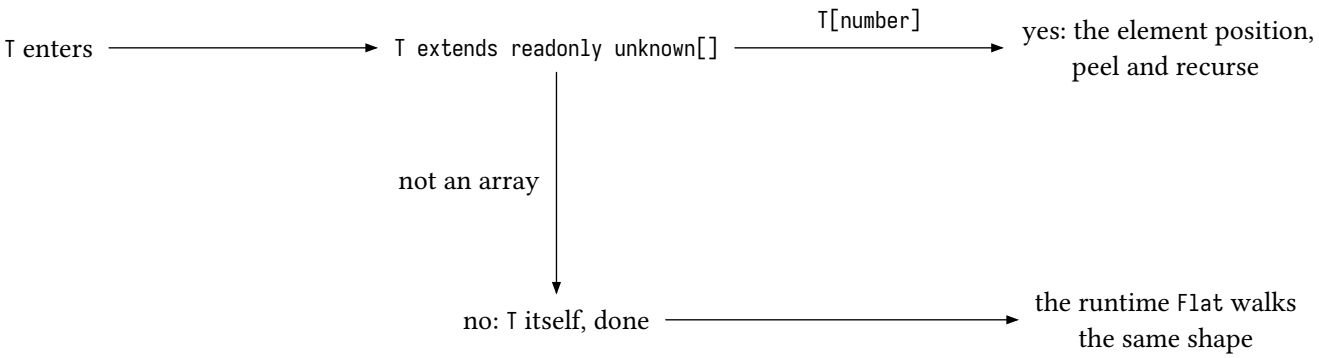


figure 91: the conditional type as routing, arrays peeled until they are not

A type alias may reference itself, but only through a deferring position, an object member or an array element, so the checker always has a smaller problem to solve before it needs the definition again. `Unwrap` defers through `T[number]`, and recursion plus `infer` is how the standard library writes its hardest utilities: `ReturnType<F>` infers a function's return, `Parameters<F>` its parameter tuple, and `Awaited<T>` is a recursive conditional that follows `then` chains of unknown depth to the value a promise eventually delivers.

17.4. template literal types

Template literal types compose strings at the type level. `Endpoint` below is four strings exactly, `"db.dev"` through `"web.prod"`, and a parameter of that type rejects every fifth. Patterns may embed `${string}` or `${number}` holes:

```
type Service = "db" | "web";
type Env = "dev" | "prod";
export type Endpoint = `${Service}.${Env}`;

export function EnvOf(endpoint: Endpoint): Env {
  return endpoint.split(".")[1] as Env;
}
```

listing 124: a domain composed from two unions (source: javascript/tssamples/ch17/typelevel.ts, lines 43-49)

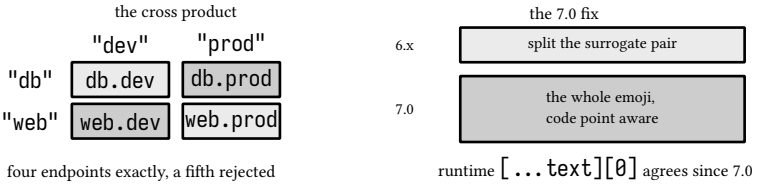


figure 92: a cross product of unions, and the 7.0 code point fix

NOTE: 7.0 makes template inference code point aware

In 6.x, `Head<"😀abc">` split the emoji's surrogate pair, yielding a broken half character. The native compiler preserves unicode code points, so the head of `"😀abc"` is `"😀"`. The runtime companion iterates code points through the string iterator, `[...text][0]`, which has agreed with the type system only since this release. The book's test pins both sides.

17.5. variadic tuples and the library operators

The variadic tuple type splices arrays inside another tuple type, `[...A, ...B]`, so `Concat` can promise the exact length and element order of its result. `PickKeys` pairs the library's `Pick` with the loop that copies the members, and `CapName` wraps the `Capitalize` intrinsic the mapped type already used:

```ts
// satisfies and const type parameters live in ch20's idiom suite
export function Concat<A extends readonly unknown[], B extends readonly unknown[]>(
  a: A,
  b: B,
): [...A, ...B] {
  return [...a, ...b];
}

export function PickKeys<T extends object, K extends keyof T>(
  obj: T,
  keys: K[],
): Pick<T, K> {
  const out: Partial<Pick<T, K>> = {};
  for (const key of keys) out[key] = obj[key];
  return out as Pick<T, K>;
}

export function CapName<S extends string>(name: S): Capitalize<S> {
  return (name.charAt(0).toUpperCase() + name.slice(1)) as Capitalize<S>;
}
```

listing 125: variadic splice, Pick, and an intrinsic wrapper (source: javascript/tssamples/ch17/typelevel.ts, lines 58-77)

`Pick` and its siblings `Omit`, `Record`, `Readonly`, `Required`, `Exclude`, and `Extract` are all built from the operators of this chapter. Two close cousins live one chapter ahead, because they are idioms rather than operators: `satisfies` checks a value against a contract without widening it, and a `const` type parameter keeps a literal's precision through a call, both chapter 20's.

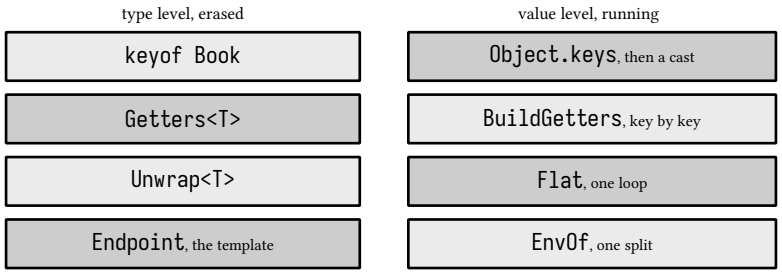


figure 93: the type level against the value level, four pairs from this chapter

The pairs are the chapter's whole claim. The left column is computation over types, finished before any code runs, and the right column is ordinary javascript from part one, correct on its own. The checker is the only thing that ever connects them, which is why the seam always shows up as a cast, and why a wrong promise on the left is caught at the boundary call site or not at all.

sources: typescriptlang.org handbook 2, mapped types, conditional types, and template literal types pages, devblogs.microsoft.com typescript 7.0 announcement for the unicode inference change, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 10 tests in ch17 green through `npm run verify`.

18. classes and decorators

The runtime half of the class keyword is part one's chapter 4: prototypes and the method chain, # private fields as runtime brands, static fields and static blocks, accessors, and the instanceof walk. Nothing here changes any of it, and the sample file deliberately repeats none of it. What this chapter owns is the type layer over that runtime: typed members, parameter properties, abstract, the polymorphic this type, generic classes, and the stage 3 decorator system. The one thing the class keyword still does not buy is nominal typing, the instance type stays structural like every other object type in chapter 15:

```
export class Bike extends Vehicle {
  #serial = "s1";
  constructor(private readonly owner: string) {
    super();
  }
  override wheels(): number {
    return 2;
  }
  get serial(): string {
    return this.#serial;
  }
  holder(): string {
    return this.owner;
  }
}
```

listing 126: parameter properties, ts private, and hash private in one class (source: javascript/tssamples/ch18/classes.ts, lines 12-26)

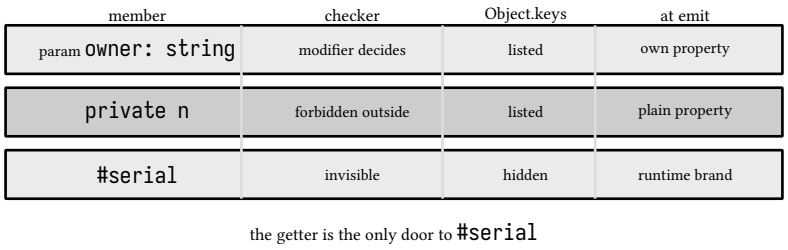


figure 94: three property systems, one bike, three visibilities

Parameter properties declare and assign in one place, and they still exist in tsc 7.0.2: the constructor line private readonly owner:
string compiles through this book's own project and emits an own data property named owner that Object.keys can see. The #serial field compiles to the runtime brand chapter 4 measured, invisible to Object.keys and unreachable by any cast, exposed only through the typed getter. The ts private modifier sits between them: forbidden by the checker, trivially present at runtime, the same erasure story chapter 11 proved against the emitted file.

18.1. abstract classes

abstract members must be overridden, and the override keyword makes the override explicit, catching renamed base members at compile time:

```ts
export abstract class Vehicle {
  abstract wheels(): number;
  describe(): string {
    return `vehicle with ${this.wheels()} wheels`;
  }
}
```

listing 127: an abstract base with one concrete requirement (source: javascript/tssamples/ch18/classes.ts, lines 5-10)

`abstract` is a checker fact only, and the sample proves it the rude way, constructing the abstract base through a cast and calling into the missing method:

```ts
// abstract is a checker fact only: the cast constructs the base
// class directly, and its describe call hits the missing method.
export function AbstractLie(): string {
  const Ctor = Vehicle as unknown as new () => Vehicle;
  const v: Vehicle = new Ctor();
  return v.describe();
}
```

listing 128: constructing the inconstructible (source: javascript/tssamples/ch18/classes.ts, lines 69-75)

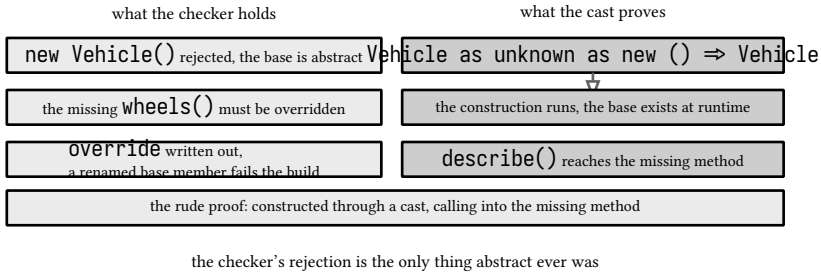


figure 95: abstract classes, a checker fact against the cast

18.2. this types

A method may return `this` as its type, and the meaning is polymorphic: through a `Loud` instance, `add` returns `Loud`, not the `Fluent` that declared it, so a chain that ends in a subclass method still compiles. A return type of the declaring class would break the chain, since `Fluent` has no `shout`:

```ts
// a this type keeps chained returns polymorphic: add on Loud
// returns Loud, not Fluent, so the subclass stays reachable
export class Fluent<T> {
  #items: T[] = [];
  add(item: T): this {
    this.#items.push(item);
    return this;
  }
  size(): number {
    return this.#items.length;
  }
}

export class Loud<T> extends Fluent<T> {
  shout(): string {
    return `${this.size()} items`;
  }
}
```

listing 129: a this typed chain that stays polymorphic through the subclass (source: javascript/tssamples/ch18/classes.ts, lines 28-45)

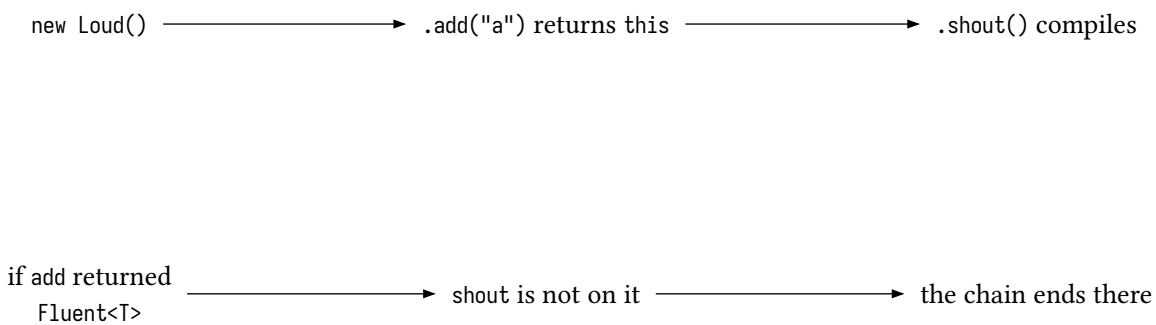


figure 96: one chain, two declarations, the subclass stays reachable

This is the class member's half of the checked `this`. Chapter 14 typed the receiver as a parameter, `this: void` on a free function, and here the same word works as a return type inside a class, one spelling, two positions, both erased.

18.3. generic classes and structure

A generic class carries its type parameter per instance, and a `Stack<T>` reads like the generic structures of the other books in this series. The structural reminder sits beside it: a plain object literal with `x` and `y` is a valid `Point2`, because instance types do not carry identity:

```ts
export class Stack<T> {
  readonly items: T[] = [];
  push(item: T): void {
    this.items.push(item);
  }
  pop(): T | undefined {
    return this.items.pop();
  }
  peek(): T | undefined {
    return this.items.at(-1);
  }
}

export class Point2 {
  constructor(readonly x: number, readonly y: number) {}
}

// class instance types stay structural: the literal satisfies Point2
export function MakePoint(p: { x: number; y: number }): Point2 {
  return p;
}
```

listing 130: a generic stack, and a literal satisfying a class type (source: javascript/tssamples/ch18/classes.ts, lines 47-67)

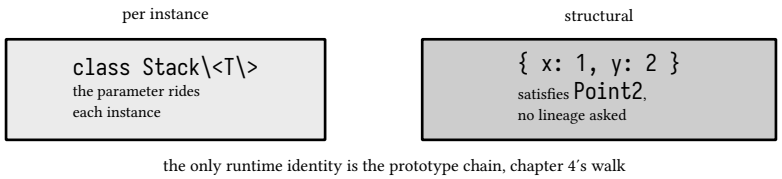


figure 97: the generic class, the structural instance type, and the abstract cast

18.4. decorators

Stage 3 decorators are functions receiving the decorated element and a context object carrying kind and name metadata. A class decorator returns the class, a method decorator returns a replacement, and both run once at class definition time, not per instance:

```
export const DecorationLog: string[] = [];

export function registered<C extends new (...args: never[]) => object>(
  value: C,
  context: ClassDecoratorContext,
): C {
  DecorationLog.push(context.name ?? "anonymous");
  return value;
}

export const CallLog: string[] = [];

export function counted(
  method: (n: number) => number,
  context: ClassMethodDecoratorContext,
): (this: unknown, n: number) => number {
  const name = String(context.name);
  return function (this: unknown, n: number): number {
    CallLog.push(`${name}(${n})`);
    return method.call(this, n);
  };
}
```

listing 131: a class decorator and a method decorator using their contexts (source: javascript/tssamples/ch18/classes.ts, lines 77-98)

```
@registered
export class Widget {}

export class Calc {
  @counted
  double(n: number): number {
    return n * 2;
  }
}

export class Meter {
  accessor value = 0;
}
```

listing 132: decorated class and method, and an auto accessor field (source: javascript/tssamples/ch18/classes.ts, lines 100-112)

The `accessor` keyword declares an auto accessor: one line of declaration, a backing store plus getter and setter at runtime. The descriptor on the prototype proves the member became an accessor pair rather than a data field. `@registered` pushes the class name into the log when `Widget` is defined, which is module evaluation time, so the test observes it without constructing anything. `@counted` wraps `double`, reads the method's name out of the context, and records each call.

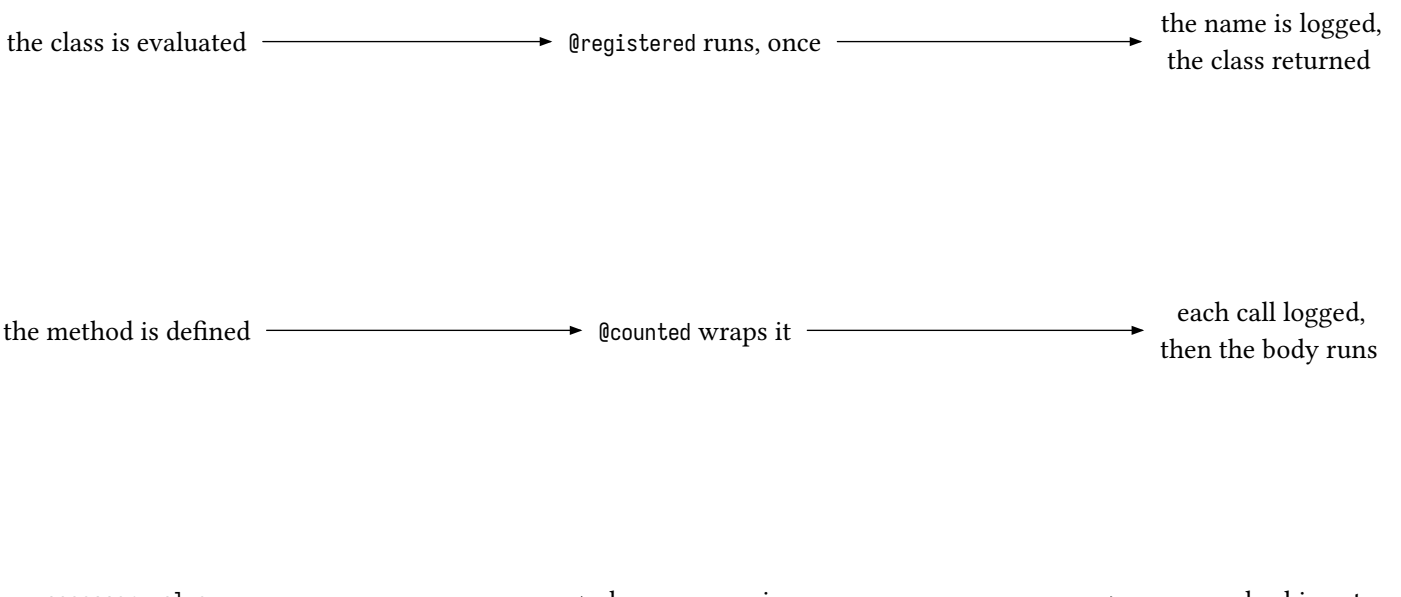


figure 98: decorators at definition time, wrappers at call time

NOTE: the measured status, 7.0.2 on this machine

Stage 3 decorators are the default and need no flag: the sample above compiles through the project's `tsc` with nothing but the book's tsconfig. The `experimentalDecorators` flag still exists in 7.0.2, it is not one of the removed options, and setting it selects the legacy one argument invocation: the same `registered` signature then fails with TS1238, which names the argument count. On the node side, 26.3.0 strip only mode rejects a decorated class with a plain `SyntaxError: Invalid or unexpected token`, without the typed `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX` diagnostic enums and parameter properties get, so decorated code must ride the `tsc` emit of chapter 19, exactly what this book's verify chain does.

18.5. implements and protected

`implements` closes the chapter the way it opens the type system: the class is checked against an interface, `public` is the default modifier written out, and `protected` reaches subclasses only, which the test file confirms by reading `prefix` through `Rush` and nowhere else:

```ts
export interface Runnable {
  run(): string;
}

// implements checks the class against the interface, public is the
// default modifier written out, and protected reaches subclasses
// only: Rush reads prefix, the test file cannot.
export class Task implements Runnable {
  protected prefix = "task:";
  public label = "sample";
  constructor(private readonly owner: string) {}
  run(): string {
    return `${this.prefix}${this.label}:${this.owner}`;
  }
}

export class Rush extends Task {
  rush(): string {
    return `${this.prefix}rush`;
  }
}
```

listing 133: implements, public, and protected in one hierarchy (source: javascript/tssamples/ch18/classes.ts, lines 114-134)

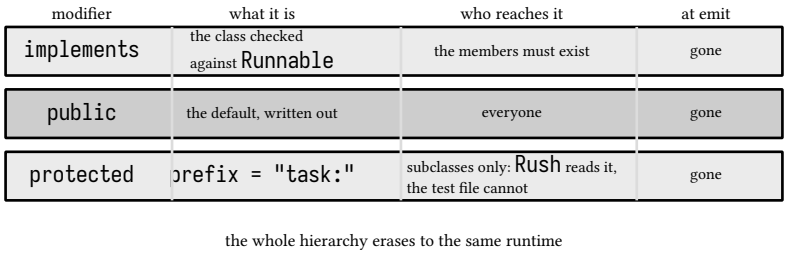


figure 99: implements, public, and protected, who reaches what

NOTE: what moved out of this chapter

The old classes chapter demonstrated static evaluation order, static blocks, and accessor pairs as runtime behavior. All of that is part one's chapter 4 now, measured on plain javascript, and the suite cases that pinned it went with it. What stays here is the declaration layer over that runtime.

sources: typescriptlang.org handbook classes and decorators pages, mdn class documentation, tc39 decorators proposal, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 11 tests in ch18 green through `npm run verify`. The decorator counterfactuals measured against the cli with `--ignoreConfig`, since 7.0 refuses a file argument beside a tsconfig otherwise: stage 3 clean with no flags, TS1238 under `--experimentalDecorators`, and the strip only `SyntaxError` reproduced on node 26.3.0 directly.

19. declarations, namespaces, and emit

Chapter 11 proved the founding asymmetry on one function: annotations are checked, then deleted. This chapter owns the seams around that fact. How the checker resolves and types the module graph part one's chapter 7 already runs, what a declaration file is and how to write one by hand for plain javascript, where the pre-module namespace still pays its way, and exactly what emit leaves behind, sourcemaps included. The parked chapter that used to carry erasure alone is dissolved into this one, and every claim below is checked against the emitted files in `dist`, not argued in prose.

19.1. resolution for the type layer

Typescript modules are ecmascript modules, one file is one module, and the checker reads the same graph node executes. The `nodenext` pair from chapter 11's tsconfig is what keeps the two honest: the checker resolves exactly the way node does, extensions included. The chapter's barrel uses every form at once:

```
import GreetDefault, { Casual, Formal } from "./greeter.js";
import { Metrics, Tree } from "./shapes.js";
import type { Lens } from "./shapes.js";
import bookData from "./data.json" with { type: "json" };

export type { Lens } from "./shapes.js";
export { Metrics, Tree } from "./shapes.js";
```

listing 134: default, named, type-only, attribute, and re-exports in one header (source: javascript/tssamples/ch19/modules.ts, lines 3-9)

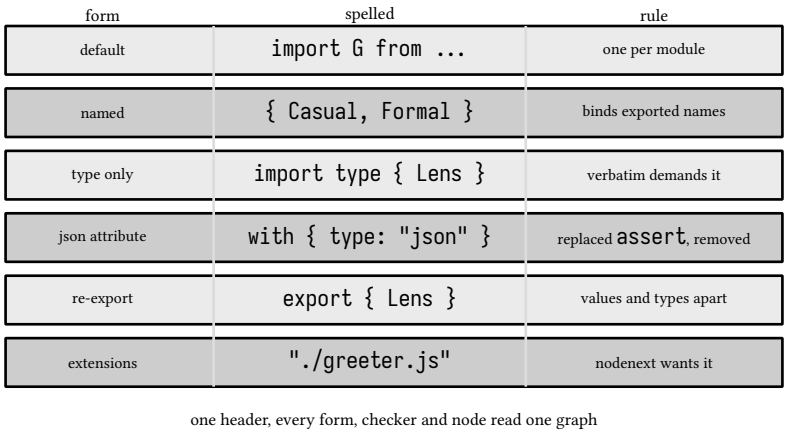


figure 100: every import form in one header, and the rule each one carries

Three details in that header carry the whole discipline. Imports name files with extensions, `./greeter.js`, because node wants the extension and the checker agrees. `import type` is not decoration: with `verbatimModuleSyntax` on, importing a type without the `type` modifier is an error, since the emitted statement would import a name that does not exist at runtime. And the json import carries an import attribute, `with { type: "json" }`, which replaced the older `assert` syntax that 7.0 removed. The re-exported type still needs its own local `import type` to be usable in the file doing the re-exporting, and dynamic import returns a promise of the module, which makes lazy loading an ordinary async function body:

```
export async function LoadGreeter(): Promise<string> {
  const mod = await import("./greeter.js");
  return mod.default("lazy");
}

export function MakeLens(): Lens {
  return { focus: 1 };
}
```

listing 135: dynamic import, and a re-exported type used locally (source: javascript/tssamples/ch19/modules.ts, lines 40-47)

19.2. namespaces against modules

Namespaces predate modules and group names under one emitted object. Nested namespaces nest objects, and members are exported from the namespace explicitly, exactly like a module:

```
export namespace Metrics {
  export function Area(w: number, h: number): number {
    return w * h;
  }
  export const Unit = "m2";
}

export namespace Tree {
  export const Root = "root";
  export namespace Branch {
    export const Leaf = 3;
  }
}
```

listing 136: nested namespaces emitting real objects (source: javascript/tssamples/ch19/shapes.ts, lines 3-15)

They remain the right tool in exactly one place, merging with an interface of the same name to attach helpers to a type, the declaration merging chapter 15 demonstrated on plain interfaces:

```
export interface Cargo {
  weight: number;
}

export namespace Cargo {
  export function Heavy(cargo: Cargo): boolean {
    return cargo.weight > 100;
  }
```

listing 137: interface and namespace merging into one entity (source: javascript/tssamples/ch19/modules.ts, lines 14-21)

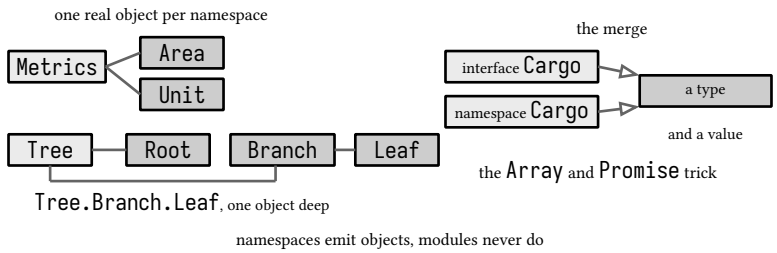


figure 101: namespaces as emitted objects, and the cargo merge

`Cargo` is now simultaneously a type, the interface, and a value, the namespace, which is how `Array` and `Promise` carry their static methods and how library types grow companion functions.

WARNING: the module keyword is gone from namespaces

7.0 rejects `declare module Foo { ... }` inside namespace declarations outright, the `module` keyword was the pre-ecmascript spelling of `namespace`. Ambient module declarations, `declare module "name"`, are unaffected, they describe a real module name.

19.3. hand written declarations

A declaration file, `.d.ts`, describes types with no implementation, and every `@types/node` import in this book is one, pulled in by the `types` array chapter 11 configured by hand since 7.0 defaults it to empty. The interesting case is authoring one. When the runtime is already plain javascript, part one's kind of file, the checker knows nothing about it until a declaration is placed beside it, and that declaration is written by hand. The boundary in miniature, three files:

```
// a plain node module: no types, no compiler, part one's javascript
export const version = 3;

export function title(id) {
  return `book-${id}`;
}
```

listing 138: the plain module, no types anywhere near it (source: javascript/tssamples/ch19/store.mjs, lines 1-7)

```
// hand written beside store.mjs: the checker's whole knowledge of
// that file. nothing here emits, nothing here runs
export declare const version: number;
export declare function title(id: string): string;
```

listing 139: the hand written declaration beside it, the checker's whole knowledge of that file (source: javascript/tssamples/ch19/store.d.mts, lines 1-5)

```
// the declaration boundary: this module calls the plain store.mjs,
// and every call is checked against the hand written store.d.mts
// beside it. the project build typechecks this file, and the test
// runs it under node's own type stripping, where ./store.mjs
// resolves beside it, because tsc emits nothing for the plain pair
import { title, version } from "./store.mjs";

export function StockTitle(id: string): string {
  return `${title(id)} v${version}`;
}
```

listing 140: a typed consumer, checked against the d.mts, run by node's stripping (source: javascript/tssamples/ch19/declarations.ts, lines 1-11)

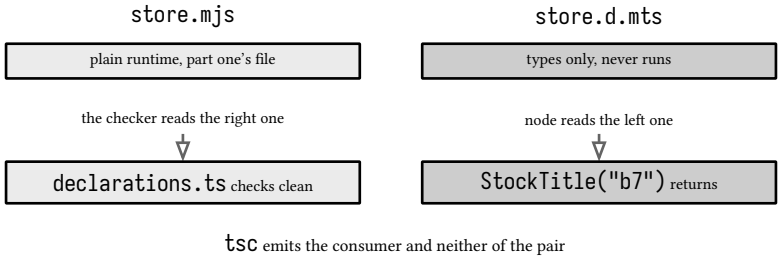


figure 102: the declaration boundary, who reads which file

The test runs the consumer under node's own type stripping, where `./store.mjs` resolves beside it, and the emitted `dist` proves the asymmetry the diagram claims: the compiled `declarations.js` lands in `dist`, while neither `store.mjs` nor `store.d.mts` rides the emit, so a compiled consumer ships with an import that only resolves beside the sources. The extension math is worth stating exactly: node maps `.mjs` to the `.d.mts` declaration, `.js` to `.d.ts`, and `.cjs` to `.d.cts`, and chapter 38's capstone uses this same move, a hand written `store.d.ts` over a plain `.mjs` store, to type its persistence layer.

```
// ambient.d.ts, for a dependency with no types of its own
declare module "legacy-markdown" {
  export function render(text: string): string;
}
```

fragment, illustrative, not executed

For a package rather than a relative file, the ambient `declare module "name"` block types every import of that name across the project, which is also what a `@types` package is: a folder of such files plus a package manifest. Declaration emit, `--declaration`, runs the other direction and generates a `.d.ts` beside every emitted file from the sources themselves. It is off by default, and the suite confirms no `erasure.d.ts` lands in `dist`.

19.4. what emit leaves behind

The audit module reads its own emitted file from `dist` and checks what the compiler left in and took out, building its search needles by concatenation so the emitted checker cannot match its own strings:

```
export enum Emitted { A, B }

export function Typed(words: readonly string[]): Map<string, number> {
  const counts: Map<string, number> = new Map();
  for (const word of words) counts.set(word, (counts.get(word) ?? 0) + 1);
  return counts;
}

const emittedPath = join(import.meta.dirname, "erasure.js");
const sourcePath = join(import.meta.dirname, "..", "..", "ch19", "erasure.ts");
const declarationsPath = join(import.meta.dirname, "..", "..", "ch19", "declarations.ts");

export function EmitSelfCheck(): { annotations: boolean; enumVar: boolean } {
  const text = readFileSync(emittedPath, "utf8");
  // the needles are concatenated so this checker's own emitted text
  // cannot contain them and match itself
  const annotations =
    text.includes(":" + " string") || text.includes(":" + " number");
  return { annotations, enumVar: /var Emitted/.test(text) };
}

export function EmitSizes(): { src: number; out: number } {
  return {
    src: readFileSync(sourcePath, "utf8").trim().split("\n").length,
    out: readFileSync(emittedPath, "utf8").trim().split("\n").length,
  };
}
```

listing 141: a module that audits its own compiled output (source: javascript/tssamples/ch19/erasure.ts, lines 10-36)

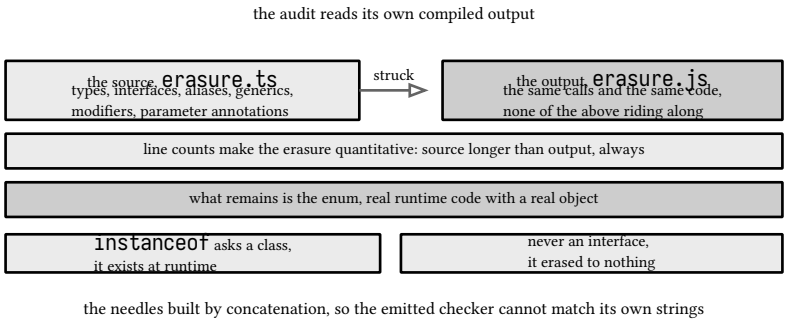


figure 103: what emit leaves behind, the struck against the survivor

Types, interfaces, aliases, generics, modifiers, and parameter annotations all vanish, and the line counts make the erasure quantitative, source longer than output, always. What remains is the enum, real code with a real object, because an enum is a runtime construct wearing type syntax. The same question, which promises have runtime shape, decides `instanceof`: it walks a prototype chain, part one's chapter 4 territory, so it can ask a class, which exists at runtime, and cannot ask an interface, which erased to nothing:

```
// instanceof walks a runtime prototype chain, so it can ask a
// class, which exists at runtime, and cannot ask an interface,
// which erased
export interface Shaped {
  area(): number;
}

export class Square implements Shaped {
  area(): number {
    return 1;
  }
}

export function ShapeGuards(shape: Shaped): [boolean, boolean] {
  return [shape instanceof Square, typeof shape.area === "function"];
}
```

listing 142: instanceof asks classes, never interfaces (source: javascript/tssamples/ch19/erasure.ts, lines 99-114)

```
const s: Shaped = new Square();
console.log(s instanceof Shaped);
// error TS2693: 'Shaped' only refers to a type, but is being
// used as a value here.
```

fragment, illustrative, not executed

This is why chapter 16's narrowing ladder leans on runtime facts, `typeof`, `in`, `Array.isArray`, real constructors, and why a shape that must be distinguished at runtime carries a discriminator or a brand rather than an interface name.

The one emit behavior that reaches into field declarations: since the es2022 target, and 7.0's target floor makes it the only behavior left, class fields install with `Object.defineProperty` semantics rather than assignment. The sample installs a throwing setter on a base prototype after declaration, invisible to the checker, then declares a field over it in a subclass:

```
// a prototype accessor installed after declaration, invisible to
// the checker's view of Plain
Object.defineProperty(Plain.prototype, "farewell", {
  get() {
    return "bye";
  },
  set() {
    throw new Error("prototype setter ran");
  },
});

class Chatty extends Plain {
  farewell = "hello";
}

class Sparse {
  declared: number | undefined;
}

// with es2022+ targets class fields use define semantics: Chatty's
// field is installed on the instance without invoking the prototype
// setter, which would throw, and Sparse's bare declaration still
// creates an own property. assignment semantics would do neither.
export function ClassFieldDefine(): { chatty: string; declared: string[] } {
  const chatty = new Chatty();
  return { chatty: chatty.farewell, declared: Object.keys(new Sparse()) };
}
```

listing 143: define semantics: the setter never runs, bare declarations still create properties (source: javascript/tssamples/ch19/erasure.ts, lines 116-142)

The field lands on the instance without invoking the prototype setter, which assignment semantics would have called, and even the bare `declared` field with no initializer creates an own property. The checker blocks the worst version of the trap outright: overriding an accessor with a field is error TS2610.

19.5. node's type stripping

Node 26 runs typescript files directly by stripping the erasable syntax itself, no compiler in the loop. The sample writes a probe with a `bigint` annotation, executes it with plain `node`, and gets
42. The same treatment rejects the audit module itself, because its

enum is real runtime code:

```
export function StripFixture(): { status: number | null; stdout: string; stderr: string } {
  const probe = join(tmpdir(), "jsbook-erasable.ts");
  writeFileSync(probe, "const x: bigint = 1n;\nconsole.log(Number(x) + 41);\n");
  const run = spawnSync(process.execPath, [probe], { encoding: "utf8", env: ProbeEnv });
  return { status: run.status, stdout: run.stdout, stderr: run.stderr };
}

// the same treatment rejects this file, because Emitted above is
// real runtime code, not a type.
export function StripRejectsEnum(): { status: number | null; stderr: string } {
  const run = spawnSync(process.execPath, [sourcePath], { encoding: "utf8", env: ProbeEnv });
  return { status: run.status, stderr: run.stderr };
}
```

listing 144: a stripped probe that runs, and the audit module which does not (source: javascript/tssamples/ch19/erasure.ts, lines 46-58)

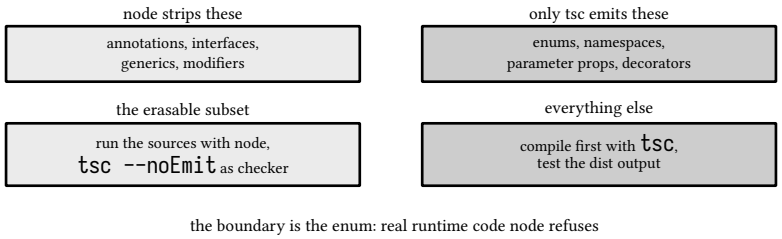


figure 104: the strip only subset, and the workflow split

The rejection names the enum specifically, `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX: TypeScript enum is not supported in strip-only mode`, and parameter properties get the same shaped diagnostic. Decorators do not even get that: measured on 26.3.0, a decorated class under stripping dies with a plain `SyntaxError: Invalid or unexpected token` at the `@`. Node strips, it does not transform, and there is no transform flag anymore.

NOTE: two runtimes, one language

Strip only mode defines a practical subset: if a codebase keeps to erasable syntax, it can run its sources directly with node and use `tsc --noEmit` as a pure checker, the modern library workflow. The moment it needs enum like emit, namespaces, parameter properties, or decorators, it must compile first, like this book's typescript samples do, and the declaration boundary above is the same trick in reverse, plain javascript that never compiles at all.

19.6. the emitted companions

With `sourceMap` on, every emitted file grows a `.map` companion whose `sources` point back at the typescript and whose `mappings` carry the position translation, which is why the verify script runs node with `--enable-source-maps` and test failures report source lines, not dist lines. The suite reads the map for this chapter's own emitted file and checks both facts. Declaration emit stays off, so a `.d.ts` is a thing this book writes by hand or reads from `@types`, never a build artifact, and the plain `store` pair from the boundary section rides no artifact at all.

```
export function SourceMapFacts(): { source: string; hasMappings: boolean } {
  const map = JSON.parse(readFileSync(`${emittedPath}.map`, "utf8")) as {
    sources: string[];
    mappings: string;
  };
  return {
    source: map.sources[0] ?? "",
    hasMappings: map.mappings.length > 0,
  };
}
```

listing 145: the sourcemap reader the test asserts against (source: javascript/tssamples/ch19/erasure.ts, lines 60-69)

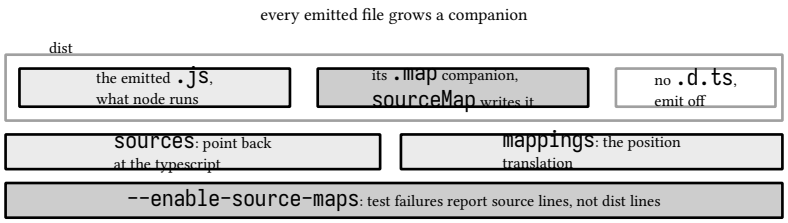


figure 105: the emitted companions, what lands in dist beside the javascript

sources: typescriptlang.org handbook modules, namespaces, and declaration files pages, tsconfig reference for declaration emit and erasable syntax, nodejs.org esm, import attributes, and type stripping documentation,

accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 18 tests in ch19 green through `npm run verify`, the declaration boundary and the strip mode probes executed end to end, the TS2693 counterfactual confirmed by `tsc --noEmit` before being shown as a fragment.

20. typescript idioms

The language gives structure no identity and the runtime no checks, so the idioms of typescript are the patterns that restore both at the boundaries where it matters, and nowhere else. Chapter 13 set the rule, `unknown` at the edges, ordinary types inside, and chapter 14 built the guard machinery, predicates, assertions, the discriminator. This chapter assembles those parts into the shapes a codebase actually keeps: branded ids, the exhaustiveness alarm, errors as values, deep freezing, state machines, typed events, and the two precision tools that arrived from chapter 17, `satisfies` and the `const` type parameter.

20.1. branding

A brand is a phantom member carrying a name in the type:

```
export type Brand<T, B extends string> = T & { readonly __brand?: B };

export type UserId = Brand<string, "UserId">;
export type OrderId = Brand<string, "OrderId">;

export function UserId(raw: string): UserId {
  return raw as UserId;
}

export function OrderId(raw: string): OrderId {
  return raw as OrderId;
}
```

listing 146: branded ids over one primitive (source: javascript/tssamples/ch20/idioms.ts, lines 4-15)

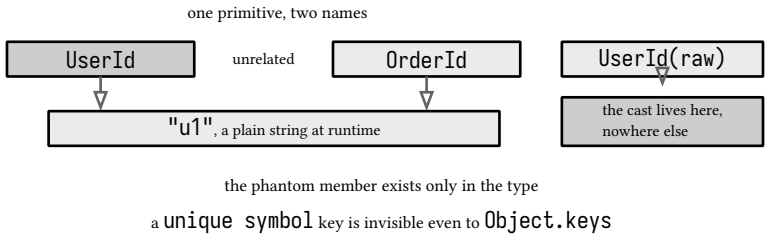


figure 106: the phantom member, two ids over one string

`UserId` and `OrderId` are both strings at runtime, the test confirms the erasure, but they are unrelated types to the checker, and a function demanding one rejects the other without a cast. This is the deliverable chapter 15's callout promised when it said identity is buyable: construction goes through the smart constructors, which are the only place the cast appears, and the boundary validation inside them is chapter 16's type predicate, so the branded type is earned by a runtime check exactly once. The alternative spelling uses a `unique symbol` key instead of a string one, invisible even to `Object.keys`, at the cost of one extra declaration.

20.2. the exhaustiveness alarm

Chapter 16 ended switches with a `never` default. The idiom extracts it into a helper, so every switch in a codebase fails the same loud way when a union grows:

```
export function assertNever(value: never): never {
  throw new Error(`unhandled: ${String(value)}`);
}
```

listing 147: the exhaustiveness helper (source: javascript/tssamples/ch20/idioms.ts, lines 17-19)

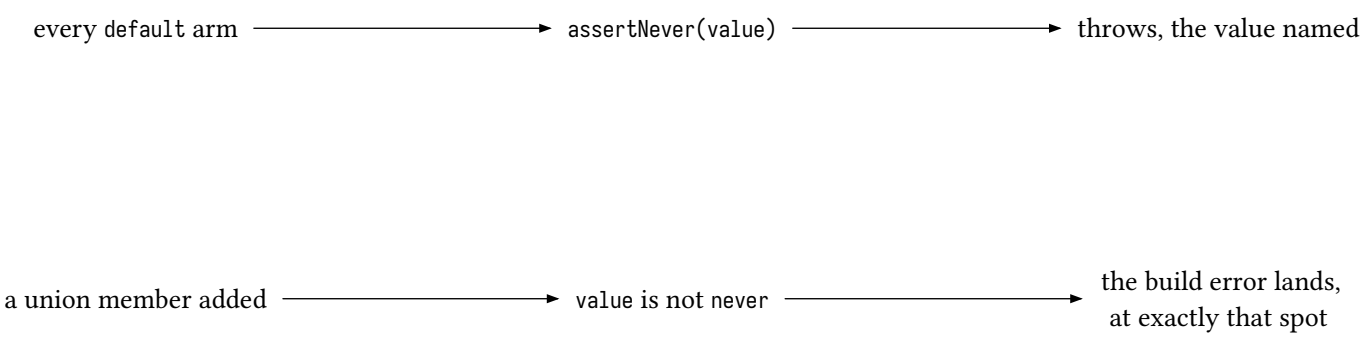


figure 107: assertNever as the alarm circuit

The rest of the boundary discipline is chapters 13 and 16's, restated as one sentence each because it is the sentence people skip: untrusted input arrives as `unknown` and leaves through a predicate or assertion, `any` is quarantined to the legacy file that forced it, and member access that has not been narrowed does not compile. No idiom in this chapter relaxes that, they all build on it.

20.3. errors as values

Throwing is for exceptional paths. For expected failures the idiom is a result union, a discriminated pair the caller must narrow before touching the payload:

```ts
export type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };

export function Ok<T>(value: T): Result<T, never> {
  return { ok: true, value };
}

export function Err<E>(error: E): Result<never, E> {
  return { ok: false, error };
}

export function TryInt(text: string): Result<number, string> {
  const parsed = Number(text);
  return Number.isInteger(parsed) ? Ok(parsed) : Err(`not an int: ${text}`);
}

export function ValueOr<T, E>(result: Result<T, E>, fallback: T): T {
  return result.ok ? result.value : fallback;
}
```

listing 148: result constructors, a parser, and a defaulting consumer (source: javascript/tssamples/ch20/idioms.ts, lines 21-40)

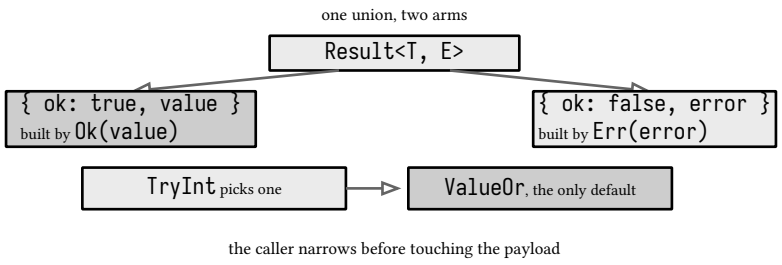


figure 108: the result union, two arms one contract

This is the same shape book 4, ch 7 (go 1.27, error handling) builds from multiple returns and the comma ok test, and it composes with every narrowing tool from chapter 16.

20.4. freezing deep

`readonly` stops at the first object. The deep version is a recursive mapped type over a recursive `Object.freeze`, type and runtime walking the same shape, and every operator in it is chapter 15's, `extends` for the object test, the mapped type for the rebuild, recursion through the member position:

```
export type DeepReadonly<T> = T extends object
  ? { readonly [K in keyof T]: DeepReadonly<T[K]> }
  : T;

export function DeepFreeze<T extends object>(value: T): DeepReadonly<T> {
  const record = value as Record<string, unknown>;
  for (const key of Object.keys(record)) {
    const inner = record[key];
    if (inner !== null && typeof inner === "object") {
      DeepFreeze(inner as object);
    }
  }
  return Object.freeze(value) as DeepReadonly<T>;
}
```

listing 149: DeepReadonly over DeepFreeze (source: javascript/tssamples/ch20/idioms.ts, lines 42-55)

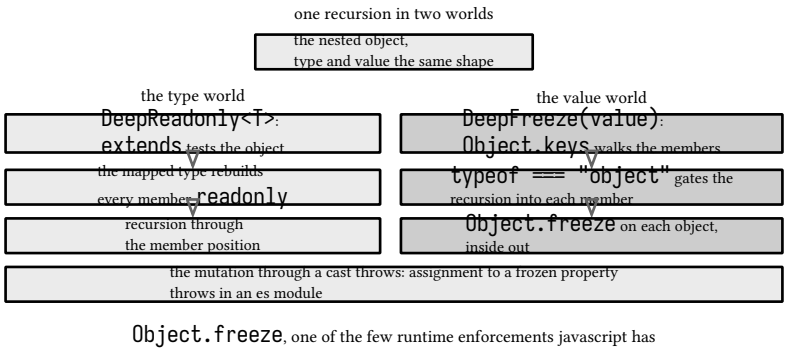


figure 109: freezing deep, one recursion in two worlds

The test mutates through a cast and gets a `TypeError`, because `Object.freeze` is one of the few runtime enforcements javascript has, and in an es module assignment to a frozen property throws.

20.5. state machines

A discriminated union of states plus a transition function is a finite state machine the checker audits: every state and event pair is either handled or it throws, and adding a state stops the build until every transition table row exists:

```ts
export type MachineState =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "loaded"; items: string[] }
  | { status: "failed"; error: string };

export type MachineEvent = "start" | "success" | "failure" | "reset";

function Invalid(state: MachineState, event: MachineEvent): never {
  throw new Error(`invalid ${state.status} + ${event}`);
}

export function Next(state: MachineState, event: MachineEvent): MachineState {
  switch (state.status) {
    case "idle":
      return event === "start" ? { status: "loading" } : Invalid(state, event);
    case "loading":
      if (event === "success") return { status: "loaded", items: ["a"] };
      if (event === "failure") return { status: "failed", error: "boom" };
      return Invalid(state, event);
    case "loaded":
    case "failed":
      return event === "reset" ? { status: "idle" } : Invalid(state, event);
  }
}
```

listing 150: states, events, and one exhaustive transition function (source: javascript/tssamples/ch20/idioms.ts, lines 57-81)

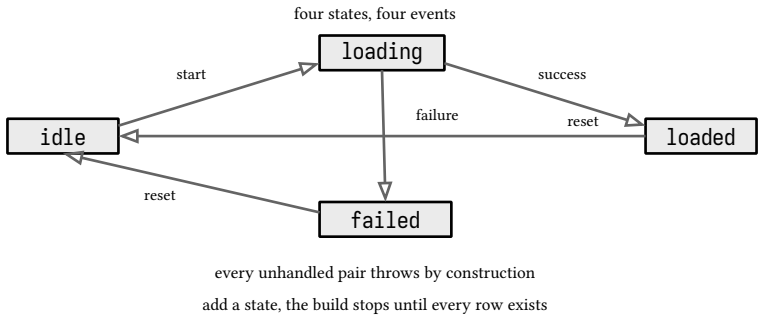


figure 110: the transition table as a machine, four states

20.6. typed events

An interface of tuple-valued keys parameterizes a pub sub, so `emit` and `on` agree on arity and types per event name, with no stringly typed dispatch anywhere. The map over `keyof E` is chapter 15's mapped type doing real work:

```ts
export interface OrderEvents {
  created: [id: string];
  shipped: [id: string, at: string];
}

export class Emitter<E extends { [K in keyof E]: unknown[] }> {
  #handlers: { [K in keyof E]?: ((...args: E[K]) => void)[] } = {};
  on<K extends keyof E>(event: K, handler: (...args: E[K]) => void): void {
    (this.#handlers[event] ??= []).push(handler);
  }
  emit<K extends keyof E>(event: K, ...args: E[K]): void {
    for (const handler of this.#handlers[event] ?? []) handler(...args);
  }
}

export function OrderLog(): string[] {
  const log: string[] = [];
  const bus = new Emitter<OrderEvents>();
  bus.on("created", (id) => log.push(`+${id}`));
  bus.on("shipped", (id, at) => log.push(`>${id}@${at}`));
  bus.emit("created", "o1");
  bus.emit("shipped", "o1", "t0");
  return log;
}
```

listing 151: a typed emitter and one wired consumer (source: javascript/tssamples/ch20/idioms.ts, lines 83-106)

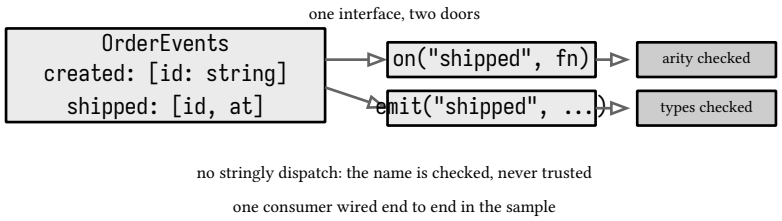


figure 111: the emitter, one interface behind both doors

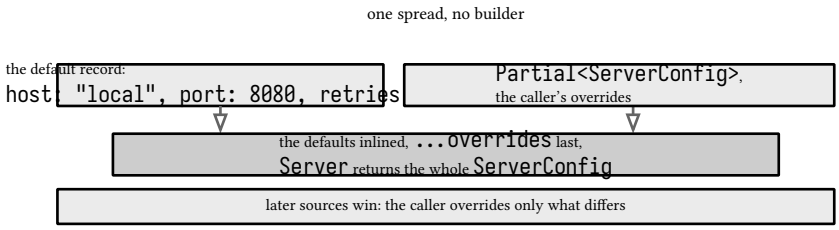
20.7. defaults

The functional options pattern in typescript is a `Partial<Config>` merged over defaults, one spread, no builder. `Partial` is chapter 17's one line mapped type, and the spread is part one's chapter 4:

```ts
export interface ServerConfig {
  host: string;
  port: number;
  retries: number;
}

export function Server(overrides: Partial<ServerConfig>): ServerConfig {
  return { host: "local", port: 8080, retries: 1, ...overrides };
}
```

listing 152: Partial overrides over a default record (source: javascript/tssamples/ch20/idioms.ts, lines 108-116)



`Partial` is chapter 17's one line mapped type, the spread is part one's chapter 4

figure 112: defaults, Partial over one spread, no builder

20.8. satisfies and const parameters

The two precision tools that moved here from chapter 17's operators. `satisfies` checks a value against a contract without widening it, and pairing it with `as const` keeps the literals while the assignment is still verified, `port` stays `8080`, not `number`, so `config.port + 1` is fine. A `const` type parameter does the same for function arguments, supported in tsc 7.0.2 and pinned by this suite: the rest parameter of `TupleOf` keeps its exact tuple type, length and literal elements included, so destructuring the result is precise:

```
// satisfies checks without widening, and a const type parameter
// keeps the argument's literal and tuple shape.
export const config = {
  host: "local",
  port: 8080,
} as const satisfies Record<string, string | number>;

export function TupleOf<const T extends readonly unknown[]>(...items: T): T {
  return items;
}
```

listing 153: satisfies plus as const, and a const type parameter (source: javascript/tssamples/ch20/idioms.ts, lines 126-135)

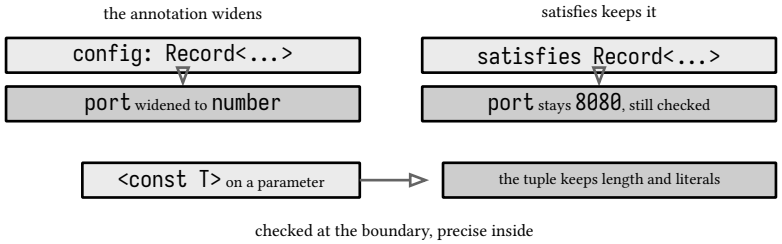


figure 113: widening against satisfies, literals kept while checked

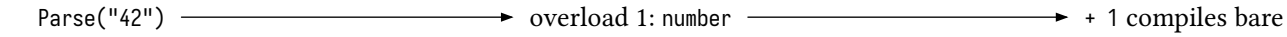
20.9. overloads against unions, and generics economy

Chapter 14 taught the overload mechanics, one implementation under signatures callers see, and the default is still a union: most functions with several input shapes want `input: string | readonly string[]` and one body, because the caller who narrows the input can narrow the output. The overload earns its place exactly when the input shape decides the output shape, and no union signature can say that:

```ts
// the one overload that earns its place: the input shape decides
// the output shape, which a plain union signature cannot say
export function Parse(input: string): number;
export function Parse(input: readonly string[]): number[];
export function Parse(input: string | readonly string[]): number | number[] {
  return Array.isArray(input) ? input.map(Number) : Number(input);
}
```

listing 154: the one overload that earns its place, input shape deciding output shape (source: javascript/tssamples/ch20/idioms.ts, lines 118-124)

With the two overloads, `Parse("42")` is `number` and arithmetic on it compiles bare, while `Parse(["1", "2"])` is `number[]` and has a length. With the union signature alone, both callers would own `number | number[]` and narrow it themselves, so the overloads spend two lines to delete two guards at every call site.



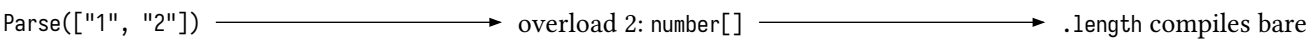


figure 114: one body, two promises, the caller's guards deleted

The same economy question governs type parameters. Chapter 14's rule, a generic must connect two positions or license member access through a constraint, is the whole test, and most violations fail it by being casts in disguise:

```ts
export function parse<T>(text: string): T {
  // compiles, and that is the problem: T connects nothing,
  // it is a double cast the caller gets to aim
  return JSON.parse(text) as T;
}
```

fragment, illustrative, not executed

A generic that appears once in a signature buys no checking, hides the real return type from inference, and pushes a cast onto every caller. Write the concrete type, `unknown` plus a chapter 16 predicate when the value is untrusted, or a real parameter that relates two positions the way `Parse` relates input to output.

sources: typescriptlang.org handbook more on functions, satisfying the contract with satisfies, and const type parameters pages, the community branding and result type conventions, accessed 2026-09-13. Behavior verified live with tsc 7.0.2 and node 26.3.0 on windows, 12 tests in ch20 green through `npm run verify`, const type parameter support confirmed by the compiling suite itself.

21. react 19: components as functions

Part three opens where part two closed, with a typed program that runs, and asks the question the first twenty chapters deferred: what happens when the target is a browser. The answer this chapter and the next build on is react 19.3.0, measured on this machine, installed into a fourth npm project, `javascript/fwsamples/`, beside the samples, tssamples, and capstone projects. Nothing about the runtime changes. React is a library the module system of chapter 7 imports like any other, its components are functions part two could have typed, and the one new tool in the layer, vite, enters exactly once per build, never per run: `node --test` stays the only runner.

The project pins `react` and `react-dom` at 19.3.0 and `vite` at 8.3.0, probed 2026-09-21, with `@sveltejs/vite-plugin-svelte` 7.3.0 waiting for chapter 22. The verify chain adds a third stage between compile and test: `tsc -p` typechecks the `.tsx` sources, the vite builds produce the bundles the assertions read, then node runs the unit tests against the emitted `dist` plus smoke tests that import the built output. This chapter's slice is 31 unit tests and 6 smokes, 37 in all, green through `npm run verify`.

21.1. a function that returns elements

A react component is a function from props to elements. That is the whole definition, no decorator, no base class, no registration, and the type layer of part two describes it with an ordinary interface:

```
}

export function Stat({ label, value }: { label: string; value: ReactNode }) {
  return (
    <p className="stat">
      <span className="stat-label">{label}</span>{" "}
      <strong>{value}</strong>
    </p>
  );
}

export function DocTable({ docs, children }: { docs: Doc[]; children?: ReactNode }) {
  return (
    <section className="doc-table">
      <table>
        <thead>
          <tr>
            <th>title</th>
            <th>pages</th>
            <th>tags</th>
          </tr>
        </thead>
        <tbody>
          {docs.map((doc) => (
            <DocRow key={doc.id} doc={doc} />
          ))}
        </tbody>
      </table>
      {children}
    </section>
  );
}

function DocRow({ doc }: { doc: Doc }) {
  return (
    <tr>
      <td>{doc.title}</td>
      <td>{doc.pages}</td>
      <td>{doc.tags.join(", ")}</td>
    </tr>
  );
}
```

listing 155: one component per concern, props in, elements out, keys where lists render (source: javascript/fwsamples/ch21-react/components.tsx, lines 11-52)

`Stat` takes two props and returns a tree. `DocTable` takes `docs` and `children`, and `children` is not syntax, it is a prop the caller fills by nesting. The `key` on each row is react's reconciliation hint, stable identity across renders, and it never reaches the dom. Below them the file keeps one class component on purpose:

```
// the class form, for contrast only. react 19 still runs class
// components and error boundaries still need them, but nothing else
// in this chapter is written as a class.
export interface ToggleState {
  open: boolean;
}

export class LegacyToggle extends Component<{ label: string }, ToggleState> {
  state: ToggleState = { open: false };

  toggle = () => {
    this.setState((prev) => ({ open: !prev.open }));
  };

  render() {
    const { label } = this.props;
    return (
      <button type="button" onClick={this.toggle} aria-expanded={this.state.open}>
        {label}: {this.state.open ? "open" : "closed"}
      </button>
    );
  }
}
```

listing 156: the class form still runs in 19, kept here for contrast and for the boundary pattern later (source: javascript/fwsamples/ch21-react/components.tsx, lines 54-76)

React 19 did not remove class components. It removed almost every reason to write one, and the two survivors are visible in this chapter: the demo above, and the error boundary of the sixth section, which in 19.3.0 still has no function-component equivalent. The suite renders the class's first paint through the server and moves on, because everything else here is a function.

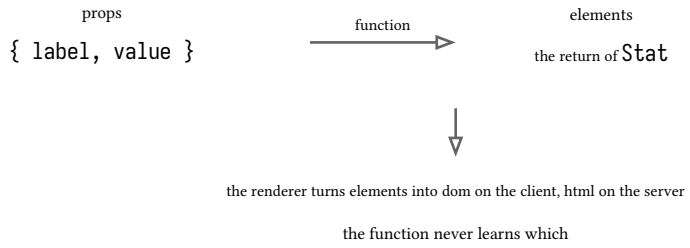


figure 115: props to elements, one direction

21.2. hooks and their rules

State lives in hooks because a function component reruns whole. The useState call returns a value from the run's own slot, and the rules exist because the slots are positional, three of them, calls at the top level, unconditional, same order every run. A custom hook is only a function whose name starts with use and whose body follows the rules, and it moves logic between components the way chapter 3 moved any other function:

```tsx
// a custom hook: ordinary function, hooks inside, reusable across
// components. the rules bind the caller too, no conditional hook
// calls around it either.
export function useSearch(docs: Doc[], initial = "") {
  const [term, setTerm] = useState(initial);
  const results = useMemo(() => filterDocs(docs, term), [docs, term]);
  return { term, setTerm, results };
}

export function SearchPanel({ docs, initial = "" }: { docs: Doc[]; initial?: string }) {
  const { term, setTerm, results } = useSearch(docs, initial);
  return (
    <div className="search">
      <label>
        search
        <input value={term} onChange={(event) => setTerm(event.target.value)} />
      </label>
      <DocTable docs={results}>
        <p>{results.length} of {docs.length} documents</p>
      </DocTable>
    </div>
  );
}
```

listing 157: the custom hook, state plus memoized derivation behind one call (source: javascript/fwsamples/ch21-react/hooks.tsx, lines 19-42)

`filterDocs` stays a pure function outside the component so the suite drives it without a renderer, and the hook composes it with `useState` and `useMemo`. The panel renders its initial state through the server, `value="systems"`, two of three documents, and the effect-shaped parts of react are simply absent here: the server never runs effects, which is why this chapter's oracle can say nothing about them and says so.

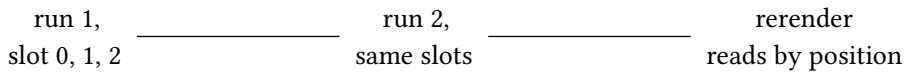


figure 116: the rules as one invariant

Break the order, a conditional call between renders, and run 2 reads slot 2 where run 1 left a different hook's state. The rule is not etiquette, it is the addressing scheme.

21.3. derive first, memo on evidence

Two tools that look interchangeable and are not. Derivation computes from props during render, `const sorted = sortByPages(docs)`, and it cannot go stale because it is recomputed every run. Memoization keeps a cached copy keyed on dependencies and pays only when the same component rerenders with unchanged inputs. The file counts the factory runs so the claim is measured, not argued:

```tsx
export let sortCalls = 0;

export function resetSortCalls() {
  sortCalls = 0;
}

function sortByPages(docs: Doc[]): Doc[] {
  sortCalls += 1;
  return docs.toSorted((a, b) => a.pages - b.pages);
}

// derivation: computed during render, no hook, no cache, correct by
// construction because it cannot go stale
export function DerivedList({ docs }: { docs: Doc[] }) {
  const sorted = sortByPages(docs);
  return (
    <ul>
      {sorted.map((doc) => (
        <li key={doc.id}>{doc.title}: {doc.pages}</li>
      ))}
    </ul>
  );
}

// memoized derivation: the factory runs only when the inputs move,
// which pays on a re render with unchanged deps, never on a first
// render
export function MemoList({ docs }: { docs: Doc[] }) {
  const sorted = useMemo(() => sortByPages(docs), [docs]);
  return (
    <ul>
      {sorted.map((doc) => (
        <li key={doc.id}>{doc.title}: {doc.pages}</li>
      ))}
    </ul>
  );
}
```

listing 158: the same derivation, plain and memoized, one shared counting factory (source: javascript/fwsamples/ch21-react/derived.tsx, lines 10-46)

Two `renderToString` calls run the factory twice in both columns, one per render, because memo has no earlier run to reuse on a first paint. The server oracle pins the floor: memo changes nothing until a rerender happens, `React.memo` on a component is the same trade one level up, skipping a subtree on shallow-equal props, and the honest order of operations is derive, measure, then memo the leaf that measured expensive.

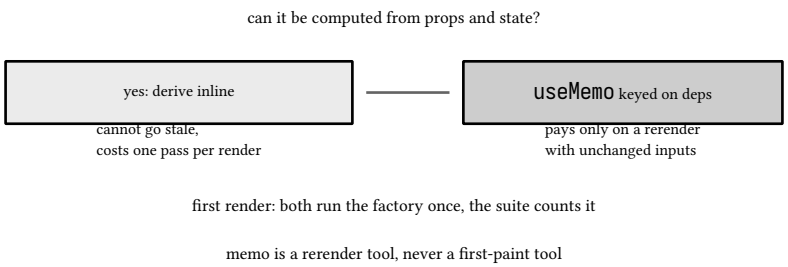


figure 117: the decision, derive against memoize

21.4. what 19 changed

The 19 specifics were probed against the installed `@types/react` 19.3.0 before this section was written. `ref` arrives as an ordinary prop, so the wrapper component `forwardRef` is no longer needed, it still exists in 19.3.0, superseded rather than removed, and the sample spells the new form directly:

```
// ── ref as a prop ──────────────────────────────────────────

export function FancyInput({
  label,
  ref,
}: {
  label: string;
  ref?: Ref<HTMLInputElement>;
}) {
  return (
    <label>
      {label}
      <input ref={ref} name={label} />
    </label>
  );
}

// ── use() with a context ───────────────────────────────────

export const ThemeContext = createContext<string>("light");

export function ThemedLabel({ children }: { children: ReactNode }) {
  const theme = use(ThemeContext);
  return <p data-theme={theme}>{children}</p>;
}

// ── use() with a promise ───────────────────────────────────

// the honest limit, measured in the suite: under the synchronous
// renderToString even an already settled promise suspends, use with
// a promise wants the streaming renderer or react's own server
// bookkeeping, so this component is a probe rather than a pattern
export function SettledValue({ promise }: { promise: Promise<number> }) {
  const value = use(promise);
  return <p>value {value}</p>;
}
```

listing 159: ref as a prop, use with a context, and use with a promise as the probe that fails (source: javascript/fwsamples/ch21-react/react19.tsx, lines 18-53)

`use()` stands in for `useContext` and reads either a context or a promise. With a context it unwraps synchronously, provider or default, both rendered by the suite. With a promise it is the honest measured limit: under the synchronous `renderToString`, even an already settled `Promise.resolve(42)` suspends and the render throws `A component suspended`, because a plain promise carries no status react can read and the sync renderer cannot wait. Promise reading belongs to the streaming renderer or to react's own server bookkeeping, and this book's oracle says so instead of demonstrating a pattern it cannot run.

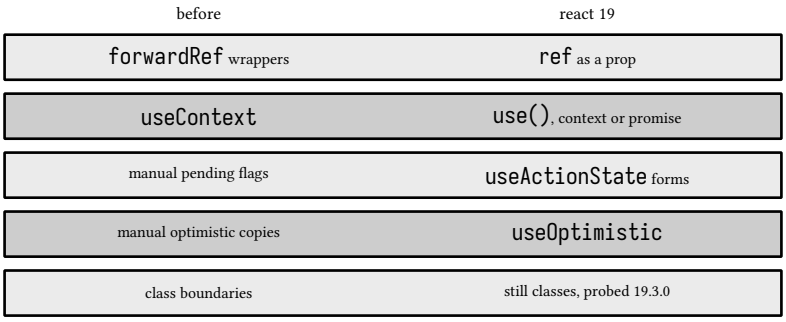


figure 118: the 19 surface, what replaced what

21.5. forms as actions

A form action in 19 is a function, and the function receives the form's own `FormData`, not synthetic arguments. `useActionState` wires the transition to the submit, hands back state, a dispatch, and a pending flag, and the transition function stays pure so the suite drives it with a constructed `FormData` without a form anywhere:

```
// ── form actions, useActionState ─────────────────────────────

// the transition function is pure so the suite drives it directly,
// the same reducer the hook wires to the form below. the payload is
// the form's own FormData, the 19 spelling, the action reads its
// fields instead of receiving synthetic arguments
export interface SearchState {
  query: string;
  log: string[];
}

export async function searchAction(
  state: SearchState,
  form: FormData,
): Promise<SearchState> {
  const query = String(form.get("q") ?? "");
  if (query === "") {
    throw new Error("empty query");
  }
  return { query, log: [...state.log, query] };
}

export function SearchForm({ initial }: { initial: SearchState }) {
  const [state, submit, pending] = useActionState(searchAction, initial);
  return (
    <form action={submit}>
      <input name="q" defaultValue={state.query} />
      <button disabled={pending}>{pending ? "searching" : "search"}</button>
      <output>{state.log.length} searches</output>
    </form>
  );
}
```

listing 160: the action reads its fields from FormData, the hook owns pending (source: javascript/fwsamples/ch21-react/react19.tsx, lines 55-86)

The server render of that form is a measured fact worth reading closely, because it is react's own honesty about the seam: the action attribute renders as a guard, `javascript:throw new Error('React form unexpectedly submitted.')`, and a replay script follows the form so a submit that races hydration is captured

and replayed instead of silently navigating. The suite asserts the guard string and the marker comments react leaves between adjacent text nodes, `2<!-- -->` searches, which exist so client hydration does not merge text nodes the server laid out separately.

`useOptimistic` layers the same shape over a vote panel: the optimistic reducer applies the vote immediately, the real action's landed state replaces it when it settles, and the initial render shows the passthrough state because there is nothing pending yet:

```
// ── useOptimistic ──────────────────────────────────────────

export interface VoteState {
  score: number;
}

// the optimistic reducer applies the vote immediately, the real
// action's state replaces it when it lands
export function voteReducer(state: VoteState, vote: number): VoteState {
  return { score: Math.max(0, state.score + vote) };
}

export async function castVote(state: VoteState, vote: number): Promise<VoteState> {
  if (vote === 0) {
    throw new Error("zero vote");
  }
  return voteReducer(state, vote);
}

export function VotePanel({ initial }: { initial: VoteState }) {
  const [real, submit] = useActionState(castVote, initial);
  const [shown, apply] = useOptimistic(real, voteReducer);
  const formRef = useRef<HTMLFormElement>(null);

  function onSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    apply(1);
    void submit(1);
  }

  return (
    <form ref={formRef} onSubmit={onSubmit}>
      <output>score {shown.score}</output>
    </form>
  );
}
```

listing 161: the optimistic reducer pure and exported, the panel composing both hooks (source: javascript/fwsamples/ch21-react/react19.tsx, lines 88-123)



figure 119: one vote, two states

21.6. boundaries catch, but not here

The error boundary is still a class component in 19.3.0, `getDerivedStateFromError` to choose the fallback and `componentDidCatch` to observe, and its honest limit is measured in the suite rather than waved at: server rendering catches nothing. `renderToString` over a boundary-wrapped throwing component lets the throw escape to the caller,

both boundary methods are client lifecycle, so the server error path belongs to the surrounding server, which is what the capstone's `server.mjs` will do with a `try` around a render:

```
export interface BoundaryState {
  error: Error | null;
}

export class ErrorBoundary extends Component<{ children?: ReactNode; fallback: ReactNode }, BoundaryState>
{
  state: BoundaryState = { error: null };

  static getDerivedStateFromError(error: Error): BoundaryState {
    return { error };
  }

  componentDidCatch(error: Error) {
    // observation point only, the fallback decision already happened
    console.error(`boundary caught: ${error.message}`);
  }

  render() {
    if (this.state.error =/= null) {
      return this.props.fallback;
    }
    return this.props.children;
  }
}

export function Boom({ message }: { message: string }): never {
  throw new Error(message);
}
```

listing 162: the boundary class, and the component that throws on purpose (source: javascript/fwsamples/ch21-react/boundary.tsx, lines 7-34)

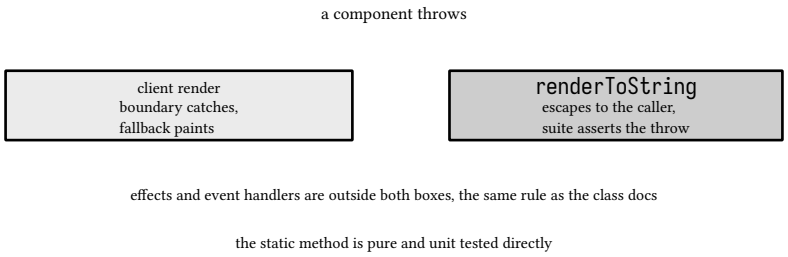


figure 120: where a throw is caught

21.7. the island seam

The browser integration point this book commits to is not an application, it is an island: a custom element whose lifecycle owns a react root, `connectedCallback` mounts, `disconnectedCallback` unmounts. The wrapper that implements it is deliberately framework free, it knows the lifecycle mapping and nothing else, and the browser globals plus the real `createRoot` are injected by the caller, which is what lets the node suite drive the contract with fakes:

```ts
// one root per connected host, created on first connect, rendered
// once, unmounted on disconnect, and a reconnect mounts fresh rather
// than reusing a dead root
export function islandHooks<Base extends object>(
  make: () => unknown,
  createRoot: (host: Base) => IslandRoot,
): IslandHooks<Base> {
  const roots = new WeakMap<Base, IslandRoot>();
  return {
    connect(host) {
      let root = roots.get(host);
      if (root === undefined) {
        root = createRoot(host);
        roots.set(host, root);
      }
      root.render(make());
    },
    disconnect(host) {
      roots.get(host)?.unmount();
      roots.delete(host);
    },
  };
}

export function defineIsland<Base extends object>(
  tag: string,
  make: () => unknown,
  env: IslandEnvironment<Base>,
  createRoot: (host: Base) => IslandRoot,
): void {
  const hooks = islandHooks(make, createRoot);
  // the class extends whatever base the environment provides. the
  // cast through a plain constructor type is the honest shape: the
  // checker cannot prove an arbitrary generic base stays compatible
  // with the lifecycle methods added here, the runtime never cares
  const base = env.base as new () => object;
  class Island extends base {
    connectedCallback() {
      hooks.connect(this as unknown as Base);
    }
    disconnectedCallback() {
      hooks.disconnect(this as unknown as Base);
    }
  }
  env.registry.define(tag, Island as unknown as new () => Base);
}
```

listing 163: one root per connected host, weakly held, unmounted on disconnect (source: javascript/fwsamples/ch21-react/wrap.ts, lines 22-67)

The rules the wrapper keeps are the ones the fakes count: a root is created once per connected host, a reconnect after an unmount mounts a fresh root because the old one is dead, and disconnect unmounts exactly once. The browser entry is three lines of injection:

```
defineIsland(
  "x-react-panel",
  () => createElement(QueryPanel, { docs }),
  { base: HTMLElement, registry: customElements },
  (host: HTMLElement): IslandRoot => createRoot(host),
);
```

listing 164: the browser entry, globals and root handed to the framework-free wrapper (source: javascript/fwsamples/ch21-react/island.tsx, lines 17-22)

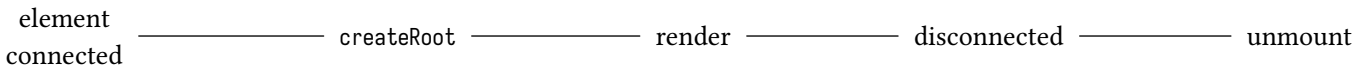


figure 121: the island lifecycle

This file is browser only and the node suite never imports it. Vite is the tool that turns it into something a script tag can load, and that is the whole of vite's role in this book: `vite build` bundles `island.tsx` with react inside it into one self-contained esm file, 861.84 kB minified, 218.69 kB gzip, measured 2026-09-21, machine checked by a smoke that reads the bytes, asserts no import statement survives, and leaves the number where the comparison with chapter 22 can find it.

21.8. the server as oracle

Every render claim in this chapter was checked by `react-dom/server`'s `renderToString` against code that exists in two compiled forms: the tsc emit the unit tests import, and the vite `--ssr` bundle the smokes import. The second form is the oracle that matters, because it is the bundler's own output, dependencies externalized exactly as a server would resolve them, and the smoke renders the island component out of the built barrel:

```
describe("ch21 vite ssr bundle", () => {
  it("renders the island component out of the built barrel", () => {
    const html = renderToString(createElement(QueryPanel, { docs }));
    assert.match(html, /data-framework="react"/);
    assert.match(html, /documents/);
    assert.match(html, /<td>lua<\/td>/);
  });
```

listing 165: the smoke, react-dom/server over the vite ssr bundle (source: javascript/fwsamples/ch21-react/render.smoke.mjs, lines 23-29)

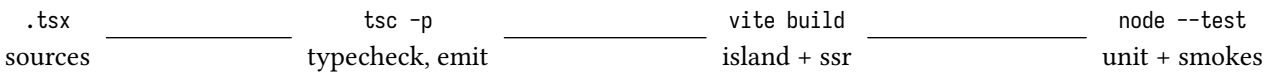


figure 122: the chapter verify chain

The one config file builds both ways, `vite build` for the browser island and `vite build --ssr` for the oracle barrel, switching entry and output directory on the build mode, and the emptyOutDir flag stays off because all the chapter builds share one `dist` the verify script clears once up front:

```
import { defineConfig } from "vite";

export default defineConfig(({ isSsrBuild }) => ({
  build: {
    outDir: isSsrBuild ? "dist/vite/react-ssr" : "dist/vite/react-island",
    emptyOutDir: false,
    lib: {
      entry: isSsrBuild ? "ch21-react/render.ts" : "ch21-react/island.tsx",
      formats: ["es"],
    },
    rollupOptions: {
      output: { entryFileNames: "[name].js" },
    },
  },
}));
```

listing 166: one config, two builds, the mode switch in three lines (source: javascript/fwsamples/ch21-react/vite.config.ts, lines 7-21)

21.9. the honest trade

React's account of a ui is a function rerunning whole, a tree diffed against the previous run, and a runtime that owns scheduling, reconciliation, and hydration. The costs and gains both follow from that shape. The measured cost on this machine: an island whose component is 29 lines ships 861.84 kB minified because the runtime travels with it, and the gzip line, 218.69 kB, is the number a network actually moves. The gains are equally concrete: the whole program is ordinary typed javascript, any state shape renders, the ecosystem is the largest of the three frameworks, and the model survives contact with teams.

The next chapter compiles the same island shape with svelte 5 and measures the same bundle, and the numbers will not be close. The argument that keeps react honest here is not size, it is that the book's own oracle, `renderToString`, the text markers, the form guard, the suspended `use`, reads react's model back to the caller with unusual candor, and a tool that tells you its seams is one you can keep on the critical path.

sources: react.dev reference pages for components, hooks, `use`, useActionState, useOptimistic, error boundaries, and custom elements, react.dev blog react 19 release post, vite.dev guide and library mode docs, accessed 2026-09-21. Verified live with react 19.3.0, react-dom 19.3.0, vite 8.3.0, and tsc 7.0.2 on node 26.3.0, windows, the chapter's 37 tests green through `npm run verify` in `javascript/fwsamples`, bundle sizes read from the vite build output 2026-09-21.

22. svelte 5: runes and compiled reactivity

React ships a runtime and reruns it on every change. Svelte 5.57.1, the version this chapter measures, ships a compiler that reads the same components and emits imperative updates, a write to `count` becomes an assignment that touches exactly the dom nodes reading it, and the reactive words that drive it are six runes, compiler instructions that look like globals and compile away. The chapter lives in the same `javascript/fwsamples/` project as chapter 21, one `@sveltejs/vite-plugin-svelte` 7.3.0 config file per concern, and the chapter's slice of the suite is 16 tests, 9 against compiled reactivity imported by node and 7 against the built output.

The part one discipline holds: no framework magic the book cannot open. The runes compile to signals, the suite drives the compiled modules directly, the same `renderToString` style oracle runs through `svelte/server`, and the island bundle is measured as bytes beside react's.

22.1. a compiler, not a runtime

The component is a script plus markup, no function returns a tree, and the reactive declarations are instructions to the compiler:

```
  }

  interface Props {
    title?: string;
    docs: Doc[];
    onselect?: (id: string) => void;
    children?: Snippet;
    row?: Snippet<[Doc, string]>;
  }

  let { title = "svelte island", docs, onselect, children, row }: Props = $props();

  let term = $state("");

  let results = $derived(
    term.trim() === ""
      ? docs
      : docs.filter(
          (doc) =>
            doc.title.toLowerCase().includes(term.trim().toLowerCase()) ||
            doc.tags.some((tag) => tag.toLowerCase().includes(term.trim().toLowerCase())),
        ),
  );

  let pages = $derived(results.reduce((sum, doc) => sum + doc.pages, 0));
</script>
```

listing 167: props through `$props`, state, two derivations, all compiler instructions (source: javascript/fwsamples/ch22-svelte/QueryPanel.svelte, lines 12-37)

Nothing here imports a reactivity library. `$props`, `$state`, and `$derived` are recognized at compile time, and the emitted javascript creates signals and wires them into the markup the compiler also emits. The react chapter's component was a function the runtime calls again on every change; this one is code that runs once and then mutates. The honest shape of what `$state` becomes, from the actual build output of this project, simplified:

```
// let count = $state(4), what the compiler emits
let count = $.source(4);
// and a write, count = 5
$.set(count, 5);
// a read inside the markup's reactive statement
$.get(count);
```

fragment, illustrative, not executed

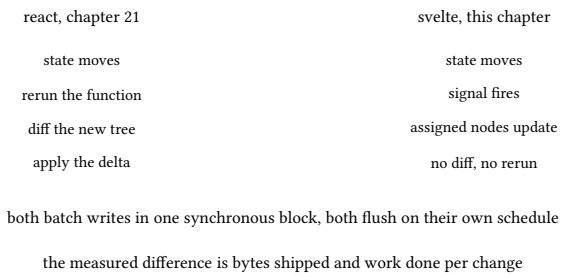


figure 123: two models of the same change

22.2. runes, outside any component

Svelte 5's runes are not component-only vocabulary. A `.svelte.js` module compiles with the same reactivity, and because the compiled output is plain esm over the signal runtime with no dom reference, node imports it and the reactivity is live, which is how this book tests reactivity without a browser:

```
export function createCounter(start = 0) {
  let count = $state(start);
  let history = $state([start]);

  let doubled = $derived(count * 2);
  let sign = $derived(count < 0 ? "negative" : count === 0 ? "zero" : "positive");

  let changes = 0;
  // an effect outside a component needs a root to own it, the
  // compiler's $effect.root, and the returned teardown is the
  // counter's dispose
  const dispose = $effect.root(() => {
    $effect(() => {
      // reads count, so the effect reruns on every change
      count;
      changes += 1;
    });
  });
```

listing 168: state, derivations, and an effect root, the whole reactive vocabulary in 18 lines (source: javascript/fwsamples/ch22-svelte/counter.svelte.js, lines 8-25)

Three rules the suite pins. `$derived` is synchronous, a write to `count` updates `doubled` on the next line with no flush. `$effect` is batched, it runs nothing until a flush, and two writes inside one synchronous block produce one rerun, the same coalescing rule react applies to events. And an effect outside a component needs an owner, `$effect.root`, whose teardown is the factory's `dispose`, otherwise the runtime answers with the `effect_orphan` error this chapter met on its first run and kept as a finding.

One boundary fact earned the hard way: the bundle carries its own copy of the svelte scheduler, so `flushSync` must come out of the built barrel too, flushing through the outer `svelte` package is a no-op on a second instance.

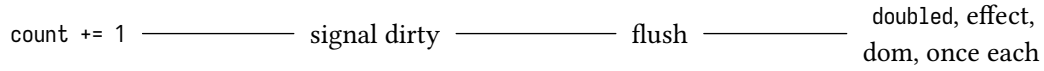


figure 124: one write, one flush

22.3. stores against runes

Svelte 4 ran reactivity through stores, containers with `subscribe` that components read with a `$` prefix. Svelte 5 keeps them working and the file builds both answers to the same counter so the trade is readable rather than asserted:

```
// ch22: stores against runes for the same job. a writable store is
// the svelte 4 answer, a container with subscribe, and it still
// runs. the runes answer owns the same state with $state and lets
// $derived replace the store map, no subscription list to maintain.
import { derived, get, writable } from "svelte/store";

export function createStore(initial = 0) {
  const count = writable(initial);
  const doubled = derived(count, (value) => value * 2);
  return {
    subscribe: count.subscribe,
    get value() {
      return get(count);
    },
    get doubled() {
      return get(doubled);
    },
    subscribeDoubled: doubled.subscribe,
    set(value) {
      count.set(value);
    },
    increment(by = 1) {
      count.update((value) => value + by);
    },
  };
}

// the runes twin, same surface, no subscribers to notify by hand
export function createRuneStore(initial = 0) {
  let count = $state(initial);
  let doubled = $derived(count * 2);
  return {
    get value() {
      return count;
    },
    get doubled() {
      return doubled;
    },
    set(value) {
      count = value;
    },
    increment(by = 1) {
      count += by;
    },
  };
}
```

listing 169: the store twin and the rune twin, one surface, two mechanisms (source: javascript/fwsamples/ch22-svelte/store.svelte.js, lines 1-46)

The store notifies a subscriber list the runtime maintains, `get` reads a current value, `derived` chains a second container off the first. The rune twin has no list, the getters read signals, and the suite drives both: the store's subscribers see `2, 3, 10` across a set and an update, the rune twin's `doubled` follows `value` through the same moves. The honest difference is machinery, not capability, and the runes win on one axis that matters to this book, they are the compiler's own path and stores are the compatibility layer.

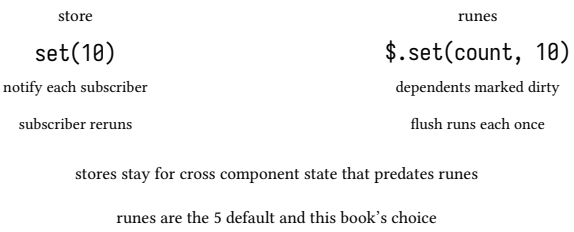


figure 125: two notification paths

22.4. props, callbacks, and bindable

Component events are gone in svelte 5, and the replacement is the simplest thing that could work: a prop holding a function. The panel takes `onselect` and calls it, the parent passes it like any prop, and there is no event system, no dispatch, no listener cleanup:

```
<div class="query-panel" data-framework="svelte">
  <h2>{title}</h2>
  <p class="stat"><span class="stat-label">documents</span> <strong>{results.length}</strong></p>
  <p class="stat"><span class="stat-label">pages</span> <strong>{pages}</strong></p>
  <input type="text" placeholder="filter" bind:value={term} />
  <table>
    <thead>
      <tr><th>title</th><th>pages</th><th>tags</th></tr>
    </thead>
    <tbody>
      {#each results as doc (doc.id)}
        <tr class="row" onclick={() => onselect?.(doc.id)}>
          {#if row}
            {@render row(doc, doc.tags.join(", "))}
          {:else}
            <td>{doc.title}</td>
            <td>{doc.pages}</td>
            <td>{doc.tags.join(", ")}</td>
          {/if}
        </tr>
      {/each}
    </tbody>
  </table>
  {@render children?.()}
</div>
```

listing 170: the markup, an onclick row, the row call, children rendered (source: javascript/fwsamples/ch22-svelte/QueryPanel.svelte, lines 39-63)

`$bindable` is the two way door, the child declares it with a default, the parent may `bind:` it, and writes through the local alias flow back. The suite renders the default through the server, `zoom` with `0`, before any binding exists:

```
<script lang="ts">
  // ch22: $bindable. a prop the parent may bind both ways, the child
  // declares the default and owns writes through the local alias.
  interface Props {
    label: string;
    value?: number;
    onchange?: (value: number) => void;
  }

  let { label, value = $bindable(0), onchange }: Props = $props();

  function bump(by: number) {
    value += by;
    onchange?.(value);
  }
</script>

<label>
  {label}: <output>{value}</output>
  <button type="button" onclick={() => bump(1)}>more</button>
  <button type="button" onclick={() => bump(-1)}>less</button>
</label>
```

listing 171: a bindable prop with its default and one writer (source: javascript/fwsamples/ch22-svelte/Binding.svelte, lines 1-22)

22.5. snippets

Slots take one shape, the parent's markup dropped in one place. Snippets are values: declared with `#snippet`, passed like any prop, rendered with `@render`, parameterized, and usable more than once. The host passes both a parameterized `row` snippet and a `children` snippet to the same panel component from chapter 21's twin:

```
<QueryPanel title="snippets" {docs} onselect={select}>
  {#snippet row(doc, tags)}
    <td><em>{doc.title}</em></td>
    <td>{doc.pages}</td>
    <td>{tags}</td>
  {/snippet}
  {#snippet children()}
    <p>picked: {picked}, last click: {last}</p>
  {/snippet}
</QueryPanel>
```

listing 172: two snippets passed as props, one parameterized (source: javascript/fwsamples/ch22-svelte/SnippetHost.svelte, lines 23-33)

The server render shows the composition working, `go` arrives italicized from the parent's snippet, `picked: b1` from the children snippet, and the panel's own derivation still runs between them. Where react solved the same problem with the `children` prop and render callbacks, snippets are the compiler's answer, they can be passed around, kept in variables, and they carry their rendering context with them.



figure 126: one component, two snippets

22.6. the island seam

The same seam react used, a custom element, with none of the glue code. `svelte:options customElement` tells the compiler to emit an element class instead of a component, props become attributes and properties, and the entry file is one registration line:

```
<svelte:options
  customElement={{
    tag: "x-svelte-panel",
    shadow: "none",
  }}
```

listing 173: one line of options and the compiler writes the element class (source: javascript/fwsamples/ch22-svelte/Island.svelte, lines 39-43)

```
// ch22: the svelte island entry, browser only, never imported by
// the node suite. vite bundles it with the compiled component and
// the svelte runtime into one esm file a script tag can load, and
// the custom element class the compiler emitted is what gets
// registered. mount and destroy are the non island path, shown once
// for contrast, the element owns its own lifecycle here.
import { mount } from "svelte";
import Island from "./Island.svelte";

// the island path: the compiled custom element class
customElements.define("x-svelte-panel", Island as unknown as CustomElementConstructor);

// the plain mount path this chapter does not ship, kept as the
// contrast line: mount(Island, { target: document.body, props }) and
// a destroy(root) when the host goes away
export const mountIsland = (target: HTMLElement) => mount(Island, { target });
```

listing 174: the entry, registration and the mount contrast in 16 lines (source: javascript/fwsamples/ch22-svelte/main.ts, lines 1-16)

The react chapter needed `wrap.ts`, a framework free lifecycle mapper with injectable globals so node could test it. Svelte's compiler writes that class, shadow dom or not as the options say, and the honest comparison is not that react was wrong, it is that react stops at `createRoot` and svelte's compiler knows the target. The capstone will ship both islands over one engine and let the reader open both files side by side.

22.7. transitions

Transitions are the demo feature of every framework and svelte's are css custom properties the runtime animates, `fade` and `fly` from the standard library. The measured server fact is the honest one: ssr renders the settled state, no intro classes anywhere in the output, because an intro animation is a client behavior that starts when the element mounts:

```
<script lang="ts">
  // ch22: transitions. a transition is a pair of css custom
  // properties the runtime animates, fade and fly from the standard
  // library, and the honest server note: ssr renders the settled
  // state, intro transitions are a client behavior.
  import { fade, fly } from "svelte/transition";

  interface Doc {
    id: string;
    title: string;
  }

  let { docs }: { docs: Doc[] } = $props();

  let shown = $state(true);
</script>

<p>
  <button type="button" onclick={() => (shown = !shown)}>
    {shown ? "hide" : "show"}
  </button>
</p>
{#if shown}
  <ul>
    {#each docs as doc (doc.id)}
      <li transition:fly={{ x: 12, duration: 150 }}>{doc.title}</li>
    {/each}
  </ul>
  <p class="note" transition:fade={{ duration: 120 }}>{docs.length} rows</p>
{/if}
```

listing 175: fly on list rows, fade on the count line, both client only (source: javascript/fwsamples/ch22-svelte/Transitions.svelte, lines 1-30)

22.8. the server as oracle

The same oracle as chapter 21, aimed at the svelte build. `svelte/server`'s `render` turns a compiled component into a head and body pair, the smokes import the vite `--ssr` output, and the assertions read the html the way react's did:

```
describe("ch22 vite ssr bundle", () => {
  it("renders the panel with props, derivation, and keyed each", () => {
    const { body } = render(QueryPanel, { props: { title: "ssr panel", docs } });
    assert.match(body, /data-framework="svelte"/);
    assert.match(body, /<h2>ssr panel<\/h2>/);
    // the two derived stats, 3 documents, 321+258+412 pages
    assert.match(body, /<strong>3<\/strong>/);
    assert.match(body, /<strong>991<\/strong>/);
    assert.match(body, /<td>lua<\/td>/);
    // hydration anchors around the keyed each, svelte's own text seam
    assert.match(body, /<!--\[-->/);
  });
```

listing 176: the oracle smoke, svelte/server over the built components (source: javascript/fwsamples/ch22-svelte/render.smoke.mjs, lines 15-26)

The output carries svelte's own hydration anchors, `<!--[-->` around each block and key hints inside the keyed each, the twin of react's `<!-- -->` text markers, and the suite asserts them rather than stripping them, they are the

contract the client bundle hydrates against. The runes get the same treatment through a second config file that builds the `.svelte.js` modules client side, because server compiled runes are inert and the reactivity tests need live signals:

```
describe("ch22 runes", () => {
  it("$state starts at its argument and $derived reads from it", () => {
    const counter = createCounter(4);
    assert.equal(counter.count, 4);
    assert.equal(counter.doubled, 8);
    assert.equal(counter.sign, "positive");
  });

  it("writes update $derived immediately, synchronously", () => {
    const counter = createCounter(1);
    counter.increment();
    assert.equal(counter.count, 2);
    assert.equal(counter.doubled, 4);
    counter.increment(-5);
    assert.equal(counter.count, -3);
    assert.equal(counter.sign, "negative");
  });
```

listing 177: compiled reactivity driven in node, no browser anywhere (source: javascript/fwsamples/ch22-svelte/runes.smoke.mjs, lines 16-32)

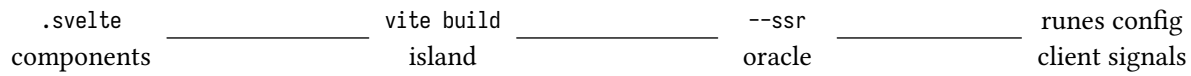


figure 127: the chapter builds

22.9. when svelte beats react

The same island, the same three documents, the same filter input, measured as bytes on this machine, 2026-09-21: the react island bundle is 861.84 kB minified, 218.69 kB gzip. The svelte island bundle is 57.79 kB minified, 18.09 kB gzip. A fifteenth of the bytes, and the reason is the model, react ships a runtime that diffs trees, svelte ships the compiler's output and a signal runtime measured in kilobytes.

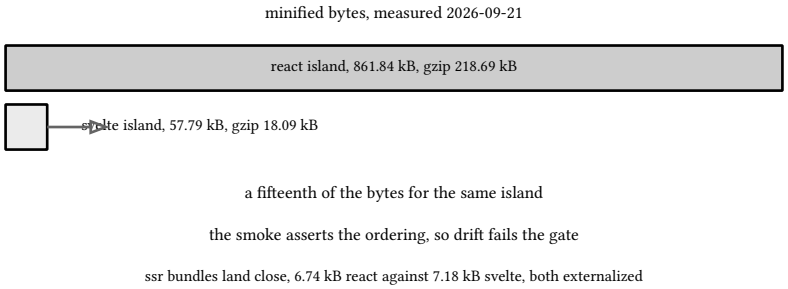


figure 128: the two island bundles, to scale in bytes

Bytes are where svelte wins, and they are not the whole ledger. The react chapter's honest trade holds here in reverse: the ecosystem, the hiring pool, and the third party component surface all favor react, svelte's compiler means the interactive words in your file are not plain javascript, and the error messages come from the compiler rather than a stack you can step through. This book's capstone ships both islands anyway, one engine under two frameworks, because the seam that made both possible, a custom element over a built bundle, is the framework free part, and that part belongs to the platform, not to either camp.

sources: svelte.dev docs on runes, snippets, stores, custom elements, and transitions, svelte.dev blog svelte 5 announcement, npmjs.com package pages for the pinned versions, vite.dev guide, accessed 2026-09-21. Verified

live with svelte 5.57.1, the vite-plugin-svelte 7.3.0 and vite 8.3.0 on node 26.3.0, windows, the chapter's 16 tests green through `npm run verify` in `javascript/fwsamples`, bundle sizes read from the vite build output 2026-09-21.

23. api kernel

Everything else in this service part sits on one small package, the same position the go book's `httpx` holds there. This chapter builds it in `books/javascript/api/src/kernel/`: the handler contract every route shares, the hand-rolled route table with method plus `:param` patterns, the constant error envelope every non-2xx response answers with, the `JSON.stringify` wire codec and its one decode path, request ids born once at the edge, and the composition file later chapters wire into. The vehicle is bare `node:http` on node 26 with zero packages, every ruling below probed on 26.3.0 first.

23.1. the handler contract

A kernel handler is one async function of one context object, plain data: `req`, `res`, the `params` the router fills, the `query` as a `URLSearchParams`, the request `id`. A handler that fails throws an `ApiError` instead of writing a failure body, so no route formats an envelope by hand and no route can answer a 500 with the wrong shape. That is the go contract, but the mechanics differ because the error channel differs: go handlers return an error value and an adapter converts it, while an escaped rejection in node is an `unhandledRejection`, which kills the process. The kernel's root handler therefore owns a try/catch at the dispatch boundary, part contract conversion, part survival guarantee:

```
handler() {
  let next = (ctx) => router.dispatch(ctx);
  for (let i = layers.length - 1; i ≥ 0; i--) next = layers[i](next);
  const stack = next;
  return async (req, res) => {
    const id = resolveRequestId(req.headers["x-request-id"], newId);
    res.setHeader("X-Request-ID", id);
    const ctx = { req, res, params: {}, query: new URLSearchParams(), id };
    try {
      await stack(ctx);
    } catch (err) {
      // the kernel's own backstop: a stack mounted without the
      // adapter layer still answers in shape, and a rejection
      // never escapes to kill the process
      writeError(res, id, err);
    }
  };
```

listing 178: the root handler: ctx birth, the fold, the backstop conversion (source: javascript/api/src/kernel/kernel.mjs, lines 30-46)

The backstop renders through the one failure writer everything else uses, so a stack without the next chapter's adapter still answers in shape. It cannot log, the kernel holds no logger: conversion with logging is the middleware layer's job, the backstop is the net.

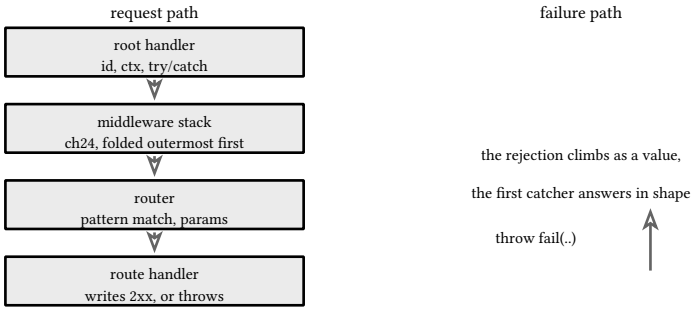


figure 129: the request climbs down the stack, a failure climbs back as a rejection

23.2. the route table

Node ships no router, only `createServer` with one request callback, and the platform ruling is that the gap is filled bottom-up rather than by adopting express. `createRouter` holds one flat array of routes, each pattern compiled once into segments where a `:name` segment becomes a parameter slot. Dispatch splits the percent-decoded path, filters the table, then picks the most specific method match. Specificity is one bit per segment, static beats param, compared left to right, and that vector totally orders any candidate set matching one path: two patterns with equal scores on the same path have the same statics everywhere, the same pattern twice. The go mux's registration-time tie panic has no js counterpart because the tie is unrepresentable:

```
  let route = null;
  let score = null;
  for (const candidate of onPath) {
    if (!methodMatches(candidate, req.method)) continue;
    const s = candidate.segments.map((segment) => (segment === null ? 0 : 1));
    if (score === null || moreSpecific(s, score)) {
      route = candidate;
      score = s;
    }
  }
  if (route === null) {
    const allow = new Set(onPath.map((candidate) => candidate.method));
    if (allow.has("GET")) allow.add("HEAD");
    res.setHeader("Allow", [...allow].sort().join(", "));
    writeError(
      res,
      ctx.id,
      fail(CODES.not_found, "method not allowed", { status: 405 }),
    );
    return;
  }
```

listing 179: dispatch: path candidates, the specific method match wins, 405 carries Allow (source: javascript/api/src/kernel/router.mjs, lines 90-110)

Two unmatched answers stay inside the contract. An unknown path is 404 as the envelope, and a known path with the wrong method is 405, also as the envelope, plus the `Allow` header the table computed with `HEAD` added wherever `GET` lives. Both carry the code `not_found` because the code set is closed at eleven, the message plus `Allow` carrying the distinction, a ruling every lane inherits. A `HEAD` request dispatches to the `GET` handler because node itself suppresses the body on `HEAD`, verified on the probe server: the handler calls `end` with bytes and the client reads zero. Trailing slashes and malformed percent escapes answer 404 rather than redirecting or throwing.

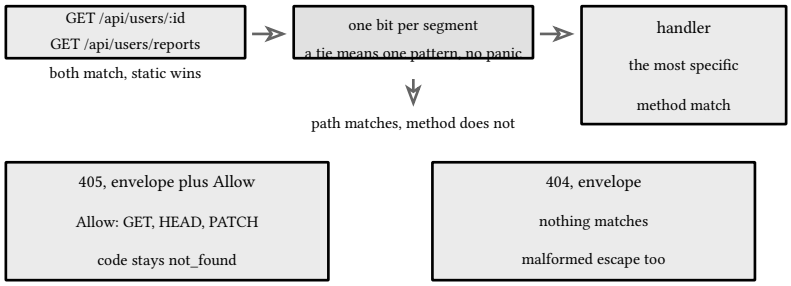


figure 130: static beats param, ties are unrepresentable, leftovers split into 405 and 404

23.3. the error envelope

The contract freezes one failure shape for the whole api: an `error` object with a `code` from a closed set of eleven, one human sentence, an optional `details` array on `validation_failed` only, and the request id. The set and the status

table live in codes.mjs, the table the source of truth, the code names derived from its keys. Two codes are split: invalid_json rides 400 and 415, not_found rides 404 and 405, and each split site passes its own status the way the 405 site did.

The writer is one function and the member order is the contract's order, code, message, details when present, request_id. That order is the byte order the golden vectors freeze, and the tests pin it against vector 05: the writer fed the vector's own code, message, and frozen request id must emit the vector's exact body bytes. An unknown thrown value renders as internal with the detail kept off the wire, because a leaked sqlite: database is locked says too much:

```
// writeError is the one failure writer. a thrown ApiError renders as
// itself, anything else answers internal with its detail kept off the
// wire. member order is code, message, details when present,
// request_id.
export function writeError(res, requestId, err) {
  const known = err instanceof ApiError;
  const e = known ? err : new ApiError(CODES.internal, "internal error");
  const error = { code: e.code, message: e.message };
  if (e.details) error.details = e.details;
  error.request_id = requestId;
  return writeJson(res, e.status, { error });
}
```

listing 180: the one failure writer: classify, order the members, delegate to the success writer (source: javascript/api/src/kernel/envelope.mjs, lines 36-47)

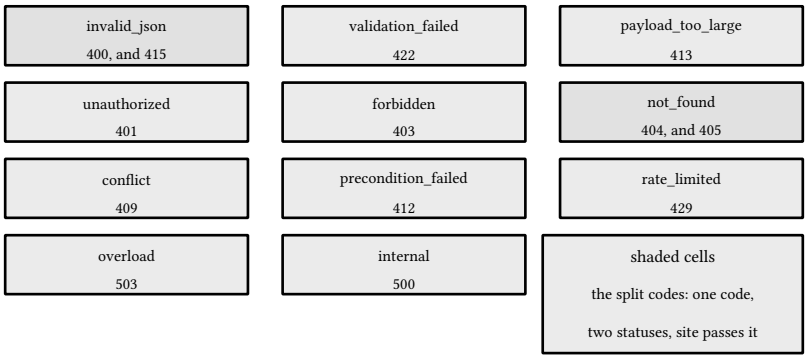


figure 131: eleven codes, eleven statuses in the table, thirteen on the wire

23.4. the wire codec

The encoder is JSON.stringify and the dto discipline is the object literal itself. Javascript specifies insertion order for string keys, so a literal in the contract's member order serializes in that order, keys hand-named in snake_case, timestamps preformatted as the RFC 3339 strings the vectors freeze.

Reading is stricter than writing because request bodies are input. readJson is the one decode path and enforces three rules in a fixed order, each a thrown ApiError: the media type must be application/json with parameters allowed, so a charset suffix passes and anything else is 415 invalid_json before a single body byte is read, a fact the codec test proves with a request double whose iterator fails the test if it runs. The body then streams through a for await over the request, counted per chunk against the contract's 1 MiB cap, and a body past it is 413 payload_too_large, always, never invalid_json. What survives goes to JSON.parse, and bytes that do not parse are 400 invalid_json:

```
export async function readJson(source) {
  const req = source?.req ?? source;
  if (contentTypeOf(req.headers) !== "application/json") {
    throw fail(CODES.invalid_json, "content type must be application/json", { status: 415 });
  }
  if (source?.req && source.body !== undefined) {
    return source.body;
  }
  const chunks = [];
  let size = 0;
  for await (const chunk of req) {
    size += chunk.length;
    if (size > MAX_BODY_BYTES) {
      throw fail(CODES.payload_too_large, "request body exceeds 1 MiB");
    }
    chunks.push(chunk);
  }
  try {
    return JSON.parse(Buffer.concat(chunks).toString("utf8"));
  } catch {
    throw fail(CODES.invalid_json, "malformed json body");
  }
}
```

listing 181: readJson: 415 before the first byte, 413 past 1 MiB, 400 on parse, a keyed layer's stash reused (source: javascript/api/src/kernel/codec.mjs, lines 20-42)

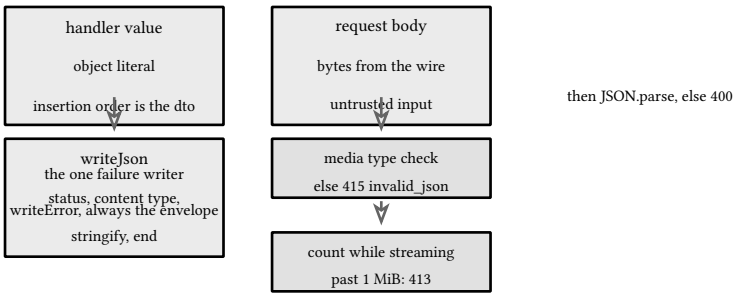


figure 132: two write sites through one writer, one decode path, four contract statuses

23.5. request ids

Every request needs one id the log line, the error envelope, and later the trace metrics all agree on. The kernel resolves it once, at the edge: the client's `X-Request-ID` when it is uuid-shaped, else a fresh v7. The uuid test is a plain regex, case-insensitive, any version, the acceptance domain of the go lane's `uuid.Parse`. Parsing is contract, not politeness: the envelope promises `request_id` is a uuid, and a free-text header echoed into logs is log injection. Garbage is replaced, never echoed.

The fresh id comes from `crypto.randomUUIDv7`, platform surface, so this lane does not hand-roll a v7 constructor. The test asserts the shape on a hundred draws, version nibble 7 and variant bits 10xx. The root handler echoes the id as `X-Request-ID` so a client correlates its own logs with the server's, and everything downstream reads `ctx.id` without touching headers again:

```
import { randomUUIDv7 } from "node:crypto";

const UUID = /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;

export function isUuid(value) {
  return typeof value === "string" && UUID.test(value);
}

export function resolveRequestId(headerValue, newId = randomUUIDv7) {
  return isUuid(headerValue) ? headerValue : newId();
}
```

listing 182: the resolver: uuid header honored, garbage replaced, platform v7 as the fallback (source: javascript/api/src/kernel/requestid.mjs, lines 6-16)

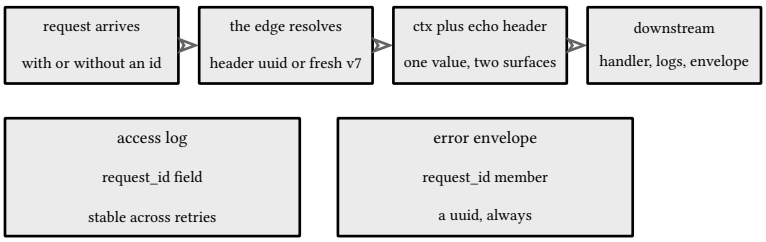


figure 133: one id, born at the edge, quoted by everything downstream

23.6. the app skeleton and the composition file

`createKernel` returns the app object the whole part builds on: `route` registers one handler, `use` appends one middleware layer, first used outermost, `handler` folds the stack over dispatch once at call time, and `listen` starts a server on an ephemeral port by default. The kernel owns the two deployment routes. `GET /healthz` answers liveness with exactly `{"status":"ok","version":".."}`, the version an injected dependency so a deployment can prove what it runs. `GET /readyz` answers 200 while the injected `ready` probe says ready and throws `overload` at 503 the moment it does not, the hook the load chapter's drain wiring flips and the handler contract from route one.

Configuration arrives as injected dependencies, the kernel never reads ambient env, and the ship chapter passes env values in from the edge. `src/server.mjs` is the composition file and has one writer, this chapter: create the app, mount each family's stack, hand the app back. Every later family lands as a `wire-<family>.mjs` function the file calls, one import and one call, in chapter order with the operations layers interleaved where the stack names. Nothing listens on import:

```
export function createApp(deps = {}) {
  // chapter 28's store family constructs first because the
  // single-store ruling's shared store is its port adapter whenever a
  // database path is given: sqlite-backed when deps.storePath is set,
  // the ch25 memory store for unit tests otherwise. one construction
  // point, one key, no family call changes:
  const wired = wireStore({ path: deps.storePath, now: deps.now });
  const d = {
    ...deps,
    store: deps.store ?? (deps.storePath ? wired.users : createMemoryStore()),
    // the keyed runner over the store's idem keys, register's replay
    // property consumed where the conc chapter pinned it, one runner
    // per construction site per js-store's ruling:
    runKeyed: deps.runKeyed ?? createKeyedRunner(wired.idem),
  };
```

listing 183: the composition file: path-aware single store, the keyed runner handed to register (source: javascript/api/src/server.mjs, lines 20-34)

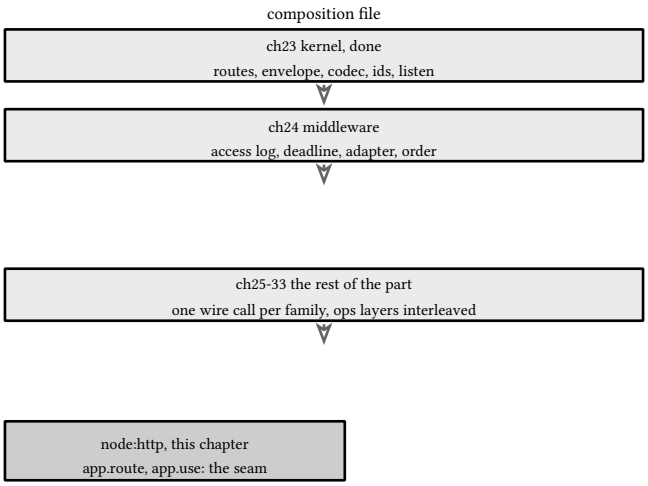


figure 134: every chapter lands as one import and one call, in chapter order

23.7. testing the kernel

Go's kernel had `httptest` and tested everything without a socket. Node has no recorder type, and the lane answers with two paths. The socketless path rolls a hand-rolled response double, `FakeRes` in `test/kernel/helpers.mjs`: it records the status, the headers, and the body chunks the kernel writes, and it refuses writes past an `end`, so a double-write bug fails the assertion it was heading for. The request side is a plain object with `method`, `url`, and lowercase headers. Router matching, envelope bytes, the codec ladder, and the id resolver all test through these doubles, no listener, no port, no timing.

The socket path is for what only the wire proves: the platform's HEAD body suppression, header casing, the id echo. Each such test calls `app.listen()`, fetches against the returned port, and closes in `t.after()`, the close calling `closeAllConnections` before `server.close` because a kept-alive socket holds a bare close open, the standing `node --test` orphan trap on this machine. The property check is a seeded loop, fixed seed 20261001, node having no native fuzzing: fifty rounds of random uuids under both registration orders, asserting the static route answers and the param route never does, the invariant named in the test title:

```
  on(event, listener) {
    (this.listeners[event] ??= []).push(listener);
    return this;
  }
  emit(event) {
    for (const listener of this.listeners[event] ?? []) listener();
  }
  write(chunk) {
    if (this.writableEnded) return false;
    this.chunks.push(Buffer.isBuffer(chunk) ? chunk.toString() : String(chunk));
    return true;
  }
  end(body) {
    if (this.writableEnded) return this;
    if (body !== undefined && body !== null) this.chunks.push(String(body));
    this.headersSent = true;
    this.writableEnded = true;
    this.emit("finish");
    return this;
  }
```

listing 184: the response double: an event surface with finish on end, past which nothing writes (source: javascript/api/test/kernel/helpers.mjs, lines 22-41)

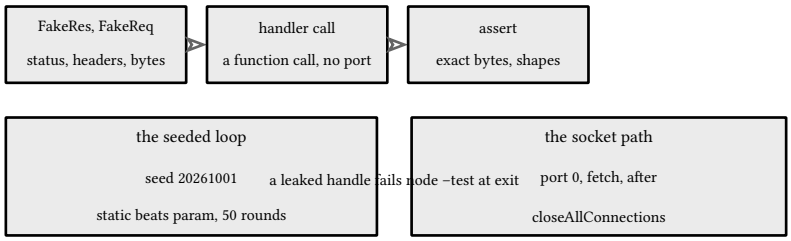


figure 135: two test paths: doubles for logic, an ephemeral socket for the wire

sources: nodejs.org/api/http.html for createServer, ServerResponse, server.close and closeAllConnections, nodejs.org/api/crypto.html for randomUUIDv7, nodejs.org/api/url.html for URL and URLSearchParams, nodejs.org/api/test.html for the runner, and developer.mozilla.org for JSON.parse, JSON.stringify, and the async iteration protocol, all accessed 2026-09-26, re-verified by the probes cited in the text on node 26.3.0. Verified by `books/javascript/api/test/kernel` tests, 33 of them, plus node –test and prettier.

24. middleware

Cross-cutting behavior does not belong inside the handlers, and node does not make anyone put it there. This chapter composes the service's middleware stack over the previous chapter's kernel, in `books/javascript/api/src/middleware/`: the chain type that folds layers into one handler, the access log with its byte counting, the handler deadline raced over an injected timer, the adapter that turns exceptions into the contract envelope, and the wiring order that makes the stack correct rather than merely present.

24.1. the chain type

One layer is one function: given the next handler, return the handler that wraps it. `chain` folds a list of layers by wrapping right to left, so the first argument ends up outermost, sees the request first and the response last, which is the onion picture every middleware chapter draws because it is the true one. A chain is itself a layer, so chains compose into bigger chains, and the kernel's `use` seam appends onto the same fold, first used outermost:

```
export function chain(...layers) {
  return (next) => {
    let handler = next;
    for (let i = layers.length - 1; i ≥ 0; i--) handler = layers[i](handler);
    return handler;
  };
}
```

listing 185: chain: one layer type, folded so the first argument is outermost (source: javascript/api/src/middleware/chain.mjs, lines 5-11)

The loop runs backwards for a reason worth saying out loud: folding left to right would make the last layer outermost, and the wiring order in this chapter's fifth section is a contract, not an accident. A test pins it, asserting the pass order `outer, middle, inner,`
`handler` and the return order back out, from a context object the layers share:

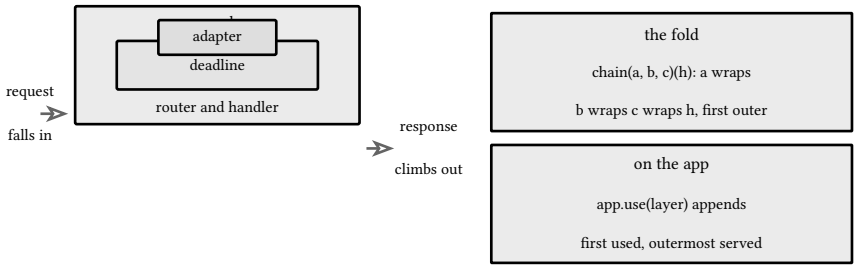


figure 136: the onion: first argument outermost, request in, response out

24.2. the access log

The access log writes one structured line per request: method, path, the status the wire actually served, the byte count, the duration, the request id. Node keeps the served status on the response object, so the layer reads `res.statusCode` after the stack below answered, and that difference from go is worth a sentence: go handlers write a status the wrapper must intercept because nothing reports it back, while node's response is a readable object. The double-write trap also differs, and the probes on 26.3.0 say how: the first `writeHead` commits immediately, a second throws `ERR_HTTP_HEADERS_SENT`, a `statusCode` assignment after commit is a silent no-op, and a second `end` before the stream finishes emits `ERR_STREAM_WRITE_AFTER_END` asynchronously while one after `finish` is silently ignored. Loudness varies trap by trap, so the capture happens after the answer, not during it.

Byte counting wraps the `write` and `end` methods on the response instance, holding the originals and restoring them in a `finally` block, which is also what guarantees the line lands when the stack below throws. The count is wire

bytes, `Buffer.byteLength` on each chunk, true for multi-byte bodies: a register whose display name is all umlauts logs the same number the client measured, and the regression test pins exactly that:

```
    const rawWrite = res.write;
    const rawEnd = res.end;
    res.write = function countingWrite(chunk, ...rest) {
      if (isBodyChunk(chunk)) bytes += Buffer.byteLength(chunk);
      return rawWrite.apply(res, [chunk, ...rest]);
    };
    res.end = function countingEnd(chunk, ...rest) {
      if (isBodyChunk(chunk)) bytes += Buffer.byteLength(chunk);
      return rawEnd.apply(res, [chunk, ...rest]);
    };
    try {
      await next(ctx);
    } finally {
```

listing 186: counting wrappers on the instance, wire bytes, originals restored in the finally (source: javascript/api/src/middleware/accesslog.mjs, lines 26-38)

The sink is a dependency, one function per line, defaulting to a hand-rolled JSON line on stdout because node ships no structured logger and the ruling is zero packages. The clock is a dependency too, so tests inject two timestamps and assert the exact `duration_ms`, and `mock.fn` is the counting spy that proves one line per request:

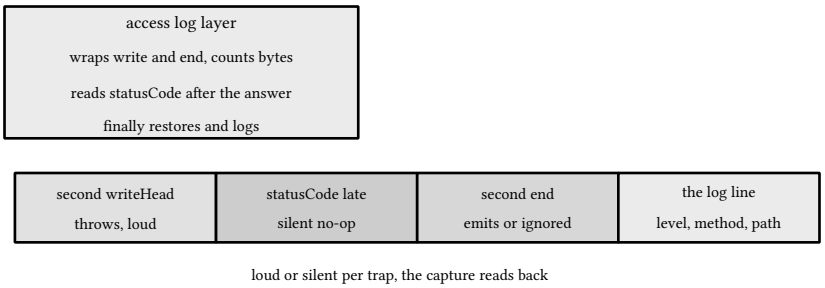


figure 137: status read after the answer, bytes counted at the writes, one line per request

24.3. the handler deadline

A service needs timeouts at two layers because the layers fail differently. The transport layer is node's own server clock, probed defaults on 26.3.0: `headersTimeout` 60 seconds, `requestTimeout` 300 seconds, `keepAliveTimeout` 5 seconds. These are coarse, they protect the process from slow clients, and when one fires the connection just dies, plain by design. The handler layer is finer: one request's handler time, after which the client deserves the contract's `overload` envelope rather than a dropped connection, and that is this layer.

The mechanism is a race the language gives directly: `Promise.race` between the handler's promise and a timer promise. The handler wins, its answer stands and the unused timer is cleared. The deadline wins, the layer answers the overload envelope and the abandoned handler keeps running, its later writes refused by the writers' already-answered guard from the kernel chapter, the cousin of go's detached writer whose buffer is dropped. The abandoned handler's failure is observed through a `catch` on the run promise, because an unobserved rejection is an `unhandledRejection` and kills the process no matter how late it lands:

```
return (next) => async (ctx) => {
  let handle;
  const expiry = new Promise((resolve) => {
    handle = timers.setTimeout(() => resolve(), ms);
  });
  const run = Promise.resolve().then(() => next(ctx));
  const winner = await Promise.race([run.then(() => "handler"), expiry.then(() => "deadline")]);
  if (winner === "handler") {
    timers.clearTimeout(handle);
    return;
  }
  writeError(ctx.res, ctx.id, fail(CODES.overload, "request timed out"));
  // the abandoned handler's failure is observed, never unhandled
  run.catch(() => {});
};
```

listing 187: the race: handler or timer, overload on expiry, the late failure observed (source: javascript/api/src/middleware/deadline.mjs, lines 15-29)

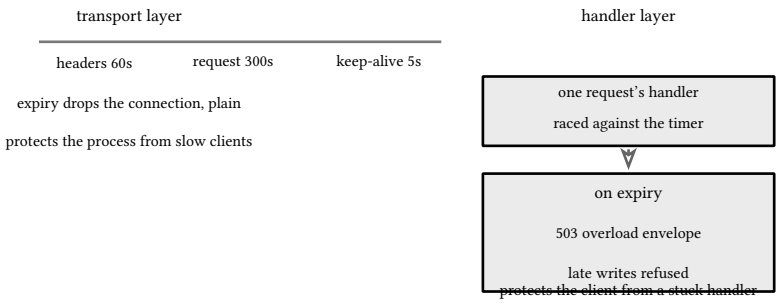


figure 138: two clocks: transport deadlines kill connections, the handler deadline answers

24.4. the exception-to-envelope adapter

The kernel's backstop already converts any rejection that escapes the stack. The adapter is the same conversion done properly, innermost, with the log line the backstop cannot write: a thrown `ApiError` renders as its own code, message, and status, and anything else answers `internal` with the stack and the request id logged and the detail kept off the wire. Known failures are not error-level noise, so a 404 or a validation failure converts silently and only an unknown exception produces an error line:

```
export function recover({ log = stdoutJsonSink } = {}) {
  return (next) => async (ctx) => {
    try {
      await next(ctx);
    } catch (err) {
      if (!(err instanceof ApiError)) {
        log({
          level: "error",
          msg: "exception recovered",
          request_id: ctx.id,
          error: err?.stack ?? String(err),
        });
      }
      writeError(ctx.res, ctx.id, err);
    }
  };
}
```

listing 188: the adapter: known failures render, unknown ones log then render (source: javascript/api/src/middleware/adapter.mjs, lines 10-26)

Placement carries one cross-language truth. The go lane had to put its recover inside the timeout layer because a go panic never crosses a goroutine boundary, and the timeout moved the handler onto another goroutine. Node has no such rule: a rejection climbs the awaited chain no matter which layer scheduled the work, so the adapter's innermost position is about the deadline race, not about reachability. Innermost, every handler failure settles the race as a resolved promise and the deadline never answers over a failure the adapter already answered. Go also rethrows one sentinel, `http.ErrAbortHandler`, the server's own connection-drop signal, and node has no counterpart to rethrow: socket teardown is not an exception here, so the adapter catches everything:

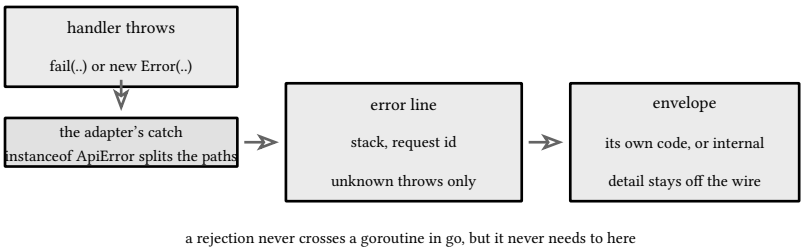


figure 139: one path to the log, one to the envelope, nothing rethrown

24.5. wiring order and why

The order of the three layers is contract, and each position is a sentence. The access log outermost, so it sees the answer everything below produced: a recovered 500 is logged as the 500 the client saw, not the status a wrapper never observed. The deadline inside the log, so the envelope the deadline writes is also logged. The adapter innermost, so the race always settles and handler failures never become deadline failures. There is no request-id layer in this stack because the kernel resolves the id at the edge, before any layer runs, and the echo header is already set by then, so the stack starts at the log. `wireMiddleware` is the family's one call in the composition file, mounting the three layers in that order, with the operations families' layers interleaved at their canonical positions through the optional mounts:

```
export function wireMiddleware(
  app,
  { log, clock, deadlineMs, timers, outermostLayer, afterLogLayer } = {},
) {
  if (outermostLayer) app.use(outermostLayer);
  app.use(accessLog({ log, clock }));
  if (afterLogLayer) app.use(afterLogLayer);
  app.use(handlerDeadline({ ms: deadlineMs, timers }));
  app.use(recover({ log }));
}
```

listing 189: the wiring: log, deadline, adapter, the ch31-33 layers interleaved at their positions (source: javascript/api/src/middleware/wire-middleware.mjs, lines 13-22)

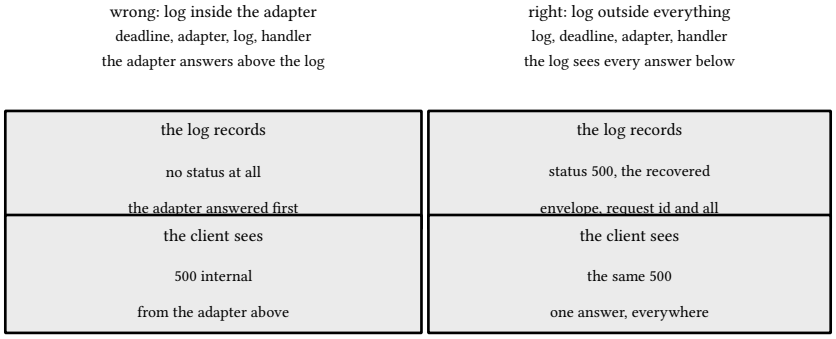


figure 140: wrong order versus right: where the log sits decides what it sees

24.6. testing middleware

Middleware tests run at two levels, and the clock discipline differs at each. The layer level drives everything through `mock.timers`: `enable` with the `setTimeout` api, `tick` to fire the deadline, `reset` in a `finally`, and no test ever sleeps. The handler that must not finish is a promise that never settles, the handler that finishes late is held on a gate the test releases after the assertions, and the late-failure test awaits the gate's rejection so nothing is left dangling. The sink is `mock.fn`, the counting spy: zero calls asserted on the quiet paths, exactly one line and its exact fields on the loud ones.

The socket level exists because the wired stack should answer over the wire once, and there the timer is a hand-rolled double injected through the layer's dependency rather than mock.timers, because mock.timers patches the global `setTimeout` and the fetch pipeline shares that global. The test's double captures the deadline callback, the handler signals when it is provably inside the route, and the test fires the captured callback at that moment: deterministic, zero wall time. The stack test is the one that fails when anyone reorders the wiring, and it is the same test that proves log and envelope agree on the request id:

```
  const lines = log.mock.calls.map((call) => call.arguments[0]);
  const infoLines = lines.filter((line) => line.level === "info");
  const errorLines = lines.filter((line) => line.level === "error");
  assert.equal(infoLines.length, 1, "one access line per request");
  assert.equal(infoLines[0].status, 500, "the log sits outside the adapter, it saw the answer");
  assert.equal(infoLines[0].request_id, body.error.request_id);
  assert.ok(infoLines[0].bytes > 0);
  assert.equal(errorLines.length, 1, "the adapter logged the exception");
  assert.ok(errorLines[0].error.includes("Error: handler bug"), "with its stack");
  assert.equal(errorLines[0].request_id, body.error.request_id);
```

listing 190: the stack test: the log saw the 500 the adapter answered (source: javascript/api/test/middleware/stack.test.mjs, lines 23-32)

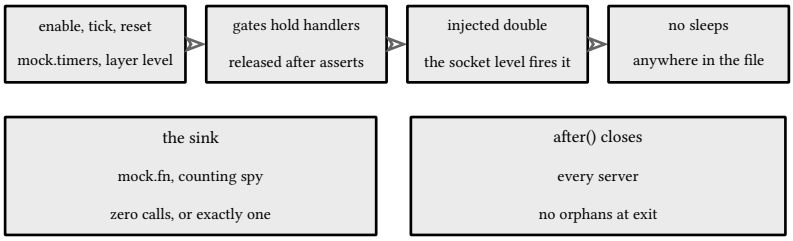


figure 141: two clock styles: mock.timers at the layer, an injected double at the socket

sources: nodejs.org/api/test.html for the runner, mock.fn, and mock.timers, nodejs.org/api/http.html for ServerResponse, the commit semantics of writeHead and end, and the server timeout clocks, and developer.mozilla.org for Promise.race and Function.prototype.apply, all accessed 2026-09-26, with the writeHead, end, and timeout facts probed on node 26.3.0 before being written here. Verified by `books/javascript/api/test/middleware` tests, 16 of them, plus node –test and prettier.

25. the user resource

The service's first real resource is the user: a record with an id, an email, a display name, roles, a creation time, and a version. This chapter builds the whole resource family in `books/javascript/api/src/users/`: the representation and its two projections as object literals, strict decoding on top of the kernel's codec, validation from first principles, merge-patch semantics for updates, keyset pagination for lists, the status contract that binds failures to codes, and the golden handler tests that pin all of it against the frozen vectors. The go mirror built the same layer over typed structs and `json/v2` tags, and node has none, so the discipline moves into the literals themselves, insertion order is serialization order, and the vectors prove the bytes.

25.1. the representation

A resource is three things at once: the bytes on the wire, the fields each reader may see, and the strong etag that names one exact version of the body. The stored record keeps a `passwordHash` field for the authn family, and the projection literals simply never mention it, so the hash cannot leak by construction, the same trick the go lane played with a dash tag. One function chooses between the two projections:

```
export function projectValue(user, admin) {
  if (admin) {
    return {
      id: user.id,
      email: user.email,
      display_name: user.displayName,
      roles: [...user.roles],
      created_at: rfc3339(user.createdAtMs),
      version: user.version,
    };
  }
  return {
    id: user.id,
    display_name: user.displayName,
    created_at: rfc3339(user.createdAtMs),
    version: user.version,
  };
}

// wireBody marshals the full representation. the stored record keeps
```

listing 191: the two projections, one chooser, field-level authorization lives here (source: javascript/api/src/users/model.mjs, lines 25-44)

The projection is where field-level authorization lives. Email and roles are admin-only data, so the list handler never decides what to hide, it calls one function and writes whatever comes back, and a rule enforced in one function cannot be forgotten in a second handler. The literal's insertion order is the wire order, javascript specifies it for string keys, so `wireBody` is `JSON.stringify` over this literal and nothing else. The `version` member bumps exactly once per successful write, and the concurrency chapter promotes it into the compare-and-swap substrate.

The etag is not stored anywhere, it is computed: `etagOf` takes the exact body string, sha256s it through `createHash`, and quotes the hex. Because the etag is a function of the body, the two can never disagree, and a client that saves the etag has saved the body's fingerprint.

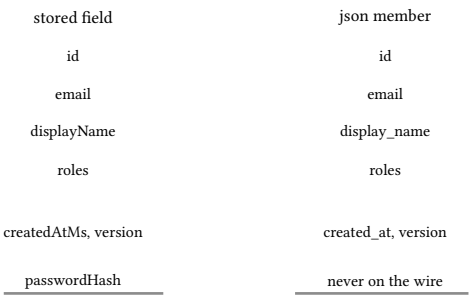


figure 142: the record's fields against the wire members, one field never sent

25.2. decoding with strictness

Input is hostile until proven otherwise, and the decode ladder answers four questions in a fixed order: is the media type json at all, is the body inside the 1 MiB cap, does it parse, and does it carry only members the contract knows. The kernel's `readJson` owns the first three, chapter 23 built them. The family owns the fourth, because the accepted member set differs per route and only the route knows it:

```
function bindStrict(body, known) {
  if (body === null || typeof body !== "object" || Array.isArray(body)) {
    throw fail(CODES.invalid_json, "body must be a json object");
  }
  for (const [key, value] of Object.entries(body)) {
    if (!known.has(key)) {
      throw fail(CODES.invalid_json, `unknown member ${key}`);
    }
    if (typeof value !== "string") {
      throw fail(CODES.invalid_json, `${key} must be a string`);
    }
  }
  return body;
}
```

listing 192: the schema rung: unknown member or wrong type is a decode failure, 400 (source: javascript/api/src/users/handlers.mjs, lines 50-63)

The strictness that pays rent is the unknown-member refusal. A client that sends a `pet` member against a contract with no `pet` gets a 400, not a silently ignored field, so a typo surfaces at the border instead of corrupting data six layers in. Go bought this with `RejectUnknownMembers` on the typed decode, and the js vehicle buys it with a plain loop over `Object.entries`, the wrong-type check riding the same loop because a registration email that arrives as a number is a decode problem, not a validation rule. A missing member arrives as `undefined` and the presence rules answer 422, so the rungs keep the go lane's split, decode failures 400, rule failures 422.

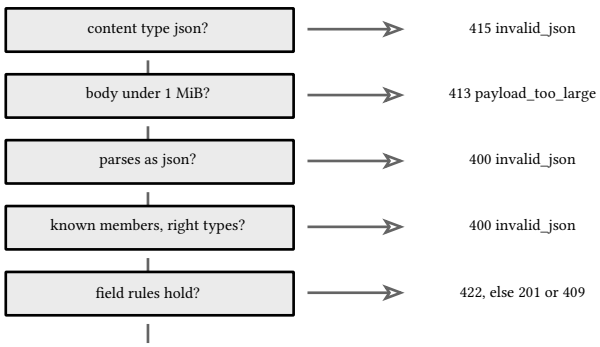


figure 143: the decode ladder: each rung rejects a class of bad input

25.3. validation from first principles

Validation rules come from the contract, not from a library: passwords at least 12 characters and at most 128, one email shape, display names 1 to 64 characters. Stating them as plain code keeps every number greppable and every message owned. The function returns one detail per violated rule rather than failing on the first, so the 422 body's `details` array tells the whole story and a client fixes everything in one round trip:

```
export function validateRegister(email, password, displayName) {
  const details = [];
  if (email === "") {
    details.push("email is required");
  } else if (!validEmail(email)) {
    details.push("email is not a valid address");
  }
  if (password === "") {
    details.push("password is required");
  } else if (password.length < MIN_PASSWORD_LENGTH) {
    details.push(`password must be at least ${MIN_PASSWORD_LENGTH} characters`);
  } else if (password.length > MAX_PASSWORD_LENGTH) {
    details.push(`password must be at most ${MAX_PASSWORD_LENGTH} characters`);
  }
  if (displayName === "") {
    details.push("display_name is required");
  } else if (displayName.length > MAX_DISPLAY_NAME) {
    details.push(`display_name must be at most ${MAX_DISPLAY_NAME} characters`);
  }
  return details;
}
```

listing 193: one detail per broken rule, in field order, every bound a named constant (source: javascript/api/src/users/validate.mjs, lines 14-34)

Three classes of rule run here. Presence rules catch missing members. Shape rules catch an email that parses as nothing, where the go lane reached for `net/mail` and its `ParseAddress`, node ships no mail parser, so the shape is stated directly: exactly one at sign, a non-empty local part, a dotted domain with no leading or trailing dot and no doubled dot, no whitespace, a length cap. It approximates the real grammar without rejecting addresses people hold. Range rules catch lengths, and the bounds live in named constants next to the rules so the handler, the tests, and this chapter quote one set of numbers.

The 422 and the 409 are different animals and the boundary is worth stating. A rule violation is about the request alone, checkable without touching state, so it is 422. The email conflict is about the store's current contents, so it is 409, and it surfaces only after validation passed and the create call answered its conflict outcome. Same request shape, two different truths. Normalization closes one gap before it opens: emails lower-case and trim on the way in, so `Ada@Example.ORG` cannot dodge a conflict or mint a second account.

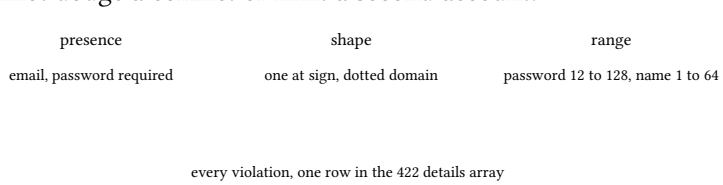


figure 144: three rule classes, one details array

25.4. patch merge semantics

RFC 7386 merge-patch gives updates three-way semantics a fixed member list cannot express: an absent member leaves the field alone, null deletes it, a value replaces it. So the patch decodes into a plain parsed object, and the handler walks the members in sorted order so details are deterministic:

```
  for (const key of Object.keys(patchBody).sort()) {
    const value = patchBody[key];
    if (key === "display_name") {
      if (value === null) {
        details.push("display_name cannot be removed");
      } else if (typeof value !== "string") {
        details.push("display_name must be a string");
      } else {
        const bad = validateDisplayName(value);
        if (bad.length === 0) {
          name = value;
        }
        details.push(...bad);
      }
    } else if (
      ["id", "email", "created_at", "version", "roles"].includes(key)
    ) {
      details.push(`${key} is immutable`);
    } else {
      details.push(`unknown field ${key}`);
    }
  }
```

listing 194: the member walk: null on the name refuses, immutable members refuse, unknown refuse (source: javascript/api/src/users/handlers.mjs, lines 220-241)

The contract makes one member writable, the display name, and the walk shows the full policy in one `if` chain. A null on it answers 422, because deleting the display name would leave the record invalid. The immutable members, `id`, `email`, `created_at`, `version`, `roles`, refuse the moment they appear, null or not: patching them is not a validation typo, it is an attempt to write a field the contract closed, and the same strictness the schema rung gave register arrives here through the walk because a parsed object cannot carry that rung. The empty patch object is legal and answers 200 with the current representation, nothing bumped.

After the walk, the precondition and the write run in order. The client's `If-Match` must equal the etag of the current body, else 412, and the update carries the expected version into the store so a write that lost a race answers 412 rather than clobbering. The no-op case short-circuits: the store is never called, the current body answers.

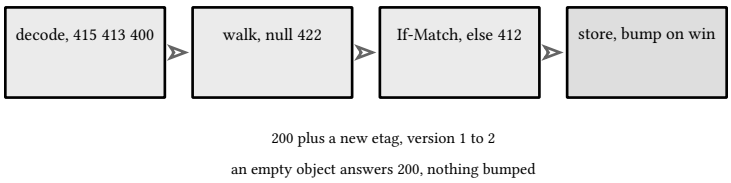


figure 145: a patch walks the members, then the precondition, then the store

25.5. keyset pagination

Offset pagination asks for row 86041 onward, and it fails twice. The store still walks and discards every skipped row, so page 4000 costs what a table scan costs. And offsets are positions in a moving file: one registration between requests and a row appears twice or vanishes. Keyset pagination fixes both by remembering where it stopped rather than counting from the front. The order is the pair `created_at` then `id`, the cursor encodes the last row's position, and the next page is everything strictly after it:

Encoding is three statements, stringify the two-member literal, wrap it in `Buffer.from`, call `toString("base64url")`, and vector 13 freezes the exact bytes. Decoding answers null for every malformed shape:

```
export function decodeCursor(raw) {
  if (typeof raw !== "string" || raw === "" || raw.includes("=")) {
    return null;
  }
  let payload;
  try {
    payload = JSON.parse(Buffer.from(raw, "base64url").toString("utf8"));
  } catch {
    return null;
  }
  if (
    payload === null ||
    typeof payload !== "object" ||
    !Number.isSafeInteger(payload.t) ||
    payload.t ≤ 0 ||
    payload.t > MAX_CURSOR_MS ||
    typeof payload.i !== "string" ||
    payload.i === ""
  ) {
    return null;
  }
  return { t: payload.t, i: payload.i };
}
```

listing 195: decode: every malformed shape answers null, the timestamp bounded to the renderable range (source: javascript/api/src/users/pagination.mjs, lines 22-44)

The payload is deliberately tiny, two members, because the cursor is a position, not a page number. Time is milliseconds since the epoch, the id breaks ties when two users share a creation millisecond, and uuid v7 ids sort by their own embedded time, so the composite order is total. The encoding is `Buffer.toString("base64url")`, node's one-call form of what the go lane built from `encoding/base64`, opaque to clients and trivially decodable in a test. Decode rejects padding, non-objects, non-integer `t`, and an empty `i`, all with null, because garbage is a 422, never a 404. The limit must be an integer in 1..100 with a default of 20, and one probe from the full stack earned its keep here: `URLSearchParams.get` answers null for a missing key, not undefined, so the absence check reads both.

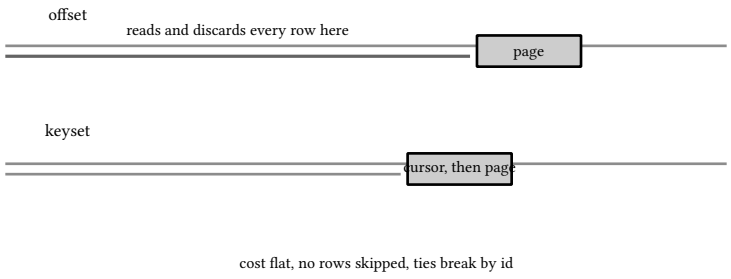


figure 146: offset walks the whole prefix, keyset jumps to the remembered position

25.6. the status contract

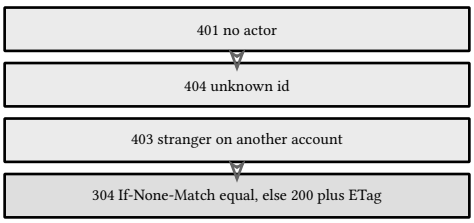
Every failure this layer can produce already has a code in the kernel's table, so the resource layer states outcomes and the table states statuses. The get handler is the whole ladder in one screen: 401 without an actor, 404 for an unknown id, 403 for a stranger touching someone else's record, then the etag work:

```
export async function get(ctx, d) {
  const actor = requireActor(ctx);
  let user;
  try {
    user = d.store.get(ctx.params.id);
  } catch (err) {
    throw mapStoreError(err);
  }
  if (actor.userId !== user.id && !adminActor(actor)) {
    throw fail(CODES.forbidden, "not the owner");
  }
  const etag = etagOf(wireBody(user));
  if (ctx.req.headers["if-none-match"] === etag) {
    ctx.res.setHeader("ETag", etag);
    writeNoContent(ctx, 304);
    return;
  }
  writeAnswer(ctx, 200, projectValue(user, true), etag);
}
```

listing 196: the failure ladder, then the one status that is not a failure, 304 (source: javascript/api/src/users/handlers.mjs, lines 136-154)

After the ladder comes the 304. If the client's `If-None-Match` equals the computed etag, the body it already holds is current, and the service answers the header and nothing else. The order inside the ladder is a policy choice this chapter owns: the 404 answers before the 403, so an unknown id and a stranger's id are distinguishable, and the authz chapter walks that tradeoff explicitly. What the contract forbids is a third path, no route here invents a status or an envelope shape, because every failure goes through one `fail` throw the kernel renders.

The store underneath speaks outcomes, not statuses, `StoreError` codes mapped once in one helper. And node's single thread deletes a whole go concern: the memory store is a map, an email index, and a sorted id array with no mutex anywhere, because every method is synchronous and nothing awaits mid-mutation. The store chapter revisits the point when the sqlite engine lands, `DatabaseSync` is synchronous by design.



patch adds 412 above the ladder, one envelope for every failure

figure 147: one request climbs the ladder, each rung one contract code

25.7. golden handler tests

The frozen vectors under `contract/testdata` are the fixture source of truth, and the tests read them as data: ids, timestamps, etag strings, cursor bytes all come from the JSON files, so a test never re-derives a constant the contract already froze. The harness is the kernel chapter's recorder pair plus a request double that carries its body as one async-iterable chunk, `readJson` runs its real ladder over it, and a `drive` helper runs one handler the way the stack runs it, the recover adapter rendering a thrown `ApiError` into the recorder:

```
test("the list walk reproduces vector 13's two pages exactly", async () => {
  const fx = fixture();
  seedWalk(fx);
  const page1 = ctxFromVectorRequest(v13, { actor: graceActor });
  await drive(list, page1, fx.deps);
  assert.equal(page1.res.statusCode, 200);
  assert.equal(page1.res.body, JSON.stringify(v13.response.body));

  const page2 = ctxFromVectorRequest(v13.response.then, { actor: graceActor });
  await drive(list, page2, fx.deps);
  assert.equal(page2.res.statusCode, 200);
  assert.equal(page2.res.body, JSON.stringify(v13.response.then.response.body));
});
```

listing 197: the vector walk: two pages, cursor bytes exact, items exact (source: javascript/api/test/users/handlers.test.mjs, lines 314-327)

Exactness is the point. The etag assertion proves the body bytes byte for byte, the etag is the sha256 of those bytes, and the cursor assertion proves the encoding down to the missing padding. The fixture injects everything time-dependent: a mutable millisecond clock advanced by hand, an id source handing out the vector's uuids, a fake hasher whose value never reaches the wire, and every response comes out of the recorder. Vector 02's replay is asserted here as the property the concurrency chapter's keyed wrapper replays, the same inputs produce the same snapshot bytes. The rest of the slice covers register 201 and 409, the 422 details array, unknown and immutable patch members, the stale `If-Match` 412, and the ownership grid. When a vector changes the code is wrong, and drift has nowhere to hide.

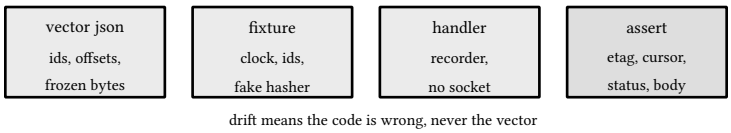


figure 148: a golden test: the vector file drives the fixture and judges the recorder

sources: RFC 7386, JSON Merge Patch, and RFC 9110 sections 8.8.3 and 13.1.1 on etags and conditional requests, at rfc-editor.org. nodejs.org/api/crypto.html for `createHash` and `digest`. nodejs.org/api/buffer.html for `Buffer.from` and the base64url family. nodejs.org/api/url.html for `URLSearchParams.get` answering null on a miss. developer.mozilla.org for `Object.entries`, `JSON.stringify` own-key ordering, and `String.includes`. Accessed 2026-09-26. Verified by `books/javascript/api/test/users` tests, 50 of them, under `npm run verify` with prettier clean.

26. authn

Authentication answers one question, who is calling, and this chapter builds every piece of the answer in `books/javascript/api/src/authn/`: scrypt password storage with parameters carried in the hash string, a login path flattened so unknown emails and wrong passwords cost the same, server-side sessions behind a store port, the cookie that carries the refresh token, HS256 and EdDSA json web tokens built from cryptographic primitives rather than a library, rotation with reuse detection on the refresh family, and the injected-clock test discipline that keeps all of it under a second. The go lane hashed with argon2id because go's platform extension ships it. Node ships `crypto.scrypt` natively and ships no argon2, so scrypt is the platform's memory-hard lane at zero dependencies, ratified for lesson parity, and the construction is decomposed below the way the argon2id lesson was.

26.1. password storage with scrypt

Passwords are never stored, only proofs that cost an attacker real time and memory to reproduce. Scrypt is the memory-hard answer: it fills and re-reads a block of memory a set number of times, so a gpu farm cannot parallelize guesses the way it parallelizes sha256, and the parameter set travels inside the encoded string. The tiers are named values, not magic numbers:

```
export const DEFAULT_PARAMS = {
  N: 131072,
  r: 8,
  p: 1,
  keylen: 32,
  maxmem: 160 * 1024 * 1024,
};

// TEST_PARAMS is the reduced test tier, a named confession: 1 MiB and
// the identical code path, each hash around two milliseconds on this
// machine, so the suite stays fast without forking the logic.
export const TEST_PARAMS = {
  N: 1024,
  r: 8,
  p: 1,
  keylen: 32,
```

listing 198: the issue tier is the owasp floor, the test tier is the same code path at 1 MiB (source: javascript/api/src/authn/password.mjs, lines 21-36)

The issue tier follows the OWASP password storage cheat sheet's scrypt floor, `N=2^17` at 128 MiB with `r=8` and `p=1`, and one probed fact on node 26.3.0 matters more than the numbers: the default `maxmem` budget of 32 MiB rejects that floor outright with `memory limit exceeded`, so the tier carries an explicit 160 MiB budget or the strongest parameters never run at all. The test tier is deliberately named `TEST_PARAMS`, a confession, 1 MiB and roughly two milliseconds per hash on this machine while exercising the identical code path. A structural test pins the issue floor, `N` above the test tier and the budget above 128 MiB, without ever running it.

Scrypt is worth naming as a construction, because it is a schedule over primitives, not a new primitive. The inner KDF is pbkdf2 with HMAC-SHA256, itself a chain of sha256 compressions over the salt and a counter, and the same pbkdf2 is the comparison the cheat sheet offers at 600000 iterations as the non-memory-hard fallback. The schedule around it is ROMix: fill a memory array of `128 * N * r` bytes by walking a pseudorandom index chain, each new block mixing one block from earlier in the chain, then re-read the whole array in the same order and re-mix, so an attacker who shrinks the array cannot answer the mixing chain. BlockMix is the permutation that shuffles each round of `2r` blocks through salsa20/8, and `N`, `r`, `p` travel in the PHC-style string so verification replays the parameters the hash was written with, never today's defaults. Node's `scryptSync` takes exactly those three plus `maxmem`:

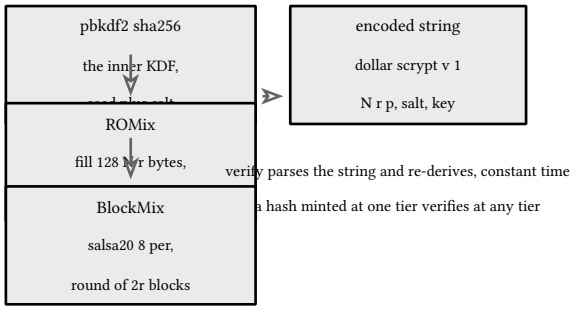


figure 149: scrypt: pbkdf2 seeds it, ROMix fills and re-reads memory, the params ride along

26.2. login and timing flattening

A login endpoint that answers fast for unknown emails and slowly for wrong passwords hands out an account census to anyone with a stopwatch. The flattening fix sits in the unknown-email branch: run the same scrypt verification the wrong-password path runs, against a dummy hash computed once at service construction, discard the result, and answer the identical bytes:

```
  let user = null;
  try {
    user = d.users.getByEmail(normalizeEmail(email));
  } catch (err) {
    if (!(err instanceof StoreError) || err.code !== "not_found") {
      throw err;
    }
    // unknown email: run the same verify against the dummy hash and
    // discard the result, then answer the identical bytes
    d.hasher.verify(d.dummyHash, password);
    throw fail(CODES.unauthorized, "invalid credentials");
  }
  if (!d.hasher.verify(user.passwordHash, password)) {
    throw fail(CODES.unauthorized, "invalid credentials");
  }
  return issue(ctx, d, user.id);
}

// issue mints the session, the refresh token, the cookie, and the
```

listing 199: both failure branches answer the same 401 after the same verify work (source: javascript/api/src/authn/handlers.mjs, lines 99-117)

Both branches return the same envelope, code, message, and no distinguishing header, and the rate limiting chapter adds the same three rate headers to both when its wrapper wraps this route. The only observable difference left is network jitter. A counting double in the suite pins the flatten itself, exactly one verify call on each failure branch. The success path runs on through `issue`, which mints the session, the refresh token, the cookie, and the access token in one place, so login stays readable as two facts: prove, then issue.

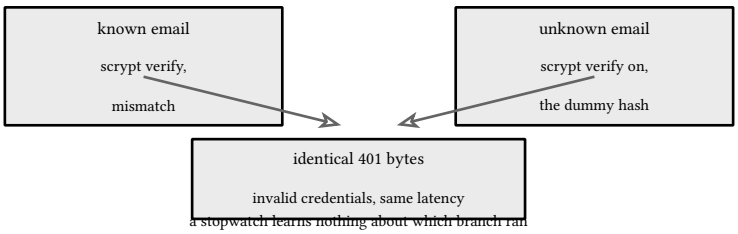


figure 150: the timing-equalized login: unknown email pays what a wrong password pays

26.3. server-side sessions

A session is the server's own record of one login: an id, the user, the expiry, a revoked flag, and the sha256 of the current refresh token. The token itself is never stored, only its hash, so a stolen store dump cannot be replayed against the live service. The store port mirrors the user family's: the shape frozen here, the in-memory implementation in this chapter, the engine implementation in the store chapter. The lifetime rules stay with the caller, on purpose: the store knows structural state, the service knows the time, and `alive` is one boolean expression both share:

```
  rotate(currentHash, nextHash, expiresAtMs) {
    const rec = tokens.get(currentHash);
    if (rec === undefined) {
      throw new SessionError("unknown_refresh", "unknown refresh token");
    }
    const s = sessions.get(rec.sid);
    if (rec.superseded || s.revoked) {
      s.revoked = true;
      throw new SessionError("session_revoked", "session revoked");
    }
    tokens.set(currentHash, tokenRec(s.id, true));
    tokens.set(nextHash, tokenRec(s.id, false));
    s.refreshHash = nextHash;
    s.expiresAtMs = expiresAtMs;
    return { ...s };
  },
```

listing 200: rotate: superseded tokens revoke the family, unknown ones miss (source: javascript/api/src/authn/session.mjs, lines 79-94)

Rotation is where the design earns its keep. A superseded token's record is kept, not deleted, because presenting one is the strongest theft signal the design gets: the legitimate holder moved on, so someone else is replaying. The answer is family revocation, the session row flips to revoked and every token it ever minted dies, the replayed one and its successor together.

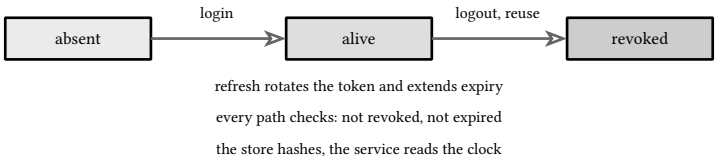


figure 151: the session lifecycle: login creates, refresh rotates, logout or reuse revokes

26.4. cookie mechanics

The browser carries the refresh token in a cookie, and the cookie's attributes are its entire protection. The builders are plain string templates because the line is frozen by the vectors, order included:

```
export function sessionCookie(token) {
  return `${COOKIE_NAME}=${token}; Path=/api; HttpOnly; Secure; SameSite=Lax`;
}

// clearSessionCookie expires the cookie: Max-Age 0 rendered exactly,
// attributes kept so the deletion overwrites the exact cookie it
// replaces.
export function clearSessionCookie() {
  return `${COOKIE_NAME}=; Path=/api; Max-Age=0; HttpOnly; Secure; SameSite=Lax`;
}

// readCookie pulls one cookie's value out of an incoming Cookie
// header. pairs split on the first equals sign, the first occurrence
```

listing 201: the session cookie and its deletion, four attributes, one line each (source: javascript/api/src/authn/cookie.mjs, lines 11-23)

`Path` scopes it to the api tree so it never rides on requests to other origins hosted beside the service. `HttpOnly` keeps scripts from reading it, so an injected snippet cannot exfiltrate what it cannot see. `Secure` keeps it off plain http. `SameSite=Lax` blocks the cookie on cross-site posts while allowing ordinary top-level navigation. Logout clears it with `Max-Age=0` rendered exactly, the attributes staying set so the deletion overwrites the exact cookie it replaces, and the tests assert both lines byte for byte against the vectors. Reading is a small parser of its own: pairs split on the first equals sign, first occurrence wins, and a longer name sharing a prefix must not match.

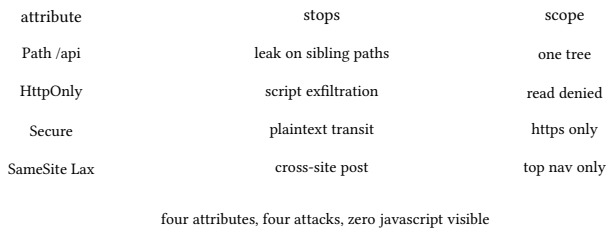


figure 152: the cookie attribute matrix: what each one stops

26.5. jwt from scratch with hmac

A json web token is three base64url parts, header, claims, signature, and the whole format fits in two functions of cryptographic primitives. HMAC-SHA256 signs the first two segments with a shared secret, through `createHmac` and `timingSafeEqual`, and the claims run after the signature, never before:

```
function parse(token, wantAlg, signatureOk, nowSec) {
  const seg = decodeSegments(token);
  if (seg === null) {
    return null;
  }
  let headerBytes;
  try {
    headerBytes = Buffer.from(seg.header, "base64url");
  } catch {
    return null;
  }
  let header;
  try {
    header = JSON.parse(headerBytes.toString("utf8"));
  } catch {
    return null;
  }
  // the pin runs before any payload byte is read: a token claiming
  // another algorithm, none included, dies here
  if (header.alg !== wantAlg || header.typ !== TYP_JWT) {
    return null;
```

listing 202: parse: the alg pin runs before any payload byte is read (source: javascript/api/src/authn/jwt.mjs, lines 95-115)

The claims literal is alphabetical by key, `exp`, `iat`, `iss`, `sid`, `sub`, insertion order is serialization order, so the same facts marshal to the same bytes and the golden test reproduces the frozen vector 04 token exactly, signature included. There is no roles claim on purpose: authorization reads the store, and the `sid` claim makes every outstanding token revocable by killing one session row. The parser refuses what the classics try: an alg none header dies at the pin before its payload is touched, a tampered claim or a wrong secret fails the constant-time compare, an expired or foreign-issuer token fails the claim checks. Every refusal answers null, one boolean fact in the node idiom, and the tests pin which check refused by constructing each input, not by an error taxonomy the handler would only collapse into one 401.

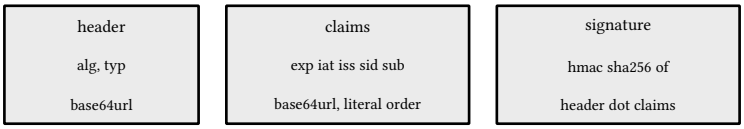


figure 153: three segments, the signature covers exactly the first two

26.6. jwt with eddsa

HMAC has one secret and both sides hold it, so whoever can verify tokens can also mint them. In one process that is fine, and it is why the issued lane stays HS256: one signer, one verifier, one process. The moment verification spreads, a public-key scheme earns its complexity. EdDSA splits the pair, the private key signs, the public key verifies, and the verification side provably cannot forge:

```
export function signEd25519(claims, privateKey) {
  const { header, payload } = segments(claims, ALG_EDDSA);
  const sig = edSign(null, Buffer.from(`${header}.${payload}`), privateKey);
  return `${header}.${payload}.${sig.toString("base64url")}`;
}

// parseHS256 checks segments, algorithm, signature, issuer, then
```

listing 203: the same envelope, an ed25519 signature (source: javascript/api/src/authn/jwt.mjs, lines 65-71)

This is the true ed25519 lesson, and it is cross-language: the c sharp lane could not deliver it. The windows base class library has no ed25519, so that lane teaches ES256 over its ecdsa classes instead, while node's `node:crypto` ships ed25519 keypairs and `sign` and `verify` calls natively, probed on 26.3.0. The two parsers cross-check each other in the tests: the HS256 parser refuses the EdDSA token on the alg pin, the EdDSA parser refuses the HS256 token the same way, and the alg confusion attack dies at the header. The claim checks are shared through one `parse`, so issuer and expiry rules never fork between algorithms.

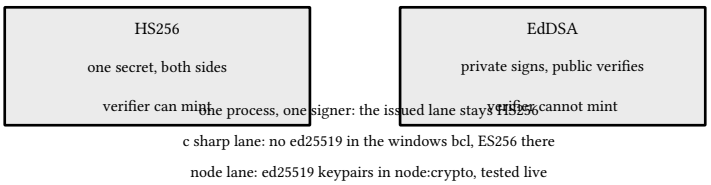


figure 154: shared secret versus split key: who can mint, who can check

26.7. refresh rotation and reuse detection

Access tokens live fifteen minutes, the refresh token is the long-lived credential, and it rotates on every use. Rotation means the store remembers the token it just retired, marked superseded, because presenting a superseded token is the theft signal, and the answer is family revocation:

```
  const next = newRefresh(d);
  let session;
  try {
    session = d.sessions.rotate(hash, hashToken(next), nowMs + SESSION_TTL_MS);
  } catch (err) {
    if (!(err instanceof SessionError)) {
      throw err;
    }
    if (err.code === "session_revoked") {
      // presenting a superseded token revokes the family: the rotated
      // token dies here too, and the freshly minted next never enters
      // the store
      throw fail(CODES.unauthorized, "session revoked");
    }
    throw fail(CODES.unauthorized, "invalid credentials");
  }
  const access = accessOf(d, session.id, session.userId, nowMs);
  ctx.res.setHeader("Set-Cookie", sessionCookie(next));
  return writeJson(ctx.res, 200, { access_token: access });
}
```

listing 204: the reuse answer: the family dies, the fresh successor never enters the store (source: javascript/api/src/authn/handlers.mjs, lines 155-174)

The vector pair pins the whole story. Refresh at plus twenty returns a new cookie and a new access token, same session id. Replay the old cookie at plus twenty-one and both die: the replay answers 401 `session revoked`, and the successor, still perfectly formed, answers 401 `session revoked` one second later. An attacker who stole the token before the holder refreshed gets one use, then loses the family along with the victim. One wrinkle the handler carries: a merely expired session does not revive, while a revoked one keeps flowing into rotate so its holders read `session revoked`, not a generic miss, the message the vector freezes.

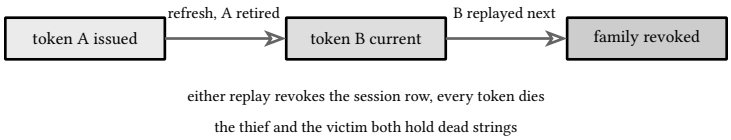


figure 155: the refresh family: rotation forks, reuse revokes everything

26.8. testing auth without walls

Every time-dependent behavior here is tested as arithmetic. The clock is a mutable millisecond the fixture advances by hand, so token expiry is one addition and the exp boundary is exact to the second. The id source hands out the vector's uuids, so the `sid` claim lands exact. The random source hands out the vector's 32-byte refresh tokens, so the `Set-Cookie` line reproduces the frozen bytes, and a counting double proves the dummy verify ran on the unknown-email path:

```
function fixture({
  ids = [sid],
  randoms = [tokenA, tokenB],
  offsetSeconds = 0,
} = {}) {
  let atMs = T0_MS + offsetSeconds * 1000;
  const idSeq = [...ids];
  const randSeq = [...randoms];
  const users = createMemoryStore();
  users.create({
    id: adaId,
    email: "ada@example.org",
    displayName: "ada lovelace",
    roles: [],
```

listing 205: the fixture: clock, id source, random source, all injected (source: javascript/api/test/authn/handlers.test.mjs, lines 84-97)

No test sleeps, no test opens a socket, and the scrypt test tier keeps the whole authn suite in tens of milliseconds. The identity layer tests beside them: one actor per credential channel, the bearer lane checking signature then session, the cookie lane resolving the opaque token by hash, garbage and expiry answering nothing, and the bearer dying the second its session row revokes. When the vectors change the code is wrong, and nothing in this chapter has a second source of truth to drift toward.

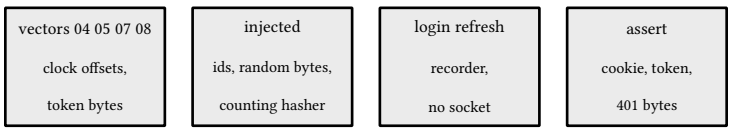


figure 156: an auth golden test: fixed inputs in, frozen bytes out

sources: cheatsheetseries.owasp.org, Password Storage Cheat Sheet, for the scrypt floor `N=2^17`, `r=8`, `p=1` and the pbkdf2-HMAC-SHA256 600000-iteration comparison, accessed 2026-09-26. nodejs.org/api/crypto.html for `scryptSync` with its options and `maxmem`, `createHmac`, `timingSafeEqual`, `generateKeyPairSync`, `sign`, `verify`, and `randomBytes`. RFC 7914, The scrypt Password-Based Key Derivation Function, and RFC 7519 plus RFC 8032 for jwt structure and EdDSA, at rfc-editor.org. developer.mozilla.org for `Set-Cookie` attributes and `Math.floor`. Accessed 2026-09-26. Verified by `books/javascript/api/test/authn` tests, 46 of them, under `npm run verify` with prettier clean.

27. authz and rbac

Authentication settled who is calling. Authorization decides what that caller may do, and this chapter builds the whole answer in `books/javascript/api/src/authz/`: the distinction between the two questions, roles as stored data with a grant and revoke surface, policies as plain functions, the 401 versus 403 discipline, object-level authorization on the routes that carry ids, a deny-by-default guard with a route audit that keeps every pattern honest, and the role by endpoint matrix test that pins the whole grid in one table. The actor is chapter 26's product, the roles live in chapter 25's store, and this chapter is the thin, total layer between them and the routes.

27.1. authentication versus authorization

Two questions, two layers, two failure codes. Authentication runs first and installs an actor or does not, the identity layer from the last chapter writing `ctx.actor`. Authorization reads that actor against the route and the object. A policy never parses a token, checks a signature, or reads a cookie, because by the time one runs the actor question is settled:

```
export function anyone() {}

// authenticated requires a resolved actor and nothing more: the caller
// proved who they are, and the route holds no object.
export function authenticated(ctx) {
  if (ctx.actor === undefined || ctx.actor === null) {
    throw fail(CODES.unauthorized, "authentication required");
  }
}
```

listing 206: the two cheapest policies: open, and any-actor (source: javascript/api/src/authz/policy.mjs, lines 12-21)

Conflating the two questions produces the classic broken answer, a 403 for an anonymous request. A caller that never proved anything has no authorization to deny, only an authentication demand, and the distinction is observable: 401 means try again with credentials, 403 means do not bother, this identity will never pass. Policies deny the way handlers fail, one thrown `ApiError` the kernel renders, so a policy is three lines of plain code and nothing else.

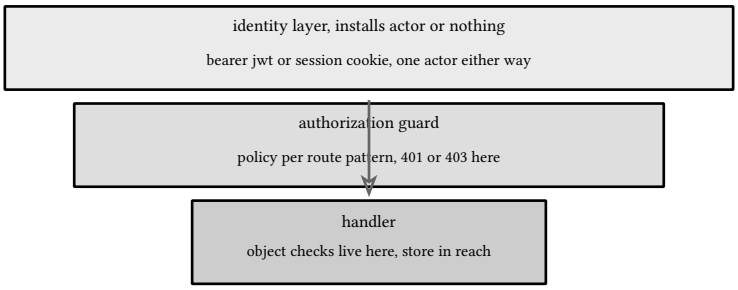


figure 157: two layers in wiring order: identity outside, guard inside, handler last

27.2. roles and the grant table

Roles are rows, not magic. Each account carries a set, the bootstrap rule grants the first registration admin, and the admin routes add and remove one role at a time, answering the resulting set. The invariant that matters is never zero admins, and it lives in the user store where both mutating verbs run it, grant and revoke alike:

```
  function adminsRemainWith(candidate) {
    for (const u of byId.values()) {
      const effective = u.id === candidate.id ? candidate : u;
      if (effective.roles.includes(ROLE_ADMIN)) {
        return;
      }
    }
    throw new StoreError("last_admin", "last admin");
  }

  return {
```

listing 207: the admin invariant: the scan substitutes the candidate, a rejected demotion leaves no trace (source: javascript/api/src/users/store.mjs, lines 54-64)

A revoke computes the post state, checks that some admin still stands across the whole table, and only then persists, so the rejected demotion leaves no trace. The frozen vector 14 walks the story: demoting the only admin answers 409 `last admin`, granting admin to a second user succeeds, and revoking that second admin then succeeds too, because the first one is standing again. The bootstrap rule makes the lockout unreachable in practice, the invariant makes it unreachable in theory.

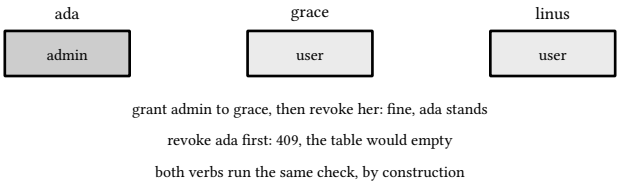


figure 158: the grant table: one column per user, the admin row must never empty

27.3. policies as functions

A policy is one function from the context to a verdict: return to allow, throw to deny. The whole surface composes from four of them, open, any-actor, a required role, an owner-or-admin check, and each is short enough to read in one breath:

```
export function requireOwner(param) {
  return (ctx, { params }) => {
    authenticated(ctx);
    const id = params[param];
    if (
      id === undefined ||
      (ctx.actor.userId !== id && !adminActor(ctx.actor))
    ) {
      throw fail(CODES.forbidden, "not the owner");
    }
  };
}
```

listing 208: the object policy: own the id or hold admin (source: javascript/api/src/authz/policy.mjs, lines 35-46)

The pattern's params arrive as an argument because of one subtlety: the guard layer runs before the router dispatches, `ctx.params` fills only at dispatch, so the guard resolves the wildcard itself, walking path and pattern in lockstep with the same static-beats-param rule the router holds. Plain functions beat a rules dsl or decorator mining at this size because the table is data the audit test can walk, and a new rule is a function next to its three siblings.

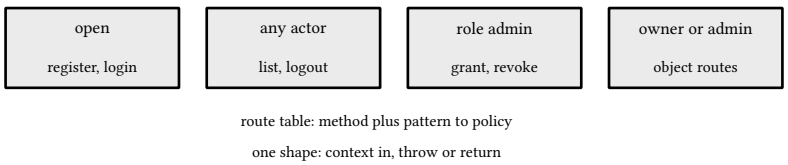


figure 159: four policies, one shape, the table maps routes onto them

27.4. 401 versus 403

The boundary lives in one place per policy: the actor check answers 401, the rule check answers 403, and nothing in between exists. Read it as a state machine. Anonymous gets 401 with the same message every time, `authentication required`, no hint about the route's existence or its rules. A present actor that fails the rule gets 403 with the rule's own words, `insufficient role` or `not the owner`, a statement about the actor, never about the resource. The one deliberate blur is existential: this service answers 404 for an unknown id before 403 for a stranger's id, which reveals that an id exists to anyone authenticated. Hiding it instead means answering 404 for both, and the tradeoff is real, disclosure against debuggability. What is never acceptable is mixing the codes by accident, a 403 for a missing `Authorization` header, a 401 for a demoted admin:

```
export function requireRole(role) {
  return (ctx) => {
    authenticated(ctx);
    if (!ctx.actor.roles.includes(role)) {
      throw fail(CODES.forbidden, "insufficient role");
    }
  };
}

// requireOwner builds the object-level policy: the actor owns the
// path's id, or holds admin. this is the check whose absence the
```

listing 209: the discipline: no actor is 401, wrong actor is 403 (source: javascript/api/src/authz/policy.mjs, lines 23-33)

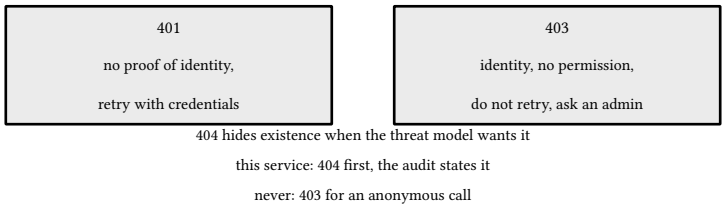


figure 160: the two failure codes, and the one deliberate blur

27.5. object-level authorization

Route rules cannot guard a route whose danger lives in the path parameter. `GET /api/users/` followed by someone else's id is the IDOR class, insecure direct object reference, and it survives every route-level check because the route is the same for everyone. The check must happen after the fetch, where the code knows whether the object exists and who owns it, which is why the object routes carry only the any-actor row in the table and the ownership rule runs in the handler:

```
test("the idor grid: a stranger on every object verb, unknown id 404", async () => {
  const { app, users } = compose();
  seedActors(users);
  // linus registers himself, then reads ada's record: 403 on every
  // object verb whatever etag he presents, 404 on an unknown id
  const linusBearer = await bearerOfRegisterLinus(app, users);
  const linus = { authorization: `Bearer ${linusBearer}` };

  const read = await call(app, {
    method: "GET",
    path: `/api/users/${adaId}`,
    headers: linus,
  });
  assert.equal(read.statusCode, 403);
  const write = await call(app, {
    method: "PATCH",
    path: `/api/users/${adaId}`,
    headers: { ...linus, "if-match": '"x"' },
    body: { display_name: "nope" },
  });
  assert.equal(write.statusCode, 403);
  const del = await call(app, {
```

listing 210: the idor test: a stranger on every object verb, unknown id 404 (source: javascript/api/test/authz/authz.test.mjs, lines 295-316)

The test pins both edges. A stranger touching ada's id gets 403 on read, write, and delete, whatever etag he presents, and an unknown id gets 404 for everyone, the ordering the last section discussed. The stranger is a real account, registered through the open route and logged in through the real stack, holding a living bearer token the identity layer resolves, because a hand-built actor would test the policy table but not the wiring. Putting the owner check in the guard instead would flatten unknown and forbidden into one 403, leak existence, and duplicate the store read the handler needs anyway, so the layering is not style, it is the only arrangement that answers all three questions correctly.

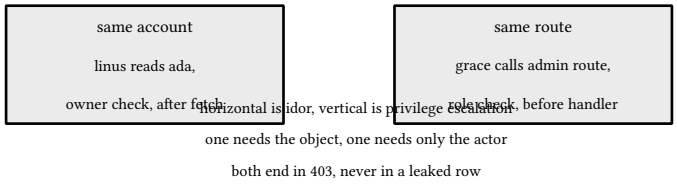


figure 161: horizontal versus vertical: two axes of privilege, two different checks

27.6. deny by default with a route audit

One table maps every registered pattern to its policy. The guard sits in the middleware chain inside the identity layer, resolves the pattern itself, runs the row's policy, and refuses what the table never saw:

```
export function guardLayer({ table, registered = null } = {}) {
  return (next) => async (ctx) => {
    const url = new URL(ctx.req.url, URL_BASE);
    const parts = url.pathname.slice(1).split("/");
    let policy = null;
    let params = null;
    let pathKnown = false;
    for (const [key, rowPolicy] of table) {
      const [rowMethod, rowPattern] = splitKey(key);
      const found = pathMatch(rowPattern, parts);
      if (found === null) {
        continue;
      }
      pathKnown = true;
      if (
        rowMethod === ctx.req.method ||
        (rowMethod === "GET" && ctx.req.method === "HEAD")
      ) {
        policy = rowPolicy;
        params = found;
      }
    }
    if (policy =/= null) {
      policy(ctx, { params, table });
      return next(ctx);
    }
    if (pathKnown) {
      // the path carries rows only under other methods: the router
      // owns the answer, 405 with Allow
      return next(ctx);
    }
    if (
      registered =/= null &&
      registered().some(({ pattern }) => pathMatch(pattern, parts) =/= null)
    ) {
      // a registered route with no row: deny by default
      throw fail(CODES.forbidden, "no policy for route");
    }
    return next(ctx);
  };
}
```

listing 211: the guard: a row runs its policy, a known path defers to the router, an unaudited route answers 403 (source: javascript/api/src/authz/guard.mjs, lines 50-90)

Three lanes leave the loop. A matched row runs its policy and passes to the router. A path that carries rows only under other methods defers, 405 with `Allow` is the router's answer, not an authorization verdict, and so is the 404 for a path nobody registered. The third lane is the deny-by-default leg: a route registered on the router but absent from the table answers 403 `no policy for route` before a handler byte runs. The shipped guard runs row-driven, `registered` null: the guard wraps the dispatch and cannot see the router's registrations, and the kernel's pattern list stays closed this pass, so a constructor-injected oracle seam carries the leg, filled by the tests and reserved for a later wiring. The audit test closes the other direction only, and says so: every table row must resolve to a living route, probed request by request through the full stack, none answering 404 or 405, so a row cannot point at nothing. A route registered without a row is the oracle's half, and until that wiring lands the audit does not claim it:

```
test("the audit: every table row resolves to a living route, none 404", async () => {
  const { app, users } = compose();
  seedActors(users);
  const adaBearer = await bearerOf(
    app,
    "ada@example.org",
    "correct horse battery staple",
  );
  for (const key of routePolicies.keys()) {
    const [method, pattern] = splitKey(key);
    const concrete = pattern.replace(":id", adaId).replace(":role", "admin");
    const res = await call(app, {
      method,
      path: concrete,
      headers: { authorization: `Bearer ${adaBearer}` },
      ...(method === "POST" && pattern.includes("roles")
        ? { body: { role: "admin" } }
```

listing 212: the audit: every row probed through the stack, none dead (source: javascript/api/test/authz/authz.test.mjs, lines 450-466)

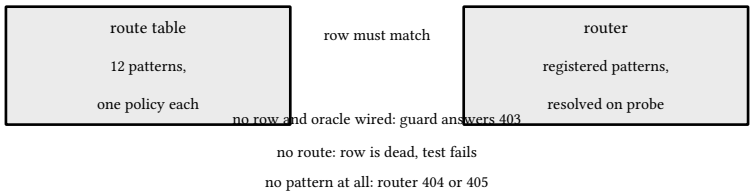


figure 162: the audit: table and router face each other, both directions checked

27.7. the rbac matrix test

The grid is the proof: every route times every actor kind, anonymous, regular user, the stranger on someone else's object, admin, one expected status per cell, asserted through the full guarded stack with living bearer credentials, the same path production traffic takes:

```
test("the rbac matrix: one status per route per actor, through the full stack", async () => {
  const { app, users } = compose();
  seedActors(users);
  const linusBearer = await bearerOfRegisterLinus(app, users);
  const adaBearer = await bearerOf(
    app,
    "ada@example.org",
    "correct horse battery staple",
  );
  const graceBearer = await bearerOf(
    app,
    "grace@example.org",
    "grace hopper rules",
  );
  users.create({
    id: "01a0f5d5-ffff-7000-8000-00000000d4",
    email: "fourth@example.org",
    displayName: "fourth account",
    roles: [],
    createdAtMs: T0_MS + 5000,
    passwordHash: "h:never logs in",
  });
  const asAda = { authorization: `Bearer ${adaBearer}` };
  const asGrace = { authorization: `Bearer ${graceBearer}` };
  const asLinus = { authorization: `Bearer ${linusBearer}` };
```

listing 213: the matrix: actor times route, one status each, real logins (source: javascript/api/test/authz/authz.test.mjs, lines 357-381)

Each row is a cell with a name that fails loudly. The grid runs the vector's last-admin chain alongside, so the 409 and the two 200s around it are pinned by frozen bytes, not just by intent. Adding a route costs three lines, one case per interesting actor, and the audit plus the matrix together answer the two questions reviewers actually ask: is every route covered, and does every covered route behave for every kind of caller.

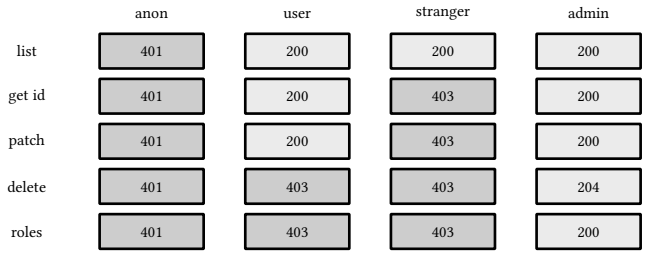


figure 163: the matrix: routes one way, actors the other, statuses in the cells

sources: OWASP, Insecure Direct Object Reference Prevention Cheat Sheet and Access Control Cheat Sheet, at cheatsheetseries.owasp.org, for the idor class and the deny-by-default guidance, accessed 2026-09-26. RFC 9110 sections 15.5.1 to 15.5.4 on 401, 403, and 404 semantics, at rfc-editor.org. developer.mozilla.org for `Map` iteration order and `Array.prototype.some`. Accessed 2026-09-26. Verified by `books/javascript/api/test/authz` tests, 12 of them, under `npm run verify` with prettier clean.

28. store

The service has run so far on in-memory maps, which tests love and restarts hate. The platform answer is `node:sqlite`, the synchronous sqlite binding shipped inside the runtime, zero dependencies, one import. This chapter owns the machinery outright: one connection per database, prepared statements with placeholders only, migrations as append-only records, the transaction idiom, the isolation wal actually gives you, the compare-and-swap that makes lost updates visible, and analytics in one query.

28.1. node:sqlite from first principles

`DatabaseSync` is synchronous on purpose. `prepare` hands back a `StatementSync`, the statement answers `run`, `get`, and `all`, and that is the whole surface the store uses: node 26.3.0 exposes no source-hash property and no reprepare hooks, so the chapter pins nothing of the sort. The probe on this machine, 2026-09-26, answers the stability question empirically: the import prints no `ExperimentalWarning` on 26.3.0, and `process.features.sqlite` does not exist here, so the code never cites that flag. Inside one process the sync shape is the design, one event loop schedules every call, each call is short, no pool to size, no promise surface to await. The engine underneath answers `sqlite_version()` as 3.53.4 on node 26.10.0, matching this corpus's pins; it answered 3.53.1 through the node 26.3.0 era, the known node delta, stated once here.

Values ride placeholders, never concatenation. `exec` runs bare sql and exists for ddl and transaction control alone, `stmt` prepares once and binds forever, and the constructor names the two settings that matter: foreign keys on, and wal for every database that lives in a file.

```
  constructor(path = ":memory:", { now = Date.now } = {}) {
    this.#path = path;
    this.now = now;
    this.#db = new DatabaseSync(path, { enableForeignKeyConstraints: true });
    // wal gives readers a snapshot without blocking the writer, the
    // isolation chapter's substrate. in-memory databases keep their
    // own journal mode.
    if (path !== ":memory:") {
      this.#db.exec("pragma journal_mode = wal");
    }
  }
```

listing 214: the constructor: one connection, foreign keys on, wal for files (source: javascript/api/src/store/store.mjs, lines 19-29)

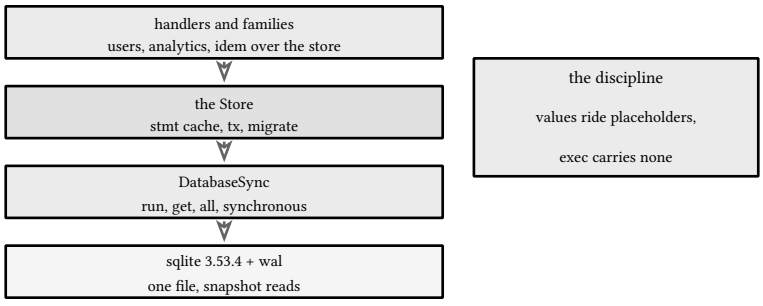


figure 164: the stack the store sits on

28.2. migrations test driven

Schema changes are code changes, so they ship as records, and the history is append-only by contract: a shipped version is frozen, a correction is a new numbered version, no record is ever edited. The registry is a table the database keeps about itself, `schema_migrations`, and `migrate` applies every pending record at most once, each inside its own transaction, so a version that fails halfway leaves nothing durable and can simply be retried.

This wave ships three records. `0001_init` marks the base vocabulary, users, roles, events, reports, one record for the whole scheme. `0002_user_versions` adds the `version` column and stands alone because it is the substrate the compare-and-swap stands on. `0003_idempotency_keys` adds the table the concurrency chapter fills. The test drives the runner from an empty directory, proves all three applied in order, re-runs and proves nothing applies twice, then closes and reopens to prove the registry came back with it.

```
  migrate() {
    this.exec(
      "create table if not exists schema_migrations (name text primary key, applied_ms integer not null)",
    );
    const done = new Set(this.applied());
    for (const m of migrations) {
      if (done.has(m.name)) continue;
      this.tx(() => {
        this.exec(m.sql);
        this.stmt(
          "insert into schema_migrations (name, applied_ms) values (?, ?)",
        ).run(m.name, this.now());
      });
    }
  }
```

listing 215: one version, one transaction, one registry row, applied at most once (source: javascript/api/src/store/store.mjs, lines 75-90)

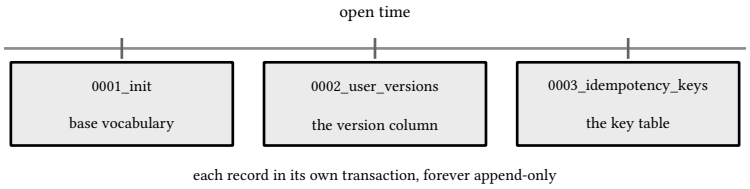


figure 165: the version list only grows right

28.3. one statement, prepared once

Preparing a statement is work: the engine parses the sql, plans it, and hands back a compiled object. Doing that per call is a tax, and the idiom is the cache: a `Map` keyed by the exact sql string, first caller prepares, every later caller receives the same object. The sql text is a constant in the source, so the same string always means the same statement. The test holds the cache to its promise, two `stmt` calls with one string answer one object.

```
  stmt(sql) {
    let s = this.#stmts.get(sql);
    if (s === undefined) {
      s = this.#db.prepare(sql);
      this.#stmts.set(sql, s);
    }
    return s;
  }
```

listing 216: the statement cache: one prepare per distinct sql string (source: javascript/api/src/store/store.mjs, lines 34-41)

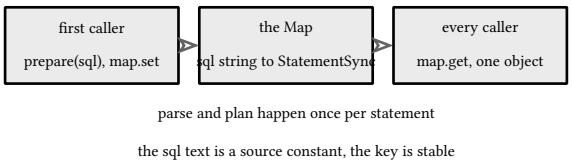


figure 166: same string in, same statement out

28.4. the transaction idiom

One shape covers every write in the family: `begin immediate`, run, `commit`, and the `catch` rolls back. Node has no unwinding and no cancellation context, a synchronous call either returns or throws, so the catch is the rollback and there is no third exit to guard. The function never manages its own transaction, the wrapper owns all three statements, and the discipline pays off twice: a handler that throws mid-group discards the whole group, and a constraint violation rolls back and surfaces raw, the caller above decides what it means. `immediate` takes the write lock up front, so writers serialize on the engine and the ordering questions never reach the family.

The suite proves the exits. The happy path lands the whole group. The throwing path discards it and leaves the connection free, the next write succeeds, a leaked transaction would refuse it. And the error the caller threw surfaces unwrapped, the marker object the test throws is the marker object the test catches.

```
tx(fn) {
  this.#db.exec("begin immediate");
  try {
    const out = fn(this);
    this.#db.exec("commit");
    return out;
  } catch (err) {
    this.#db.exec("rollback");
    throw err;
  }
}
```

listing 217: the idiom: begin immediate, commit on return, rollback on throw (source: javascript/api/src/store/store.mjs, lines 53-63)

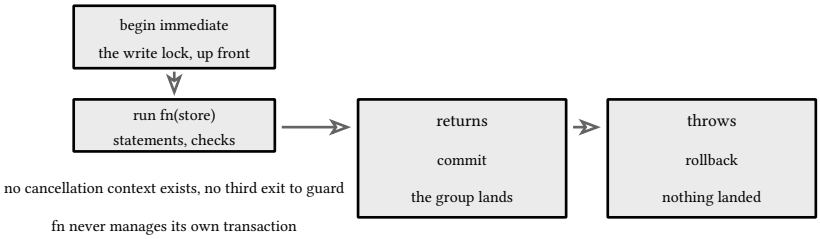


figure 167: a transaction has exactly two exits

28.5. the snapshot sqlite actually gives you

The textbook menu, read uncommitted through serializable, is a menu of guarantees a general engine picks among. What this engine serves is a fact about two settings the code states itself: wal journal mode, and `begin deferred` on the reader. A deferred read transaction pins its snapshot at the first read, every later read inside it sees the same view, and the writer commits past it without blocking, which is what wal buys. No dirty reads, because a reader's view is a state that was committed. No non-repeatable reads inside one transaction, because the view never moves. Write skew stays possible, two writers can act on disjoint stale reads and both commit clean. Writers serialize on `immediate`. That is snapshot isolation, and no configuration file stands between the claim and the two statements that make it true.

The demo stays deterministic, and in node the determinism is free: the interleave is the statement order, one thread, no callbacks, no timers. The reader opens and reads 1. The writer commits 2. The reader reads again inside its transaction and still sees 1. A fresh read sees 2. The reader commits and sees 2:

```
const reader = openStore(file);
reader.exec("begin deferred");
assert.equal(
  reader.stmt("select version from u where id = ?").get("ada").version,
  1,
);

writer.tx((s) => {
  s.stmt("update u set version = ? where id = ?").run(2, "ada");
});
```

listing 218: pin a view, commit past it, read again: the statement order is the schedule (source: javascript/api/test/store/isolation.test.mjs, lines 20-29)

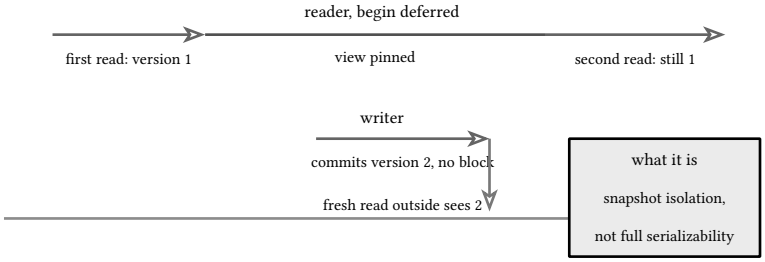


figure 168: one timeline: the reader's view holds

28.6. the lost update and the compare-and-swap answer

Two writers read the same user, both compute, both write, and one update silently vanishes. The engine reproduces it with two blind updates: both succeed, the second name is all that survives, and the `version` column never moved, so nothing anywhere records that anything was lost. Serialization does not fix this, the write lock guarantees each write lands whole and says nothing about the stale read the write was computed from. That invisibility is what makes the lost update the classic bug, the writer that caused it gets a success answer.

The version column is the detector and the compare-and-swap is the fix. The whole exchange runs inside one `begin immediate` transaction: the update refuses anything but the expected version, bumps and writes in the same statement, and `changes === 0` means a refusal, the current version read back so the caller can answer precisely. Of two writers holding version 1 exactly one wins, and the loser learns the fact, which over http is exactly the 412 the next chapter stages with two concurrent PATCHers.

```
    updateCas(id, expectedVersion, patch) {
      return store.tx((s) => {
        const res = s
          .stmt(
            "update users set display_name = ?, version = version + 1 where id = ? and version = ?",
          )
          .run(patch.displayName, id, expectedVersion);
        if (res.changes === 0) {
          const cur = s.stmt("select version from users where id = ?").get(id);
          if (cur === undefined) {
            return { ok: false, reason: "missing" };
          }
          return { ok: false, reason: "version", current: cur.version };
        }
        const row = s.stmt(`select ${COLS} from users where id = ?`).get(id);
        return { ok: true, row: plain(row) };
      });
    },
```

listing 219: the cas: refuse, bump, and write in one statement inside one transaction (source: javascript/api/src/store/users.mjs, lines 78-96)

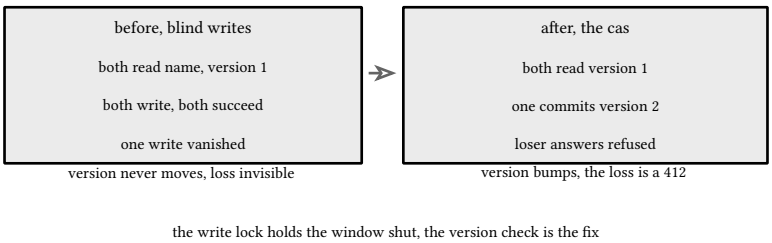


figure 169: same race, two outcomes: silent loss versus a refused writer

28.7. analytics in one query

Reports are sql's home turf, and the engine answers them natively. The events table carries the day and the user beside every fact, so the daily active count is one `group by` over distinct users in day order, and the frozen report rows fall out exactly: seed the three events the vector's setup names and the query answers the pair of rows the contract freezes. The window functions are where one query replaces a loop, and the probe pinned them on this build's engine, on 3.53.1 at first measure and again on the 3.53.4 that node 26.10.0 bundles: the running total is `sum(active) over (order by day)`, and each user's first day is `row_number() over (partition by user_id)`. Both answered in the probe, so the chapter promises them.

The queries stay in sql because the engine's own aggregation is the honest tool, and the rows leave the family as plain objects, the null-prototype rows `node:sqlite` answers are absorbed at the boundary.

```
    runningActive(fromDay, toDay) {
      return store
        .stmt(
          "select day, active, sum(active) over (order by day) as running from (select day, count(distinct
user_id) as active from events where day ≥ ? and day ≤ ? group by day) order by day",
        )
        .all(fromDay, toDay)
        .map(plain);
```

listing 220: the window function: a running total across the window in one query (source: javascript/api/src/store/analytics.mjs, lines 32-38)

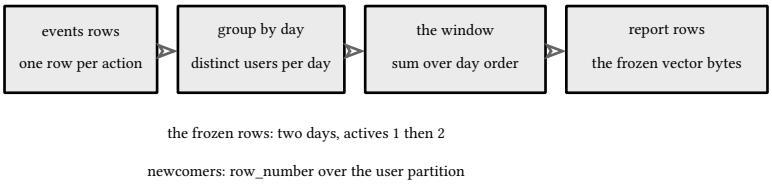


figure 170: keys to window to the report body

28.8. store tests without docker

Every test in the family starts the same way: open a store, an in-memory database or a fresh temp file, migrate, run, close. No container, no shared server, no ordering between tests, and nothing sleeps: the api is synchronous, the clock is injected, and the one clock-dependent path in the family, the idempotency key's 24 hour window, runs under `mock.timers` with `tick` as the only clock. The file facts are deterministic by construction, and the recovery questions the go lane asked its own engine do not exist here because the engine is sqlite's, not ours.

The property loop rides the prng the book already states, splitmix32, seed committed in the file. The invariant is named in the source: paging the keyset scan with arbitrary limits and the previous page's last row as the cursor concatenates to exactly the single full ordered scan, and the seed shuffles insertion order and forces timestamp collisions so the id half of the keyset carries weight. Node has no native fuzzing, so the loop is the property check and the seed is the case.

```
test("keyset paging round-trips the full scan at any limit", (t) => {
  const rng = splitmix32(0x12345678);
  const store = openStore(":memory:");
  store.migrate();
  t.after(() => store.close());
  const users = createUsers(store);
```

listing 221: the property loop: fixed seed, named invariant, one store per test (source: javascript/api/test/store/property.test.mjs, lines 24-29)

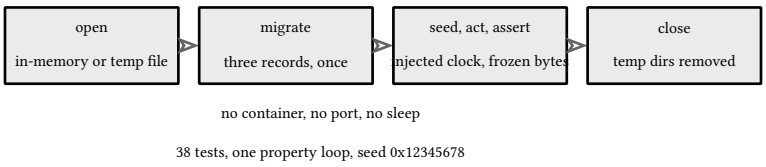


figure 171: a store test's lifecycle, nothing shared between runs

sources: nodejs.org/api/sqlite.html for the `DatabaseSync` constructor, `StatementSync`, and the run, get, all contracts, accessed 2026-09-26, nodejs.org/api/test.md for the `mock.timers` clock the idempotency window rides, accessed 2026-09-26, and sqlite.org/wal.html with sqlite.org/lang_transaction.html for the snapshot semantics of wal readers and the begin modes, accessed 2026-09-26. The stability and version facts are the T0b probe on node 26.3.0, 2026-09-26. Verified by the 38 `test/store` tests under `node --test` in `npm run verify`.

29. concurrency control

The store chapter's compare-and-swap gave one writer the win and the other the fact. This chapter moves that fact to the http edge and builds everything around it, re-derived for one thread instead of many: why node has no locks and still has races, strong etags over the exact bytes with the conditional GET and the guarded PATCH, the duel two concurrent PATCHers settle over real tcp, idempotency keys with stored response snapshots so a retried register replays instead of double-creating, and the in-flight window a promise map closes.

29.1. one thread, many interleavings

Node executes every statement of js on one thread, so the shared-memory data race, two writers inside one field with no ordering, does not exist here. What survives is the race condition, because `await` is a preemption point. Two async handlers interleaving at their awaits can read the same row, both compute, both write, and one update silently vanishes, exactly the lost update the go lane needed goroutines to stage. The interleave points are countable, everything between two `await`s is atomic, and that fact is the whole locking story of this vehicle: no mutexes because no second thread exists, and still protocols, because interleaving does.

So js needs the same three answers, for a different reason. The compare-and-swap in the store is the atomic layer. Etags carry versions to the client. Idempotency keys make retries safe. And the code that coordinates them leans on the atomic stretch: the keyed runner below keeps claim and lookup inside one await-free stretch, a critical section nobody had to lock, because the scheduler cannot enter it.

```
export function createKeyedRunner(keys, { routes = IDEMPOTENT_ROUTES } = {}) {
  const flights = new Map();

  return async function runKeyed({ route, identity, key, body, handler }) {
    if (!routes.has(route) || key === undefined || key === "") {
      return handler();
    }
    const requestHash = sha256Hex(body);
    const triple = route + "\0" + identity + "\0" + key;
```

listing 222: the runner head: ungoverned routes pass through, the triple names one flight (source: javascript/api/src/conc/guard.mjs, lines 34-42)

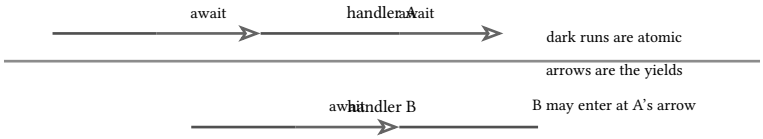


figure 172: one thread, preemption only at awaits

29.2. strong etags over the exact bytes

The strong etag of a representation is the sha256 hex of the exact response body, quoted. The exactness is the contract: the body is stringified in the contract's member order, insertion order is serialization order, and one byte of drift is a different representation, the test proves it by upper-casing one letter of a frozen body and refusing the equal etag. The frozen vectors carry the proof the other way: the hash of vector 09's body equals the ETag header the vector freezes, and vector 11's patched body hashes to its own header, so the mechanism reproduces the contract's bytes, not approximately but exactly.

The conditional GET spends the etag. An `If-None-Match` that equals the current etag answers 304 with the etag stamped and no body, the client already holds the body the etag names, and a 304 with a body would poison the client cache into holding the wrong thing for the wrong reason. Vector 10 freezes that exchange, the same etag

in, 304 out. The decision is pure, `null` means run the handler, anything else is the answer, and stale, absent, or a missing resource all pass through.

```
export function etagOf(body) {
  return '"' + createHash("sha256").update(body).digest("hex") + '"';
}
```

listing 223: the strong etag: sha256 hex of the exact body, quoted (source: javascript/api/src/conc/etag.mjs, lines 10-12)

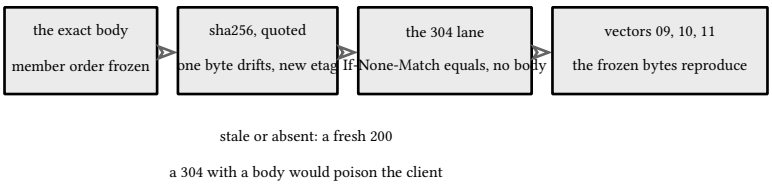


figure 173: body bytes to etag to the 304 lane

29.3. the duel: two patchers, one version

The unsafe side requires the precondition. `If-Match` is mandatory, the contract chose 412 over the 428 the RFC suggests, and a value stale against the current representation refuses the request before the handler burns its decode. Vector 12 freezes the refusal, code `precondition_failed`, message "etag mismatch", and the test pins the message from the vector's own bytes. The fast fail is the cheap layer, the store's compare-and-swap remains the guarantee, because a route can exist without the wrapper and the check and the write must still be one atomic exchange.

The duel stages the whole chain over real tcp: a `node:http` server mounting the real store, two PATCHers carrying the same valid etag, fired together through `Promise.all`. Both may pass the fast-fail check, the check reads the row before either write lands, and the compare-and-swap still crowns exactly one. The assertion is on the counts, one 200 and one 412, never the order, and version 2 standing after, which is what makes the test deterministic under any interleaving the loop picks.

```
  const etag = etagOf(representation(current));
  const shoot = (name) =>
    fetch(`http://127.0.0.1:${port}/api/users/u-ada`, {
      method: "PATCH",
      headers: { "Content-Type": "application/json", "If-Match": etag },
      body: JSON.stringify({ display_name: name }),
    }).then((r) => r.status);

  const statuses = await Promise.all([shoot("writer a"), shoot("writer b")]);
  assert.deepEqual(statuses.slice().sort(), [200, 412]);
  assert.equal(users.byId("u-ada").version, 2);
});
```

listing 224: the duel: two shooters, one Promise.all, counts asserted, never order (source: javascript/api/test/conc/duel.test.mjs, lines 88-100)

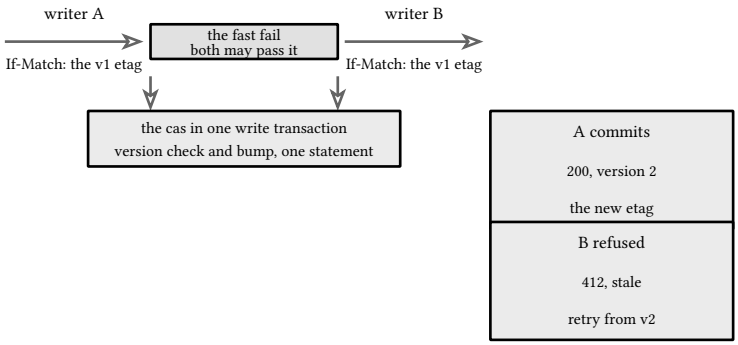


figure 174: two patchers, one fast fail, one cas, one winner

29.4. idempotency keys and stored snapshots

A client that times out retries, and a register route that recreates on retry is a double-charge machine. The contract's answer is the `Idempotency-Key` header. The exact request bytes are hashed under sha256, whitespace drift is a different request, and the key is scoped by route and caller identity, so two clients may use the same key freely. The first successful run is stored as a snapshot, status, headers, body, and a later request with the same key and the same hash replays those bytes at the stored status: vector 02's retried register returns the original 201, Location and ETag included, byte for byte.

Two failure paths are as deliberate. The same key under a different hash is refused with a thrown failure the kernel's one writer renders at 422, details array beside the code in the writer's member order, the one place the envelope carries details, and the guard never draws a body itself. The stored snapshot is untouched. A failing first run is never stored, a retry after a validation error deserves a fresh run, not a replayed failure, so only outcomes under 400 become snapshots. Keys live for 24 hours in the store chapter's `idem` table, expiry lazy on read. The reports route is deliberately not on this guard: replaying a completed job is a read, so a stored 202 upgrades to the completed report at 200, the one sanctioned divergence, and the test asserts the upgrade against vector 16's frozen bytes.

```
    for (;;) {
      const found = keys.lookup({ route, identity, key, requestHash });
      if (found.state === "mismatch") {
        throw fail(CODES.validation_failed, "validation failed", {
          details: [MISMATCH_DETAIL],
        });
      }
      if (found.state === "match") {
        return found.snapshot;
      }
```

listing 225: the two verdict exits: mismatch throws for the one writer, match replays the snapshot (source: javascript/api/src/conc/guard.mjs, lines 44-54)

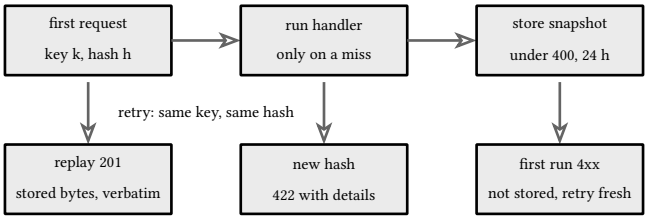


figure 175: one key's life: first run, snapshot, replay, refusal

29.5. the in-flight window

The missing half was concurrency, and the window is real even on one thread. A lookup that misses, a handler that awaits, a save that lands only after the handler resolves: two callers under the same key inside that window both miss, both run, and the loser answers the handler's own conflict for an operation that succeeded and is recorded under its key. The guard closes the window with a promise map keyed by the same route, identity, and key triple the stored rows live under. The first caller claims the slot, runs, and settles a promise on its way out, and a later caller awaits that promise and reads the settled row again. The await-free stretch from lookup to claim is the critical section, one thread cannot enter it, so there is no scheduling window at all.

The tests hold the window open with a gate, a promise the test resolves by hand, zero timers, zero sleeps. Two callers under one key collapse into one handler run and both receive the stored snapshot. The darker path is pinned too: a waiter whose first run failed, a 422 is never stored, wakes to find no row and proceeds with a fresh run, so a failed first attempt cannot wedge the key forever. And the map entry is gone the moment the run settles, so the herd that arrives after the lull fills again.

```
    const existing = flights.get(triple);
    if (existing === undefined) {
      // the first caller claims the slot and runs
      let settle;
      const promise = new Promise((resolve) => (settle = resolve));
      flights.set(triple, { promise });
      try {
        // the failure policy: only a run under 400 becomes a
        // snapshot, a retry after a validation error deserves a
        // fresh run, not a replayed failure
        const res = await handler();
        if (res.status < 400) {
          keys.save({ route, identity, key, requestHash, snapshot: res });
        }
        return res;
      } finally {
        flights.delete(triple);
        settle();
      }
    }
    await existing.promise;
  }
```

listing 226: claim or wait: first caller runs and settles, later callers await then re-read (source: javascript/api/src/conc/guard.mjs, lines 55-76)

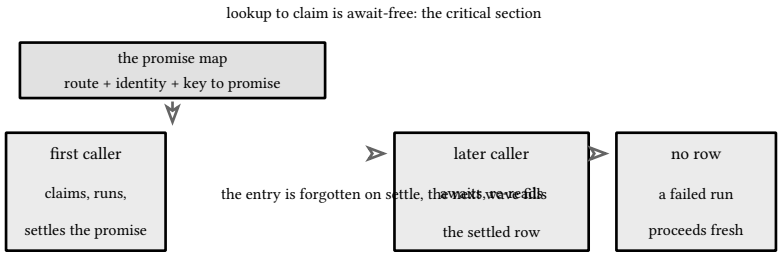


figure 176: the window, closed by a promise map

29.6. the reports route over the wire

The reports pair is the conc family's own route. `POST /api/reports` runs the ladder the contract orders: the actor first, 401 with no identity, the `Idempotency-Key` second, 422 with one detail when it is missing, then the body read once under the codec's cap with the exact bytes hashed before anything else looks at them, the strict member set

so an unknown member is a decode failure, and the window rules, one detail per broken rule, kind, day shapes, ordering, with days checked as real calendar days because 2026-13-99 has the right shape and no day behind it.

The interesting half is what the route does not need. From the lookup to the save, miss, insert the pending report row, store the 202 snapshot under the key, the whole exchange is one synchronous stretch, no `await` anywhere inside it, so the single thread makes it atomic and the concurrent double submit simply serializes: the second caller's lookup reads the first caller's row and replays. The register guard needed a promise map because its handler awaits, the reports route needs nothing because it does not. `GET` computes the series on first read and flips the row to done, and the replay answers the upgraded 200, vector 16 walked end to end over a real kernel app with the identity seam stubbed at the same layer the composition mounts:

```
app.route("POST", "/api/reports", async (ctx) => {
  const actor = requireActor(ctx);
  const key = ctx.req.headers["idempotency-key"];
  if (key === undefined || key === "") {
    throw fail(CODES.validation_failed, "validation failed", {
      details: ["idempotency key is required"],
    });
  }
  const { raw, body } = await readForHash(ctx);
  bindStrict(body);
  const details = validateWindow(body);
  if (details.length > 0) {
    throw fail(CODES.validation_failed, "validation failed", { details });
  }
  const requestHash = sha256Hex(raw);

  const found = idem.lookup({
    route: ROUTE,
    identity: actor.userId,
    key,
    requestHash,
```

listing 227: the create ladder: actor, key, exact-byte hash, strict bind, window rules (source: javascript/api/src/conc/reports.mjs, lines 105-125)

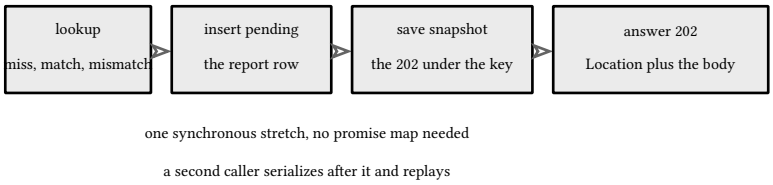


figure 177: claim to settle with no await inside: the loop cannot enter

sources: nodejs.org/api/crypto.html for `createHash` and the sha256 digest contract, accessed 2026-09-26, nodejs.org/api/http.md for the duel's `node:http` server, accessed 2026-09-26, nodejs.org/api/test.md for `mock.fn` and the timer discipline the gates replace, accessed 2026-09-26, and the `If-Match` and `If-None-Match` semantics read from developer.mozilla.org/docs/Web/HTTP/Headers with RFC 9110 sections 13.1.1 and 13.1.2 at rfc-editor.org, accessed 2026-09-26. Verified by `books/javascript/api` tests, 28 of them in `test/conc`, the duel over real tcp and the vector 16 route walk included, under `node --test` in `npm run verify`.

30. caching

A cache is a promise to answer from a copy, and every promise has a price. This chapter prices it honestly in the node idiom: what caching promises and what it silently trades away, the LRU over one `Map` where the go book needed a map and a list, TTL beside LRU so age evicts as well as crowding, the stampede guard as a singleflight hand rolled from a promise map, invalidation by delete after write, and the discipline of what never gets cached at all.

30.1. what caching promises

A cache answers from a copy to save the origin the work, and both sides of the bargain matter: the copy must be cheap to find, and the answer must be one the origin would still give. Every caching decision is a bet that the read will be asked again before it changes, and the three terms of the bet are capacity, how many copies, lifetime, how long each stays, and invalidation, what a write owes the copies it made wrong. Miss any term and the cache stops being an optimization and becomes a second source of truth, which is the failure mode every caching chapter exists to prevent.

This service's copy lives in one layer, a bounded store of responses in front of the route table, constructed with its three terms up front and an injected clock so every one of them is testable as arithmetic rather than waiting. The wiring names the whole contract in one call:

```
export function wireCache({ max = 512, ttlMs = 5_000, now = Date.now } = {}) {
  const cache = createLruCache({ max, ttlMs, now });
  const loader = createLoader();
  return { cache, loader, layer: createLayer({ cache, loader }) };
}
```

listing 228: the layer at construction: capacity, lifetime, and a clock, three terms up front (source: javascript/api/src/cache/wire-cache.mjs, lines 8-12)

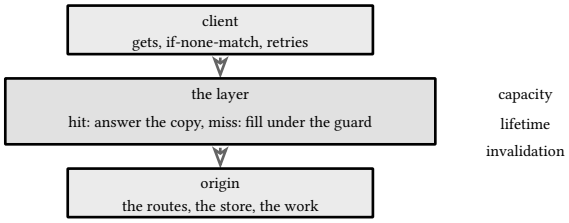


figure 178: the layer sits between client and origin

30.2. LRU over one Map

Least recently used is one `Map`. The js `Map` already remembers insertion order, the iteration walks entries oldest first, and that single fact collapses the two structures the textbook pairs: `delete` then `set` moves a key to the most recent position, and the first key the iterator offers is the eviction. Every operation is constant time, and there is no hand-threaded node list to keep honest, the ordering lives inside the structure the language guarantees.

The proof is the scramble test: fill to capacity, touch a key from the middle of the order, and the next insert evicts the true least recently used entry, not the oldest insert. A plain object with string keys would evict nothing and grow forever, and a second list would re-implement what `Map` iteration already answers. The property loop drives it further, a seeded splitmix32 stream of two thousand mixed set, get, and delete ops against a deliberately naive reference model, an array scanned front to back, and the two agree on every answer and every size, the bound never blown.

```
    set(key, value) {
      entries.delete(key);
      entries.set(key, { value, expiresAt: now() + ttlMs });
      while (entries.size > max) {
        entries.delete(entries.keys().next().value);
      }
    },
```

listing 229: the write: re-insert at the end, evict the first key while over capacity (source: javascript/api/src/cache/lru.mjs, lines 31-37)

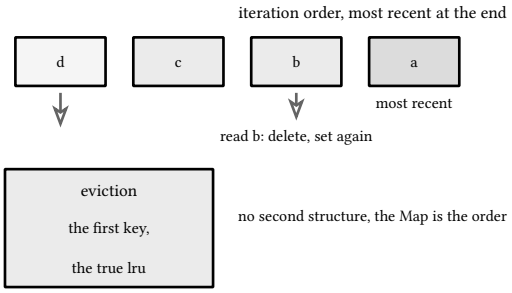


figure 179: one Map, oldest first, hottest last

30.3. TTL beside LRU

LRU answers crowding, not age. An entry nobody reads can be least recently used forever and still hold its bytes, and an entry everybody reads can be hot long after the origin moved on. Every entry carries its own expiry stamped from the injected clock at write time, and reads check it lazily: an expired entry evicts itself on contact, so every read is its own eviction decision and no sweep is needed for correctness.

The sweep is for memory, not correctness, and it is nothing but a loop over the same expiry test, runnable on demand or on an interval the wiring owns. Because the clock is injected, the tests drive whole lifetimes with `tick` and never sleep: the sweep test writes two entries, advances the clock to the exact expiry, shows both still held until something touches them, then sweeps and shows the store empty, hours of wall time inside microseconds.

```
test("the sweep removes expired entries nobody read", (t) => {
  t.mock.timers.enable({ now: 1_000_000 });
  const cache = createLruCache({ max: 5, ttlMs: 1_000, now: () => Date.now() });
  cache.set("a", 1);
  cache.set("b", 2);
  t.mock.timers.tick(1_000);
  assert.equal(cache.len(), 2); // lazy expiry keeps them until touched
  cache.sweep();
  assert.equal(cache.len(), 0);
});
```

listing 230: the sweep test: tick is the only clock, no sleep anywhere (source: javascript/api/test/cache/lru.test.mjs, lines 48-57)

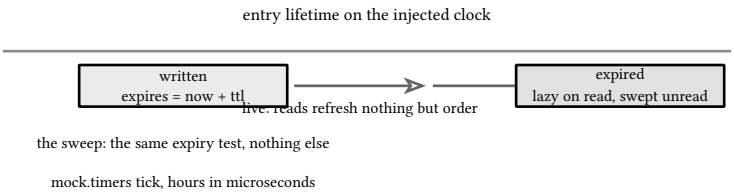


figure 180: two pressures push entries out: age and crowding

30.4. the stampede guard: singleflight from a promise map

Every entry expires eventually, and the moment a hot one does, the herd arrives together: a hundred concurrent misses, a hundred identical origin reads, the exact load spike the cache existed to prevent, arriving at the worst moment, the cold one. The guard is the singleflight pattern, and in js it is a promise map, hand rolled: the first caller for a key starts the fill and parks the promise in the map, every concurrent caller receives that same promise, and the entry is forgotten the moment the call settles.

The properties the tests pin are three. The collapse: three callers registered together run the fill exactly once and all receive the same answer, and because registration is synchronous, one `Map.get` and one `Map.set` with no `await` between them, callers that arrive together have provably joined one flight, no scheduling window. The forgetting: after the settle the entry is gone, so the herd that arrives after the lull fills again, a collapse tool, not a cache. And the errors: a rejected fill rejects for every caller and is never memoized, so an origin outage cannot be baked into the key, the retry after recovery fills fresh.

```
  load(key, fill) {
    const existing = inFlight.get(key);
    if (existing !== undefined) {
      return existing;
    }
    const promise = Promise.resolve()
      .then(fill)
      .finally(() => {
        inFlight.delete(key);
      });
    inFlight.set(key, promise);
    return promise;
```

listing 231: the loader: one promise per key, forgotten on settle, errors never stored (source: javascript/api/src/cache/singleflight.mjs, lines 17-28)

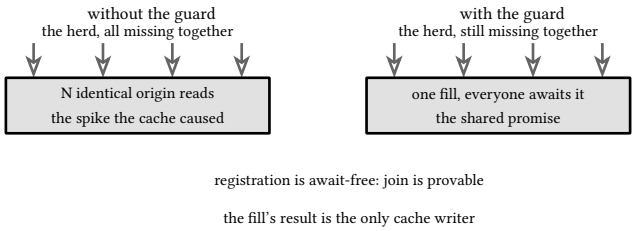


figure 181: the same expiry, with and without the guard

30.5. invalidation on write

A write changes the origin, and the copies of what it changed are now wrong. The discipline is delete after write, never update: the writer drops the affected entries and lets the next read refill them through the guard. Deleting is idempotent and always safe. Updating instead, pushing the writer's view into the entry, sounds cheaper, one write instead of a miss, but the writer does not know what a concurrent reader cached mid-flight, and the staged test shows the ending, an entry holding the stale bytes with a fresh ttl, the worst of both worlds, while its deleted twin refills the origin's new truth.

The rule is path plus collection: a write to a row invalidates the row, the list that reads it, and any child under it, nothing else. The test stages four entries and one write and shows exactly the three affected gone and the stranger still hit. Failed writes invalidate nothing, the origin never moved.

```
  invalidate(path) {
    const slash = path.lastIndexOf("/");
    const parent = slash > 0 ? path.slice(0, slash) : "";
    cache.deleteWhere((key) => {
      const keyPath = key.split("\0")[0];
      const affected =
        keyPath === path ||
        keyPath === parent ||
        keyPath.startsWith(path + "/");
      return !affected;
    });
  },
```

listing 232: delete after write: the path, its collection, its children, nothing else (source: javascript/api/src/cache/layer.mjs, lines 68-79)

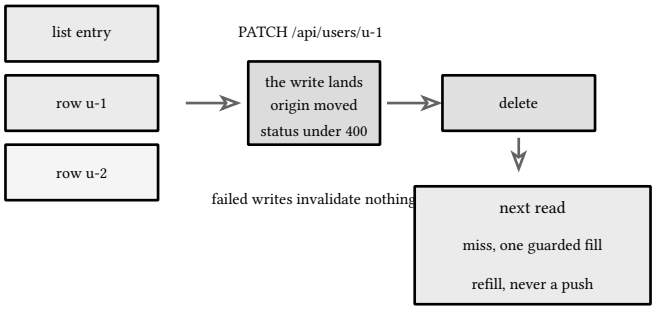


figure 182: write to a row: the row, the list, and children go, the refill is the only writer

30.6. what never gets cached

The last decision is the one that prevents the incidents. Credential routes, login and refresh, carry secrets and must never be replayable from a copy. Rate-limited responses, the 429 lane, must always reach the client, because a cached budget answer is a lie about money. Personalized bodies without a vary key to separate identities are a cross-user leak waiting for traffic, which is why the entry key carries the vary values, the bearer token and the session cookie, so two identities reading the same url hold two entries. The policy table is deny by default, a route is cacheable only when a row says so, and the two rows that say so are the reads, `GET /api/users` and `GET /api/users/:id`, both telling clients `private, max-age=0, must-revalidate`, nothing fresh for long, every reuse round-trips its etag through the 304 lane the previous chapter built.

The exclusion test pins the posture end to end: a route with no row, login, asked three times reaches the origin three times, answers from itself every time, and never once says hit. The cheap check is the one worth keeping in the gate, because the day someone adds a row to the wrong route, this is the test that says which lane they put it in.

```
test("a route without a policy row never caches, deny by default", async (t) => {
  const { layer } = layered(t);
  const fill = t.mock.fn(async () => ({
    status: 200,
    headers: {},
    body: "ok",
  }));
  const login = { ...ctx(), route: "POST /api/auth/login", fill };
  for (let n = 0; n < 3; n++) {
    const res = await layer.handle(login);
    assert.equal(res.headers["X-Cache"], undefined);
    assert.equal(res.body, "ok");
  }
  assert.equal(fill.mock.callCount(), 3);
});
```

listing 233: the exclusion test: no row, no copy, the origin answers every time (source: javascript/api/test/cache/layer.test.mjs, lines 65-79)

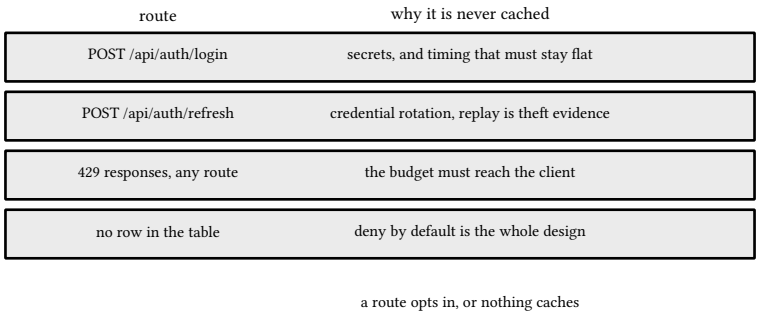


figure 183: the exclusion list, each row with its reason

sources: `developer.mozilla.org/docs/Web/JavaScript/Reference/Global_Objects/Map` for the insertion-order iteration guarantee the LRU stands on, accessed 2026-09-26, nodejs.org/api/test.md for `mock.timers` and the tick discipline every ttl path rides, accessed 2026-09-26, and the `Cache-Control`, `Vary`, and `no-store` semantics quoted from RFC 9111 section 5.2 at rfc-editor.org, accessed 2026-09-26. Verified by `books/javascript/api` tests, 19 of them in `test/cache`, the seeded property loop with its committed 0x12345678 seed included, under `node --test` in `npm run verify`.

31. rate limiting

A service that answers every request it receives eventually answers none of them: password guessing arrives as a flood of logins, a runaway client loop arrives as the same request ten thousand times, a retry storm arrives the moment a deploy slows one route down. This chapter builds the service's rate limiting in `api/src/limit`: the token bucket derived from first principles over an injected clock, the gate that keeps one bucket per key and sweeps the idle ones, the 429 contract with its headers and the one envelope writer, what to key a limit on and what never to trust, and the test discipline that keeps every refill and every reset second off the wall clock.

31.1. why limits exist

Capacity is finite and failure is nonlinear. Past some offered load a service does not degrade gently, it falls off a cliff: queues grow, latency climbs past every client timeout, memory fills with requests waiting on answers that can no longer arrive, and the health check itself starts timing out. A limit is the refusal to stand on that cliff. It converts an overload that would take the whole service down into a fast, honest rejection of the excess, and the client that honors it gets a working service at one request per second instead of a dead one at zero.

The two limited routes carry numbers chosen for what they protect. Login allows a burst of five per address, then one per second, because every login attempt pays the scrypt verification budget and a credential guesser does not need more than five tries to be dangerous. Register allows ten per client ip for the same reason plus its idempotency bookkeeping. Both refill one token per second, so a client that trips the limit is back to full capacity five and ten seconds later, which is the difference between a limit and a lockout:

```
// the contract's two limited routes, verbatim numbers: login bursts 5
// per lower(email), register bursts 10 per client ip, both refill one
// token per second.
export const LOGIN_BURST = 5;
export const REGISTER_BURST = 10;
export const REFILL_PER_SECOND = 1;
export const LOGIN_MESSAGE = "too many login attempts";
export const REGISTER_MESSAGE = "too many registration attempts";
// a key's bucket retires after this much disuse.
export const DEFAULT_IDLE_TTL_MS = 10 * 60 * 1000;
// the kernel's body cap from the wire contract, 1 MiB. this copy is
// the fallback reader's bound until the composition shares one source.
export const MAX_BODY_BYTES = 1024 * 1024;
```

listing 234: the contract's two rules as one constant block, numbers and messages (source: javascript/api/src/limit/limiter.mjs, lines 10-22)

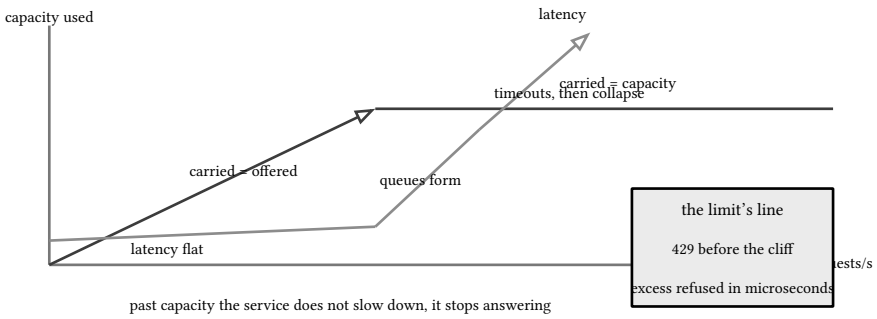


figure 184: offered load versus what the service can carry, the cliff between them

31.2. the token bucket from first principles

A token bucket holds a floating level, a capacity, and a refill rate. It starts full, every allowed request removes one token, and time adds tokens continuously at the configured rate until the capacity caps them. That is the entire algorithm, and its two knobs split the two jobs cleanly: capacity is the burst allowance, how much may arrive at once, and the refill rate is the sustained allowance, how much may arrive per second forever. The contract's login rule, capacity five at one per second, permits five requests in the first instant, a sixth at the one second mark, and then one per second for as long as the client cares to wait.

The arithmetic of a denial is the part worth deriving rather than reciting. After the fifth same-instant draw the level is exactly zero, so the next token exists one full second in the future, and that instant is both the reset header and the retry hint. Half a second later the level is 0.5: still not spendable, and the next whole token is now only half a second away. A whole level always waits a full period, a fraction waits its complement, which is the wait rule in the listing: the wait is `1 - fraction(level)` over the refill rate, and a denial never spends anything. The level never leaves this module as a float: the wire sees whole remaining tokens, a unix second for reset, and a ceiling of seconds for retry:

```
  // draw one token at t. an allowed draw costs exactly one token, a
  // denial costs nothing and reports the wait for the level to reach
  // the next whole token, the earliest the caller may retry.
  allow(t) {
    refill(t);
    const allowed = level ≥ 1;
    if (allowed) {
      level -= 1;
    }
    const remaining = Math.floor(level);
    const fraction = level - remaining;
    const waitMs = ((1 - fraction) / perSecond) * 1000;
    return {
      allowed,
      limit: capacity,
      remaining,
      resetAtMs: t + waitMs,
      retryAfterMs: allowed ? 0 : waitMs,
    };
  },
```

listing 235: allow: one draw, one decision, the wait computed from the fractional level (source: javascript/api/src/limit/bucket.mjs, lines 30-49)

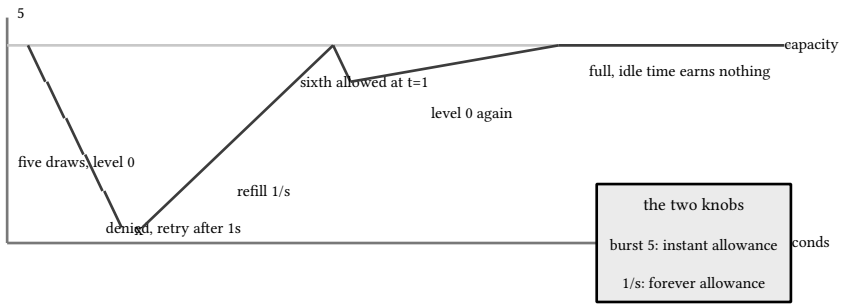


figure 185: bucket level over one draw timeline, the refill climbing between bursts

31.3. one bucket per key

One bucket for the whole service would let one attacker spend every other client's tokens, so the gate keeps one bucket per key in a plain `Map`, created on first use and stamped with the instant it was last touched. Every distinct

key is a cheap but unbounded allocation, so an idle ttl retires keys nobody has touched. The sweep is amortized into the request path: whenever a full ttl has passed, the lookup sweeps inside the same call, which deletes the sweeper timer entirely, no ticker to leak in the load chapter.

The go lane reached for `golang.org/x/time/rate` here and spent its section reading that library as primitives. The javascript lane has no sanctioned equivalent to adopt, and the npm convention, `bottleneck` and `rate-limiter-flexible`, is dated meta this vehicle deliberately fails to meet: both are community-owned behavioral boxes over exactly the arithmetic this chapter already built, one float, one clock read, one comparison, and adopting one would buy a dependency where the platform needs none. The teaching bucket is the production gate.

Node also removes half the go lesson for free: there is no mutex anywhere in this file. The gate runs on one thread, the only interleaving point between two requests is the `await` between the key derivation and the decision, and the decision itself is computed without yielding, so the buckets map cannot observe a torn state:

```
function entryFor(key, rule, t) {
  if (t - lastSweepMs ≥ idleTtlMs) {
    sweepLocked(t);
  }
  let entry = buckets.get(key);
  if (entry === undefined) {
    entry = {
      bucket: createBucket({
        capacity: rule.burst,
        perSecond: rule.perSecond,
        nowMs: t,
      }),
      lastMs: t,
    };
    buckets.set(key, entry);
  }
  entry.lastMs = t;
  return entry;
}
```

listing 236: entryFor: per-key buckets, created on first use, swept on the same call (source: javascript/api/src/limit/limiter.mjs, lines 125-143)

```
const { limiter, layer } = wireLimit({ fail: envelopeWriter })
app.use(layer)
```

fragment, illustrative, not executed

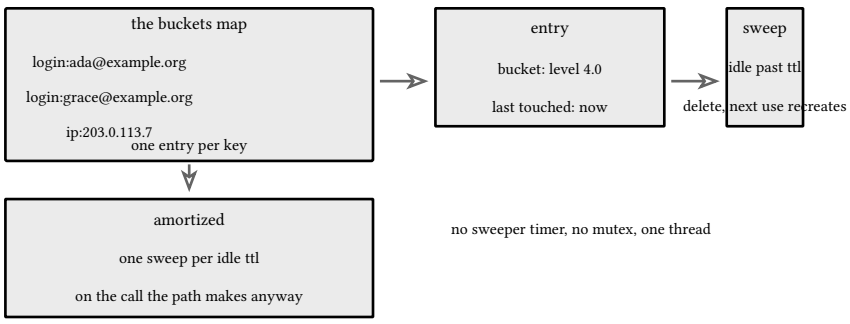


figure 186: key to bucket map, first use creates, idle age retires

31.4. the 429 contract

The wire shape is pinned by the contract and the frozen vector, and the layer writes it in one place. Every response from a limited route carries three headers: `X-RateLimit-Limit` is the burst capacity, `X-RateLimit-Remaining` is whole tokens left after this draw, `X-RateLimit-Reset` is the unix second at which the next token arrives. A denial adds `Retry-After` in seconds, the ceiling of the wait, and answers the standard envelope with code `rate_limited`, never a bare status or a bespoke body.

The denial path is a throw, not a write. The layer sets the four headers on the response object, then throws the kernel's one failure value, `fail(CODES.rate_limited, rule.message)`, and the kernel's backstop catches it and renders the envelope through the single writer every other error already flows through. That is why the request id in the 429 body is the same id the access log and the metrics series will carry: one id minted once at the edge, one writer, one shape:

```
export function limitLayer({ limiter }) {
  return (next) => async (ctx) => {
    const path = ctx.req.url.split("?")[0];
    const outcome = await limiter.allow(path, ctx);
    if (outcome === null) {
      return next(ctx);
    }
    const { decision, rule } = outcome;
    ctx.res.setHeader(HEADER_LIMIT, String(decision.limit));
    ctx.res.setHeader(HEADER_REMAINING, String(decision.remaining));
    ctx.res.setHeader(
      HEADER_RESET,
      String(Math.floor(decision.resetAtMs / 1000)),
    );
    if (!decision.allowed) {
      ctx.res.setHeader(
        HEADER_RETRY_AFTER,
        String(Math.ceil(decision.retryAfterMs / 1000)),
      );
      throw fail(CODES.rate_limited, rule.message);
    }
    return next(ctx);
  };
}
```

listing 237: the layer: three headers always, Retry-After and the thrown failure on denial (source: javascript/api/src/limit/layer.mjs, lines 14-37)

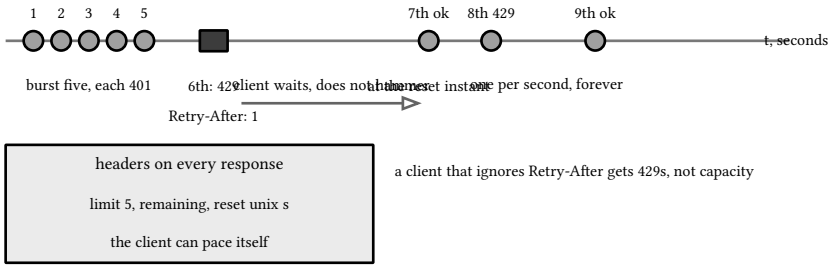


figure 187: a client that honors Retry-After: five through, one refused, the next at the reset instant

31.5. what to key on

A limit is only as good as its key, and the two routes key on different things for opposite reasons. Login keys on `lower(email)`: the attack the rule exists for is guessing one account's password, so the bucket belongs to the account, and the casing normalization means `ADA@Example.ORG` and `ada@example.org` are one bucket. The key lives

in the request body, and a node stream cannot be unread: the key function takes the body the earlier middleware parsed when there is one, reads bounded once when there is not, and stashes the parse on the context so the handler below reuses it instead of waiting on a consumed stream. Register keys on the client ip from the socket's remote address, and the proxy headers stay unread on purpose: `X-Forwarded-For` is a header any client can write, so a limiter that keys on it verbatim sells a fresh bucket of ten registrations per forged address, one per request, forever.

Two shared buckets close the dodge holes. A body the email key cannot parse lands in one `login:unparsed` bucket, so garbage cannot orbit the limit by being invalid, and a body the bounded reader cannot read lands in `login:unreadable`. The test for the spoof is the contract said out loud: eleven register posts from one real address with a different forged forwarding header on the last one, and the eleventh is still the 429, because the key read the socket, not the header:

```
export async function keyEmail(ctx) {
  let body = ctx.body;
  if (body === undefined) {
    let raw;
    try {
      raw = await readBounded(ctx.req, MAX_BODY_BYTES);
    } catch (err) {
      // a transport failure lands in the shared unreadable bucket, an
      // ApiError (the over-cap 413) propagates to the one writer
      if (err instanceof ApiError) {
        throw err;
      }
      return "login:unreadable";
    }
    try {
      body = JSON.parse(raw);
    } catch {
      return "login:unreadable";
    }
    ctx.body = body;
  }
```

listing 238: keyEmail: bounded read, the parse stashed, the over-cap 413 propagating (source: javascript/api/src/limit/limiter.mjs, lines 56-76)

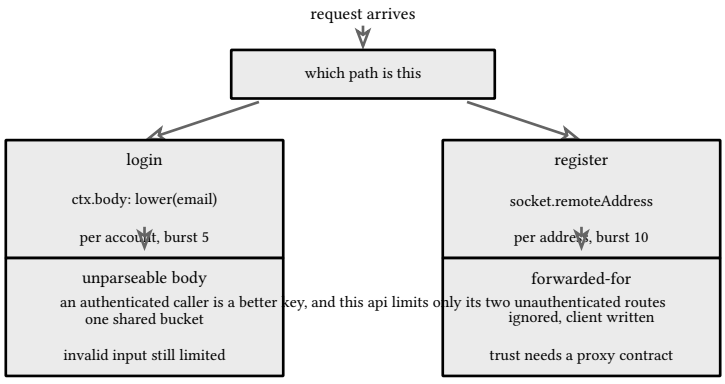


figure 188: key choice: the route decides what the bucket belongs to

31.6. limits under test

Every time-dependent behavior in this chapter is arithmetic on a fixed clock, because a rate limiter tested against the wall clock is tested against a scheduler. The bucket tests walk the whole grammar of the level in fixed draws, burst, cap, sustained rate, the half-token denial, a backwards jump, plus a seeded prng loop over five thousand

random tick and draw steps asserting the named invariants: the level stays in range, remaining is always its floor, and a denial's reset instant is exactly now plus its wait.

The vector replay runs the real layer on the real kernel against the frozen file. `mock.timers.enable` pins `Date.now` to the vector's instant, t0 plus twelve seconds, all six requests fire at that same frozen instant, and the sixth asserts every header byte and the body through the kernel's writer. One platform fact earned its place in this code the hard way: a default clock captured as the `Date.now` reference still sees the real clock after the mock replaces the global `Date`, because the capture happened first. The seam is a thunk, `now = () => Date.now()`, which resolves the global on every call, and that is the difference between a passing replay and a reset header four days off:

```
it("replays vector 06: five draws then the frozen 429, byte for byte", async (t) => {
  mock.timers.enable({ now: AT_VECTOR });
  t.after(() => mock.timers.reset());
  const handle = await app.listen();
  t.after(() => handle.close());

  const remaining = [4, 3, 2, 1, 0];
  for (const want of remaining) {
    const r = await loginAttempt(handle.port);
    assert.equal(r.status, 401);
    assert.equal(r.headers["x-ratelimit-limit"], "5");
    assert.equal(r.headers["x-ratelimit-remaining"], String(want));
    assert.equal(r.headers["x-ratelimit-reset"], "1790830813");
    assert.equal(
      r.headers["retry-after"],
      undefined,
      "allowed responses carry no Retry-After",
    );
```

listing 239: the vector replay: fake clock at t0+12s, five draws, then the frozen sixth (source: javascript/api/test/limit/layer.test.mjs, lines 55-72)

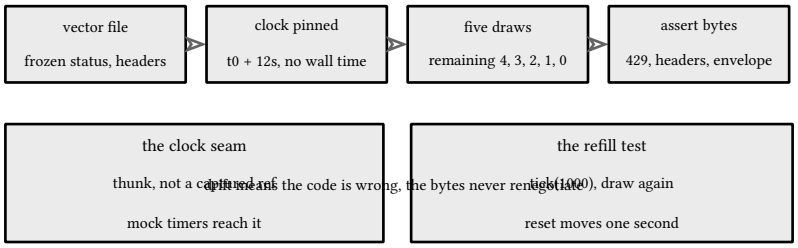


figure 189: the fixed-vector pipeline: file in, draws through the gate, bytes out

sources: nodejs.org/api/test.html for the mock timers, `enable`, `tick`, `reset`, and `t.after`, including what the mock replaces, read 2026-09-26 and probed on node 26.3.0, nodejs.org/api/http.html for `setHeader` and `IncomingMessage` url handling, nodejs.org/api/net.html for `socket.remoteAddress` and its ipv6-mapped `::ffff:` form, plus RFC 6585 for status 429 and the MDN `Retry-After` page for the delay seconds semantics, all accessed 2026-09-26. Verified by `javascript/api` tests, 21 of them under `test/limit`, green under `npm run verify`, with the sixth-login vector replayed byte for byte from `contract/testdata/06-login-429.json`.

32. observability

A service no one can see is a service no one can fix, and a service whose only story is a console log of prose is seen by whoever happens to be reading along live. This chapter builds the service's observability in `api/src/obs`: the metrics registry with the counter and latency histogram the contract pins, the prometheus text exposition rendered by hand and frozen byte for byte by a vector, the recording layer that ticks both series at the status the writer really wrote, the internal listener the exposition lives on, and the inspector protocol, the platform's own profiling oracle, proven reachable from inside the process.

32.1. what a service owes its operators

Two numbers answer the first question every incident asks, is it up and is it slow: how many requests, at what statuses, and how long did they take. Both are series, not events, and both are registry types here rather than library types. The counter is one series per route and status code, `goapi_requests_total{route,code}`, incremented once per request. The histogram is one per route, `goapi_request_duration_seconds`, with the contract's bucket ladder tight at the low end where service latencies live and geometric through the tail: one, two and a half, five, ten, twenty five, fifty milliseconds, then the hundreds, seconds, and `+Inf` implicit last. The series names are frozen because the compose checks and the lane parity tests grep for them:

```
// end where service latencies live and geometric through the tail,
// with +Inf implicit as the last bucket.
export const DURATION_BUCKETS = [
  0.001, 0.0025, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5,
];

export const REQUESTS_NAME = "goapi_requests_total";
export const DURATION_NAME = "goapi_request_duration_seconds";
export const REQUESTS_HELP = "requests by route and code.";
export const DURATION_HELP = "request latency.";

// numbers render shortest round trip without exponents: String is
// already shortest, it only switches to exponent notation outside the
// range the exposition wants, and that is repaired with a fixed-point
```

listing 240: the contract's bounds and series names, one constant block (source: javascript/api/src/obs/metrics.mjs, lines 7-20)

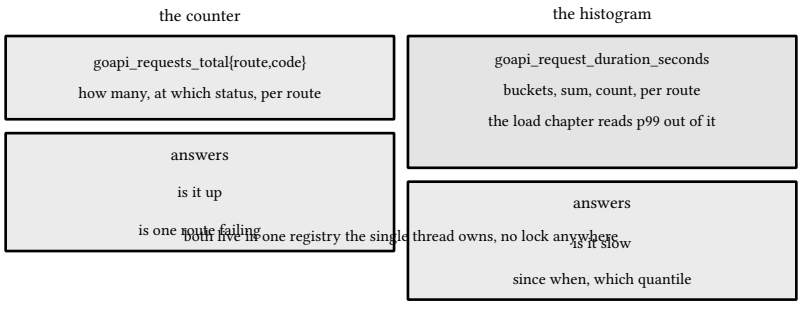


figure 190: the questions the two series answer

32.2. the metrics registry

The registry is two maps. The counter map is keyed by the route and code pair, so incrementing is one `Map.set` of the prior value plus one. The histogram map holds one entry per route: an array of cumulative counts one longer than the bounds, the extra slot for `+Inf`, plus the running sum and count. Recording one observation increments every bound the value fits under, and that is the whole trick of cumulative buckets: each bucket counts everything at or under its bound, so a scraper can subtract neighbors to recover any window, and the counts only ever grow.

One observation of three milliseconds fills `0.005` and every bound above it, touches nothing below, and adds to the sum and the count. The registry never stores individual samples, that is the resolution the format trades for constant memory, and the load chapter is the one that pays for the trade when it wants p99:

```
  // histogram: every bound the value fits under, then +Inf, the sum,
  // and the count, which is what makes the buckets cumulative.
  observe(route, seconds) {
    const h = histFor(route);
    for (const [i, bound] of DURATION_BUCKETS.entries()) {
      if (seconds ≤ bound) {
        h.cum[i]++;
      }
    }
    h.cum[h.cum.length - 1]++;
    h.sum += seconds;
    h.n++;
  },

  // histogram copies one route's cumulative counts out for the
  // quantile math. a route never observed answers zeros.
```

listing 241: observe: cumulative buckets, sum, count, one entry per route (source: javascript/api/src/obs/metrics.mjs, lines 53-68)

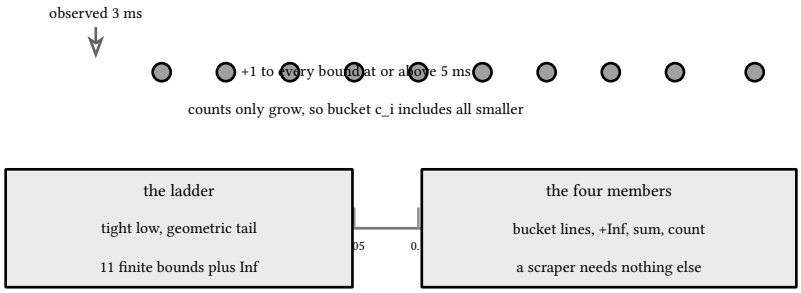


figure 191: cumulative buckets: each observation fills every bound above it

32.3. the prometheus text format

The exposition format is text, line oriented, small enough to render by hand, which is what this module does. Families come out counter first then histogram, matching the frozen vector, series sorted by label tuple within a family, bucket lines ascending with `+Inf` last. Zero buckets are elided, the one deliberate divergence from the reference client libraries, which print every finite bucket at zero as well: one slow observation renders eight lines, not twenty. Numbers render shortest round trip without exponents, and the one repair that needs code is the tiny tail, where `String` switches to exponent notation a fixed-point pass undoes, so a microsecond sum never appears as `1.5e-7`:

```
    render() {
      const lines = [];
      const countKeys = [...counts.keys()].sort();
      if (countKeys.length > 0) {
        lines.push(`# HELP ${REQUESTS_NAME} ${REQUESTS_HELP}`);
        lines.push(`# TYPE ${REQUESTS_NAME} counter`);
        for (const key of countKeys) {
          const [route, code] = key.split("\u0000");
          lines.push(
            `${REQUESTS_NAME}{route="${route}",code="${code}"} ${counts.get(key)}`,
          );
        }
```

listing 242: the counter family: sorted label tuples, one line per series (source: javascript/api/src/obs/metrics.mjs, lines 92-103)

The frozen vector pins the whole rendering. One `/healthz` observation slower than the largest finite bucket elides every finite bucket line, leaving `+Inf`, the sum marked as observed in the file, and the count, and the test replays the file byte for byte with the observed value substituted:

```
# HELP goapi_requests_total requests by route and code.
# TYPE goapi_requests_total counter
goapi_requests_total{route="/healthz",code="200"} 1
# HELP goapi_request_duration_seconds request latency.
# TYPE goapi_request_duration_seconds histogram
goapi_request_duration_seconds_bucket{route="/healthz",le="+Inf"} 1
goapi_request_duration_seconds_sum{route="/healthz"} <observed>
goapi_request_duration_seconds_count{route="/healthz"} 1
```

listing 243: the frozen exposition the render is checked against, byte for byte (source: javascript/api/contract/testdata/15-metrics.txt, lines 1-8)

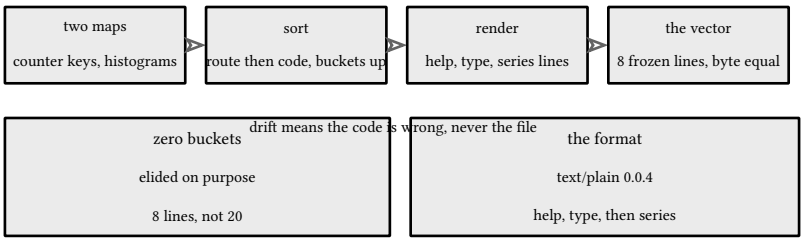


figure 192: a scrape: registry state to exposition bytes

32.4. recording every request

The layer that feeds the registry is the outermost wrapper, so every response including the kernel backstop's lands in both series. Two node facts carry the whole design. First, durations come from `process.hrtime.bigint()`, the monotonic clock that never moves for wall-clock adjustments and reads in nanoseconds, one subtraction, one division. Second, the status is read on the response's `finish` event, which the http module emits when the last segment of headers and body has been handed to the operating system, and that is the one hook that runs after every writer: when a handler under this layer throws, the backstop writes the error after this layer has already yielded, and a status captured at yield time would record 200 for a request that answered 401.

The consequence for a request that never finishes is honest by omission: no response, no tick, and the counter never counts a request the client never received. The route label is the pattern the router matched, stamped on the context at dispatch, so `/api/users/42` and `/api/users/43` share one `/api/users/:id` series instead of shredding the label space into one series per id. The layer keeps a raw-pathname fallback for contexts that reach it without a router, and the test asserts both facts, the stamped pattern present and the raw path absent:

```
export function metricsLayer({ metrics = createMetrics(), routeFor } = {}) {
  const labelFor =
    routeFor ?? ((ctx) => ctx.route ?? ctx.req.url.split("?")[0]);
  return (next) => async (ctx) => {
    const startNs = process.hrtime.bigint();
    // finish is the one hook that runs after every writer: a
    // response that never finishes never records, which is the
    // honest answer for a request that was never answered
    ctx.res.on("finish", () => {
      const seconds = Number(process.hrtime.bigint() - startNs) / 1e9;
      const route = labelFor(ctx);
      metrics.countRequest(route, ctx.res.statusCode);
      metrics.observe(route, seconds);
    });
    return next(ctx);
  };
}
```

listing 244: the recording layer: monotonic start, the finish hook, both series ticked (source: javascript/api/src/obs/layer.mjs, lines 9-25)

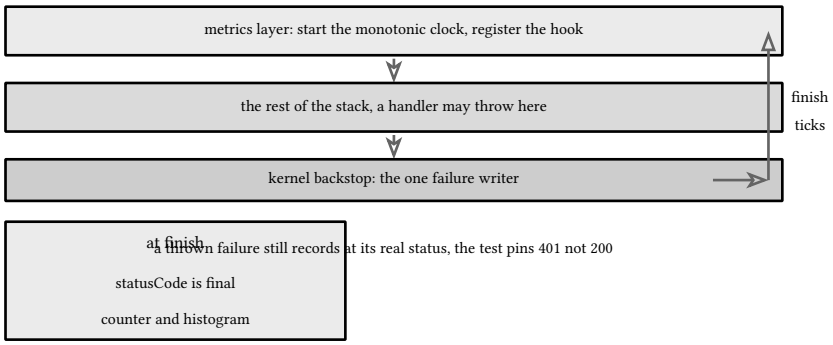


figure 193: where the finish hook sits: after every writer, including the backstop

32.5. the internal listener and the inspector

The endpoint binds on its own listener, not the public one, and the placement is the policy. The contract says `GET /metrics` carries no auth, and an unauthenticated route belongs on the socket the authz layers never wrap, reachable only from inside the deployment, with the prometheus content type carrying the format version. Everything else on that listener answers 404, because a private port that answers unknown routes is a public port that got lucky:

```
export function createInternalServer({ metrics }) {
  return createServer((req, res) => {
    if (req.method === "GET" && req.url.split("?")[0] === "/metrics") {
      res.setHeader("Content-Type", METRICS_CONTENT_TYPE);
      res.end(metrics.render());
      return;
    }
    res.statusCode = 404;
    res.end();
  });
```

listing 245: the internal listener: one route, one content type, 404 for the rest (source: javascript/api/src/obs/internal.mjs, lines 11-21)

The listener is the production mode, and the contract names a second: served on the main mux in tests, where one process answers everything. The wiring returns both, the listener and a kernel route handler rendering the same

registry through the same content type, so the composition registers the route on its mux and the deployment binds the listener on its private port in production:

```
const { metrics, layer, internal, route } = wireObs()
app.route("GET", "/metrics", route)
```

fragment, illustrative, not executed

The go lane mounts pprof handlers next to the metrics route and reads its profiles over http. Node's profiling oracle is not a route: the inspector protocol, the same chrome devtools protocol a debugger speaks, served on its own wire, and reachable in-process through `node:inspector/promises` with no flag at all. One session, three posts, a cpu profile of the very process running the api, which is the committed proof the surface exists where the chapter says it does. An inspector endpoint is an oracle, it reads where the work happens, so it stays inside the deployment exactly like the exposition:

```
export async function cpuProfile() {
  const session = new Session();
  session.connect();
  try {
    await session.post("Profiler.enable");
    await session.post("Profiler.start");
    const { profile } = await session.post("Profiler.stop");
    return profile;
  } finally {
    session.disconnect();
  }
}
```

listing 246: one session over the inspector protocol, a cpu profile of this process (source: javascript/api/src/obs/inspector.mjs, lines 9-20)

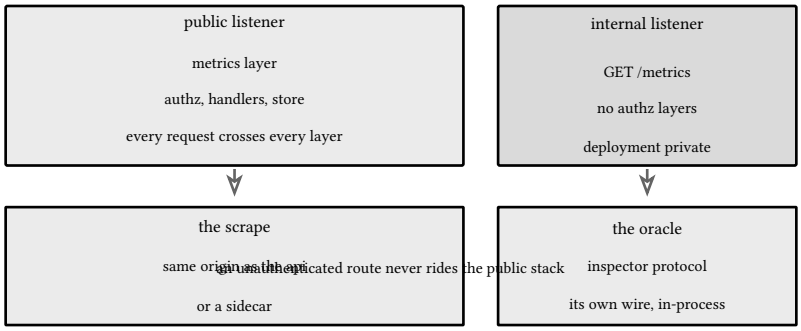


figure 194: two wires: the public stack and the private surface

32.6. one request fully observed

Wire the signals in the order the request meets them and one login tells the whole story. The metrics layer is the outermost wrapper, so the clock starts before the request id middleware mints the id, and the finish hook fires after the backstop or the handler has written. The id ties the artifacts together: the envelope body carries it, any access log line will carry it, and the counter's label tuple names the route it answered. Reading it back is what the tests do, without ceremony: the layer test drives the real kernel, asserts the 200 and the thrown 401 land as two labeled series, the registry test replays the frozen vector, and the internal test scrapes the listener over real http:

```
const { metrics, layer, internal, route } = wireObs()
wireMiddleware(app, { outermostLayer: layer })
app.route("GET", "/metrics", route)
```

fragment, illustrative, not executed

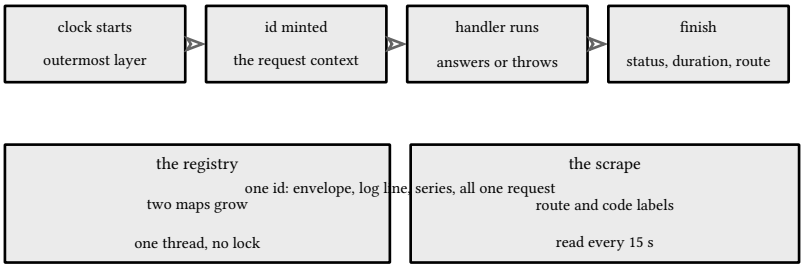


figure 195: one request id through every signal

sources: nodejs.org/api/http.html for the response `finish` event and its handed-off-to-the-operating-system wording plus `statusCode` semantics, nodejs.org/api/process.html for `process.hrtime.bigint` and `process.getActiveResourcesInfo`, nodejs.org/api/inspector.html for the promises session and the `Profiler` domain, all read 2026-09-26 on node 26.3.0, the prometheus exposition format at prometheus.io/docs/instrumenting/exposition_formats for the 0.0.4 text format, help and type lines, histogram buckets, and `+Inf`, with the client_golang zero-bucket behavior read from its source, accessed 2026-09-26. Verified by `javascript/api` tests, 14 of them under `test/obs`, green under `npm run verify`, with the exposition replayed line for line from `contract/testdata/15-metrics.txt`.

33. load and p99

A service that has never been loaded is a service whose p99 is a guess, and a deploy that has never been drained is a deploy that drops requests every time it ships. This chapter closes the operations side in `api/src/load`: the event loop as the whole concurrency story, graceful shutdown with a real drain order, the admission gate that answers overload at the cap, the two load loops over injected seams, p99 by nearest rank with the bucket approximation labeled as the bound it is, socket backpressure as the node lane's own flow control, and the orphan check that keeps every run honest after it ends.

33.1. the event loop is the worker

The go chapters spend their first lesson on goroutines, a scheduler, and the mutexes that protect shared maps. This lane needs none of that story, and the reason is structural: one thread runs every line of this api. A request handler is an async function, its awaits are the only places another request's code can interleave, and everything between two awaits is atomic by construction. The limit chapter's buckets map needed no mutex for exactly this reason, and the load harness gets its workers for free: an async function per worker under `Promise.all` is a pool, and each worker fires strictly one request at a time because its loop awaits each send.

The closed-loop test pins the serialization honestly. Three workers fire four requests each through an injected server whose sends each advance a fake clock by five milliseconds, and the elapsed total is sixty: the twelve sends ran one after another on the one thread, overlapping only between their awaits, because the fake server is one thread like the real one. What a node service parallelizes is io waiting, never compute, which is the fact the admission gate and the backpressure section both lean on:

```
test("the closed loop with concurrent workers: serialized on one thread", async () => {
  const clock = { now: T0 };
  const h = createHarness({
    send: async () => {
      clock.now += 5;
      return 200;
    },
    now: () => clock.now,
  });
  const result = await h.closedLoop({ workers: 3, perWorker: 4, warmup: 0 });
  assert.equal(result.n, 12);
  // the injected server is one thread like the real one: the 12 sends
  // serialize on the fake clock, 5 milliseconds each, workers overlap
  // only between their awaits
  assert.equal(result.elapsedMs, 60);
  assert.ok(
    result.latencies.every((ms) => ms ≥ 5 && ms % 5 === 0),
    "each latency is a whole number of sends",
  );
});
```

listing 247: three workers, twelve sends, sixty fake milliseconds: one thread serialized them (source: javascript/api/test/load/harness.test.mjs, lines 35-54)

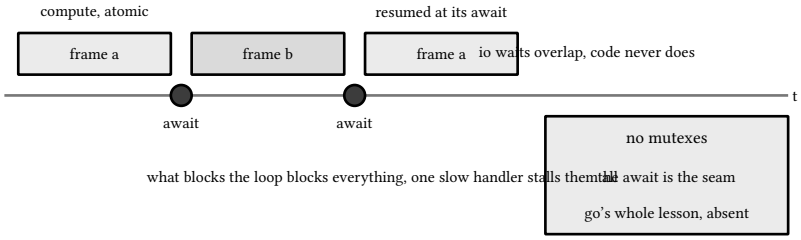


figure 196: one thread, many async frames

33.2. graceful shutdown with drain

Shutdown has an order, and the order is the contract. Readiness flips first: the drainer's state moves to draining, `/readyz` answers its 503 overload envelope, and every load balancer watching the service stops routing new requests here while the old ones are still being answered well. Then the listener stops accepting, and node needs one extra move the go lane did not: idle keep-alive sockets count as active connections, so `closeIdleConnections` retires them before the close, which is what lets the close callback mean drained. Then the drain waits for in-flight requests under a grace deadline and reports the deadline instead of pretending the drain succeeded, because the caller decides whether to force the hard exit itself.

The kernel takes the drainer's `ready` function as its readiness probe, the same hook the contract's overload ruling flows through, and the grace timer is a thunk seam for the same reason the limit clock is, a captured reference escapes `mock.timers`:

```
export function drain(
  server,
  drainer,
  {
    graceMs = DEFAULT_GRACE_MS,
    setTimeoutRef = (fn, ms) => setTimeout(fn, ms),
    clearTimeoutRef = (t) => clearTimeout(t),
  } = {},
) {
  drainer.startDrain();
  // idle keep-alive sockets count as active connections to close, so
  // retiring them first is what lets the close callback mean drained
  server.closeIdleConnections();
  return new Promise((resolve) => {
    const timer = setTimeoutRef(
      () => resolve({ timedOut: true, drained: false }),
      graceMs,
    );
    server.close(() => {
      clearTimeoutRef(timer);
      resolve({ timedOut: false, drained: true });
    });
  });
}
```

listing 248: drain: flip readiness, retire idle sockets, wait under grace, report the deadline (source: javascript/api/src/load/shutdown.mjs, lines 27-50)

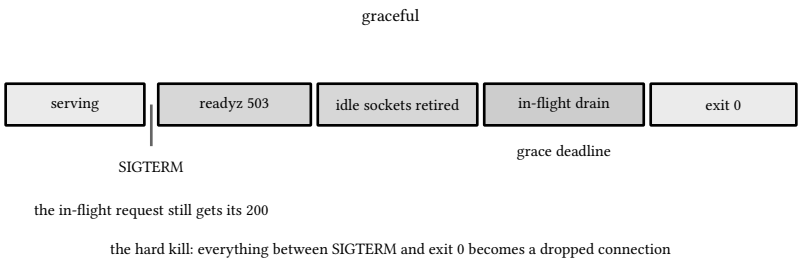


figure 197: SIGTERM to drain to exit, against the hard kill

33.3. admission control

The rate limit bounds arrivals over time. The admission gate bounds requests inside the service at once, and little's law states the trade in one line: the average number inside equals the arrival rate times the time each spends there. Offered load raises that product two ways, more arrivals or slower service, and past the point where every arrival finds the gate full, latency rises steeply while throughput stops moving, the hockey stick the harness measures. The gate caps the inside count directly, one plain number up on admission and down in a `finally`, and a caller at the cap gets the `overload` envelope immediately, because queueing it would spend memory the service just proved it cannot keep up with.

The go gate is a channel of slots, a buffered send to take and a deferred receive to release. The node gate is a number and a try, and the panic-proofing transfers exactly: the release rides the `finally` of an async function, so a handler that throws still gives the slot back, and the barrier test observes `inFlight` at exactly the cap before the third request eats its 503:

```
export function createGate(capacity = DEFAULT_ADMISSION_CAPACITY) {
  let inFlight = 0;
  return {
    // inFlight reports how many slots are held right now, the number
    // the little's law arithmetic in the chapter is about.
    inFlight: () => inFlight,

    layer: (next) => async (ctx) => {
      if (inFlight >= capacity) {
        throw fail(CODES.overload, OVERLOAD_MESSAGE);
      }
      inFlight++;
      try {
        return await next(ctx);
      } finally {
        inFlight--;
      }
    },
  };
}
```

listing 249: the gate: a plain count, finally release, overload thrown at the cap (source: javascript/api/src/load/admission.mjs, lines 15-34)

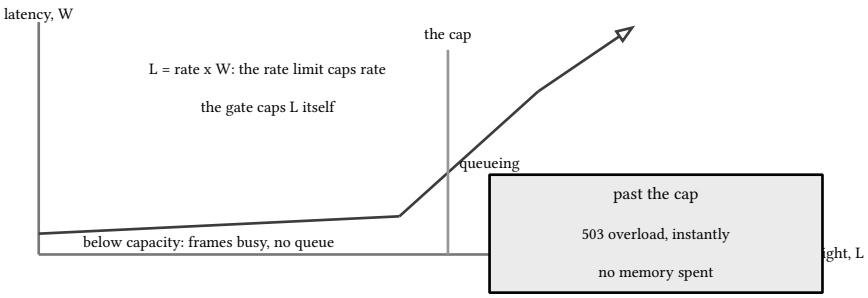


figure 198: latency versus in-flight: the hockey stick the gate cuts

33.4. the load harness

The two loops differ in what they hold constant. The closed loop holds concurrency constant: workers each firing one request at a time, so offered load is whatever the server can keep up with and never exceeds the worker count. The open loop holds arrival rate constant: request `i` is scheduled at `i` over rate seconds past the start, fired whether or not anything has come back, so when the server slows, requests pile up exactly the way they do in production, and the cliff becomes visible. Warmup runs first on every loop and never touches the sink, because the first request pays for the dial and the cold caches.

Three seams make the whole scheduler testable: `send`, so tests drive deterministic responses with no socket, `now`, so latency is arithmetic, and `sleep`, so the pacing is a function the test replaces. Two platform facts earned their way into this module's comments. An `async` sleep wrapper adds a microtask hop between the timer firing and the caller resuming, and under fake timers that hop lets the test's next tick advance the clock before the fired request observes its own deadline, so the default sleep is a plain promise. Fake timers replace `setImmediate` too, so tests under mocks signal through latches, never through immediate polls:

```
    async openLoop({ ratePerSec, total, warmup = 0 }) {
      for (let i = 0; i < warmup; i++) {
        await fire(null);
      }
      const result = newResult();
      const start = now();
      await Promise.all(
        Array.from({ length: total }, async (_, i) => {
          const wait = scheduleMs(i, ratePerSec);
          if (wait > 0) {
            await sleep(wait);
          }
          await fire(result);
        }),
      );
      result.elapsedMs = now() - start;
      result.achievedRps = result.n / (result.elapsedMs / 1000);
      return result;
    },
  };
}
```

listing 250: the open loop: schedule math, concurrent pacing, independent of every response (source: javascript/api/src/load/harness.mjs, lines 111-131)

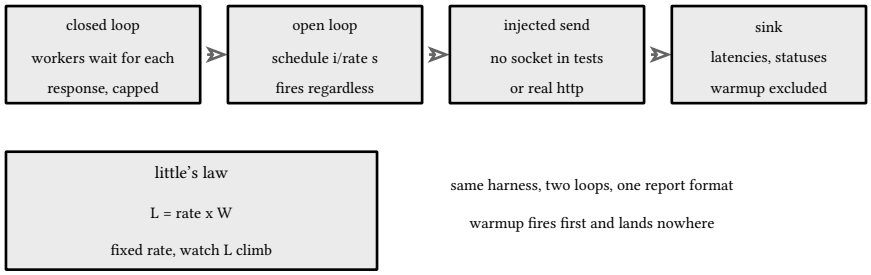


figure 199: workers, server, latency sink: one harness, two clocks

33.5. p99 from nearest rank

The percentile definition is nearest rank: sort the latencies, take the ceiling of p over 100 times n as the rank, answer that rank's value. Its honesty is in what it refuses to do: no interpolation between samples, so p99 of one hundred values is the ninety ninth smallest, not an average of two neighbors, and a rank that runs past the data clamps to the largest, because a percentile past the sample is a bound, not a measurement. P99.9 of ten samples says "the worst one", and says so honestly.

The bucket approximation is the second half of honesty. A scraped histogram cannot answer nearest rank, it can only say which bucket the rank lands in, so the bucketed row of the report answers the smallest bound whose cumulative count reaches the rank, an upper bound on the true value, and not knowable when the rank falls into `+Inf`. The report table states the difference instead of hiding it: the golden set of six latencies, 1, 1, 2, 3, 5, 8 milliseconds, renders an exact row with p50 of 2 and p99 of 8, and a buckets row with p50 of 2.5 and p99 of 10, the source column naming which row is which:

```
export function nearestRank(sorted, p) {
  if (sorted.length === 0) {
    return undefined;
  }
  let rank = Math.ceil((p / 100) * sorted.length);
  if (rank < 1) {
    rank = 1;
  }
  if (rank > sorted.length) {
    rank = sorted.length;
  }
  return sorted[rank - 1];
```

listing 251: nearest rank: ceil the rank, clamp the ends, no interpolation anywhere (source: javascript/api/src/load/quantile.mjs, lines 7-18)

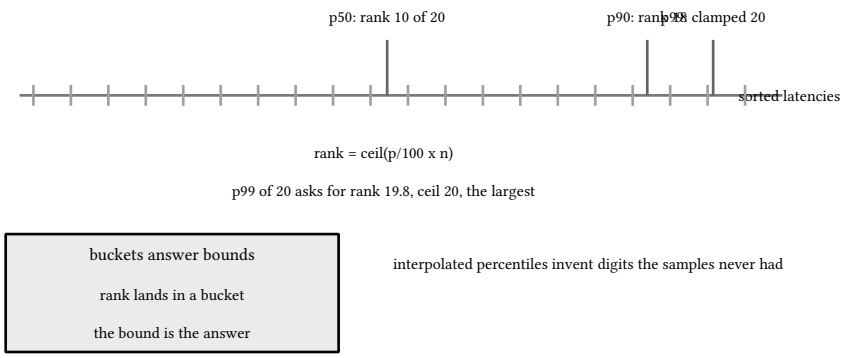


figure 200: the tail ladder: where p50, p90, p99 sit in one sorted sample

33.6. backpressure and writable sockets

Node's own flow control is the one thing the go chapters never had to name, because go's goroutines made every write look free. A response is a writable stream: `res.write` returns false when the socket's buffer has reached the high watermark, `writableLength` is how many bytes sit in that buffer, and the drain event is the socket saying it caught up. Ignoring the return value is the classic node memory leak, bytes accumulating for a client that has stopped reading, and the honest response to a false write is to stop offering until the drain:

```
export async function writeWithBackpressure(res, chunks) {
  for (const chunk of chunks) {
    if (!res.write(chunk)) {
      await once(res, "drain");
    }
  }
}
```

listing 252: the whole rule: a false return waits for the drain event (source: javascript/api/src/load/backpressure.mjs, lines 10-16)

The test pins the rule with a stalled writable: a four byte watermark, a first chunk parked in a consumer whose callback waits on a latch, and the proof the second chunk is never offered until the latch opens and the drain fires, with no wall time anywhere:

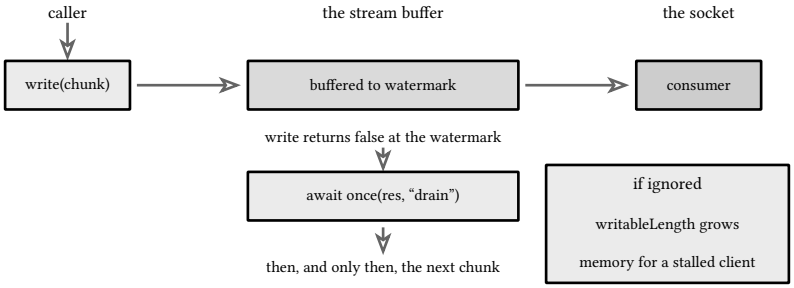


figure 201: the write path: offer, buffer, drain, offer again

33.7. no orphans left behind

A load tool that leaks sockets or timers per run silently slows every later run and lies about the service it measures. The go lane asks its runtime for leaked goroutines after a run, and node's documented equivalent is `process.getActiveResourcesInfo`, the strings naming every handle and request still holding the event loop open. The check runs a real closed loop against a real server over real sockets, closes everything, and asserts that no `TCPSocketWrap`, no `Timeout`, and no `Immediate` remains. Teardown is asynchronous, sockets wind down over the next loop turns, so the check polls the real state until it is quiescent instead of sleeping a guess. This is the same discipline the runner enforces on this machine, a leaked handle fails the suite's exit, and this test is the committed proof the harness leaks nothing:

```
  // teardown is asynchronous: sockets wind down over the next loop
  // turns, so the check polls the real state until it is quiescent
  // instead of sleeping a guess
  const strays = () =>
    process
      .getActiveResourcesInfo()
      .filter(
        (kind) =>
          kind === "TCPSocketWrap" ||
          kind === "Timeout" ||
          kind === "Immediate",
      );
  for (let i = 0; i < 1000 && strays().length > 0; i++) {
    await new Promise((resolve) => setImmediate(resolve));
  }
  assert.deepEqual(strays(), [], "orphaned handles after the run");
```

listing 253: run, close everything, then ask the process what is still alive (source: javascript/api/test/load/leak.test.mjs, lines 26-41)

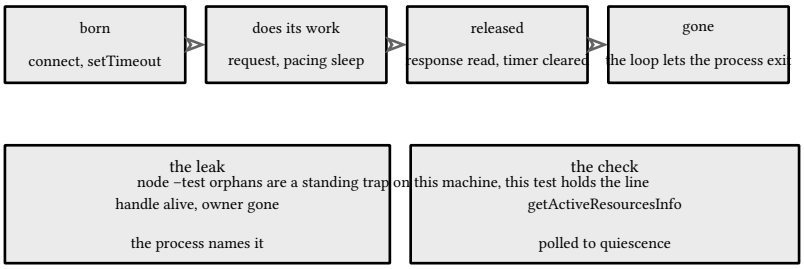


figure 202: a handle lifecycle: born, released, or leaked

sources: nodejs.org/api/http.html for `server.close`, `closeIdleConnections`, and `closeAllConnections`, plus nodejs.org/api/process.html for `process.hrtime.bigint` and `process.getActiveResourcesInfo`, nodejs.org/api/stream.html for the writable `write` return value, `writableLength`, and the `drain` event, nodejs.org/api/events.html for `events.once`, and nodejs.org/api/test.html for what the mock timers replace including `setImmediate`, all read 2026-09-26 on node 26.3.0, with the microtask-hop and immediate-replacement behaviors probed on this machine the same day. Verified by `javascript/api` tests, 26 of them under `test/load`, green under `npm run verify`, on the real-socket closed loop with the orphan assertion green after every run.

34. ship it

The module ends as a directory of tests unless it also ships. This chapter turns the service into an artifact and a lane: an image built from one stage, one compose command that brings it up healthy, and a make gate that replays the contract over that stack. Everything leans on machinery the earlier chapters built: the health and ready routes chapter 23 owns, the strong ETags chapter 29 computes, the drain state chapter 33 wires, the metrics exposition chapter 32 binds. The platform envelopes around a shipped service, the ci workflow, the helm chart, and the terraform slice, are taught in full in book 4, ch 29 (go 1.27, ship it) and mirrored in book 6, ch 33 (c# 15, f# 11, and .net 11, ship it), and this third lane keeps one executing lane per vehicle instead of re-deriving them. What this chapter owns is the node truth of each layer: what the image is when there is nothing to compile, what the probe runs when the runtime is the only executable.

34.1. the image

One stage, because node has no build to stage. The go lane's chapter crossed from a builder into `scratch` with one static file, legal because its store engine was pure go, and the `c#` lane needed the aspnet base because its sqlite bundle is a native library the runtime must carry. This service's store is `node:sqlite`, a module compiled into the runtime itself, and its dependency list is empty, so the smallest correct image is the runtime alone. A builder stage would carry a compiler the image never runs, and the image states that trade instead of paying it. The whole image is 24 lines and the service adds about 4 MB over its base:

```
# single stage, node honesty: no compile step exists, npm ci over the
# committed lockfile is a no-op that exists to prove it, so a builder
# stage would carry a toolchain the image never uses. the tag is the
# exact version the book builds under, on glibc slim.
FROM node:26.3.0-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY src src
# the typed tree rides the same image: no build product exists to
# copy instead, stripping runs the .ts sources, so the ts lane is the
# same image with the entrypoint pointed at the typed entry
COPY src-ts src-ts
# /data is created with an owner because a named volume inherits the
# image content's ownership on first mount, and a volume over a
# root-owned directory would deny the container user its database.
# the user is numeric, 1000, the uid the image ships as node.
RUN mkdir /data && chown 1000:1000 /data
ARG VERSION=dev
ENV GOAPI_VERSION=${VERSION}
ENV NODE_ENV=production
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["node", "src/ship/main.mjs"]
```

listing 254: the whole image: one stage, the runtime, the no-op install proof, both source trees, user 1000 (source: javascript/api/Dockerfile, lines 1-24)

Four decisions earn a second look. The manifests copy precedes the source copy so the install layer caches independently of code edits, the same layer economy the other lanes pay for, except that here the install is deliberately a proof: `npm ci --omit=dev` over the committed lockfile resolves zero packages, and the step exists so the day a dependency sneaks into the manifest the build fails loudly at the layer that introduced it. The source copy takes both trees, `src` and `src-ts`, because the typed lane of chapter 37 runs the same image with the entrypoint pointed at

the typed entry, node stripping the `.ts` sources at run, so no second image exists to maintain. The `/data` directory is created with an owner because a named volume inherits the image content's ownership on first mount, and a volume mounting onto a root-owned directory would deny the container user its database. The user is numeric, `1000:1000`, the uid and gid the node image ships as `node`, read off the pulled image rather than assumed from docs. And only 8080 is declared, because the metrics listener on 6090 is the observability chapter's internal surface, published by the lane that asserts against it and by nothing else.

The tag is exact, `node:26.3.0-slim`, verified against the registry on the day the chapter was written, and it matches the machine the book builds under at the major and the minor. The slim variant is glibc, the same libc family the local suite runs on, at a fraction of the full image, and alpine's musl would make the shipped platform a different platform than the tested one to save megabytes this service does not care about.

The healthcheck has the shape problem both earlier lanes met: the slim image carries no curl and no wget, both verified absent from the pulled image, so nothing exists to probe with except the one executable that does exist, the runtime itself. `src/ship/probe.mjs` reads `GOAPI_ADDR`, rewrites the host to loopback because a bind address of `0.0.0.0` names the side that listens and is not a host anyone can dial, and asks its own `/readyz`. Ready is 200 and only 200, a draining 503 means do not route traffic here, and a refused dial is a failed probe. The probe mirrors the go lane's `-health` flag and the `c#` lane's `--ready-probe` with the one difference that node's global `fetch` makes the probe a module of ten lines rather than a flag on the entrypoint.

The `GOAPI_` names carry over from the go and `c#` lanes deliberately. The contract pins the `goapi_` metric series and the token issuer, so one parameter family and one frozen contract serve all three lanes, each with its own compose file, the ruling the `c#` lane's chapter states and this lane inherits rather than re-argues. The vehicle reads five names, `GOAPI_ADDR`, `GOAPI_METRICS_ADDR`, `GOAPI_DB`, `GOAPI_JWT_SECRET`, and `GOAPI_VERSION`, and nothing else.

A `.dockerignore` earns its place here where the go module's did not: a node vehicle grows `node_modules` from local runs, and riding it into the build context would both bloat the upload and undermine the install proof. The excluded set is the local installs and every tree that is not service source.

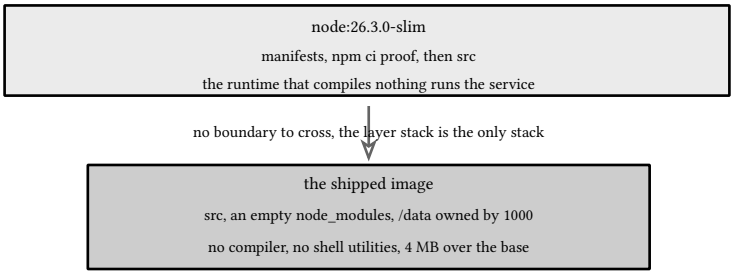


figure 203: one stage, nothing to cross: the runtime is the toolchain is the app

34.2. compose one command

The compose file is one service and its volume, and the contract it enforces is the return value of a single command. `docker compose up -d --build --wait` builds the image, creates the container, starts it, and blocks until the healthcheck passes, so a green up means the image built, the migrations ran against a fresh volume, and the listener answers probes. `down -v` removes the volume in the same breath, so every lane run starts from migration zero, which is what keeps the suite behind it deterministic:

```
services:
  api:
    build: .
    environment:
      GOAPI_JWT_SECRET: dev-only-not-a-secret
      GOAPI_DB: /data/goapi.db
      # both listeners bind every interface inside the container,
      # node's default host is loopback so the bind side is explicit
      GOAPI_ADDR: 0.0.0.0:8080
      GOAPI_METRICS_ADDR: 0.0.0.0:6090
    # host ports move off 8080 and off the c# lane's 18080/16090:
    # shared dev machines carry loopback shadows on 8080, and a lane
    # must be the only thing under its ports
    ports:
      - "19080:8080"
      - "19090:6090"
    volumes:
      - data:/data
    healthcheck:
      # no curl, no wget in the image: the runtime probes its own
      # readyz on loopback, 200 and only 200
      test: ["CMD", "node", "src/ship/probe.mjs"]
      <<: *hc
```

listing 255: the whole system: both listeners, the volume, the probe, host ports off 8080 (source: javascript/api/compose.yml, lines 14-36)

The environment block carries exactly what construction demands. The secret must be present or the process refuses to start, loudly, a behavior the chapter verified by running the image without it and reading the crash: the error names `GOAPI_JWT_SECRET`, the orchestrator restarts, an operator reads the log. A missing secret is a crash, never a running service signing tokens with an empty key. The value in the file is a development constant, and the platform envelopes in the earlier lanes exist partly to replace it. Both listeners bind every interface inside the container because the kernel's listen default in this vehicle is loopback, so the compose file states the bind side explicitly rather than inheriting a default chosen for tests.

The host side of both mappings sits on 19080 and 19090. The container ports keep the family shape, 8080 for the api and 6090 for the metrics listener, but the host ports move off 8080, where shared dev machines carry loopback shadows, and off the `c#` lane's 18080 and 16090, because two lanes must never race for one port on one machine. A lane is the only thing under its ports.

The environment is read exactly once, at the process boundary. `src/ship/config.mjs` is the vehicle's one reader of ambient environment, every family takes its configuration as injected dependencies, and the entrypoint passes those injections to the composition the same way the tests do. One more fact of the entry is stated plainly rather than hidden: the users port rides one store handle the entry opens, and the composition opens a second handle to the same file for idempotency keys and analytics, two handles the store chapter's wal mode lets coexist. The single-store ruling names the day the composition constructs both from one open, and until that day the entry is the honest description of what runs.

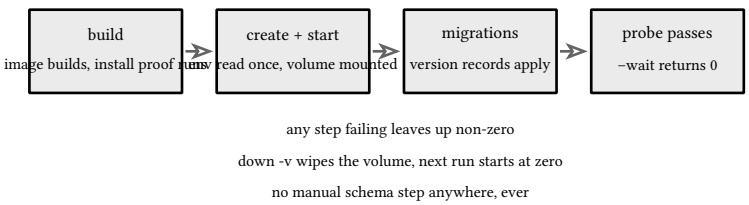


figure 204: what a green up proves, in order

34.3. the docker lane as a gate

The lane is one make target, and its shape mirrors the two-gate rule book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems) owns: the plain module gate never needs a docker daemon, and the docker lane never runs silently, a missing daemon a loud red with the instruction to start it, never a skip. The integration suite lives under `integration/` with its own script entry, outside the `test/**` glob `npm run verify` runs, the faithful mirror of go's build tag and the c# lane's project outside the solution:

```
verify-jsapi-docker:
  @docker info >/dev/null 2>&1 || { \
    echo "docker daemon not reachable." >&2; exit 1; }
  @cd books/javascript/api && docker compose down -v >/dev/null 2>&1 || true && \
    trap 'docker compose down -v' EXIT && \
    docker compose up -d --build --wait && \
    JSAPI_URL=http://127.0.0.1:19080 \
    JSAPI_METRICS_URL=http://127.0.0.1:19090 \
    npm run test:integration; rc=$$?; exit $$rc
```

fragment, illustrative, not executed

Two environment families live in this lane and never mix. The `GOAPI_` names are the vehicle's configuration surface, the same keys in the compose file, read once by the entry. The `JSAPI_URL` and `JSAPI_METRICS_URL` names are the lane's own probe coordinates, read by the integration suite and set by the make target, never by the service. The split guards a failure mode the c# lane named: renaming one family without the other makes the lane silently probe its defaults, a green suite pointed at nothing, so the two lists change together or not at all. The suite also refuses a target that carries a path, because `new URL(path, base)` would silently drop a prefix and the suite would stay green while probing the wrong namespace.

Six tests cover the contract's scenarios: both probes and the two routing envelopes, including the ruling that a 405 carries the `not_found` code plus an `Allow` header, the register replay and login walk with its cookie attribute line and its identical 401 bytes, the etag duel with its one 200 and one 412, the rate drain to a 429, the reports idempotent pair with the sanctioned 202-to-200 upgrade, and the metrics exposition off the internal listener. The ETag rule is the sharpest, a strong ETag is the sha256 of the exact response body, so the suite recomputes the hash over the wire bytes and compares:

```
// the strong etag rule: the header is the sha256 hex of the exact
// response body, recomputed over the live wire bytes rather than read
// off any frozen vector
function wantStrongETag(result) {
  const etag = (header(result, "etag") ?? "").replace(/^"|"$/g, "");
  assert.ok(etag.length > 0, `no ETag header, body ${result.body}`);
  const want = createHash("sha256").update(result.body, "utf8").digest("hex");
  assert.equal(
    etag,
    want,
    `ETag ${etag} is not the sha256 of the body ${result.body}`,
  );
  return etag;
}
```

listing 256: the etag check recomputed over live bytes, not frozen ones (source: javascript/api/integration/lane.test.mjs, lines 67-80)

The rate drain is where the lane tells a node truth the in-process suites cannot. The issue-tier scrypt the authn chapter pins costs about 0.4 seconds per verify on this machine, measured, and the bucket refills one token per second, so a counted drain of exactly five draws races the refill and flakes. The suite draws until the 429 is a fact,

bounded at twelve draws, and asserts the denial's `Retry-After` of at least one second and a remaining count of zero. The clock under test is real, refill is part of the behavior under it, and the assertion names the fact rather than the interleaving.

The lane earned its keep before it ever booted a container. Two hand probes against the composed app over a real socket found failures no family's own suite had walked, because no in-process test registers through the users family and then logs in through the authn family over the composition: the login first read a second, empty user store the composition had accidentally allowed each family to default, and after that fix the login answered 400 because the limiter's email key had consumed the request body that the handler then tried to read a second time from a spent stream. Both were one-line composition fixes, and both were the argument for this chapter: the composed service over a real socket is a different test object than any family's stack, and the contract only holds when the composition holds.

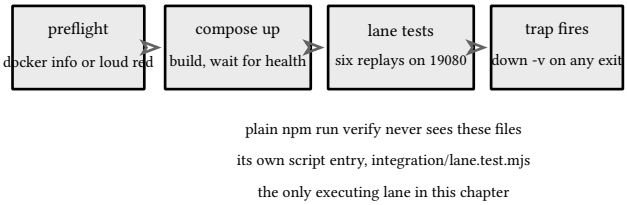


figure 205: the gate: one lane, one trap, no skips

34.4. the shipped checklist

The lane ends where it started, with the property each chapter contributed, and the ship family added one artifact of its own, the process entry. The entry is the composition's last consumer: it reads the environment once, fails closed on the secret, opens and migrates the store, hands the composition its injected dependencies, and boots through the ship family's own wiring, so the file holds no listening of its own:

```
export async function runMain({ env = process.env } = {}) {
  const config = readShipEnv(env);
  // the users port rides this handle; the composition opens its own
  // to the same file for idem and analytics, wal lets the two
  // coexist, and the single-store ruling names the day the
  // composition constructs both from one open
  const wired = wireStore({ path: config.dbPath });
  const app = createApp({
    version: config.version,
    secret: config.jwtSecret,
    storePath: config.dbPath,
    store: createUsersPort(wired.store),
  });
  if (typeof app.boot !== "function") {
    wired.store.close();
    throw new Error(
      "createApp has no boot seam: wireShip must be mounted before main runs",
    );
  }
  await app.boot(config);
  return { app, store: wired.store };
}
```

listing 257: the process entry: env once, fail closed, injection, boot (source: javascript/api/src/ship/main.mjs, lines 11-32)

The boot the entry calls owns the ordered shutdown. A `SIGTERM`, which is what a container stop sends, runs the load family's drain over the real server, readiness flips first so balancers stop routing while in-flight requests finish, and the exit code reports what the drain earned: zero when it drained, one when the grace deadline passed and

the hard kill had to run, with `closeAllConnections` executed only after that deadline, the load chapter's rule carried to the process edge, reported rather than pretended.

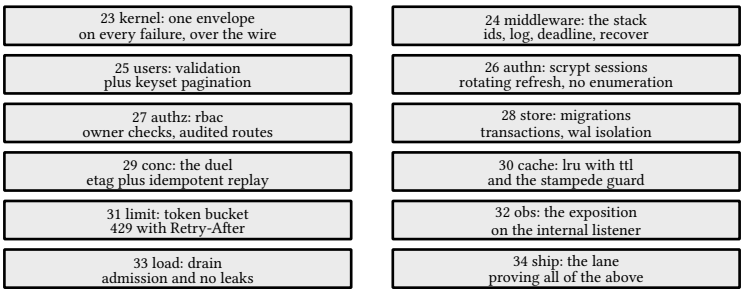


figure 206: the readiness table: twelve chapters, twelve properties, all proven

The chapter declares what it does not carry. The ci workflow, the helm chart, and the terraform slice are the go lane's teaching and the `c#` lane's mirror, and the corpus keeps one executing lane per vehicle. The parameter family is what transfers: the earlier lanes' envelopes configure this image by the same `GOAPI_` names, with the image reference and the host ports as the only edits. Docker's storage model, the container runtime, and compose internals belong to book 13, ch 3 (infrastructure: docker, databases, and testing, containers: processes, signals, pid 1) and book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems). What this chapter owns is the narrow band between a green module and a running service, and the suite chapter that follows hardens the whole set.

sources: the node image tag verified against the registry at hub.docker.com/v2/repositories/library/node/tags, `26.3.0-slim` present, accessed 2026-09-26, and the image facts read off the pulled image the same day, user `node` at uid and gid 1000, `curl` and `wget` absent, `node -v` answering 26.3.0. Docker docs on build cache ordering, healthchecks, and `docker compose up --wait` semantics at docs.docker.com, and node docs on process signal events, `import.meta.main`, the global fetch, and the test runner under nodejs.org/docs/latest/api, both accessed 2026-09-26. The scrypt figure is this machine's own measurement at the chapter's pinned parameters. Verified by `docker compose build`, a run of the image failing closed without the secret and naming the missing name, the unit suite at 317 green, and the compose lane end to end under `make verify-jsapi-docker`.

35. the test suite

The service part ends with the service tested, and this chapter turns the suite around and examines it as its own artifact. What is left is the discipline the whole suite runs on: `node:test` with `assert/strict` as the only framework, doubles that are arguments instead of loader patches, every deadline owned by a fake clock, invariants held by seeded loops because node ships no fuzz engine, and a coverage lane that stays opt-in. The standing rule governs here too: no third party adopted, every habitual package taught as a dated fact instead.

35.1. what the suite holds

The inventory first, because the chapter builds on all of it. The suite holds 60 test files across the family directories plus this chapter's own `test/suite`, 317 test functions and 6 describe suites at this landing, and all sixteen frozen vectors are pinned by a single file whose job is the spine: each vector's bytes hashed with sha256, drift in either direction failing the run. Around the vectors sit the techniques each chapter contributed, the router precedence, parity, and bound loops, the limiter's 429 replayed at the vector's frozen instant, the load lane's orphan check. Zero tests sleep, every test that opens a server closes it in `after()` or a `finally`:

```
// the 16 golden vectors are the lane's spine: byte-frozen, sha256
// pinned, one lane's copy must equal every other lane's. drift here
// means the code is wrong, never the vectors.
const pins = {
  "01-register-201.json":
    "486c37f3aa30ddcc3b7119870af4a07e83b8906c74a25549ec3e30b267cc54f1",
  "02-register-replay.json":
    "40aaded599035926a9383ebc631f42b5b3271b97005cbfc738d97a7385773afe",
  "03-register-409.json":
    "11e89367420ece00721d73e68377b5d063763fcf4328c8c13484168021d3f85f",
  "04-login-200.json":
    "9dce77524259b2856e12a6a30b9839256b7ba60f6b18a65afdacf33236882b23",
  "05-login-401.json":
    "3d1a35f30c792218edc009d4afe3aa72a49f253444d4f66a394d2ac090c1638a",
  "06-login-429.json":
    "0254246013fc76d4d89d5b89b42154294120af4664c144009e3bce72ec68e84c",
  "07-refresh-rotate.json":
```

listing 258: the sixteen vectors, sha256 pinned: the spine every family test hangs from (source: javascript/api/test/contract.test.mjs, lines 8-24)

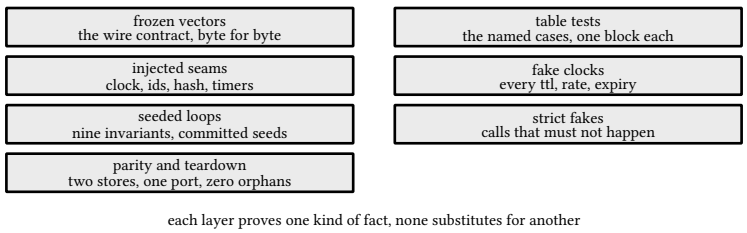


figure 207: the suite's layers, each against the one thing it proves

35.2. the surface

`node:test` is the vehicle's only test framework and the whole surface fits in one import. The unit is `test()`, the grouping form is `describe()` with `it()` inside: 6 describe suites carry the 17 layered tests that mount a real kernel, and the other 300 ride bare `test()` calls. Subtests exist on the surface as `t.test()` and the vehicle does not use them, `describe` and `it` nest identically in the runner's report and keep the arrange and assert phases in separate scopes. The assertion half is `assert/strict`, and the strict is load-bearing: plain `node:assert` runs legacy mode where

`equal` is == and `assert.equal(1, "1")` passes, while the strict import rebinds `equal` to `strictEqual` and `deepEqual` to `deepStrictEqual`. Each identity gets one job: `assert.equal` compares wire bytes as strings, strict identity on strings being content equality and the vectors content, and `assert.deepEqual` compares parsed shapes like the router's params, where prototype and type must match too:

```
describe("the metrics layer mounted on the real kernel", () => {
  it("ticks both series at the status the writer really wrote", async () => {
    const { metrics, layer } = wireObs();
    const app = createKernel();
    app.use(layer);
    app.route("GET", "/healthz", async (ctx) => {
      writeJson(ctx.res, 200, { status: "ok" });
    });
    app.route("GET", "/boom", async () => {
      // the failure is thrown under the metrics layer, so the kernel
      // backstop writes the 401 after this layer has already yielded:
      // the finish hook is what sees the real status
      throw fail(CODES.unauthorized, "invalid credentials");
    });
    const handle = await app.listen();
```

listing 259: a describe suite over the real kernel, the handle closed in the same block (source: javascript/api/test/obs/layer.test.mjs, lines 9-23)

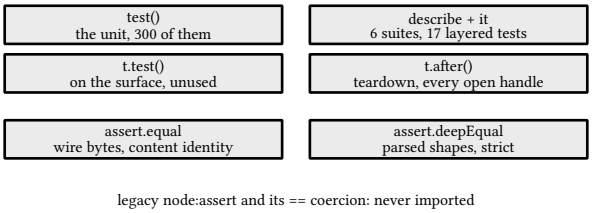


figure 208: the runner surface against what the vehicle uses it for

35.3. doubles without a framework

The doubles story is two platform tools and one hand-rolled shape. `mock.fn()` builds a counting function, and the suite counts with it everywhere: handler calls in the concurrency guard, fills in the cache layer, log lines in the middleware tests, each asserted through `handler.mock.callCount()`. `mock.method()` intercepts one method of a real object in place, and the vehicle never reaches for it, because every seam the service defines is an injected argument rather than a method on a shared object, so a seam's double is a plain object the test builds whole. The hand-rolled shape is the strict fake, the one thing neither platform tool gives by default, a double where any call at all fails the test, and the concurrency chapter's `refusingKeys` is eight lines proving a negative, that the keyed runner never consults storage on the paths it does not govern:

```
}

// a strict double: any call is unexpected, proving the guard never
// consults storage on routes it does not govern.
function refusingKeys() {
  const fail = () => {
    throw new Error("keys must not be touched on this path");
  };
```

listing 260: the strict fake: any call is unexpected, the guard proven clean-handed (source: javascript/api/test/conc/guard.test.mjs, lines 19-26)

The ecosystem's habit is the reverse of this choice, `jest.fn()` and vitest's `vi.fn()`, `testdouble`, `sinon` with its spies and sandbox restore. All are taught as names and adopted as nothing, on the maintenance rule: a mock library in the graph is a dependency whose deprecation becomes your migration, while these seams are small enough that a readable object literal costs less than the tooling it replaces, and the strictness those libraries sell is the eight lines above for unexpected calls and a `mock.fn` plus an `assert.equal(callCount(), n)` for expected ones:

the job	the vehicle's answer	the ecosystem's answer
counting	mock.fn + callCount()	jest.fn, vi.fn
refusal	a throwing object	expectation APIs
interception	an injected argument	mock.method, sinon
dependency cost	zero	one module, or a framework

figure 209: four double shapes against the job each does

35.4. the fake clock owns every deadline

Every clock-dependent test runs on `mock.timers`, and the discipline has three parts. Enable names the apis to replace, `enable({ apis: ["setTimeout"] })` fakes `setTimeout` in the global context and in `node:timers` and `node:timers/promises` alike, while a bare `enable({ now })` takes the broader default and fakes `Date` too. Tick is the only clock: the idempotency key's 24 hour window is driven to its boundary from below and above, the cache ttl by a 1000ms tick, the limiter's frozen 429 at the vector's own instant, and no test anywhere sleeps. Reset restores the real clock, explicitly in a `finally` or `after()`, or implicitly because `t.mock.timers` restores itself when the test ends. The idempotency test is the whole discipline in eleven lines, a match at `ttl - 1` and a miss one tick later:

```
test("a key past its 24 hour window answers miss and deletes its row", (t) => {
  t.mock.timers.enable({ now: 1_000_000 });
  const keys = seeded(t, () => Date.now());
  keys.save(entry);
  t.mock.timers.tick(24 * 60 * 60 * 1000 - 1);
  const lastMs = keys.lookup(entry);
  assert.equal(lastMs.state, "match");
  t.mock.timers.tick(1);
  assert.deepEqual(keys.lookup(entry), { state: "miss" });
  assert.deepEqual(keys.lookup(entry), { state: "miss" });
});
```

listing 261: the 24 hour window ticked to its boundary: match below it, miss past it (source: javascript/api/test/store/idem.test.mjs, lines 43-53)

Three platform traps are already probed on the pinned 26.3.0 and each has its fix in the tree. The first is the captured reference: a module whose default clock is the `Date.now` function value, `now = Date.now`, freezes the real clock into its closure before any test enables the mock and ticks move nothing, so the vehicle's rule is a thunk, `now = () =>`
`Date.now()`, resolving `Date` at every call and seeing whatever clock is installed, a rule the limiter states in its own comment. The second is the default scope: a bare `enable()` replaces `setImmediate` too, so an un-ticked immediate poll would hang forever, and the fixes are both in the tree, the deadline tests narrow the apis list to `setTimeout` so their `setImmediate` settle steps stay real, and the load lane's tests under the full default use latches, promises the code under test releases, never polls. The third is the import: `mock` rides the named export of `node:test` itself, imported alongside `test` in the whole-clock tests:

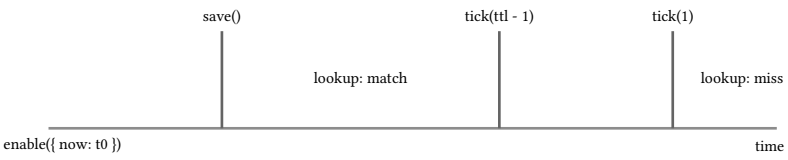


figure 210: one ttl under the fake clock, ticked across its boundary

35.5. properties as seeded loops

Go has a fuzz engine, and node does not: `node:test` ships no mutation search, no corpus directory, no coverage guided anything. The node answer in this vehicle is the seeded loop: a property is a sentence that must hold for every input, a small prng with a committed seed generates the inputs, the invariant is named in the file beside the seed, and a failure replays exactly because the seed is the case. The book's stated prng is splitmix32, and the suite holds nine loops: router precedence on an LCG with seed 20261001, cursor compose on seed `0x2a17c0ffee`, the keyset round-trip, store parity, and cache bound loops on seed `0x12345678`, bucket arithmetic on seed `0x4c494d49`, metrics monotonicity on seed `0x4f425339`, and this chapter's two additions in `test/suite`. The cursor closedness loop earned its prose the hard way, its own seed found a true fact about the codec: a flipped base64 character decoded to a carriage return between two json members, `JSON.parse` tolerates it as whitespace, and the parsed key was valid while its re-encoding differed from the corrupted string. So the invariant is stated in its honest two-leg form, byte identity pinned on issued bytes only, and the corruption leg pins what matters to the walk, that any single-character corruption either refuses or yields a well-formed key that itself round-trips, never a silently empty position that would truncate page two:

```
  for (let round = 0; round < 300; round++) {
    const key = { t: stamp(), i: id() };
    const encoded = encodeCursor(key);
    assert.equal(
      encoded.includes("="),
      false,
      "issued cursors carry no padding",
    );
    assert.deepEqual(
      decodeCursor(encoded),
      key,
      `round ${round}: round trip moved`,
    );
    assert.equal(
      encodeCursor(decodeCursor(encoded)),
      encoded,
      "re-encode is the identity on issued bytes",
    );
```

listing 262: round-trip over random sort keys, the identity leg on issued bytes (source: javascript/api/test/suite/cursor-closed.test.mjs, lines 74-91)

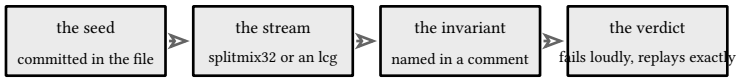


figure 211: seed to stream to invariant to verdict, the node answer to fuzzing

35.6. dated and declined

Node has a module mocking story and it is dated prose, not adopted surface. The documentation marks `--experimental-test-module-mocks` at stability 1.0, early development, added in v22.3.0 and v20.18.0, and it gates `mock.module()`, which swaps a specifier's resolved module for a fake for the remainder of the run. Verified on the pinned 26.3.0 with a probe: without the flag the `mock` object exposes no `module` member at all, with the flag it is a function. The design reason for declining it is stronger than the flag: module interception works by patching the loader, mocks a specifier for every importer at once, fights ESM's evaluated-once semantics, and couples a test to module identity rather than behavior. The vehicle's seams are constructor and function injection instead, the deadline middleware takes a `timers` object, the limiter a clock thunk, the store a `now`, so faking a dependency means handing the subject a different argument, never touching the module graph. The deadline tests show the style in one file, the seam test injecting a `timers` object built from two `mock.fn` doubles and asserting the cleared handle without any clock at all:

```
test("a handler that finishes in time passes through, its timer cleared", async () => {
  const cleared = [];
  const timers = {
    setTimeout: mock.fn(() => 7),
    clearTimeout: mock.fn((handle) => cleared.push(handle)),
  };
  const layer = handlerDeadline({ ms: 100, timers });
  const ctx = makeCtx({ url: "/fast" });
  await layer(async (c) => {
    writeJson(c.res, 200, { status: "ok" });
  })(ctx);
  assert.equal(ctx.res.statusCode, 200, "the handler's answer stands");
  assert.deepEqual(cleared, [7], "the unused timer was cleared");
});
```

listing 263: the seam as the module-mock alternative: a timers object handed in, two mock.fn doubles inside (source: javascript/api/test/middleware/deadline.test.mjs, lines 25-38)

The property library the ecosystem reaches for is `fast-check`, at version 4.10.2 on the registry the day of this writing. It offers structured generators, shrinking that walks a failing case down to its minimal form, and frequency-weighted unions, and the vehicle deliberately fails its adoption criteria: the module graph is zero dependencies by standing rule and `npm ci` over the committed lockfile is a no-op, the shapes this service generates are flat enough that a ten line splitmix32 loop already states them, and reproducibility rides the committed seed rather than a shrinker's output. The same accounting covers the frameworks the kernel chapter declined, `express` at 5.2.1 and `fastify` at 5.12.5 on the registry the same day, both would own the router, the middleware chain, and the error envelope, the three things this part exists to build by hand. The criteria are stated so the choice reads as a decision: a third party earns its keep against generator complexity a hand-rolled loop cannot express, or a framework feature the kernel would otherwise get wrong at the wire, and neither holds here:

```
  for (let round = 0; round < 100; round++) {
    const noisy = shuffled(record, rng);
    noisy[`debug_field_${round}`] = round;
    assert.equal(
      wireBody(noisy),
      canonicalAdmin,
      `round ${round}: admin bytes moved`,
    );
    assert.equal(
      publicBody(noisy),
      canonicalPublic,
      `round ${round}: public bytes moved`,
    );
    assert.ok(
      !wireBody(noisy).includes("passwordHash"),
      "the hash cannot leak by construction",
    );
  }
```

listing 264: the hand-rolled generator loop: shuffled construction, byte-identical wire answers (source: javascript/api/test/suite/envelope-order.test.mjs, lines 92-109)

the axis	mock.module, fast-check 4.10.2	express 5.2.1, fastify 5.12.5
what it offers	loader swaps, generators	router, chain, envelope
stability	a flag, stability 1.0	mature, and unnecessary
the vehicle's answer	a seam, a seed, a loop	100 lines the book teaches
dependency cost	a module in the graph	a framework owns the wire

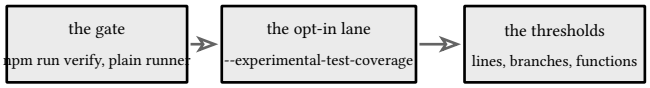
figure 212: the declined column against what each would have owned

35.7. coverage, opt-in

Node ships a native coverage lane and the flag story is verified on the pinned 26.3.0 rather than recalled: `node --test --experimental-test-coverage` prints a per-file table of line, branch, and function percentages plus the uncovered line ranges, and the flag still carries its experimental name in the v26 documentation, added in v19.7.0 and usable with `--test` since v20.1.0. Around it 26 ships a threshold family, `--test-coverage-lines`, `--test-coverage-branches`, and `--test-coverage-functions` turn a low percentage into a failure, and `--test-coverage-include` and `--test-coverage-exclude` scope the report. The lane is opt-in and stays out of the gate: `npm run verify` is the plain runner and the coverage command is typed when wanted, never wired into a script, on the ruling the go lane already made, a percentage measures the tree and the day, and a pinned number would be a lie with a date on it. No percentage appears in this book, the report's one job is the gap scan before a chapter lands, the branches the new tests never reached:

```
"scripts": {
  "test": "node --test \"test/**/*.test.mjs\"",
  "test:ts": "node --test \"test-ts/**/*.test.ts\"",
  "verify": "node --test \"test/**/*.test.mjs\" && node --test \"test-ts/**/*.test.ts\" && tsc --noEmit",
  "test:integration": "node --test integration/lane.test.mjs"
```

listing 265: the gate: one script, the plain runner, coverage never wired in (source: javascript/api/package.json, lines 9-13)



still experimental-named in the v26 docs, verified on 26.3.0

figure 213: the gate against the opt-in lane

35.8. what the suite proves

The closer maps each layer to the fact it establishes. The vectors prove the wire contract byte for byte, the table tests the named cases, the injected seams determinism, ids and hashes and clocks supplied by the test, the fake clocks every ttl, rate, and expiry boundary without one millisecond of wall time, the seeded loops nine invariants under their committed streams, the strict fakes that calls which must not happen do not, and the parity loop the deepest one, that the sqlite adapter and the frozen in-memory store answer identically through the port because the port is the oracle and disagreement names the diverging op. Two honest boundaries close the account. Node has no race detector, and the go lane's child process canary has no equivalent here, the single threaded event loop is the mitigation the concurrency chapter argued, and the load lane's orphan check polls `process.getActiveResourcesInfo()` after a run so a leaked socket or timer fails the suite instead of haunting the next one. And node has no fuzz engine, so the loops are fixed streams, not searches, which is why their seeds are committed like vector bytes, the replay is the contract:

```
// invariant, named: for any op stream, the sqlite adapter and the
// frozen in-memory store answer identically, full records and thrown
// StoreError codes alike. the port is the oracle because it is the
// contract two chapters share, and the memory store is its reference
// implementation.
test("the adapter and the memory store agree on every op, seed 0x12345678", (t) => {
  const rng = splitmix32(0x12345678);
  const mine = ported(t);
  const oracle = createMemoryStore();
  const pick = (arr) => arr[Math.floor(rng() * arr.length)];
  const codesOf = (fn) => {
    try {
      return { ok: fn() };
    } catch (err) {
      if (!(err instanceof StoreError)) throw err;
      return { threw: err.code };
    }
  };
```

listing 266: the parity loop: two stores, one port, the port the oracle (source: javascript/api/test/store/users-port.test.mjs, lines 193-210)

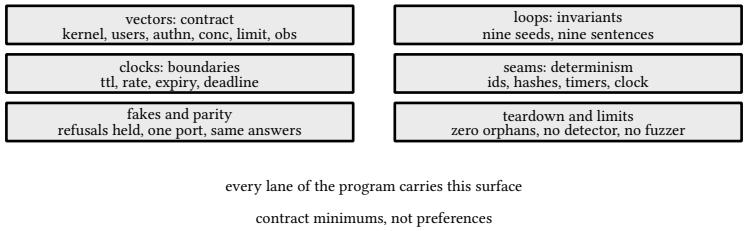


figure 214: the layer to property to where proven grid, closing the part

sources: nodejs.org/docs/latest-v26.x/api/test.html for `test`, `describe`, `it`, subtests, `mock.fn`, `mock.method`, and `mock.timers` including the `apis` option, nodejs.org/docs/latest-v26.x/api/assert.html for strict mode and the legacy == warning, and nodejs.org/docs/latest-v26.x/api/cli.html for `--experimental-test-coverage`, its threshold family, and `--experimental-test-module-mocks` at stability 1.0, all accessed 2026-09-26. The flag and mock surfaces verified against the pinned node 26.3.0 locally: the coverage table run over a family test file, `mock.module` absent without the flag and present under it, a bare `mock.timers.enable()` probed to take `setImmediate` and `Date`. The registry reads: fast-check at 4.10.2, express at 5.2.1, fastify at 5.12.5, accessed 2026-09-26, deliberately not adopted. Verified by the suite, 317 tests and 6 describe suites under `npm run verify`, zero sleeps, every handle closed.

36. the typed layer

The service part has run for 13 chapters on plain `.mjs`, and this chapter runs it again in TypeScript without forking a single decision. The vehicle is one package: `src/` stays the teaching spine every chapter before this one pinned, `src-ts/` is the same vehicle re-expressed, and the two trees share one contract directory, one set of frozen vectors, and one lockfile. Nothing about the wire changes. The typed layer is the lesson, the same lesson the `f#` tour taught over its `c#` artifacts: a second notation over the first, never a second service.

36.1. one vehicle, two languages

Node 26.3.0 runs `.ts` sources natively. The probe is one property read off the process itself, `process.features.typescript`, and it answers `"strip"` on this machine: the runtime parses type syntax and erases it, no emit step, no loader flag, no build watcher between saving a file and running it. The probe is a committed test, so the mechanism is pinned the way every other platform fact in this book is pinned, by an assertion that runs. The stripping mode also draws the one hard line the sources obey: only erasable syntax is allowed, no enums, no namespaces, no parameter properties, and chapter 35's ruling that the suite owns no compiler step stays intact because there is no step to own.

The compiler still matters, so TypeScript rides along as the package's one devDependency, pinned at 7.0.2 and lockfile-committed the way the `c#` lane pins xunit: a toolchain dependency, never a module graph dependency. Nothing in `src-ts/` imports from `typescript`. The verify script states the whole arrangement in three legs, the js suite over `src/`, the ts suite over `src-ts/`, and `tsc --noEmit` over the typed trees, and every commit of this wave ran all three.

The composition mirrors with the same discipline. `src-ts/server.ts` holds the same one-writer shape chapter 23 pinned, one import and one wire call per family at its chapter slot, the store constructed first for the single-store ruling, the operations layers mounted position-aware, the keyed runner built over the store's idem adapter. A reader who knows `src/server.mjs` can diff the two files line for line and find only the type annotations, which is the point: the typed layer claims nothing the spine did not already claim.

```
  "scripts": {
    "test": "node --test \"test/**/*.test.mjs\"",
    "test:ts": "node --test \"test-ts/**/*.test.ts\"",
    "verify": "node --test \"test/**/*.test.mjs\" && node --test \"test-ts/**/*.test.ts\" && tsc --noEmit",
    "test:integration": "node --test integration/lane.test.mjs"
  },
  "devDependencies": {
    "typescript": "7.0.2"
  }
}
```

listing 267: one package, two source trees, one toolchain dependency, three verify legs (source: javascript/api/package.json, lines 9-18)

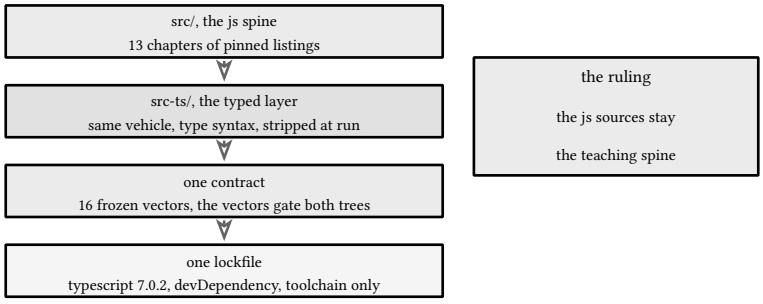


figure 215: two notations, one contract underneath

36.2. the strict tsconfig

Seventeen lines carry the whole configuration, and every flag in them paid its way during the port. `strict` turns on the family: no implicit `any`, no unchecked `null`. `exactOptionalPropertyTypes` refuses to write `undefined` into an optional member, which is why every deps bundle in `src-ts/` declares its optionals as `X | undefined` or restructures, the difference between a member that is absent and one that is present holding `undefined` is now a stated fact instead of a silent coercion. `noUncheckedIndexedAccess` reads every index lookup as possibly absent, so `ctx.params` answers `string | undefined` and the loop-bounded reads in the router carry the `!` assertion, the compiler saying out loud that a matched route guarantees presence while being unable to see the invariant itself. `verbatimModuleSyntax` forces `import type` for type-only imports, so what erases at runtime is exactly what says it erases.

The last two flags state the architecture. `erasableSyntaxOnly` refuses any syntax the stripping runtime cannot erase, the compiler enforcing the same line the runtime draws, and `allowImportingTsExtensions` with `noEmit` lets module specifiers name `.ts` files directly, which is what node's own resolution expects. `types` is the empty array, no ambient package loads, and the platform surface the sources touch is declared by hand in the next section's file.

```
{
  "compilerOptions": {
    "strict": true,
    "exactOptionalPropertyTypes": true,
    "noUncheckedIndexedAccess": true,
    "verbatimModuleSyntax": true,
    "erasableSyntaxOnly": true,
    "noEmit": true,
    "allowImportingTsExtensions": true,
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "target": "esnext",
    "lib": ["esnext"],
    "types": []
  },
  "include": ["src-ts/**/*.ts", "test-ts/**/*.ts"]
}
```

listing 268: the strict line: five families of flags, one erasable-syntax ruling, no ambient packages (source: javascript/api/tsconfig.json, lines 1-17)

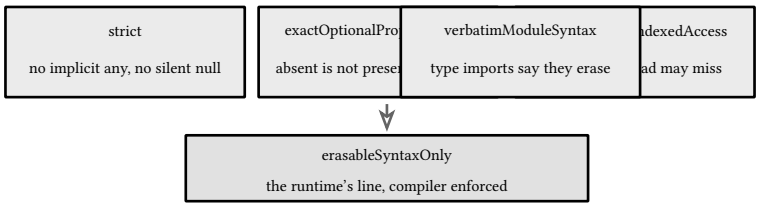


figure 216: each flag buys one stated fact

36.3. types at the kernel seams

The js chapters kept the context shape implicit, a plain object every handler happened to agree on. The typed layer names it once, in `src-ts/kernel/ctx.ts`, and every family that touches a context imports from that file, all but the cache, whose lru and singleflight never see one, so kernel, middleware, and handlers cannot drift apart the way js silently lets them. `Ctx` carries the request, the response, the params map, the query, the edge-resolved id, and the two members later layers install, the route label and the stashed body parse, plus the actor the identity layer resolves. `Handler` and `Layer` are one line each and the whole middleware discipline of chapter 24 typechecks against them.

Two seams deserve their own words. `Res` is the minimal writer contract, the status, the answered flags, the members the envelope, the access log, and the metrics recorder touch, and the real `ServerResponse`, the register replay's recording double, and the suite's `FakeRes` all satisfy it structurally, no inheritance anywhere. And because no `@types/node` rides in the toolchain ruling, `env.d.ts` declares the platform surface by hand, exactly the members the sources import and nothing more, `node:test` with its `mock` timers, `node:sqlite` with `StatementSync`, the handful of `node:crypto` primitives, the globals. Every declaration in that file is a seam this chapter can point at, and the compiler refuses a member nobody declared, which keeps the surface honest in both directions.

```ts
// the actor the identity layer installs. roles read from the stored
// record, never from a token claim.
export interface Actor {
  userId: string;
  sessionId: string;
  roles: string[];
}

// the request context the kernel births at the edge and the router
// fills in. params reads string | undefined under
// noUncheckedIndexedAccess: the router guarantees presence for a
// matched pattern, but the compiler cannot see that invariant, so
// handlers run their own presence rules, the same rules the js
// handlers already ran.
export interface Ctx {
  req: Req;
  res: Res;
  params: Record<string, string>;
  query: URLSearchParams;
  id: string;
  route?: string;
  body?: unknown;
  actor?: Actor | null;
}

export type Handler = (ctx: Ctx) => void | Promise<void>;
export type Layer = (next: Handler) => Handler;
```

listing 269: the seam module: the actor, the context every family builds against, the handler and layer types (source: javascript/api/src-ts/kernel/ctx.ts, lines 35-61)

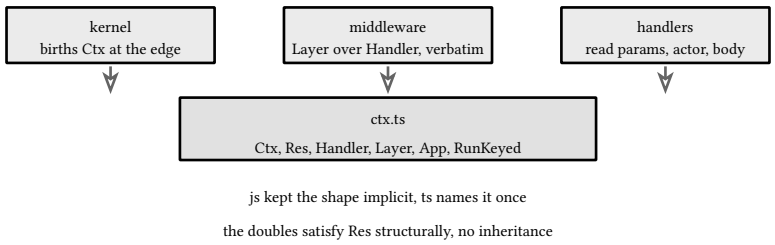


figure 217: one seam module, three consumers, no drift

36.4. the store port as interfaces

Chapter 25 froze the user store port in prose and discipline, two implementations that happened to agree. The typed layer freezes it in the compiler. `UserStore` is the interface, the memory store from chapter 25 and the sqlite adapter from chapter 28 both satisfy it, and a verb signature that drifts in either tree is a compile error before it is a bug. `StoreError` narrows its `code` to the four-outcome union, so the handlers' `mapStoreError` exhausts the arms against a closed set instead of comparing free strings. `UserRecordInput` is `UserRecord` without the version, the type

stating what the js code already did, the store stamps the version on the way in and a caller who claims one is ignored.

The parity property survives unchanged. The seeded loop from chapter 28's tests, splitmix32 at seed `0x12345678`, drives the sqlite adapter and the memory store through the same op stream and asserts identical records and identical thrown codes, and the loop runs again in the ts suite over the ts stores. The port is the oracle because it is the contract two chapters share, and now the compiler reads the contract too.

```
// the frozen port: the verbs the handlers build against, the memory
// store below and the sqlite adapter in the store family both satisfy
// this interface, and the compiler holds them to it.
export interface UserStore {
  create(u: UserRecordInput): UserRecord;
  get(id: string): UserRecord;
  getByEmail(email: string): UserRecord;
  list(q: { after: Cursor | null; limit: number }): UserRecord[];
  updateDisplayName(id: string, expectVersion: number, displayName: string): UserRecord;
  delete(id: string): void;
  grantRole(id: string, role: string): string[];
  revokeRole(id: string, role: string): string[];
}
```

listing 270: the port as an interface: both stores satisfy it, the compiler holds them to it (source: javascript/api/src-ts/users/store.ts, lines 46-58)

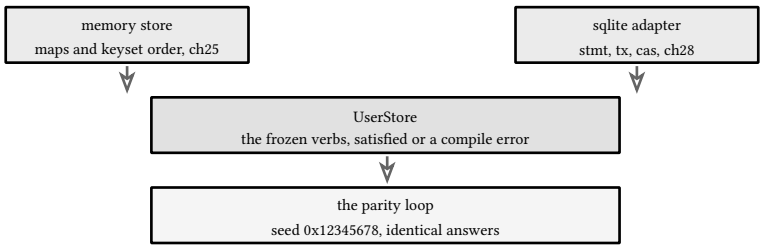


figure 218: two implementations, one interface, one property loop

36.5. what the types prove

The honest accounting has two columns. The compiler proves the internal wiring: a handler registered where a layer belongs is a compile error, a deps bundle missing its clock is a compile error, a store without `getByEmail` is a compile error, and the discriminated unions narrow without casts, the idem lookup answers miss, mismatch, or match with the snapshot riding only the last arm, and narrowing on `state` makes the snapshot readable exactly where it exists. Those are guarantees about the seams, and they hold before any test runs.

The other column is what only the runtime owns, and the port kept every one of those checks. The strict decode ladder still inspects the parsed body member by member, because the wire hands over `unknown` and the type only describes what survived the inspection. `readJson` still runs the media type, the cap, and the parse, the cookie reader still validates shape by hand, and `isUuid` still tests the string, a type can name a uuid but cannot know the runtime produced one. Where a type would make a js check redundant the check stayed anyway, commented only when the reason is not obvious, because the js sources are the spine and the two trees must answer identically. What the types prove is the wiring. What the vectors prove, next chapter, is the behavior.

```
// the discriminated lookup outcome: miss, mismatch, or match with the
// stored snapshot riding the last arm only.
export type IdemLookup =
  | { state: "miss" }
  | { state: "mismatch" }
  | { state: "match"; snapshot: ResponseSnapshot };

export interface LookupArgs {
  route: string;
  identity: string;
  key: string;
  requestHash: string;
```

listing 271: the outcome union: three states, the snapshot on one arm, narrowed without casts (source: javascript/api/src-ts/store/idem.ts, lines 10-21)

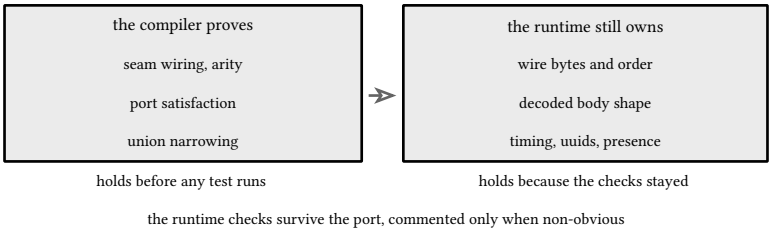


figure 219: two columns, both true

sources: nodejs.org/api/typescript.html for the stripping mode and the erasable syntax rules, accessed 2026-09-27, devblogs.microsoft.com and typescriptlang.org for the typescript 7.0.2 release and the strict-flag family, accessed 2026-09-27. The mechanism facts are the committed probe on node 26.3.0, `process.features.typescript` with the value `strip`, 2026-09-27. Verified by the live chain: 317 js tests, 319 ts tests, and `tsc --noEmit` clean in `npm run verify`.

37. the typed suite

Chapter 35 built the suite: `node:test`, `assert/strict`, hand-rolled doubles, mock timers as the only clock, seeded prng loops as the property checks, 317 tests over the js sources. This chapter runs the same suite again with the import specifiers pointed at `src-ts/`, and the claim under test is narrow and checkable: the typed vehicle behaves identically because the same assertions pass against both trees. Where the port needed a seam to speak, the seam is typed. Where it did not, the test is the same text.

The wave built the mirror the way it built the vehicle, family by family with the three verify legs green at every step. Each family's mirrors landed as one unit, the js suite proving the spine untouched, the ts suite proving the port faithful, the compiler proving the seams coherent, and the next family started only when all three held. A mirror that could not compile was never left in the shared tree, the same parking discipline the js waves held for red tests, and the ledger of those units is the commit chain this chapter's counts come from.

37.1. the same suite over the ts sources

Every one of the 63 js test files has a mirror under `test-ts/`, same file name, same test names, same seeds, same counts. The differences are exactly three. The imports name `.ts` modules, so the suite loads the stripped sources through the same runner and the same glob shape the js suite uses, `node --test` over a pattern that differs by its directory and its extension. The doubles grew the members the typed seams demand, `FakeRes` declares itself an implementation of the kernel's `Res`, and one chapter's private minimal recorder collapsed into the shared double because the compiler refused the shortcut. And the fixtures type their deps bundles, which reads as documentation and changes no runtime behavior.

The helpers came over with the same philosophy. `helpers.ts` is the js `helpers.mjs` with one addition, the request double now carries an empty async iterator because the `Req` seam names it and a router test never reads a body, and `TestCtx` types the context at the recorder's own richness so `res.json` reads without a cast, and one inert drift: `makeCtx`'s id default moved from js `null` to the empty string, unseen by every test that reads it. `ops-helpers.ts` types the socket round trip and nothing else. The suite's discipline carried over untouched: no test sleeps, every deadline and window runs under `mock.timers` with `apis: ["setTimeout"]`, every opened server closes in `after`, and the leak poll from the load chapter still asks the process for its active resources and gets silence back.

The property loops came over byte for byte. The router's static-beats-param invariant still drives seed `20261001`, the cursor codec's closure still drives seed `0x53553138`, the store parity loop still drives seed `0x12345678` against both stores, and the wire writer's member-order loop still drives seed `0x53553137`. The seeds are the cases, and a different answer from the ts tree would be a different vehicle, which is the failure this suite exists to rule out. Two tests have no js mirror, both in the probe file: one runs typed syntax through the runner, one pins the stripping mode off the process itself, so the ts suite counts 319 against the js 317, 6 describe suites in each, zero sleeps in either.

The describe suites came over with their shapes intact because the runner surface is the same surface. The limit layer's vector 06 replay still mounts the wired stack under `describe` with a `before` that builds the app once, the obs chapters still exercise the internal listener and the finish-hook recorder in their own suites, the load chapters still drive both loops, and the ship chapter still fires its hand-rolled signal double at a real listening server. The mock timers came with them, and the stripping runtime handles the double patching the same way it handles any other library call, because the doubles patch globals and the type syntax never survives to the call.

```
test("static beats param for any id, under any registration order (seeded)", async () => {
  // invariant: /api/users/reports is always served by the static
  // route and never by /api/users/:id, whichever order they were
  // registered in. seeded prng, no native fuzzing in node.
  let seed = 20261001;
  const rng = () => {
    seed = (seed * 1664525 + 1013904223) >>> 0;
    return seed / 4294967296;
  };
  const hex = "0123456789abcdef";
  const uuidOf = () => {
    let s = "";
    for (let i = 0; i < 32; i++) s += hex.charAt(Math.floor(rng() * 16));
    return `${s.slice(0, 8)}-${s.slice(8, 12)}-7${s.slice(13, 16)}-8${s.slice(17, 20)}-${s.slice(20, 32)}`;
  };
  for (let round = 0; round < 50; round++) {
    const router = createRouter();
```

listing 272: the same invariant, the same seed, the same loop, over the ts router (source: javascript/api/test-ts/kernel/router.test.ts, lines 24-40)

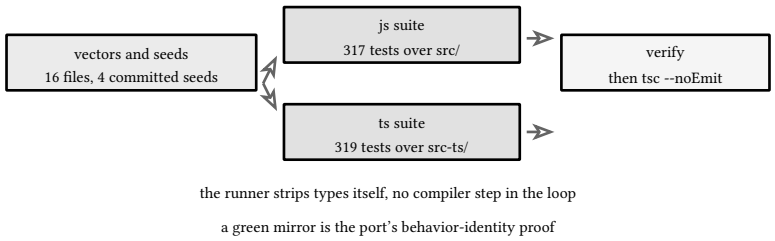


figure 220: one suite, two trees, one gate

37.2. what strictness catches

The port met, in the compiler, the classes of error the js tree can only meet at runtime, and a few of them are worth naming because they are the classes this book's js chapters guarded by hand.

The unknown catch. In js, `catch (err)` is whatever was thrown, and `err?.stack` reads it optimistically. The typed `catch` hands over `unknown`, so the recover layer's log line narrows with `instanceof Error` before it reads the stack, and the js spine's line and the ts line produce the same bytes while only one of them had to say what it knows. The index read. `noUncheckedIndexedAccess` made every params read a possible miss, which is true, a handler can be mounted under a different pattern than the one it was written for, and the presence rules the js handlers already ran are now the only thing standing between the read and the answer, stated instead of assumed. The optional member. `exactOptionalPropertyTypes` refused every hand-built deps bundle that passed an explicit `undefined` down, and the fix was to declare the optionals honestly, which is why the deps interfaces read `| undefined` throughout.

The composition felt that one twice. `createKernel` receives the spread of the whole deps bundle, and the first draft of its options type took plain optional members, so the composition's `{ ...d, ready }` failed to compile the moment any member held an explicit `undefined`. The honest fix was not a cast, it was typing the optionals as `X | undefined` and letting the spread tell the truth about what a caller may pass. The same flag caught the wire limit's key seam, where a default whose declared type accidentally read as returning `undefined` was refused until the parentheses said what the rule meant.

The tests also sharpened the seams, which runs the usual direction backwards. The keyed runner's body argument started as a buffer because the register wrapper reads bytes, and the guard's own tests hand it strings, so the seam widened to `string | Buffer`, both naming the same sha256, and the declaration now admits what the callers actually do. The session store's `alive` started on the full record and narrowed to the two members the time policy

reads, because the js test feeds it a partial and the partial is the honest input. A seam that only compiles against the fixture you imagined is a seam the suite has not met, and the port met every seam the js suite already owned.

The doubles were the quietest win. A hand-rolled response that forgot `write` or `on` compiled fine in js and failed the moment a layer touched the member. Under the `Res` seam the double declares itself an implementation and the compiler lists the missing members at authoring time. And the ambient surface pays the same rent in reverse: a test that reached for an api nobody declared, an unprobed `mock.module`, a convenience off `process` nobody measured, fails to compile until someone writes the declaration down and owns it.

```
export function recover({ log = stdoutJsonSink }: RecoverDeps = {}): Layer {
  return (next) => async (ctx) => {
    try {
      await next(ctx);
    } catch (err) {
      if (!(err instanceof ApiError)) {
        log({
          level: "error",
          msg: "exception recovered",
          request_id: ctx.id,
          error: err instanceof Error ? err.stack ?? String(err) : String(err),
        });
      }
      writeError(ctx.res, ctx.id, err);
```

listing 273: the unknown catch narrowed by instance, the stack read only where it exists (source: javascript/api/src-ts/middleware/adapter.ts, lines 15-28)

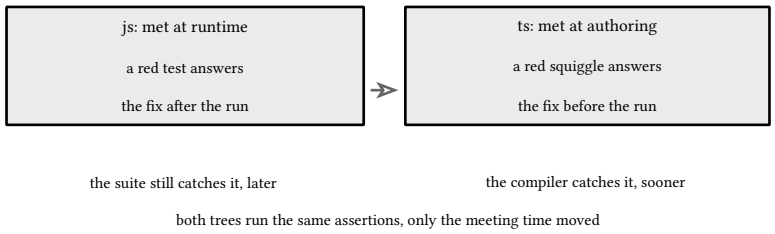


figure 221: the same mistakes, met at two different times

37.3. what the vectors still own

The types never fork the contract. The 16 frozen files stay the spine of both suites, sha256 pinned, and the pin test itself came over unchanged: the mirror reads the same `contract/testdata` directory, hashes the same bytes, and fails on the same drift. Wire member order, status mapping, the timing flatten's identical 401 bytes, the etag over exact body bytes, the replay snapshot, the metrics exposition, all of it is behavior, and behavior belongs to the vectors and the suite, not to the annotations.

The vector-driven chapters read the same in both trees for that reason. Register still answers vector 01 byte for byte, the frozen Set-Cookie line still lands with its attributes in order, the login 429 still replays its exact request id, and the report pair still walks its 202, done, upgraded 200 chain. No type participated in any of those answers, and none could: a type can promise that a writer exists, only the frozen bytes can promise what it writes. The port kept every runtime check for exactly this reason, the strict decode ladders, the uuid test, the presence rules, so that the ts handler and the js handler run the same code path with the same branches taken.

The wire claim is machine checked too, over the real socket. The ship chapter's image carries both trees, and a compose override points the entrypoint at the typed entry, so `make verify-jsapi-ts-docker` boots the same image as a typed service on host ports 19180 and 19190 and replays the same six wire tests against it, the same probes, the same register and login walk, the same etag duel and report pair and metrics exposition. Node strips the `.ts`

sources inside the container exactly as it does in the suite, no compiler rode into the image, and a green lane is the same contract served through the typed composition.

The js spine is untouched by ruling, and the ruling is enforced by the chain rather than by promise. Every commit of this wave ran the js suite first, so an edit that reached into `src/` or `test/` would have turned the first leg red before anything else ran, and no commit in the ledger carries such an edit. The two trees stay in lockstep the same way they got there, a change lands in the spine, the js suite proves it, the mirror follows it, and the ts suite plus the compiler prove the mirror. A change that lands only in the typed tree is a fork, and the shared vectors are the tripwire that fires on it.

That division is the wave's conclusion. The typed layer bought guarantees about the seams, the wiring inside one process, and paid for them with declarations the js spine never needed to write. The vectors kept everything the process says to the outside world, and they gate both languages alike, so the ts tree can never drift from the contract without failing a test the js tree already ran. Two notations, one service, 636 tests saying so on every commit.

What a reader carries out of the pair of chapters is the working method rather than a new service. Start from the spine that already runs, the js sources or the plain sources of any lane, and let the typed tree grow beside it one family at a time with both suites green between steps. Name the seams first, the context, the writer, the port, the outcome unions, because those are the places where the compiler can hold an agreement the prose otherwise enforces. Keep the runtime checks that own the boundary, the decode ladders and the shape tests, and let the frozen files arbitrate everything the process says outward. When the method is followed the second notation costs declarations and buys certainty, and the day it stops being followed the compiler says so first.

```
test("the 16 golden vectors are present and sha256 pinned", () => {
  const dir = path.join(here, "..", "contract", "testdata");
  const names = readdirSync(dir).sort();
  assert.deepEqual(names, Object.keys(pins).sort());
  for (const [name, want] of Object.entries(pins)) {
    const bytes = readFileSync(path.join(dir, name));
    const got = createHash("sha256").update(bytes).digest("hex");
    assert.equal(got, want, `${name} drifted`);
  }
});
```

listing 274: the pin test, mirrored unchanged: the same 16 hashes gate both trees (source: javascript/api/test-ts/contract.test.ts, lines 48-58)

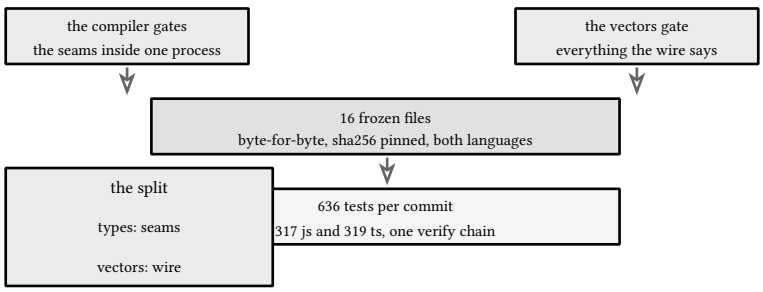


figure 222: what gates what

sources: nodejs.org/api/test.md for the runner surface the mirrors ride and nodejs.org/api/typescript.html for the stripping mode, both accessed 2026-09-27, typescriptlang.org for the 7.0.2 compiler's strict behavior on unknown catches and optional members, accessed 2026-09-27. The mechanism facts are the committed probe on node 26.3.0, `process.features.typescript` with the value `strip`, 2026-09-27. Verified by the live chain: the js suite at 317, the ts suite at 319, and `tsc --noEmit` clean in `npm run verify`, plus the six wire tests green over the typed composition under `make verify-jsapi-ts-docker`.

38. capstone: the query engine, microfrontend

The book's claim, stated once in chapter 1 and repeated since, is that javascript runs and typescript describes. The capstone is that claim as one program: a document query engine split into a plain layer under `capstone/core`, three `.mjs` files with no compiler anywhere in their path, and a typed layer under `capstone/typed` that adds the checker's guarantees on top without re-implementing any behavior. The 4.0 edition adds a third face to the same engine, `capstone/web`, a microfrontend: a hand written html shell, an import map, and two independently built islands, react and svelte, both running against one shared engine instance at runtime. Sixty-three tests drive it red to green, 28 against the plain layer, 14 against the typed one, and 21 against the web, and `npm run verify` runs the plain suite first, the web builds and suite second, the compiler last, so a regression in any face fails the same gate.

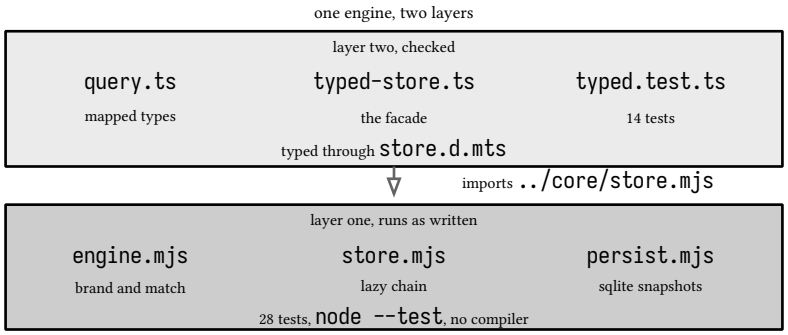


figure 223: the two layers, who runs and who checks

38.1. layer one: conditions without a compiler

A condition is a small object, a field, an operator, and a value. In the plain layer the only way to make one is `build`, which validates the shape it can check without a document, non-empty string field, known operator, an array for `in`, a number for `gt` and `lt`, then freezes the object with a symbol brand. `matches` re-checks the brand first, then re-checks the field and the domain against every document it sees:

```
const BRAND = Symbol("jsbook.condition");

const OPS = new Set(["eq", "in", "gt", "lt", "has"]);

export function build(field, op, value) {
  if (typeof field !== "string" || field === "") {
    throw new TypeError("condition field must be a non-empty string");
  }
  if (!OPS.has(op)) {
    throw new TypeError(`unknown operator ${String(op)}`);
  }
  if (op === "in" && !Array.isArray(value)) {
    throw new TypeError("in needs an array value");
  }
  if ((op === "gt" || op === "lt") && typeof value !== "number") {
    throw new TypeError(`${op} needs a number value`);
  }
  return Object.freeze({ field, op, value, [BRAND]: true });
}

export function matches(doc, cond) {
  if (cond === null || typeof cond !== "object" || cond[BRAND] !== true) {
    throw new TypeError("forged condition: only build() output runs");
  }
  const { field, op, value } = cond;
  if (!(field in doc)) {
    throw new Error(`unknown field ${field}`);
  }
  const current = doc[field];
  switch (op) {
    case "eq":
      return current === value;
    case "in":
      return value.includes(current);
    case "gt":
      return numberOrThrow(current, op, field) > value;
    case "lt":
      return numberOrThrow(current, op, field) < value;
    case "has":
      if (!Array.isArray(current)) {
        throw cannotApply(op, field);
      }
      return current.includes(value);
    default:
      throw cannotApply(op, field);
  }
}
```

listing 275: the builder and the evaluator, one brand between them (source: javascript/capstone/core/engine.mjs, lines 7-53)

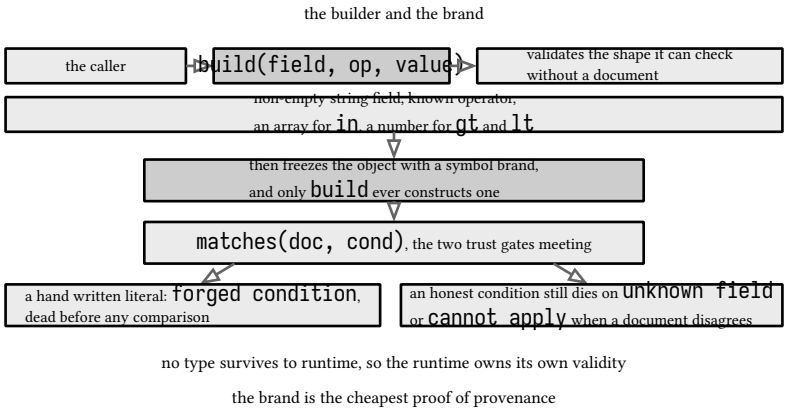


figure 224: layer one, conditions without a compiler

Nothing else in the layer ever constructs a condition, so a hand-written object literal with the same members is rejected with `forged condition` before any comparison runs. This is chapter 11's erasure rule turned into a design decision: since no type survives to runtime anyway, the runtime owns its own validity, and the brand is the cheapest proof of provenance. The two trust gates meet in `matches`: an honest condition from the builder still dies on `unknown field` or `cannot apply` when a document disagrees with it, which is what the evaluator owes a store that accepts documents of any shape.

The comparator at the bottom of the file is deliberately dull. Numbers subtract, everything else compares through `String`, and that single rule covers `Temporal.Instant`, whose string form is a fixed width iso 8601 utc text that orders lexicographically the same way it orders in time.

38.2. the lazy chain

The store is a private array with one entry point. `query()` makes one shallow copy and hands back a `Query` over it, and every stage of a chain wraps the previous iterator without pulling from it. Work starts when a terminal operation iterates, `toArray`, `count`, or a `for..of`, and a fresh `Symbol.iterator` call replays the whole chain, the protocol rule chapter 5 pinned when it built iterators by hand:

```
export class Query {
  // mk builds a fresh iterator per pass, so stages stay restartable
  #mk;

  constructor(make) {
    this.#mk = make;
  }

  #stage(wrap) {
    const mk = this.#mk;
    return new Query(() => wrap(mk()));
  }

  where(cond) {
    return this.#stage((it) => it.filter((doc) => matches(doc, cond)));
  }

  map(fn) {
    return this.#stage((it) => it.map(fn));
  }

  take(n) {
    return this.#stage((it) => it.take(n));
  }

  drop(n) {
    return this.#stage((it) => it.drop(n));
  }

  // sorting cannot stay pull based one item at a time: the stage
  // drains its source on first pull, sorts, and replays from the copy
  sortBy(key, dir = "asc") {
    return this.#stage((it) =>
      [...it]
        .toSorted((a, b) => (dir === "asc" ? compare(a, b, key) : compare(b, a, key)))
        .values(),
    );
  }

  // interning through Map.getOrInsert: the first document per key is
  // kept and later ones are dropped, decided lazily as they arrive
  dedupe(keyOf) {
    return this.#stage((it) => {
      const seen = new Map();
      return it.filter((doc) => seen.getOrInsert(keyOf(doc), doc) === doc);
    });
  }
```

listing 276: the stage pattern, each helper wraps the previous iterator (source: javascript/capstone/core/store.mjs, lines 9-55)

```
  select(keys) {
    return this.map((doc) => {
      const row = {};
      for (const key of keys) row[key] = doc[key];
      return row;
    });
  }

  // elements offset through offset+count: take wide enough first,
  // then drop the offset, so both bounds are index stable
  limit(count, offset = 0) {
    return this.take(count + offset).drop(offset);
  }

  *[Symbol.iterator]() {
    yield* this.#mk();
  }

  toArray() {
    return [...this];
  }

  count() {
    let n = 0;
    for (const _ of this) n += 1;
    return n;
  }
}

export class Store {
  #docs = [];

  insert(doc) {
    this.#docs.push(doc);
    return this;
  }

  get size() {
    return this.#docs.length;
  }

  // one shallow copy at entry: a chain never sees inserts made after
  // query() was called, and replays see the same documents
  query() {
    const docs = [...this.#docs];
    return new Query(() => docs.values());
  }
}
```

listing 277: terminals, pagination, and the store itself (source: javascript/capstone/core/store.mjs, lines 57-104)

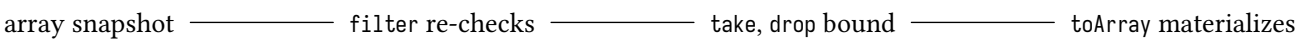


figure 225: one pull through the chain

The stages are the iterator helpers chapter 5 introduced, and they run on iterators, chapter 5's own rule, so `where` is `filter`, `select` is `map`, and pagination composes `take` with `drop`. Two details earn their comments. `sortBy` cannot stay pull based one item at a time, so its stage drains the source on first pull, sorts the copy with `toSorted`, and replays from it. And `limit` takes `count + offset` before dropping `offset`, because `take(2).drop(1)` over a sorted list would

hand back one element when the page asked for two. `dedupe` is chapter 5's `Map.getOrInsert` doing lazy interning: the first document per field value is kept, later ones are dropped as they arrive, and no set of distinct values is ever materialized.

The laziness is load bearing, not decorative. A chain over an unbounded generator runs in bounded memory because `take` stops the pull, and the test that proves it counts every yield:

```
it("pulls lazily from the source", () => {
  let pulled = 0;
  function* source() {
    for (let n = 0; ; n += 1) {
      pulled += 1;
      yield { id: `g${n}`, pages: n };
    }
  }
  const firstTwo = new Query(source)
    .where(build("pages", "gt", 0))
    .map((doc) => doc.id)
    .take(2)
    .toArray();
  assert.deepEqual(firstTwo, ["g1", "g2"]);
  assert.equal(pulled, 3); // one past the taken pair, then take stops
});
```

listing 278: an infinite source, three pulls, two results (source: javascript/capstone/core/store.test.mjs, lines 81-96)

38.3. snapshots that outlive the process

The document set saves to a real sqlite file through `node:sqlite`, which node 26 ships built in, so the layer adds no dependency. The section is short because the code carries three measured decisions. First, `DatabaseSync` on node 26.3.0 implements `Symbol.dispose`, probed on this machine, so `#open` uses a `using` declaration and the handle closes at scope exit even on throw, and no method needs a `close` call. Second, timestamps follow chapter 8's boundary rule, `Temporal.Instant` in memory and iso text in the rows, `.toString()` on the way in and `Temporal.Instant.from` on the way back. Third, the checksum:

```
// the checksum payload carries each instant as its exact
// epochNanoseconds digits through JSON.rawJSON, emit exact only:
// stringify keeps every digit, JSON.parse of the same text returns
// the double, so nothing ever parses the payload back, it is hashed
// and compared as text.
function payload(docs) {
  return JSON.stringify(
    docs.map((doc) => ({
      id: doc.id,
      title: doc.title,
      pages: doc.pages,
      added: JSON.rawJSON(String(doc.added.epochNanoseconds)),
      tags: doc.tags,
    })),
  );
}

export function checksum(docs) {
  return new TextEncoder().encode(payload(docs)).toBase64();
}

export class Snapshot {
  constructor(path) {
    this.path = path;
  }

  #open(run) {
    using db = new DatabaseSync(this.path);
    db.exec(SCHEMA);
    return run(db);
  }
```

listing 279: the checksum payload and the one place a database opens (source: javascript/capstone/core/persist.mjs, lines 29-59)

```
  save(docs) {
    return this.#open((db) => {
      const putDoc = db.prepare(
        "INSERT INTO docs (id, title, pages, added, pos) VALUES (?, ?, ?, ?, ?)",
      );
      const putTag = db.prepare(
        "INSERT INTO doc_tags (doc_id, ord, tag) VALUES (?, ?, ?)",
      );
      const putChecksum = db.prepare(
        "INSERT OR REPLACE INTO meta (key, value) VALUES ('checksum', ?)",
      );
      db.exec("BEGIN");
      try {
        db.prepare("DELETE FROM doc_tags").run();
        db.prepare("DELETE FROM docs").run();
        docs.forEach((doc, pos) => {
          putDoc.run(doc.id, doc.title, doc.pages, doc.added.toString(), pos);
          doc.tags.forEach((tag, ord) => putTag.run(doc.id, ord, tag));
        });
        putChecksum.run(checksum(docs));
        db.exec("COMMIT");
      } catch (err) {
        db.exec("ROLLBACK");
        throw err;
      }
    });
  }

  load() {
    return this.#open((db) => {
      const stored = db.prepare("SELECT value FROM meta WHERE key = 'checksum'").get();
      const docs = db
        .prepare("SELECT id, title, pages, added FROM docs ORDER BY pos")
        .all()
        .map(({ id, title, pages, added }) => ({
          id,
          title,
          pages,
          added: Temporal.Instant.from(added),
          tags: [],
        }));
      const byId = new Map(docs.map((doc) => [doc.id, doc]));
      for (const { doc_id, tag } of db
        .prepare("SELECT doc_id, tag FROM doc_tags ORDER BY doc_id, ord")
        .all()) {
        byId.get(doc_id)?.tags.push(tag);
      }
      if (stored && checksum(docs) !== stored.value) {
        throw new Error("checksum mismatch: the snapshot was modified");
      }
      return docs;
    });
  }
```

listing 280: transactional save, checksum verified load (source: javascript/capstone/core/persist.mjs, lines 61-113)

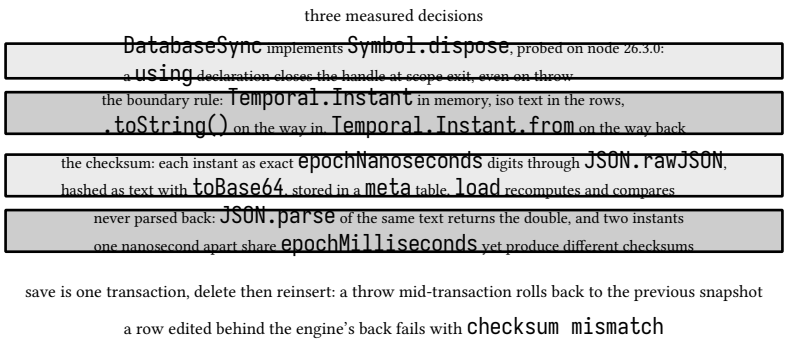


figure 226: snapshots that outlive the process, the persistence decisions

The payload serializes each instant as its exact `epochNanoseconds` digits through `JSON.rawJSON`. That is an emit exact use and only that: `stringify` keeps every digit, but `JSON.parse` of the same text returns the double, measured in the suite, so the payload is hashed and compared as text and never parsed back. The payoff is in the test: two instants one nanosecond apart share `epochMilliseconds` yet produce different checksums, because the digits differ. The hash itself is `Uint8Array.prototype.toBase64` over the encoded payload, stored in a `meta` table, and `load` recomputes and compares before returning, so a row edited behind the engine's back fails with `checksum mismatch`. Saving is one transaction, delete then reinsert, and a throw mid-transaction rolls back to the previous snapshot, which the rollback test exercises by deleting a document's instant.

The engine underneath is node's own build. `node:sqlite` embeds its sqlite inside the node binary. The corpus pins 3.53.4 for the c, go, and lua toolchains, and node 26.10.0 here reports exactly 3.53.4, caught up to the pin; node 26.3.0 reported 3.53.1, and the suite asserts an exact string, never a floor, since in this one book the version is a property of the runtime. Test files are created under the system temp directory through `mkdtempSync`, never inside the repo, and cleaned in an `after` hook. The snapshots stay on `node:sqlite` because a save is one transaction that rolls back whole, and the engine choice it would be weighed against is measured in book 13, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine).

38.4. layer two: the typed facade

The typed layer adds what the plain layer refuses to have: a compile time account of which conditions make sense. The mapped type is unchanged from the single-package capstone, because it was already the whole policy:

```ts
// capstone: the query types. one mapped type distributes conditions
// over the document's keys, binding each field name to the operators
// and values its member type admits.
export type ElementOf<V> = V extends readonly (infer E)[] ? E : never;

export type OpsFor<V> =
  V extends string
    ? "eq" | "in"
    : V extends number
      ? "eq" | "in" | "gt" | "lt"
      : V extends readonly string[]
        ? "has"
        : never;

type ConditionFor<T, K extends keyof T> = {
  field: K;
  op: OpsFor<T[K]>;
  value: T[K] | T[K][] | ElementOf<T[K]>;
};

export type Condition<T> = {
  [K in keyof T]-?: ConditionFor<T, K>;
}[keyof T];
```

listing 281: conditions distributed over the document keys (source: javascript/capstone/typed/query.ts, lines 1-24)

`OpsFor` admits `eq` and `in` for strings, adds `gt` and `lt` for numbers, gives `has` to string arrays, and maps everything else to `never`. That last branch now does visible work: `Book.added` is a `Temporal.Instant`, so no condition can mention it. It stays sortable through the comparator's string branch and unqueryable in conditions, which is the honest outcome, since no operator in the set applies to an instant. The proof that the binding holds is mechanical, from a probe compiled outside the suite:

```ts
store.query().where({ field: "title", op: "gt", value: 5 });
// error TS2345: Type '"gt"' is not assignable to
//                 type '"eq" | "in"'.
```

fragment, illustrative, not executed

The same expression with `op: "eq"` and a string value, or with `pages` and `gt`, compiles clean. Chapter 19 drew the declaration boundary in miniature with a five line `store.d.mts` beside a `store.mjs`. The capstone scales that exact shape up, one hand written `core/store.d.mts` describing the whole plain layer, the `.d.mts` spelling because nodenext maps an `.mjs` to a `.d.mts`:

```ts
// hand written beside core/store.mjs: the checker's whole knowledge
// of the plain layer. the .d.mts spelling is the nodenext correct
// declaration file for an .mjs, the same boundary chapter 19 drew in
// miniature with tssamples/ch17/store.d.mts. nothing here emits and
// nothing here runs, and conditions stay opaque: where() accepts
// only what build() returns, and layer two never forges one by hand.
export declare function build(field: string, op: string, value: unknown): unknown;

export declare class Store<T> {
  insert(doc: T): Store<T>;
  query(): Query<T>;
}

export declare class Query<T> {
  where(cond: unknown): Query<T>;
  map<U>(fn: (doc: T) => U): Query<U>;
  take(n: number): Query<T>;
  drop(n: number): Query<T>;
  sortBy(key: string, dir?: "asc" | "desc"): Query<T>;
  dedupe(keyOf: (doc: T) => unknown): Query<T>;
  select(keys: readonly string[]): Query<Record<string, unknown>>;
  limit(count: number, offset?: number): Query<T>;
  [Symbol.iterator](): Iterator<T>;
  toArray(): T[];
  count(): number;
}
```

listing 282: the checker's whole knowledge of the plain layer, hand written (source: javascript/capstone/core/store.d.mts, lines 1-27)

Nothing in the file emits and nothing in it runs. Conditions stay opaque, `where` takes `unknown`, because the declaration's job is to type the surface, and the brand has no type side worth faking. The facade on top translates between the two accounts:

```ts
export class BookQuery {
  #core: CoreQuery<Book>;

  constructor(core: CoreQuery<Book>) {
    this.#core = core;
  }

  where(cond: Condition<Book>): this {
    this.#core = this.#core.where(build(cond.field, cond.op, cond.value));
    return this;
  }

  sortBy<K extends keyof Book>(key: K, dir: "asc" | "desc" = "asc"): this {
    this.#core = this.#core.sortBy(key, dir);
    return this;
  }

  dedupe(keyOf: (book: Book) => unknown): this {
    this.#core = this.#core.dedupe(keyOf);
    return this;
  }

  limit(count: number, offset = 0): this {
    this.#core = this.#core.limit(count, offset);
    return this;
  }

  // the cast is honest: the core hands back plain projected rows and
  // the key list came from keyof Book, so the shape is exactly Pick
  select<K extends keyof Book>(keys: readonly K[]): Pick<Book, K>[] {
    return this.#core.select(keys).toArray() as unknown as Pick<Book, K>[];
  }

  *[Symbol.iterator](): Iterator<Book> {
    yield* this.#core;
  }

  toArray(): Book[] {
    return this.#core.toArray();
  }

  count(): number {
    return this.#core.count();
  }
}
```

listing 283: the typed facade, delegation with a type attached (source: javascript/capstone/typed/typed-store.ts, lines 29-73)

typed literal ——— checker admits it ——— facade rebuilds it ——— brand at runtime

figure 227: one condition, two gates

`BookQuery.where` does one thing the plain layer cannot: it takes a `Condition<Book>` and rebuilds it through the core `build`, so every typed condition crosses the brand boundary by construction and no facade method ever trusts a caller's object. A cast over the checker, the test does it on purpose, still dies at the `where()` call with `unknown operator`, because the builder validates shape before the chain stores anything. `select` carries the one honest cast in the layer, justified because the field list came from `keyof Book`, so `Pick<Book, K>` is the shape the plain `map` stage really produces.

38.5. the third face: one engine, two islands

The web layer asks the question the framework chapters set up: two frameworks, one domain, no federation machinery. The answer is the platform's own. A static shell carries one import map that binds the bare specifier `domain` to exactly one module url, each island is built separately by vite with `domain` marked external, and at runtime both bundles resolve `domain` through the map to the same module instance, so the react panel and the svelte panel read one `Store`, seeded once, and their document counts cannot disagree:

```
<script type="importmap">
  {
    "imports": {
      "domain": "/islands/domain.js"
    }
  }
</script>
```

listing 284: the import map, the whole architecture in four lines of json (source: javascript/capstone/web/shell/index.html, lines 21-27)

The machine checks are part of the suite, not prose: a test greps both built bundles for the bare specifier import, another asserts the map binds `domain` exactly once, and a third proves module identity the honest way, the same url imports to the same `library`, a cache busted second specifier makes a second instance, which is precisely why the single binding matters.

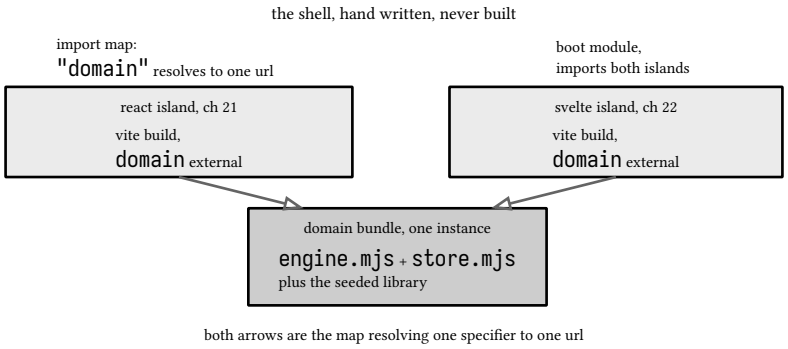


figure 228: one shell, one map, two islands, one engine

38.6. the domain and the two islands

The domain entry is a re-export plus a seed, and the one decision worth teaching sits in its comment: the documents carry `added` as iso text, not `Temporal.Instant`, because this module also runs in browsers, chapter 8's boundary rule applied to a wire. The server converts back to instants at the persist layer's edge, and the comparator's string branch sorts the text correctly everywhere else:

```
export { build, matches, compare } from "../../core/engine.mjs";
export { Query, Store } from "../../core/store.mjs";

import { Store } from "../../core/store.mjs";

export interface DomainDoc {
  id: string;
  title: string;
  pages: number;
  tags: string[];
  added: string;
}

const SEED: DomainDoc[] = [
  { id: "b1", title: "go", pages: 321, tags: ["systems", "compiled"], added: "2026-01-05T09:00:00Z" },
  { id: "b2", title: "lua", pages: 258, tags: ["scripting", "embedded"], added: "2026-02-11T09:00:00Z" },
  { id: "b3", title: "typescript", pages: 412, tags: ["systems", "web"], added: "2026-03-02T09:00:00Z" },
  { id: "b4", title: "csharp", pages: 377, tags: ["compiled", "web"], added: "2026-04-19T09:00:00Z" },
  { id: "b5", title: "javascript", pages: 389, tags: ["web", "runtime"], added: "2026-05-27T09:00:00Z" },
];

// the singleton. same module instance, same store, every island that
// imports domain reads and writes this one object
export const library: Store = new Store();
SEED.forEach((doc) => library.insert(doc));

export function allDocs(): DomainDoc[] {
  return library.query().toArray();
}
```

listing 285: the domain bundle's whole source, re-exports, seed, singleton (source: javascript/capstone/web/domain/domain.ts, lines 8-36)

The islands are deliberately boring, each framework doing what its chapter taught with the same shared documents. React mounts through the chapter 21 wrapper, svelte through the compiler's `svelte:options` custom element, and both keep `domain` external so the map stays in charge:

```
// capstone react island entry, browser only. the bare specifier
// domain is marked external in the vite config, so the built bundle
// ships `from"domain"` and the import map hands it the one shared
// instance at runtime.
import { createElement } from "react";
import { createRoot } from "react-dom/client";
import { defineIsland, type IslandRoot } from "./wrap.js";
import { QueryPanel } from "./QueryPanel.js";

defineIsland(
  "react-query-panel",
  () => createElement(QueryPanel),
  { base: HTMLElement, registry: customElements },
  (host: HTMLElement): IslandRoot => createRoot(host),
);
```

listing 286: the react island entry, injectable wrapper, external domain (source: javascript/capstone/web/react-island/island.tsx, lines 1-15)

```
// capstone svelte island entry, browser only. the compiler emitted
// a custom element class, the entry only registers it, and the bare
// specifier domain stays external for the import map to resolve.
import QueryPanel from "./QueryPanel.svelte";

customElements.define(
  "svelte-query-panel",
  QueryPanel as unknown as CustomElementConstructor,
);
```

listing 287: the svelte island entry, one registration line, 9 lines in all (source: javascript/capstone/web/svelte-island/main.ts, lines 1-9)

Both islands also build an ssr barrel with `domain` aliased to its source entry, because node has no import map, and the render smokes assert both panels produce the same five titles out of their own built output.

38.7. the server, plain node:http

One file serves everything: the shell and the three built bundles as static routes with mime types, and two apis over the same core modules the islands import. Node 26's own type stripping runs the domain's `.ts` source directly here, the erasable subset chapter 19 pinned, so the server and the browser read one seed from one file:

```
const STATIC = new Map([
  ["/", { file: here("./shell/index.html"), type: "text/html; charset=utf-8" }],
  ["/boot.mjs", { file: here("./shell/boot.mjs"), type: "text/javascript" }],
  ["/islands/domain.js", { file: here("./domain/dist/domain.js"), type: "text/javascript" }],
  ["/islands/react-island.js", { file: here("./react-island/dist/island.js"), type: "text/javascript" }],
  ["/islands/svelte-island.js", { file: here("./svelte-island/dist/island.js"), type: "text/javascript" }],
]);

const FIELDS = new Set(["title", "pages", "tags", "added"]);
const send = (res, status, text, type = "application/json; charset=utf-8") => {
  res.writeHead(status, {
    "content-type": type,
    "content-length": Buffer.byteLength(text),
  });
  res.end(text);
};
const json = (res, status, body) => send(res, status, JSON.stringify(body));

// the persist layer's memory format is Temporal.Instant, the wire
// format is iso text: convert on the way into a snapshot
function toPersistDocs(docs) {
```

listing 288: the static routes and the json seam (source: javascript/capstone/web/server.mjs, lines 20-40)

```
function toPersistDocs(docs) {
  return docs.map((doc) => ({ ...doc, added: Temporal.Instant.from(doc.added) }));
}

async function search(url, res) {
  const field = url.searchParams.get("field");
  const op = url.searchParams.get("op");
  const raw = url.searchParams.get("value");
  if (field === null || op === null || raw === null) {
    return json(res, 400, { error: "search wants field, op, value" });
  }
  if (!FIELDS.has(field)) {
    return json(res, 400, { error: `unknown field ${field}` });
  }
  let value = raw;
  if (op === "gt" || op === "lt") value = Number(raw);
  if (op === "in") value = raw.split(",").map((part) => part.trim());
  try {
    const rows = library.query().where(build(field, op, value)).toArray();
    return json(res, 200, { count: rows.length, results: rows });
  } catch (err) {
    return json(res, 400, { error: err.message });
  }
}

async function snapshot(res, path) {
  const docs = toPersistDocs(allDocs());
  const snap = new Snapshot(path);
  snap.save(docs);
  const loaded = snap.load();
  json(res, 200, {
    count: loaded.length,
    checksum: checksum(loaded),
    first: loaded[0].title,
  });
}
```

listing 289: the persist boundary, the search api through the builder, the snapshot round trip (source: javascript/capstone/web/server.mjs, lines 40-75)

The search endpoint reaches the engine exactly the way chapter 21 reached react internals, through the public surface: it validates the field, coerces the value, and calls `build`, so a bad operator answers 400 with the builder's own `unknown operator` message rather than a stack trace. The snapshot endpoint is chapter 8's boundary rule end to end, iso text on the wire, `Temporal.Instant.from` at the persist layer's edge, one transactional save, and a checksum-verified load back out, through the same `node:sqlite` the core suite pins at 3.53.4.

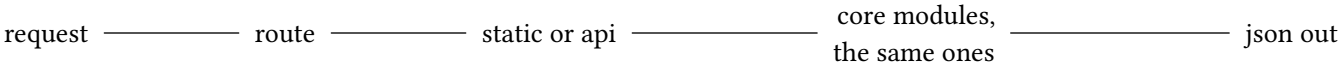


figure 229: one request through the server

38.8. profiled, and why

path	measured, this machine	the why
parse	0.04 us per condition	validation and one frozen literal, cheap enough to ignore forever
plan	stages 0.008 us each, entry copy 0.3 to 0.4 ns per doc	the plan is closures, the entry a snapshot, the price of replay safety
scan	29.4 ns per document per round	69 of 93 samples in anonymous iterator closures, the collector through ch 9
the reads	limited queries: p50 0.059, p99 0.23, p99.9 0.46 ms	corpus-shaped until `take` closes, the sqlite step the indexed alternative
the loop tail mirrors one round of synchronous work: p50 5.779 ms, p99 7.754 ms over 7 samples		

200,000 documents, 8 rounds in 48.31 ms, all measured 2026-10-04

best save of 5,000 documents 32.48 ms, best load 12.18 ms, engine 3.53.4

figure 230: profiled, and why, the four paths at 200,000 documents

The engine's own runtime, profiled by its own book: chapter 10's tools turned on this engine, and the opt-in `capstone/bench/profile.mjs` walks the four paths of a query at 200,000 documents, all measured on this machine, 2026-09-13. Parse, building the condition, is validation and one frozen literal: 200,000 `build` calls in 8.01 ms, 0.04 microseconds per condition, cheap enough to ignore forever. Plan splits in two, and the split is the finding: staging is free, 2,000-entry runs measured the stages at 0.008 microseconds each over 200,000 three-stage chains with nothing pulled, but entering `query()` copies the document array, 0.3 to 0.4 nanoseconds per document, about 70 microseconds for this corpus on every chain that starts at the store. The plan is cheap because it is closures. The entry is not, because it is a snapshot, and the snapshot is the price of replay safety, the rule chapter 5 pinned.

```
// scan: the pull, matches() over every document, profiled in-process
// with a setImmediate yield between rounds so the loop histogram lives
const session = new inspector.Session();
session.connect();
const post = (method, params) =>
  new Promise((resolve, reject) =>
    session.post(method, params, (err, res) => (err ? reject(err) : resolve(res))));
await post("Profiler.enable");
await post("Profiler.setSamplingInterval", { interval: 100 });
await post("Profiler.start");
const histogram = monitorEventLoopDelay({ resolution: 1 });
histogram.enable();
const ROUNDS = 8;
let total = 0;
const scanStart = performance.now();
for (let r = 0; r < ROUNDS; r++) {
  await new Promise((resolve) => setImmediate(resolve));
  for (const doc of store.query().where(cond)) total += doc.pages;
}
const scanMs = performance.now() - scanStart;
histogram.disable();
const { profile } = await post("Profiler.stop");
session.disconnect();
const hits = new Map();
for (const node of profile.nodes) {
  const name = node.callFrame.functionName || "(anon)";
  hits.set(name, (hits.get(name) ?? 0) + (node.hitCount ?? 0));
}
const top = [...hits.entries()].sort((a, b) => b[1] - a[1]).slice(0, 5);
console.log(`scan: ${ROUNDS} rounds over ${DOCS} docs in ${round(scanMs)} ms, ${round((scanMs * 1e6) /
(DOCS * ROUNDS))} ns per doc per round, total ${total}`);
console.log(`scan profile: ${profile.nodes.length} nodes, ${top.map(([n, h]) => `${n}=${h}`).join(", ")}`);
console.log(`scan loop tail: p50=${round(histogram.percentile(50) / 1e6, 3)} ms,
p99=${round(histogram.percentile(99) / 1e6, 3)} ms, mean=${round(histogram.mean / 1e6, 3)} ms over
${histogram.count} samples`);
```

listing 290: the scan path profiled in-process, the histogram live between rounds (source: javascript/capstone/bench/profile.mjs, lines 51-83)

Scan is where the time goes: 8 rounds over 200,000 documents in 47.04 ms, 29.4 nanoseconds per document per round, 39.63 to 47.04 ms across three runs. The in-process profile of that loop at a 100 microsecond sampling interval: 20 nodes, top frames `(anon)` with 69 of 93 hits, `matches` 15, `(garbage collector)` 4, `query` 3, `(program)` 2. The why reads through both new chapters. In a lazy chain the named functions are a minority of samples because the pull itself lives inside the iterator helpers' anonymous closures, so the hottest code in this engine has no name a sampler can show, and the profile is honest about that rather than renaming closures for the tool's sake. Chapter 9's lens reads the collector frame: four samples of garbage collection against 1.6 million condition checks is the scavenger amortizing the iterator state each pull creates and drops. And the event-loop histogram around the same loop, enabled between rounds, reads p50 5.706 ms and p99 7.143 ms over 7 samples, because the loop cannot yield inside a round: the tail mirrors one round of synchronous work, chapter 10's rule that delay is born where a long callback holds the thread.

The server-shaped read, 2,000 limited queries, lands at p50 0.073 ms, p99 0.268 ms, p99.9 0.495 ms. A limited query is still a full scan until `take` closes the pull, so its latency is corpus-shaped rather than page-shaped, and the honest fix at scale is a store with an index, which is what the sqlite step is for: the best save of 5,000 documents measured 32.48 ms and the best load 12.18 ms against engine 3.53.4 on node 26.10.0, re-measured 2026-10-04, persistence that costs about one scan of a fortieth of the corpus.

```
describe("capstone cpu profile", () => {
  it("a child --cpu-prof run names the scan path's frames", () => {
    const dir = mkdtempSync(join(tmpdir(), "capstone-cpuprof-"));
    const run = spawnSync(process.execPath, [
      "--cpu-prof", `--cpu-prof-dir=${dir}`, "--cpu-prof-name=p.cpuprofile",
      child, "200000",
    ], { encoding: "utf8" });
    assert.equal(run.status, 0, run.stderr);
    const profile = JSON.parse(readFileSync(join(dir, "p.cpuprofile"), "utf8"));
    const frames = new Map();
    let scanPathHits = 0;
    let totalHits = 0;
    for (const node of profile.nodes) {
      const name = node.callFrame.functionName || "(anon)";
      const url = node.callFrame.url ?? "";
      const hits = node.hitCount ?? 0;
      totalHits += hits;
      frames.set(name, (frames.get(name) ?? 0) + hits);
      // the scan work spans scan-child.mjs and the store iterator it drives;
      // where the samples land shifts with tier-up and load, so the assert
      // covers the whole scan path, not one function's frame
      if (name === "scanCorpus" || url.endsWith("scan-child.mjs") || url.endsWith("core/store.mjs")) {
        scanPathHits += hits;
      }
    }
    assert.ok(profile.nodes.length > 5, `nodes ${profile.nodes.length}`);
    assert.ok(scanPathHits ≥ 1, `scan path hits ${scanPathHits}, frames ${[...frames.entries()].map(([n,
h]) => `${n}:${h}`).join(", ")}`);
    assert.ok(totalHits > 0);
    rmSync(dir, { recursive: true, force: true });
  });
});
```

listing 291: the in-suite smokes, a parsed child profile and an ordered histogram (source: javascript/capstone/core/profile.test.mjs, lines 14-45)

The suite carries two deterministic smokes from this section, a child `--cpu-prof` run of the scan child parsed for the engine's own frames, and the histogram's ordered positive percentiles, both under half a second; the numbers above stay in the opt-in bench where a measurement belongs.

38.9. the suite and the gate

The counts moved from 18 tests in one package to 42 in two layers and 63 with the web face: 7 in `engine.test.mjs`, 11 in `store.test.mjs`, 8 in `persist.test.mjs`, 2 in `profile.test.mjs`, 28 plain, 14 in `typed.test.ts`, and 21 across five web suites, 8 driving the server over real http on an ephemeral port, 4 reading the shell as a contract, 4 rendering both islands through their ssr barrels, 3 on the wrapper's lifecycle contract, and 2 on the shared instance itself. The typed suite covers the same surface as the core suite plus the parts only the facade has, the `Pick` projection that typechecks without a cast, iteration through the facade's own `Symbol.iterator`, and the two-layer trust story:

```
  it("distributes conditions over the keys", () => {
    // compiling this literal array is the proof: each op binds to the
    // field its member type admits, and running them exercises the
    // same builder the plain layer uses
    const conds: Condition<Book>[] = [
      { field: "title", op: "eq", value: "typescript" },
      { field: "pages", op: "in", value: [321, 412] },
      { field: "pages", op: "gt", value: 300 },
      { field: "tags", op: "has", value: "web" },
    ];
    let q = loaded().query();
    for (const cond of conds) q = q.where(cond);
    assert.deepEqual(q.toArray().map((b) => b.id), ["b3"]);
  });

  it("rejects forged conditions at runtime", () => {
    // the cast climbs over the checker on purpose. the facade rebuilds
    // every condition through the core builder, so a bad operator dies
    // at the where() call, before any document is touched
    const forged = { field: "title", op: "sounds-like", value: "g?" };
    assert.throws(
      () => loaded().query().where(forged as unknown as Condition<Book>),
      /unknown operator/,
    );
  });
```

listing 292: the distribution proof and the forged condition (source: javascript/capstone/typed/typed.test.ts, lines 103-127)

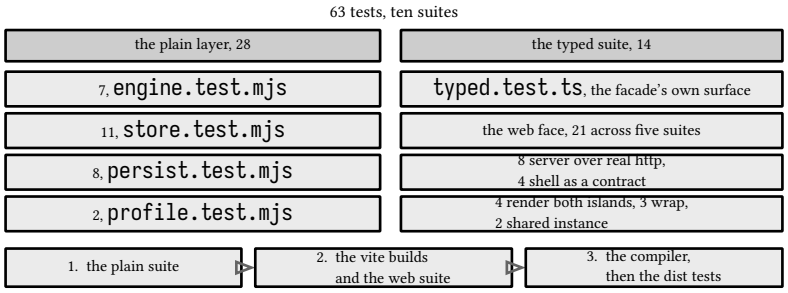


figure 231: the suite and the gate, 63 tests in ten suites

The verify script is the whole gate in one line, plain suite first, the five vite builds and the web suite second, the compiler last:

```
"verify": "... rmSync dist and the web dists ..."
  + " && node --test \"core/**/*.test.mjs\""
  + " && vite build -c web/domain/vite.config.ts"
  + " && ... four more vite builds ..."
  + " && node --test \"web/**/*.test.mjs\""
  + " && tsc -p tsconfig.json"
  + " && node --test \"dist/**/*.test.js\""
```

fragment, illustrative, not executed

The glob form is not cosmetic. `node --test` on node 26 accepts globs and fails on bare directory arguments, and the compiled tests import `../core/store.mjs`, which resolves because `rootDir` is `typed` and the compiled files land in

`dist` beside the untouched `core` directory, no copy step, no path rewriting. The suite passed on three consecutive `npm run verify` runs as on the first, and the project verifies through the repo's `make verify-ts` alongside the samples.

NOTE: what the capstone exercises

Chapter 5 carries the whole chain, the protocol, the helpers, and the replay rule, plus `Map.getOrInsert` for interning and `Uint8Array.prototype.toBase64` for the checksum, chapter 8 carries the edition APIs the persistence leans on, `Temporal.Instant` at the string boundary and `JSON.rawJSON` for emit exact digits, and chapter 19 carries the declaration boundary the typed layer stands on. Chapter 16′s narrowing runs the evaluator's switch, chapter 11′s erasure explains why the brand exists at all, and the closed set of condition shapes is the same discipline book 6, ch 6 (c# 15, f# 11, and .net 11, pattern matching and control flow) enforces with matching. The profiled section is chapters 9 and 10 turned inward, the collector frame in a real capture and the profiler tools the engine runs on itself. The web face is chapters 21 and 22 made structural, both islands over one seam, and chapter 7′s module graph is what the import map resolves. One small program, most of the book.

38.10. the service part

The service part built on the same book after this capstone is the next evidence layer: the 6.0 wave's 13 chapters at 23 through 35 ride `books/javascript/api`, one package mounting twelve families at slots 23 through 34 behind the kernel, and the wave 3b typed vehicle mirrors the js spine into a typed source tree beside it. The lane gates through `make verify-ts` in the repo chain. The audit records the js spine at 317 of 317 green at the 6.0 closeout, 2026-09-27, the typed mirror at 319 ts tests beside the js 317, and the docker lane 6 of 6 over the real image.

sources: nodejs.org node:test, node:sqlite, and iterator helpers documentation, developer.mozilla.org Temporal, Map.getOrInsert, Uint8Array toBase64, and JSON.rawJSON pages, and the typescriptlang handbook mapped types page, accessed 2026-09-13, the whatwg import maps specification for the web face, accessed 2026-09-21. Verified live with tsc 7.0.2, vite 8.3.0, react 19.3.0, and svelte 5.57.1 on node 26.3.0 on windows, 63 tests green through `npm run verify`, 28 plain, 21 web, and 14 typed, clean consecutive runs, plus one compile probe confirming operator rejection at the type level, one runtime probe confirming `DatabaseSync` implements `Symbol.dispose`, and the profiling bench runs quoted in the profiled section, measured 2026-09-13 on this machine.

39. appendices: coverage and sources

The coverage matrices are this book's audit trail. The 4.0 edition keeps the book split at the language's own seams, chapters 1 through 10 the runtime with no build step, heap and profiler included, 11 through 20 the type layer, 21 and 22 the framework layer where vite enters, 23 the capstone that runs every face of the engine, 24 these appendices, and the matrices follow that split: every word the grammar admits mapped to the chapter that teaches it, every library family placed on its edition shelf with this runtime's verdict, and the pins that freeze the whole thing in time. The build enforces honesty from the other side: every chapter listing reads a real file that an npm project in this repo executes, and this chapter counts those projects' suites exactly rather than estimating them.

39.1. keyword coverage

The census was recounted against the live lexical grammar page on 2026-09-13 and the strata held: 35 words reserved outright, 4 reserved in strict mode and module code, 7 reserved for the future, and the 29 typescript contextual words that parse as keywords only inside type position, 75 in all. Every word maps to a chapter: 0 unmapped. The grammar words concentrate in part one, 36 of the 46 plus the contextual `of`, 37 words in all, because they are the runtime's own vocabulary, and the type words all land in part two.

The exceptions carry the interesting facts. Switching lives in chapter 16, the exhaustive switch is the book's whole treatment of `case` and `switch`, and part one contains no switch at all. The `in` guard and the `is` and `asserts` predicates sit there for the same reason. `void` never executes, it is the checker's return contract, taught with the two tops and two bottoms of chapter 13. `enum` is future reserved in javascript and real runtime code in typescript, so it waits for the emit chapter. And four words never run anywhere in this book: `while`, `do`, `with`, and `package` are grammar facts only, part one iterates with `for..of` and generator pulls, `with` is an error in module code, and `package` never appears. `break` and `continue` each execute exactly once, inside the loop tests, chapter 5′s pull count and chapter 2′s table walk.

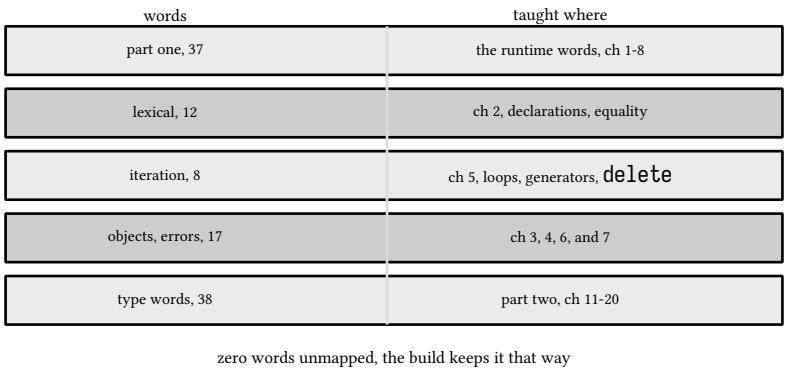


figure 232: the 75 words, where each stratum lands

word	kind	taught in
break	reserved	ch 5
case	reserved	ch 16
catch	reserved	ch 6
class	reserved	ch 4
const	reserved	ch 2
continue	reserved	ch 5
debugger	reserved	ch 2
default	reserved	ch 3
delete	reserved	ch 5
do	reserved	ch 5

word	kind	taught in
else	reserved	ch 2
export	reserved	ch 7
extends	reserved	ch 4
false	reserved	ch 2
finally	reserved	ch 6
for	reserved	ch 5
function	reserved	ch 3
if	reserved	ch 2
import	reserved	ch 7
in	reserved	ch 16
instanceof	reserved	ch 4
new	reserved	ch 4
null	reserved	ch 2
return	reserved	ch 3
super	reserved	ch 4
switch	reserved	ch 16
this	reserved	ch 3
throw	reserved	ch 6
true	reserved	ch 2
try	reserved	ch 6
typeof	reserved	ch 2
var	reserved	ch 2
void	reserved	ch 13
while	reserved	ch 5
with	reserved	ch 2
let	strict	ch 2
static	strict	ch 4
yield	strict	ch 5
await	strict	ch 6
enum	future	ch 19
implements	future	ch 18
interface	future	ch 15
package	future	ch 2
private	future	ch 18
protected	future	ch 18
public	future	ch 18
abstract	contextual	ch 18
accessor	contextual	ch 18
any	contextual	ch 13
as	contextual	ch 11
asserts	contextual	ch 16
boolean	contextual	ch 12
declare	contextual	ch 19
from	contextual	ch 19
get	contextual	ch 18

word	kind	taught in
global	contextual	ch 19
infer	contextual	ch 17
intrinsic	contextual	ch 17
is	contextual	ch 16
keyof	contextual	ch 17
module	contextual	ch 19
namespace	contextual	ch 19
never	contextual	ch 13
number	contextual	ch 12
object	contextual	ch 17
of	contextual	ch 5
readonly	contextual	ch 15
satisfies	contextual	ch 20
set	contextual	ch 18
string	contextual	ch 12
symbol	contextual	ch 12
type	contextual	ch 12
undefined	contextual	ch 12
unique	contextual	ch 20
unknown	contextual	ch 13

39.2. stdlib coverage

Component families against edition shelves, with the status column carrying this runtime's verdict row by row. ES2026 was approved 2026-06-30 as the seventeenth edition with seven features: this node runs five, ships the json half, and lacks one outright, and the matrix says which is which. Temporal runs the other direction, stage 4 and slated for the es2027 edition, yet fully present and unflagged here, typed through the `esnext.temporal` fragment ahead of any edition library.

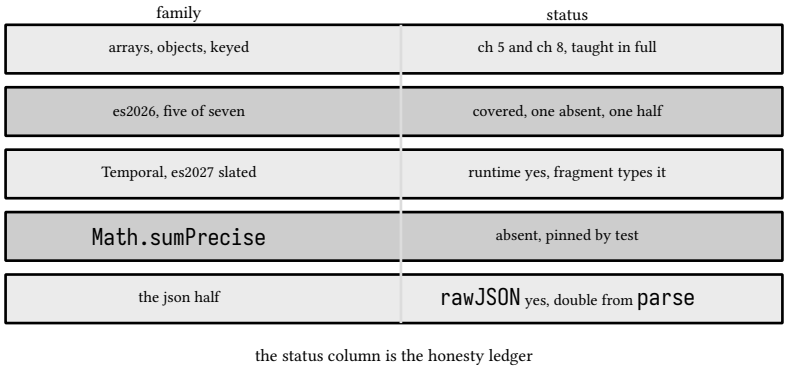


figure 233: families against shelves, the status ledger

component family	chapter	status
Array change-by-copy: toSorted, with, toReversed	ch 8	covered, es2023
Object.groupBy and its null prototype	ch 8	covered, es2024
Promise.withResolvers	ch 8	shelf row only, es2024, not demonstrated
Iterator helpers: map, filter, take, drop, flatMap	ch 5	covered, es2025, lazy by call count
Iterator.concat	ch 5	covered, es2026, lazy by tap count
Set algebra: four producers, three predicates	ch 5	covered, es2025

component family	chapter	status
iteration protocols, Symbol.iterator, replay	ch 5	covered
generators, yield delegation, two way next	ch 5	covered
Map and Set, SameValueZero, insertion order	ch 5	covered
Map and WeakMap getOrInsert, getOrInsertComputed	ch 5	covered, es2026
WeakMap and WeakRef, the deterministic facts	ch 5	covered
Float16Array, Math.f16round, DataView	ch 5	covered, es2025
Uint8Array base64 and hex codecs	ch 5	covered, es2026, byte exact
RegExp.escape, the measured escape classes	ch 8	covered, es2025
Promise.try	ch 6	covered, es2025
using, Symbol.dispose, DisposableStack	ch 7	covered, es2025
import attributes with json, the with clause	ch 7	covered, es2025, assert removed
Array.fromAsync	ch 6	covered, es2026, esnext.array fragment
Error.isError	ch 8	covered, es2026
JSON.rawJSON and isRawJSON	ch 8	covered, es2026, emit exact only
Math.sumPrecise	ch 8	absent, undefined on node 26.3.0, pinned by test
JSON.parse source text access	ch 8	absent, arity 2, parse returns the double
Temporal: Instant, PlainDate, ZonedDateTime, Duration	ch 8	covered at runtime, stage 4, es2027 slated
Temporal string boundary, toJSON rfc 9557	ch 8	covered, relational operators throw
Promise: all, allSettled, race, any	ch 6	covered
AbortSignal, timeout, any	ch 6	covered
microtask and timer ordering	ch 6	covered
for await..of, async generators, Symbol.asyncIterator	ch 6	covered
prototypes, Object.create, Symbol.hasInstance	ch 4	covered
Object statics: keys, defineProperty, freeze	ch 4	covered
string code points against utf-16 units	ch 2	covered
structuredClone boundaries	ch 8	covered
Error cause chains	ch 8	covered, es2022
Intl.Collator numeric sorting	ch 8	covered
live bindings, dynamic import, import.meta	ch 7	covered
node:test and node:assert/strict	ch 1	covered
node:fs, node:path, node:child_process probes	ch 19	covered
node:sqlite, DatabaseSync, Symbol.dispose	ch 38	covered, engine 3.53.4 measured on node 26.10.0
node:v8, getHeapSpaceStatistics, heap spaces	ch 9	covered, 15 spaces measured on 26.3.0
ArrayBuffer external memory, memoryUsage counters	ch 9	covered, arrayBuffers against heapUsed measured
WeakRef and FinalizationRegistry under --expose-gc	ch 9	covered, child run measured
Float64Array against object literals, the particle kernel	ch 9	covered, 1m and 100m measured
--cpu-prof and --heap-prof captures, parsed in suite	ch 10	covered, child runs measured
node:inspector Session, Profiler domain	ch 10	covered, in-process session asserted
monitorEventLoopDelay percentiles, perf_hooks	ch 10	covered, p50 p99 measured
AsyncLocalStorage and async_hooks tracing	ch 10	covered, trace across awaits

component family	chapter	status
node:sqlite row kernel, GROUP BY against a cursor walk	ch 10	covered, 10m and 100m measured
decorators and the accessor keyword	ch 18	covered, stage 3

39.3. pinned sources

Every chapter ends with its own sources line naming the official sections consulted while writing it. The book level pins carry three dates: the 1.0 fetches of 2026-09-08, the 2.0 re-probe of 2026-09-13, when the toolchain was measured again and the new material, the edition approval, the temporal proposal status, and the release notes, was fetched, and the 4.0 fetches of 2026-09-21 for the framework layer, react 19.3.0, svelte 5.57.1, vite 8.3.0, and the import maps specification. The four projects' suites, counted at this edition and not estimated: `javascript/samples`, part one, 133 tests in 11 suites with no build step, `javascript/tssamples`, part two, 105 tests in 10 chapter suites, checked and emitted by `tsc` before node runs them, `javascript/fwsamples`, part three, 53 tests, 37 for the react chapter and 16 for the svelte chapter, unit tests over the `tsc` emit and smokes over five vite builds, and `javascript/capstone`, 63 tests in 10 suites, 28 plain against the `.mjs` layer, 21 web against the microfrontend, and 14 typed through the facade. Together 354 tests, and the repo's `make verify-ts` runs all four projects plus the repertoire suite.

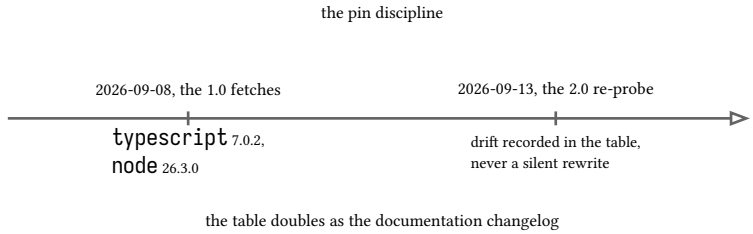


figure 234: the pins as one dated discipline

topic	url	accessed
announcing typescript 7.0	devblogs.microsoft.com/typescript/announcing-typescript-7-0	2026-09-08
typescript 6.0 stepping stone	devblogs.microsoft.com/typescript/announcing-typescript-6-0	2026-09-08
tsconfig reference	typescriptlang.org/tsconfig	2026-09-08
handbook: everyday types, narrowing	typescriptlang.org/docs/handbook	2026-09-08
handbook 2: mapped, conditional, template types	typescriptlang.org/docs/handbook/2	2026-09-08
mdn lexical grammar and reserved words	developer.mozilla.org/docs/Web/JavaScript/Reference/Lexical_grammar	2026-09-08
mdn javascript reference	developer.mozilla.org/docs/Web/JavaScript/Reference	2026-09-08
mdn iteration protocols	developer.mozilla.org/docs/Web/JavaScript/Reference/Iteration_protocols	2026-09-08
node test runner	nodejs.org/api/test	2026-09-08
node esm and import attributes	nodejs.org/api/esm	2026-09-08
node type stripping	nodejs.org/api/typescript	2026-09-08
tc39 proposals: groupby, fromAsync, withResolvers	tc39.es	2026-09-08
tc39 decorators proposal	github.com/tc39/proposal-decorators	2026-09-08
ecma 262 es2026, the 17th edition, approved 2026-06-30	ecma-international.org	2026-09-13

topic	url	accessed
tc39 temporal proposal, stage 4, slated for es2027	github.com/tc39/proposal-temporal	2026-09-13
node 26.3.0 release line, v8 14.6, npm 12.0.2	nodejs.org/en/blog/release	2026-09-13
typescript package pin, 7.0.2, es2026 rejected TS6046	npmjs.com/package/typescript	2026-09-13
v8 dev blog, tail calls and temporal shipping	v8.dev/blog	2026-09-13
tc39 proposals: iterator.concat, getOrInsert, uint8array codecs	tc39.es	2026-09-13
tc39 explicit resource management and import attributes	tc39.es	2026-09-13
react 19 reference: hooks, use, actions, error boundaries	react.dev/reference/react	2026-09-21
react 19 release post, ref as prop and form actions	react.dev/blog/2024/12/05/react-19	2026-09-21
svelte 5 docs: runes, snippets, custom elements, stores	svelte.dev/docs/svelte	2026-09-21
vite guide and library mode	vite.dev/guide	2026-09-21
whatwg import maps specification	html.spec.whatwg.org/multipage/scripting.html#import-maps	2026-09-21

NOTE: version drift rule

The 1.0 pins were fetched 2026-09-08 against typescript 7.0.2 on windows with node 26.3.0, and the 2.0 edition re-probed the same toolchain on 2026-09-13, node v26.3.0 with V8 14.6.202.34-node.20 and npm 12.0.2. The projects pin `typescript` at `^7.0.2` with `es2025` targets plus two fragments, `esnext.array` and `esnext.temporal`. The drift worth naming: `tsc` 7.0.2 knows no es2026 at all, `--target es2026` and `--lib es2026` both answer TS6046 and the list it prints ends at `es2025` then `esnext`, so the runtime is ahead of its checker's edition vocabulary and the fragments are the bridge. Re-verified 2026-09-30 under the corpus version currency law, which rides the newest installable build of every vendor, nightlies excepted: 7.0.2 is still the newest typescript on the npm dist-tags, 7.1 exists only as nightly dev builds with the es2026 target and lib committed for its beta, and both TS6046 answers reproduced verbatim on that date, so the pin stands and moves the day a newer installable build ships. Temporal is stage 4, slated for the es2027 edition, shipped unflagged in node 26.3.0, and absent from `@types/node` 26, so without the fragment a file using it fails TS2503. Decorators are still stage 3, the default in 7.0.2 with no flag. On this runtime `Math.sumPrecise` is `undefined` and `JSON.parse` takes two arguments, so `JSON.rawJSON` pins digits on the way out while parsing the same text returns the double. The 4.0 framework wave re-probed its layer 2026-09-21, react 19.3.0, svelte 5.57.1, the svelte vite plugin 7.3.0, and vite 8.3.0, all measured on the projects' installed trees, and the browser facts are node-measured, no browser ran in the suite. If an upstream page changes, the delta gets recorded here rather than silently rewriting chapters, so this table is also the book's changelog for documentation movement.